

Proiectarea cu Microprocesoare
Curs 1

Andrei Voinescu

POLITEHNICA University of Bucharest

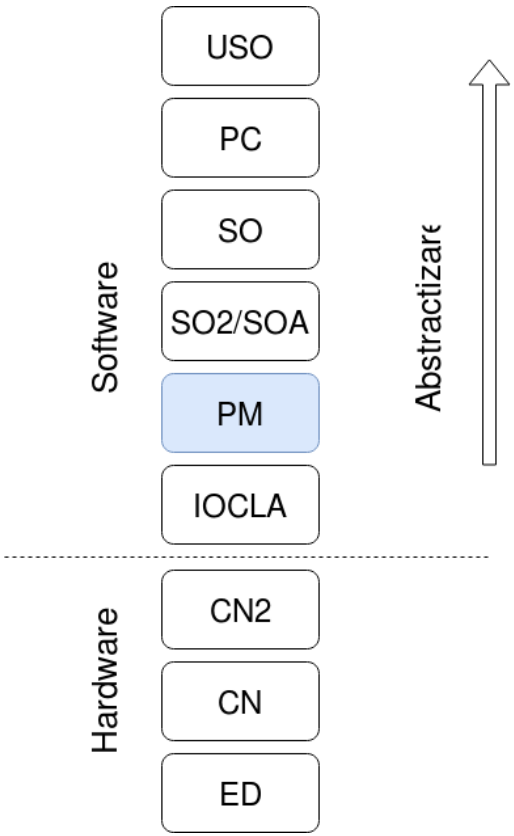
3 Martie 2021



Ce este Proiectarea cu Microprocesoare?

- Proiectarea de sisteme care să conțină cel puțin un microprocesor
  - Ce este un microprocesor?
  - În zilele de azi, toate procesoarele sunt microprocesoare
- Considerăm un sistem cu microprocesor a fi orice de la un sistem de control pentru o mașină de spălat vase, la sistemul de navigație al unui avion, la o placă de bază pentru un PC
- Accentul se pune pe proiectare, ceea ce dă un caracter profund practic materiei
- Vrem să aducem la viață sisteme de calcul, și tocmai asta veți face în cadrul laboratorului și proiectului!

PM în contextul altor materii



- ED, CN și CN2 prezintă bazele sistemelor de calcul
- USO este utilizator al unei abstracții la nivel de shell
- PC se bazează pe abstracțiile de sistem de operare și pe libc
- SO se bazează pe ceea ce oferă sistemul de operare
- SO2 interfațează hardware-ul, dar se concentrează pe a oferi funcționalități nivelelor de deasupra
- PM interfațează hardware-ul

De ce facem Proiectarea cu Microprocesoare?

- Sisteme de calcul cu microprocesoare se găsesc peste tot
- Este important pentru o formație de inginer să știe ce se află în calculatorul lui, cum se interconectează componentele măcar la un nivel teoretic
- Proiectarea de mici dispozitive hardware este din ce în ce mai relevant în contextul tehnologic actual:
  - Legea lui Moore a încetinit [1], dar tendințele de miniaturizare continuă
  - Internet of Things este buzzword-ul decadei și propune miliarde de dispozitive mici interconectate
  - Aceste dispozitive ar fi construite atât de firme mari, startup-uri sau chiar hobby-iști

[1] https://www.technologyreview.com/s/601441/moores-law-is-dead-now-what/

- Curs
  - Prezentarea diferitelor periferice și metode de comunicare folosite în laborator
  - Metode de programare pentru microcontroller-ul de laborator
  - Proiectarea unei plăci *de bază*, bazată pe 8086 :)
  - Altele
- Laborator (primele 7 săptămâni)
  - Programarea pe microcontroller-e, Arduino
- Proiect (ultimele 7 săptămâni)
  - Construirea unui dispozitiv hardware cu o anumită funcție
    - Roboți
    - Jocuri pe LCD (Gameboy, nu Nintendo DS)
    - Mașini de făcut clătite, mașinuțe teleghidate, etc

Precizări importante

- Nu tot conținutul va fi pe slide-uri
- Cursul poate fi modificat de anul trecut
- Cursul înregistrat este pe Youtube (link pe Moodle)
- Toate materialele pentru laborator și proiect se găsesc pe wiki la http://ocw.cs.pub.ro/pm





Notare

- 4p Lucrări de curs (Moodle)
- 2p Examen final (Moodle)
- 1p Activitate Laborator
- 3p Proiect
- 1p Colocviu după terminarea laboratoarelor

- 3p din punctajul final
  - 20p wiki[2]
  - 50p implementare hardware
  - dintre care 5p pentru lucrat ordonat
  - 30p implementare software
  - dintre care 5p pentru cod scris ordonat
  - -10p nerespectare milestone-uri (5p x 2)
- Motivații pentru *
  - Nu vrem documentații fără proiect
  - Placa de bază este doar un punct intermediar, nu un scop în sine

[1] Dacă proiectul este dezvoltat dincolo de placa de bază

- 2 Milestone-uri
  - Sunt aici ca să vă ajute!
  - de pe wiki:
    - *"PS:Nu lasati proiectul pe ultima suta de metri.Lucrati din timp!"*
    - *"trebuia sa ma apuc de proiect cu mult mult timp inainte"*
  - Încercați să vă țineți de ele, dar nu fiți limitați de ele
    - Puteți să le îndepliniți și mai devreme!

- Reprezintă ocazia unică din facultate de a construi un dispozitiv hardware
- Beneficiați de ajutorul echipei de PM în atingerea acestui scop
- Are un preț asociat
- Considerăm că prețul este justificat pentru cât învățați din acest proiect



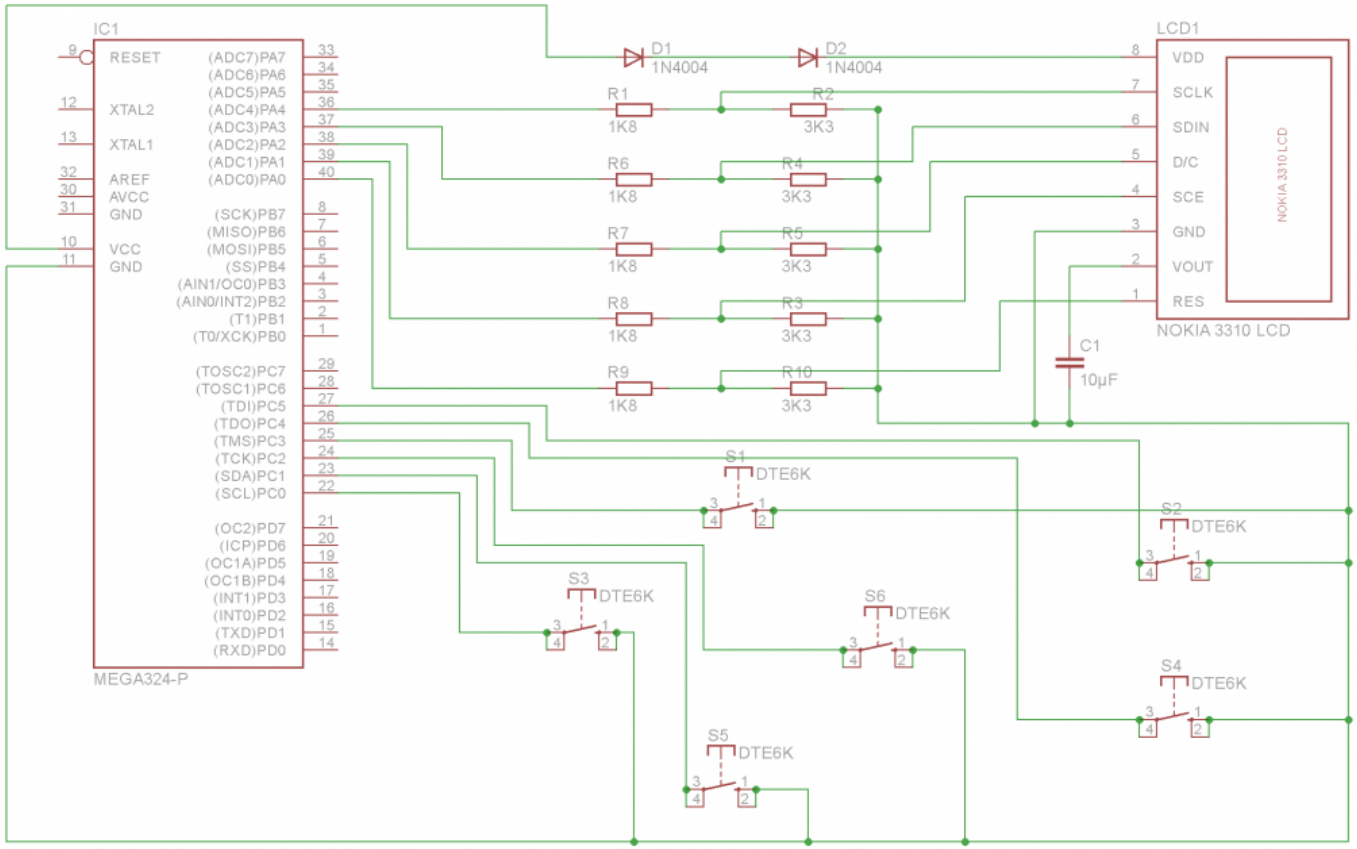
- Proiectul de PM există din timpuri imemoriale
  - No really, chiar nu știu de când e, 15 ani pe puțin
  - Cronicile pe wiki încep abia din 2009
- Aveți pe wiki un Hall of Fame
  - `http://ocw.cs.pub.ro/pm/halloffame`

Săptămâna 9 - Alegerea temei, schema bloc și lista de piese

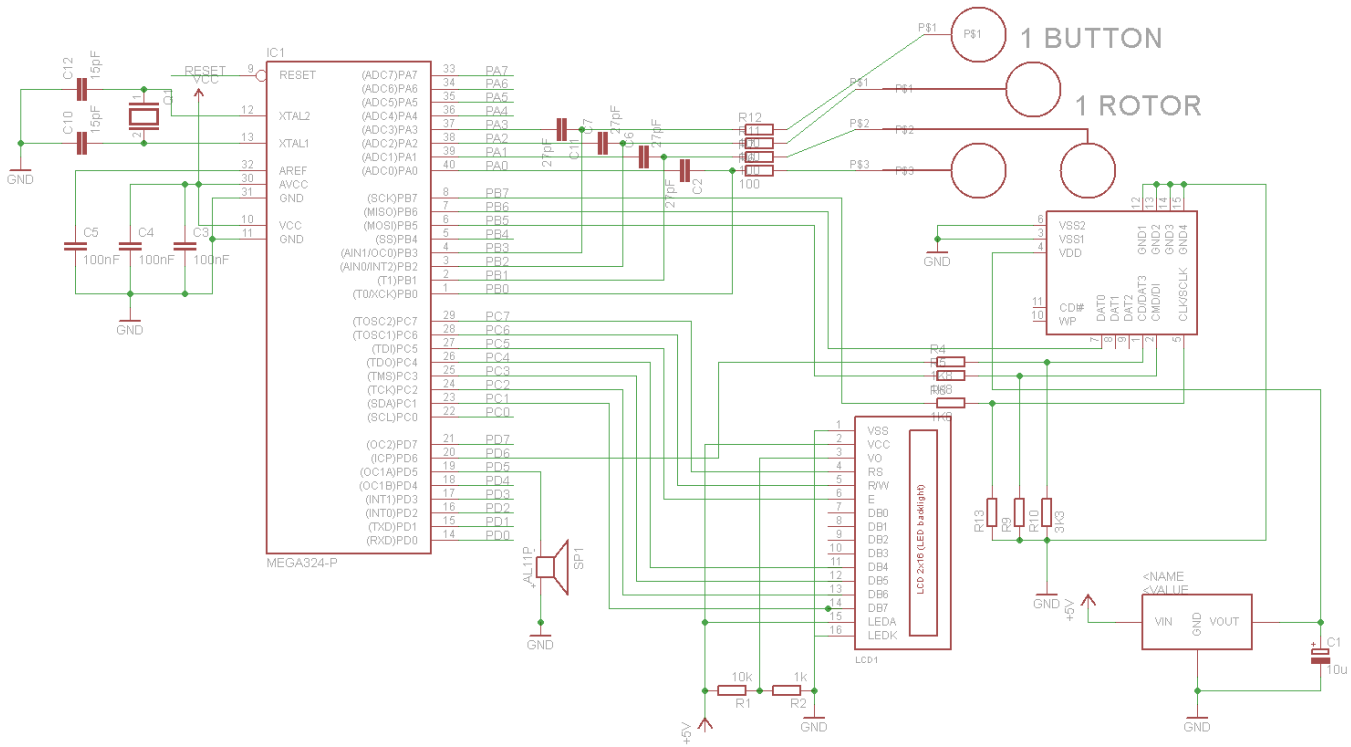
- The sky is the limit!
- Ideal - vă gândiți de pe acum
  - Aveți proiecte din 2009-2019, știți ce se poate face
  - Instructables, Sparkfun, Avrfreaks, Make, Hackaday
- Săptămâna 8 - vă întâlniți cu asistentul de proiect, discuții
- Asistentul de proiect trebuie să aprobe tema
  - Nu o să se aprobe: ceasuri digitale, termometre digitale etc.
  - Dacă totuși aprobă un proiect considerat simplu, acesta va fi considerat la 80% din notă
  - Rule of thumb: Dacă e mai simplu decât un lab de PM, nu e bine
- Schema bloc (*Atenție*, nu electrică)
  - 3-4 componente sau câte aveți în proiectul vostru, cu conexiuni simple între ele

- Nu trebuie să fie lista finală, dar trebuie să conțină ce aveți nevoie pentru partea a doua a proiectului
- Listă de piese
  - Fără piesele din prima parte
  - Așa da: *senzor de temperatură LM35, rezistență* $1k\Omega$*, fire*
  - Așa nu: *rezistențe, condensatori*
  - Implică documentare
  - Găsit supplier
- Maica Domnului, Robofun, Farnell, Okazii, Ebay, Optimus Digital
- Trade-off între timp și bani
  - Placă de motor făcută de voi ∼20RON
  - Cumpărată de pe robofun ∼80RON
- de pe wiki
  - *"Te ajută foarte mult să îți faci lista de piese înainte ca să nu fii nevoit să te dui de un milion de ori să iei piese"*[sic]

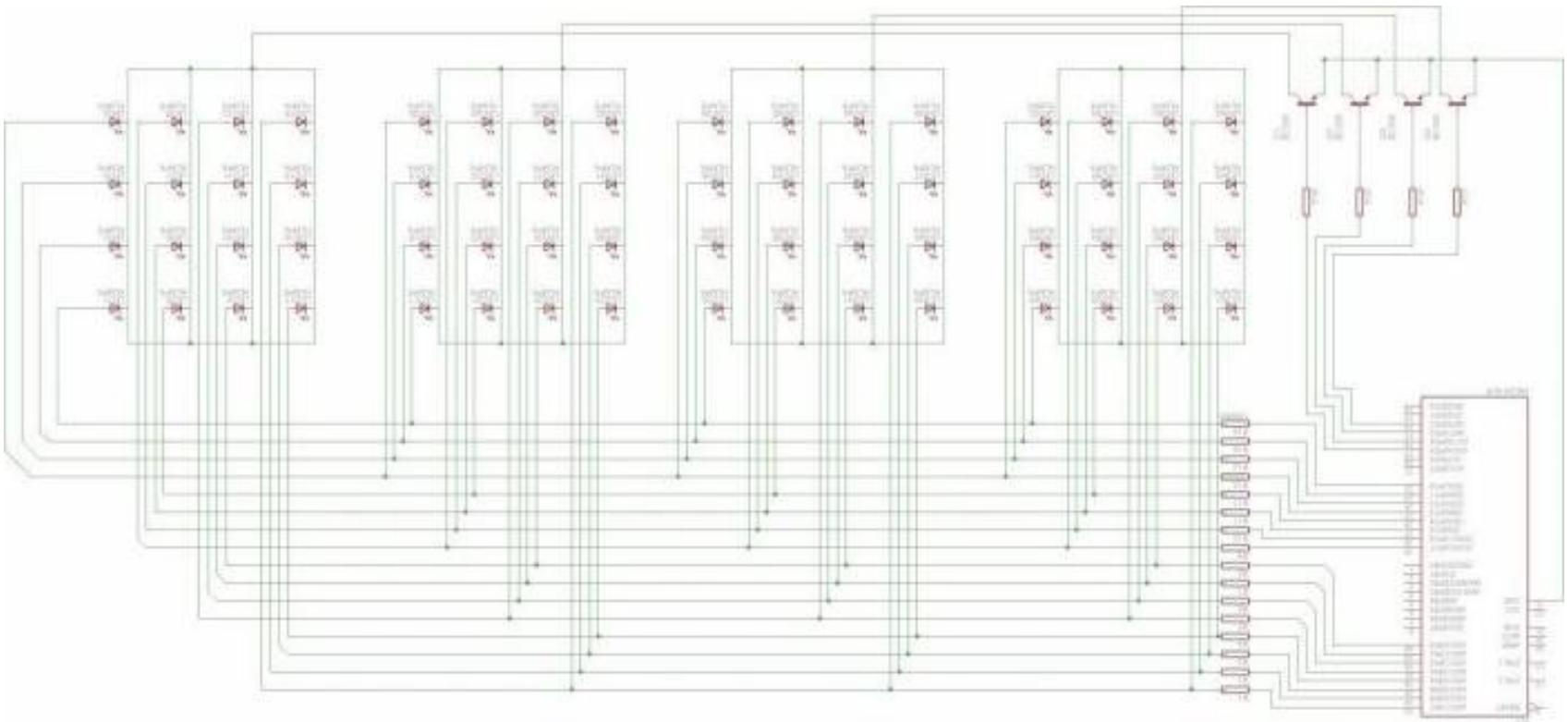
- Nu trebuie să fie finală, dar trebuie să fie completă
- Recomandăm Eagle, nu trebuie făcut decât schematic
- Poate fi mică



- Nu trebuie să fie finală, dar trebuie să fie completă
- Recomandăm Eagle, nu trebuie făcut decât schematic
- Poate fi medie



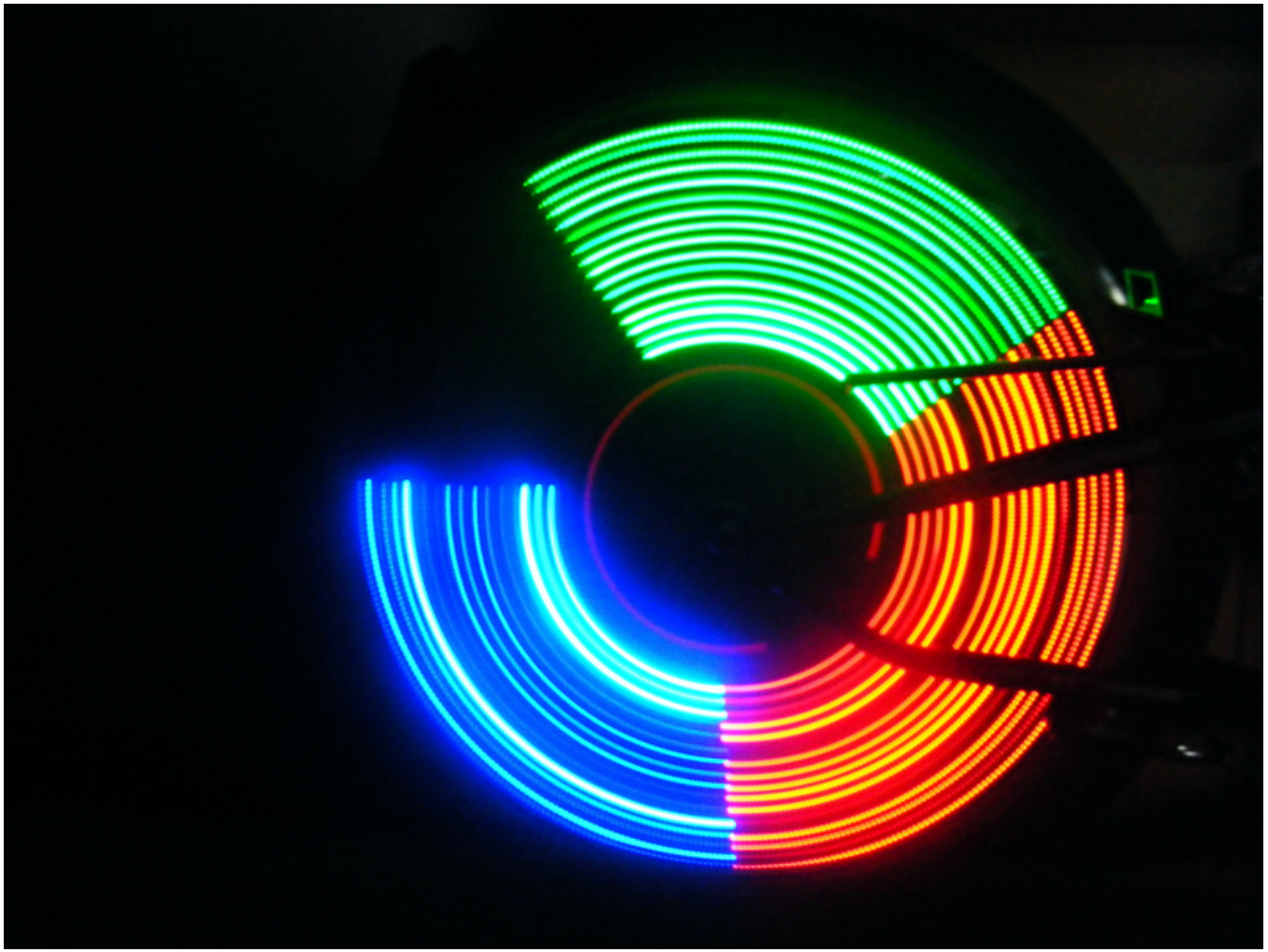
- Nu trebuie să fie finală, dar trebuie să fie completă
- Recomandăm Eagle, nu trebuie făcut decât schematic
- Poate fi mare



- Mașina de făcut clătite



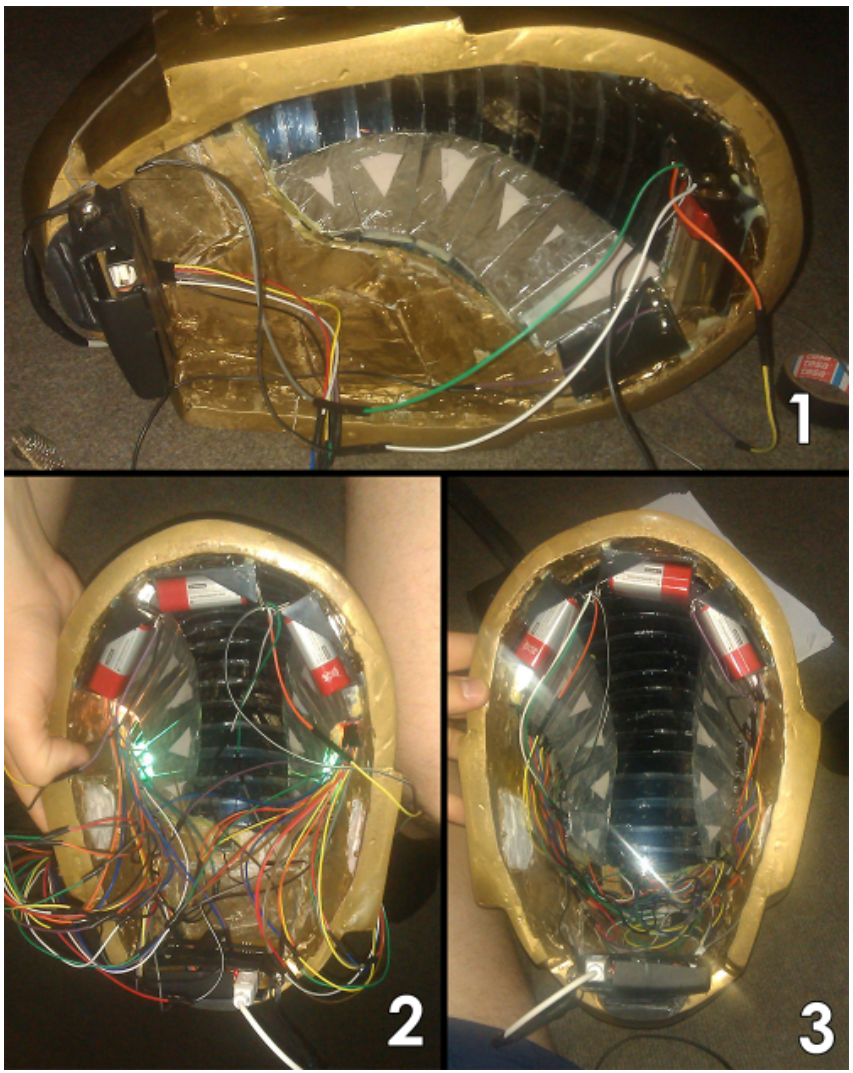
- POV pentru bicicletă - Persistence of Vision



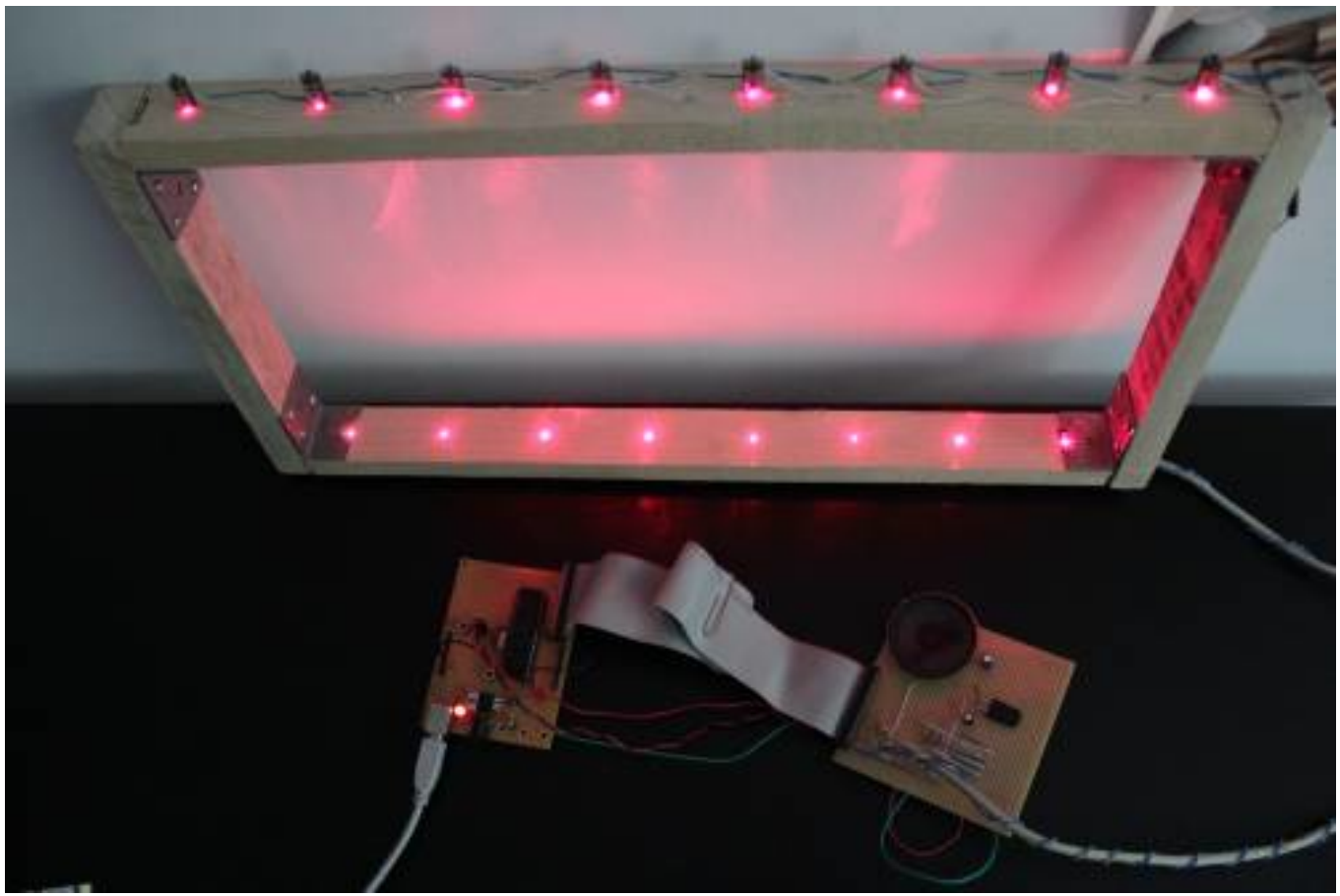
- Air POV - Persistence of Vision



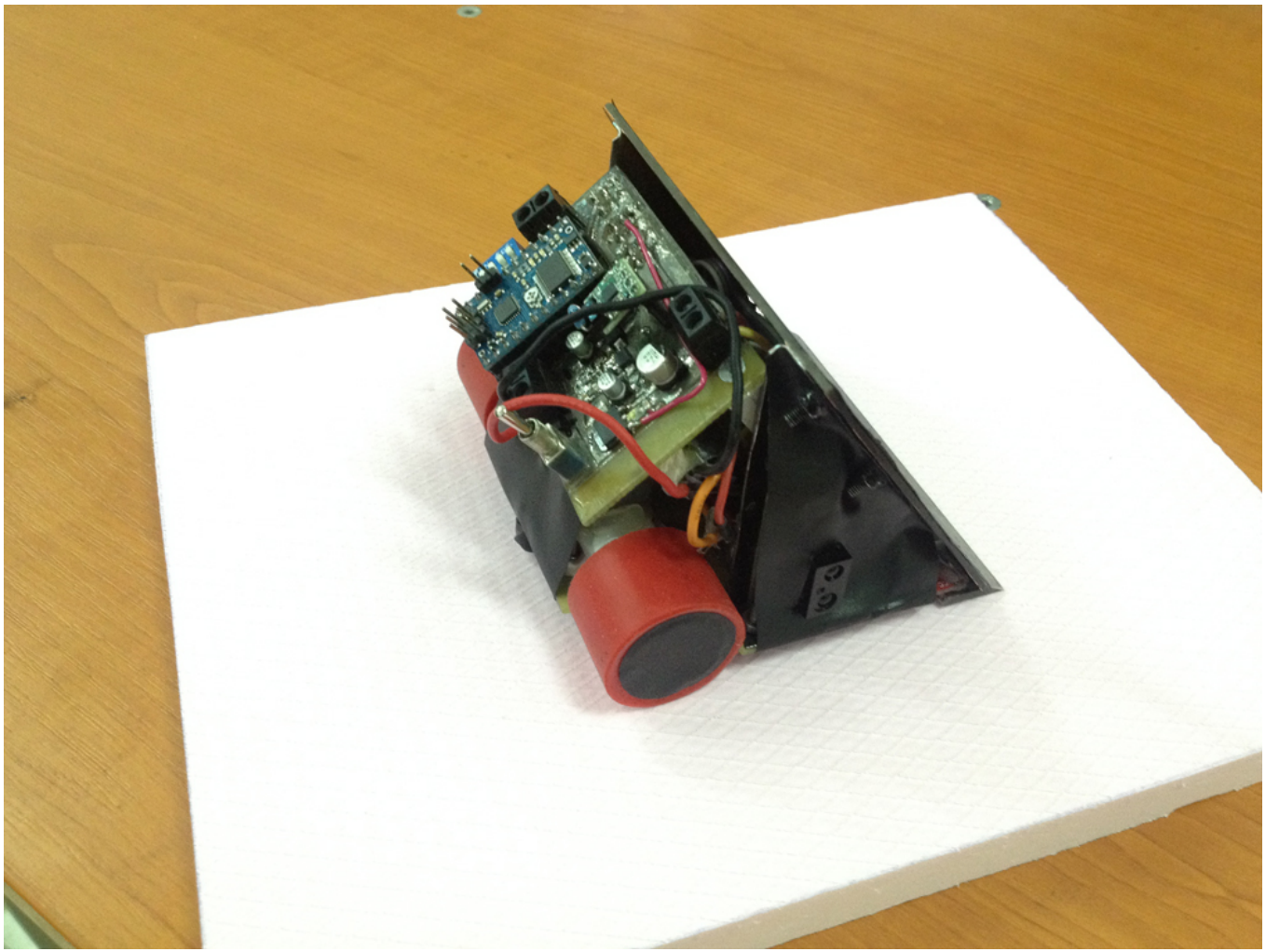
- Cască Daft Punk



- Harpă Laser



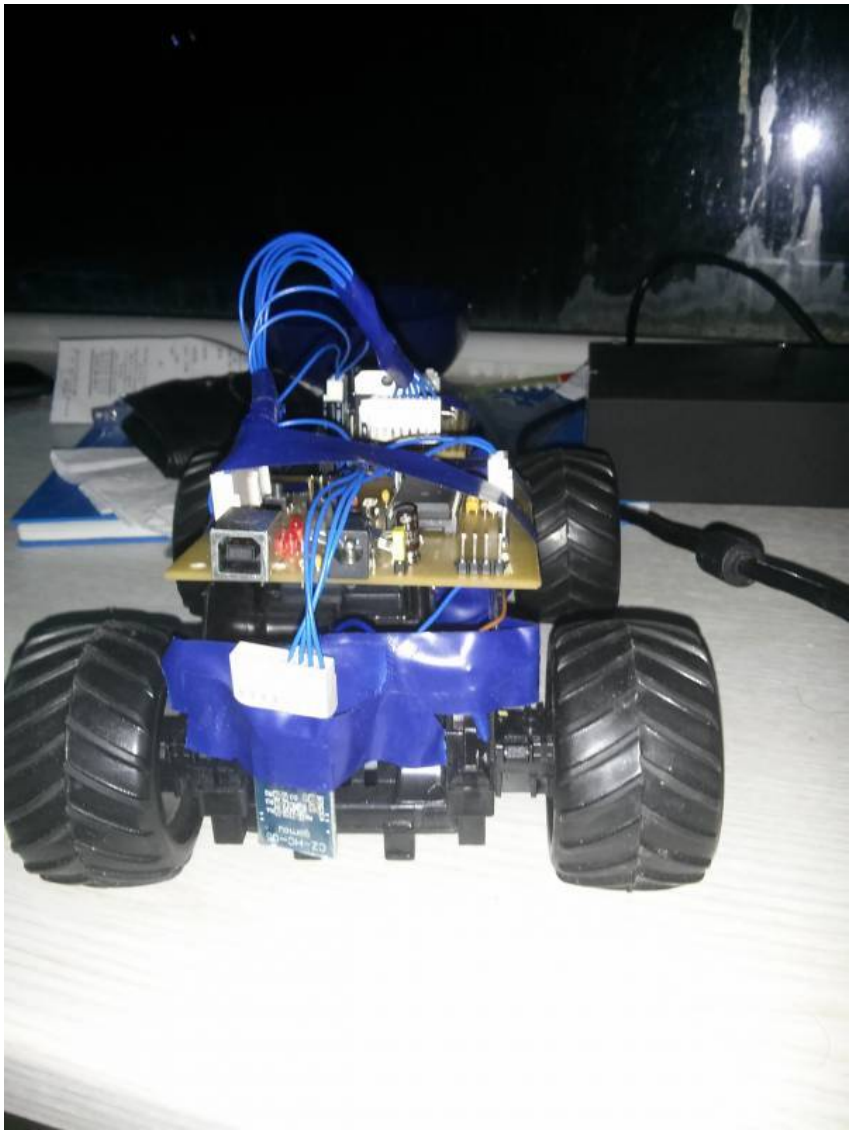
- Robot Mini-sumo



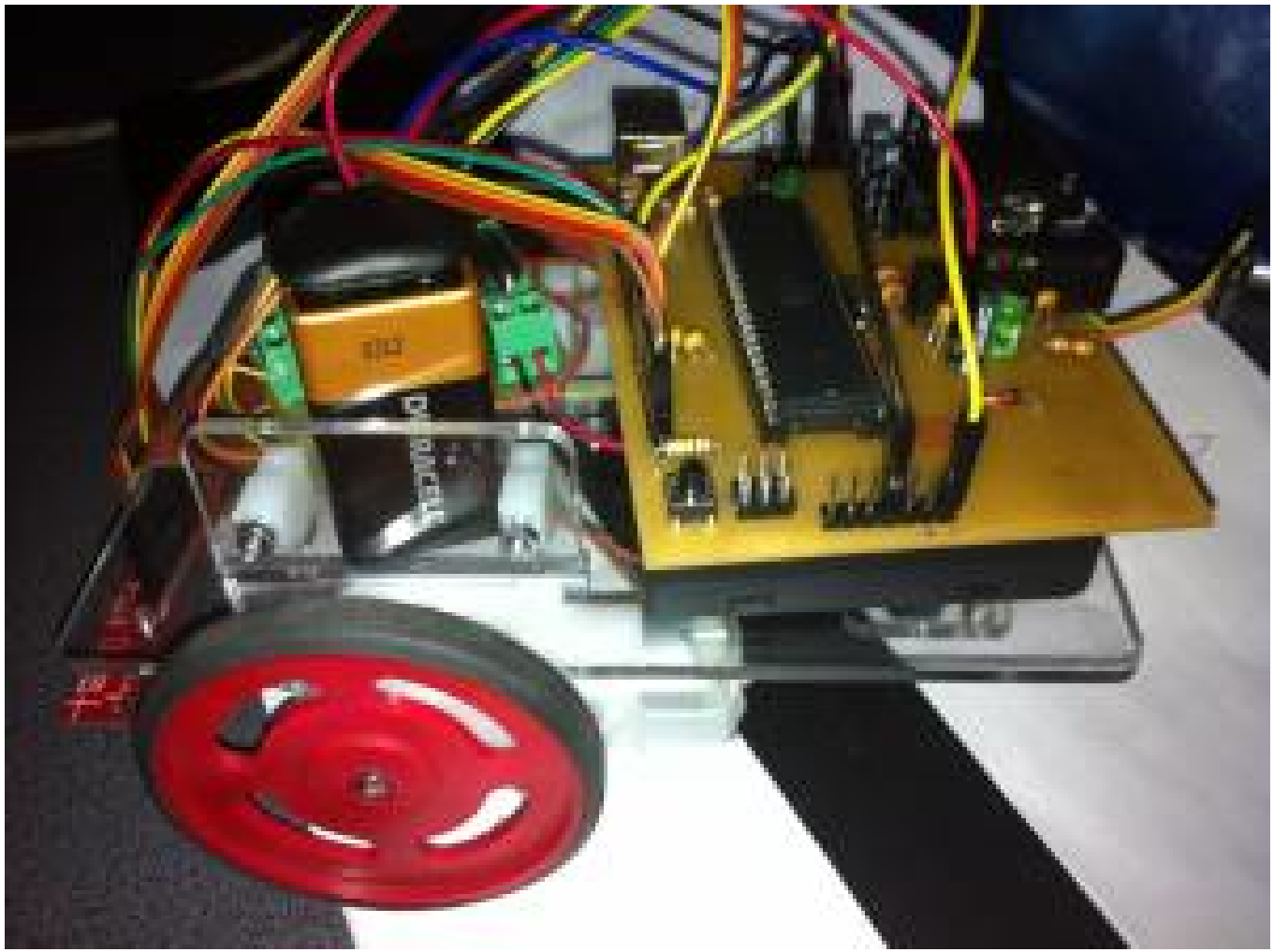
- Joc 2048 pe LCD



- Mașină comandată pe Bluetooth



- Line Follower Robot



- Braț robotic



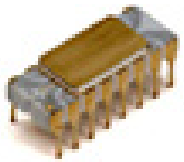
- Un dispozitiv electronic pe o singură plachetă de siliciu care execută instrucțiuni
- De unde încarcă instrucțiuni?
- De unde preia intrările?
- Unde stochează ieșirile?

- 1947 - dezvoltarea primului tranzistor în cadrul Bell Laboratories
- 1959 - prima "integrare" a tranzistoarelor pe același substrat
- 1969 - primul chip INTEL - o memorie de 1Kbit RAM
- 1971 - primul microprocesor INTEL - 4004, pe 4 biți

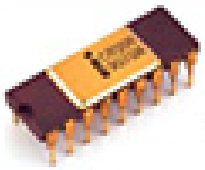


- 1972 - primul microprocesor pe 8 biți - 8008
  - 800 kHz
  - 3500 tranzistoare
  - 6 registre
  - spațiu de adrese de 64KB (PC de 14 biți)
- 1974 - Brian Kernighan și Dennis Ritchie dezvoltă C
- 1978 - Intel lansează 8086
  - Primul microprocesor pe 16 biți
  - Magistrală de date de 16 biți, dar adrese pe 20 de biți - poate accesa 1MB de memorie

Alte procesoare din aceeași perioadă



Intel 4004 (1971)
0.1 MHz, 4-bit
The world's first single-chip microprocessor



Intel 8008 (1972)
0.2 MHz, 8-bit



Intel 8080 (1974)
2 MHz
Space Invaders machines

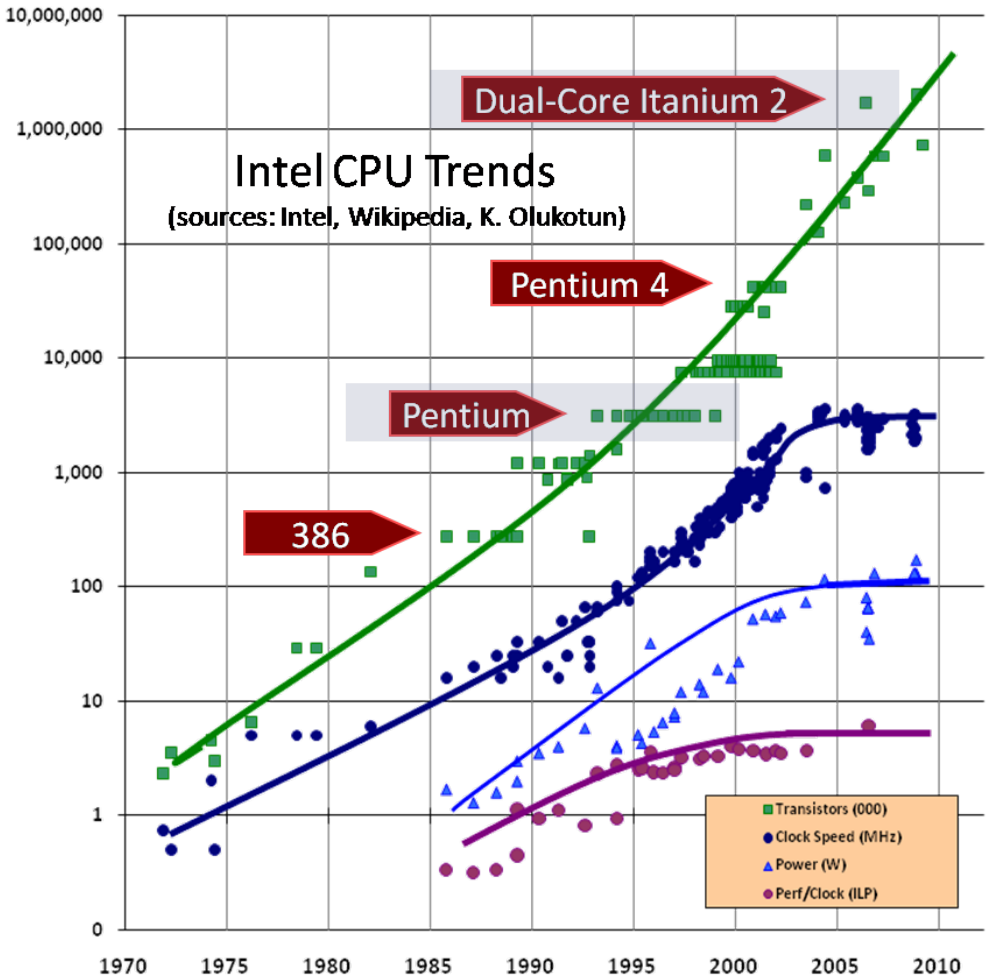


Zilog Z80 (1976)
2 – 4 MHz
ZX81 & ZX Spectrum;
TRS-80; CP/M machines
& Amstrad PCW

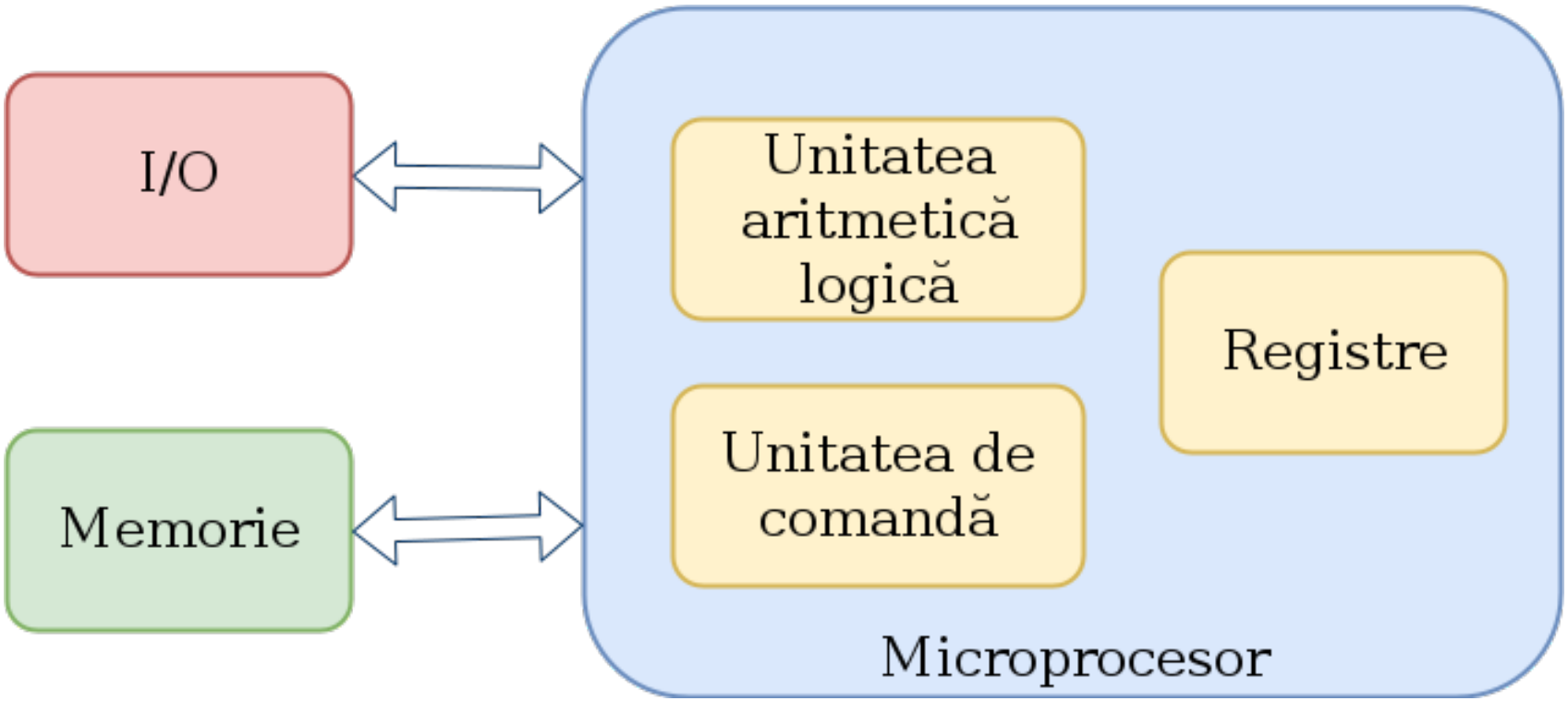


MOS 6502 (1975)
1 – 4 MHz
Apple I & II, BBC Micro,
Atari VCS, Nintendo NES,
Commodore 64, and many
early arcade machines

Creșterea procesoarelor de-a lungul timpului

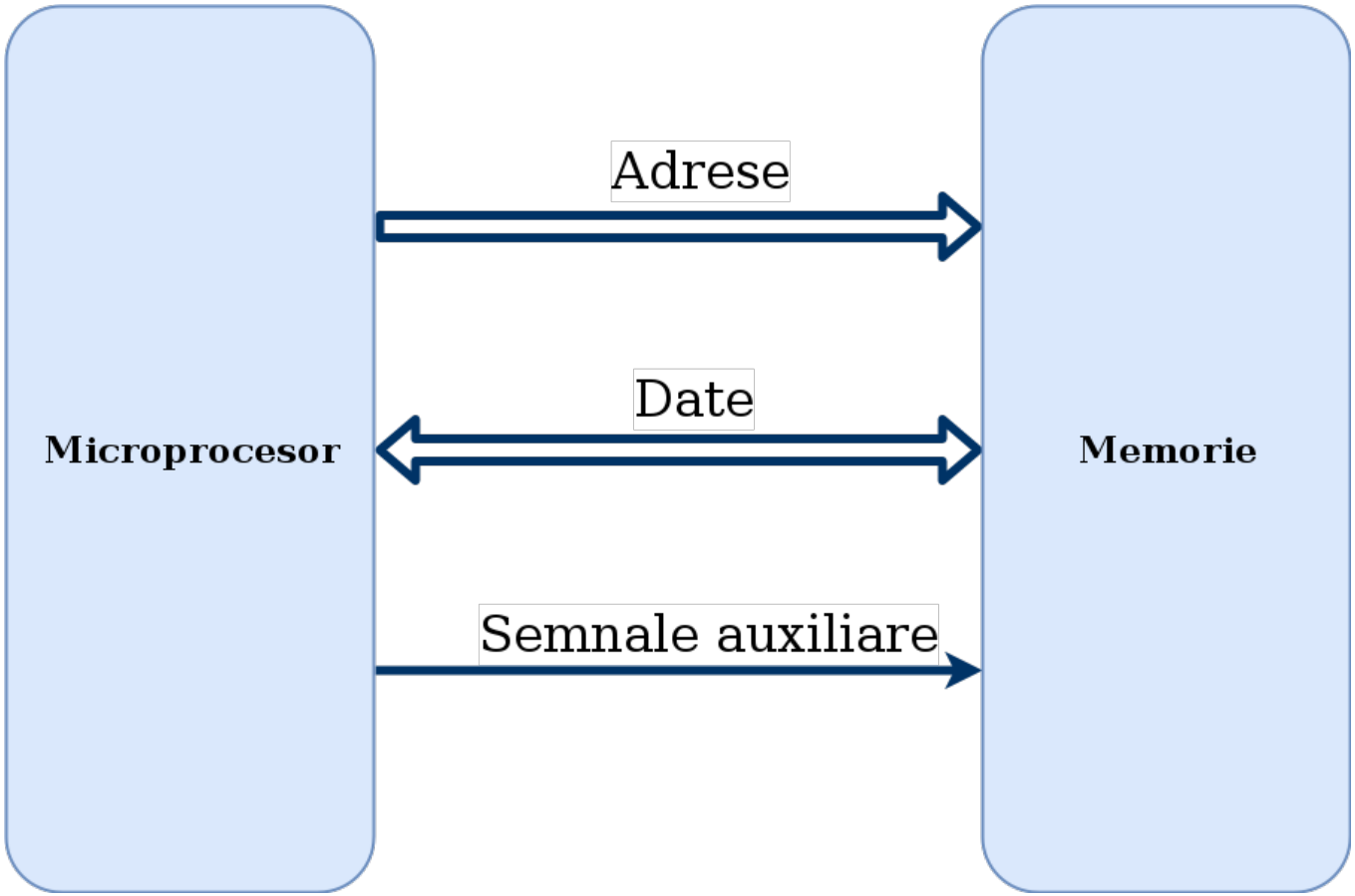


- Din punctul de vedere al accesului la memorie, sunt 2 arhitecturi:
  - *von Neumann*, unde memoria conține atât instrucțiuni cât și date. PC-urile actuale sunt toate von Neumann
  - *Harvard*, unde accesul la memorie se face pe magistrale separate, una pentru date, una pentru instrucțiuni. Microcontrollere AVR de la laborator sunt arhitectură Harvard


I/O
Memorie
Unitatea aritmetică logică
Unitatea de comandă
Registre
Microprocesor

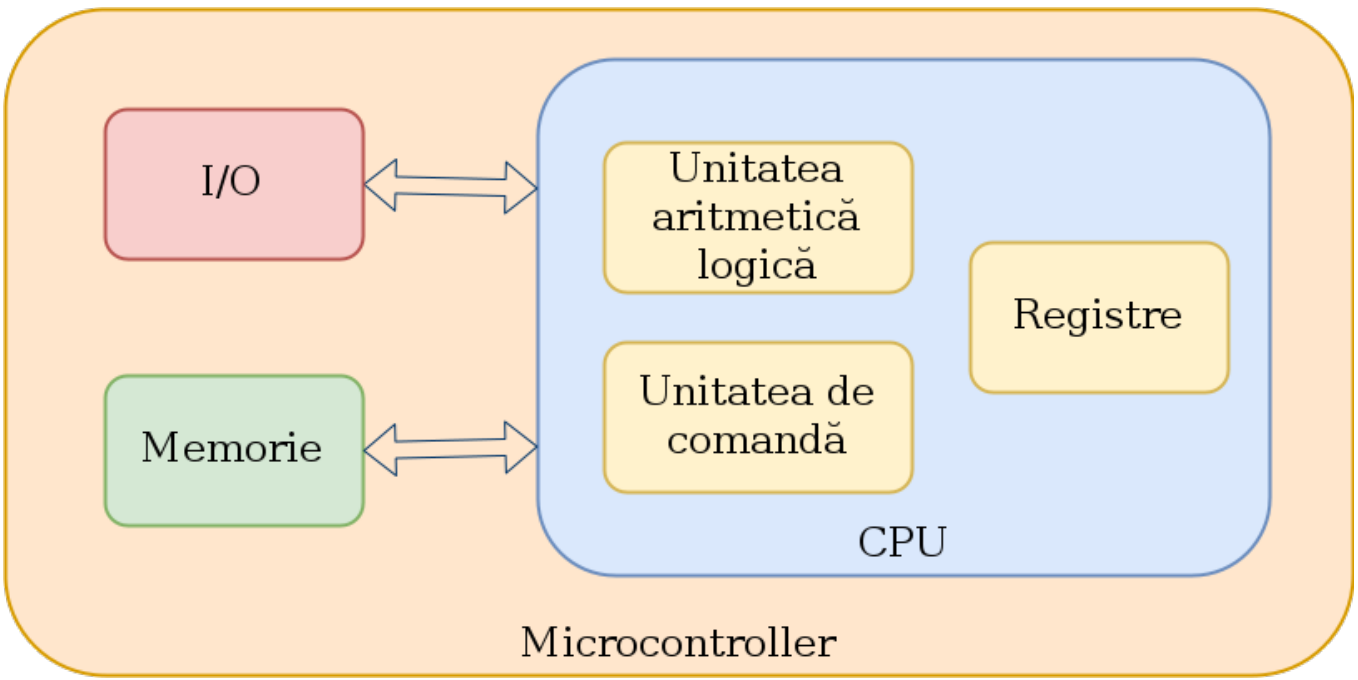
- Un microprocesor are nevoie de:
  - Memorie
  - Periferice I/O
  - Circuit de ceas
  - Circuit de RESET
  - ...
- Există componente care nu sunt strict necesare, dar fac viața mai ușoară
  - DMA
- Toate acestea presupun componente externe chip-ului care trebuie așezate pe o placă

- Magistrala asigură transferul de date bidirecțional între procesor și o locație din memorie
- Accesul procesorului la memorie are loc doar prin magistrală

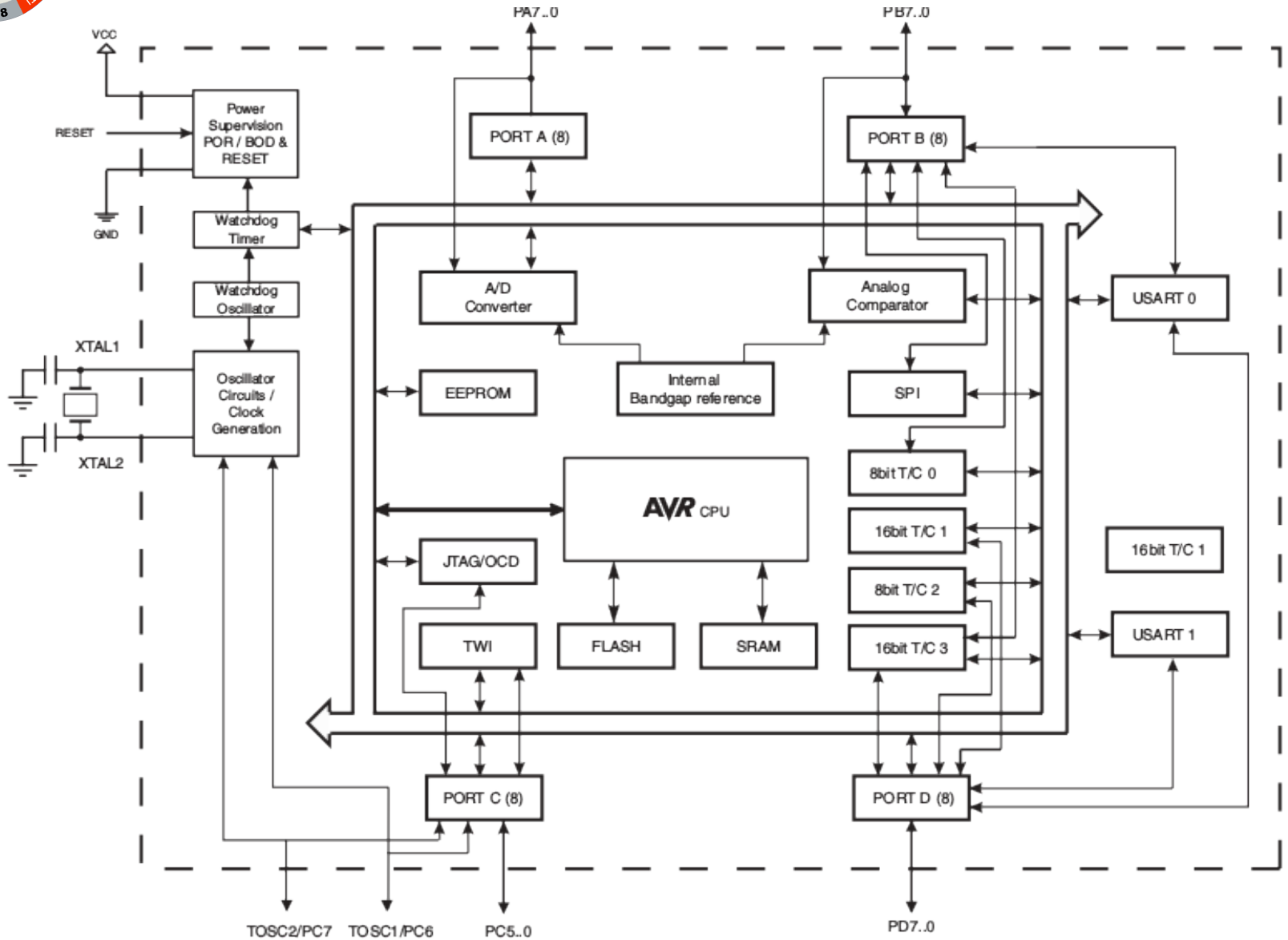


- Sunt două metode de a accesa periferice I/O:
- Memory-mapped I/O:
  - Perifericul I/O are o adresă, registrele lui sunt accesibile la acea adresă
- Port-mapped I/O:
  - Microprocesorul are instrucțiuni speciale de acces al perifericelor I/O, care se traduc la exteriorul chip-ului prin alte semnale auxiliare decât cele pentru citire/scriere din/în memorie

- Microcontroller-ul este un chip integrat care conține un microprocesor, cu memorii și cu periferice I/O
- Unde microprocesorul are nevoie de o placă de bază întreagă, microcontroller-ul este self-contained
- un uP poate fi 100$, un uC poate fi $1



Schemă bloc ATMega324a



- Arhitectură Harvard:
  - Există o memorie de program (32kB)
  - Există o memorie de date (2kB)
- Intrările și ieșirile sunt *port-mapped*
  - Există instrucțiuni speciale (`in` și `out`) de acces la memoria I/O
  - Compilatorul se ocupă singur de diferențierea între scriere în memorie și scriere către I/O

- Încărcarea programului
  - Cu ajutorul unui programator extern sau a unui bootloader, binarul de AVR trebuie să ajungă în memoria de program
- Lansarea în execuție
  - La resetarea microcontroller-ului, se pornește execuția începând cu adresa 0[3]
- Fire de execuție
  - Unul singur, dat de registrul `PC` (Program Counter)
- Interacțiune cu exteriorul
  - Doar prin instrucțiuni `in`/`out` (port-mapped I/O)

[3] diferă în cazul bootloader-ului

- În C, foarte similar cu programarea pe PC
  - Nu avem consolă
  - GCC este configurat pentru un sistem pe 8 biți (`int` și pointerii sunt pe 16 biți)
  - Avem macrodefiniții pentru toate adresele relevante de I/O, compilatorul știe automat să folosească instrucțiunile dedicate pentru scrierile și citirile de la aceste adrese

```
#define PORTC *(volatile uint8_t *) (0x020)
```

```
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>

int main()
{
    DDRC = (1 << PC4);


    while (1)
    {
        PORTC ^= (1 << PC4);
        _delay_ms(250);
    }
}
```

- Header-ul cu registrele I/O
- Frecvența de lucru (determinată de placă)
- Setarea pinului PC4 ca ieșire
- Alternarea stării pinului PC4 o dată la 250 de milisecunde

Cum arată rezultatul?

```
00000000 <__vectors>:
   0:   0c 94 3e 00     jmp 0x7c                ; 0x7c <__ctors_end>
                                                ; care face jump catre main dupa initializar
00000094 <main>:
  94:   80 e1           ldi r24, 0x10           ; (1 << 4)
  96:   87 b9           out 0x07, r24           ; scriere in DDRC
  98:   90 e1           ldi r25, 0x10           ; (1 << 4)
  9a:   88 b1           in  r24, 0x08           ; citire din PORTC
  9c:   89 27           eor r24, r25            ; r24 = r24 XOR r25
  9e:   88 b9           out 0x08, r24           ; scriere in PORTC
  a0:   2f ef           ldi r18, 0xFF           ; 255
  a2:   34 e3           ldi r19, 0x34           ; 52
  a4:   8c e0           ldi r24, 0x0C           ; 12
  a6:   21 50           subi    r18, 0x01       ; 1 ciclu
  a8:   30 40           sbci    r19, 0x00       ; 1 ciclu
  aa:   80 40           sbci    r24, 0x00       ; 1 ciclu
  ac:   e1 f7           brne    .-8             ; 0xa6 <main+0x12>
                                                ; 1 ciclu daca nu face conditia, 2 altfel
                                                ; practic se numara de la 0x0C34FF la 0
                                                ; 0x0C34FF * 5 cicli - 1 de iesire
  ae:   00 c0           rjmp    .+0             ; 0xb0 <main+0x1c>
  b0:   00 00           nop                     ; nop + rjmp = 3 cicli
  b2:   f3 cf           rjmp    .-26            ; 0x9a <main+0x6>
```



Proiectarea cu Microprocesoare
Curs 2

Andrei Voinescu

POLITEHNICA University of Bucharest

10 martie 2021

- Încărcarea programului
  - Cu ajutorul unui programator extern sau a unui bootloader, binarul de AVR trebuie să ajungă în memoria de program
- Lansarea în execuție
  - La resetarea microcontroller-ului, se pornește execuția începând cu adresa 0[1]
- Fire de execuție
  - Unul singur, dat de registrul `PC` (Program Counter)
- Interacțiune cu exteriorul
  - Doar prin instrucțiuni `in`/`out` (port-mapped I/O)

[1] diferă în cazul bootloader-ului

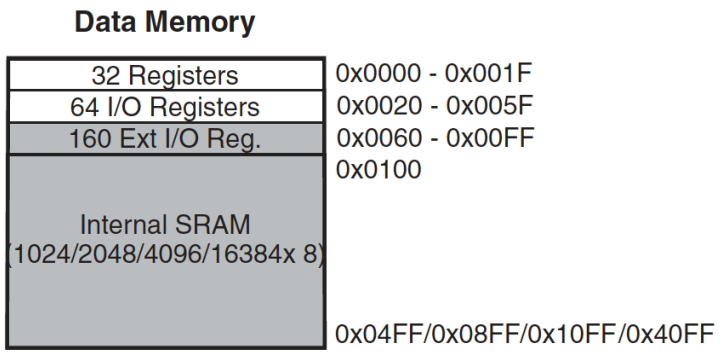
- Programul poate fi pus în memoria de program cu ajutorul unui In-System Programmer sau a unui debugger extern
- Alternativ, putem folosi un bootloader, un program care funcționează doar dintr-o parte a memoriei de program și care poate scrie în restul memoriei
  - Fiind arhitectură Harvard, în mod normal nu există instrucțiuni de acces la memoria de program pentru scriere
    - În loc de instrucțiuni de `ld` și `st` (load și store), avem `lpm` și `spm`

- Bootloaderul ocupă spațiu din memoria de program
  - Dacă bootloader-ul ocupă 2kB, rămân 30kB pentru programul nostru
- În același timp trebuie să conțină codul necesar pentru a obține programul nou
- Memoria de program este în tehnologie Flash, deci trebuie scrise datele în **pagini** (dimensiunea depinzând de chip)

- În C, foarte similar cu programarea pe PC
- Input/Output?
  - Nu avem consolă
  - Poate avem LEDuri conectate la microcontroller
  - Putem avea un LCD
  - Serială, etc.
- Variabile?
  - Le putem avea în RAM (2kB)
  - Le putem avea și în Flash (32kB, împreună cu codul)
- Pornire la reset (sau după bootloader)
- GCC este configurat pentru un sistem pe 8 biți (`int` și pointerii sunt pe 16 biți)
- Avem macrodefiniții pentru toate adresele relevante de I/O, compilatorul știe automat să folosească instrucțiunile dedicate pentru scrierile și citirile de la aceste adrese

```
#define PORTC *(volatile uint8_t *) (0x020)
```

- Cum clasificăm variabilele într-un mediu de programare pe PC în funcție de alocarea memoriei lor?
- Variabilele pot fi locale funcției → pe stivă
- Variabilele pot fi statice → secțiunea `.bss`
- Variabilele pot fi alocate dinamic → heap
  - Presupune adăugarea codului pentru alocator de memorie
  - Memory overhead pentru un dispozitiv care are foarte puțină memorie în general
- Constante! Le putem ține în memoria de program
- Stocare non-volatilă există și în EEPROM
  - Citire și scriere greoaie
  - 1kB de memorie
  - număr destul de limitat de scrieri (100 000x)
  - util pentru ce trebuie citit/scris doar o dată per restart



- 32 registre general purpose accesibile și ca memorie
- 64 registre I/O accesibile și cu `in`/`out`
- 160 de registre I/O extra
- 2048 de octeți de memorie SRAM

Hello, world!

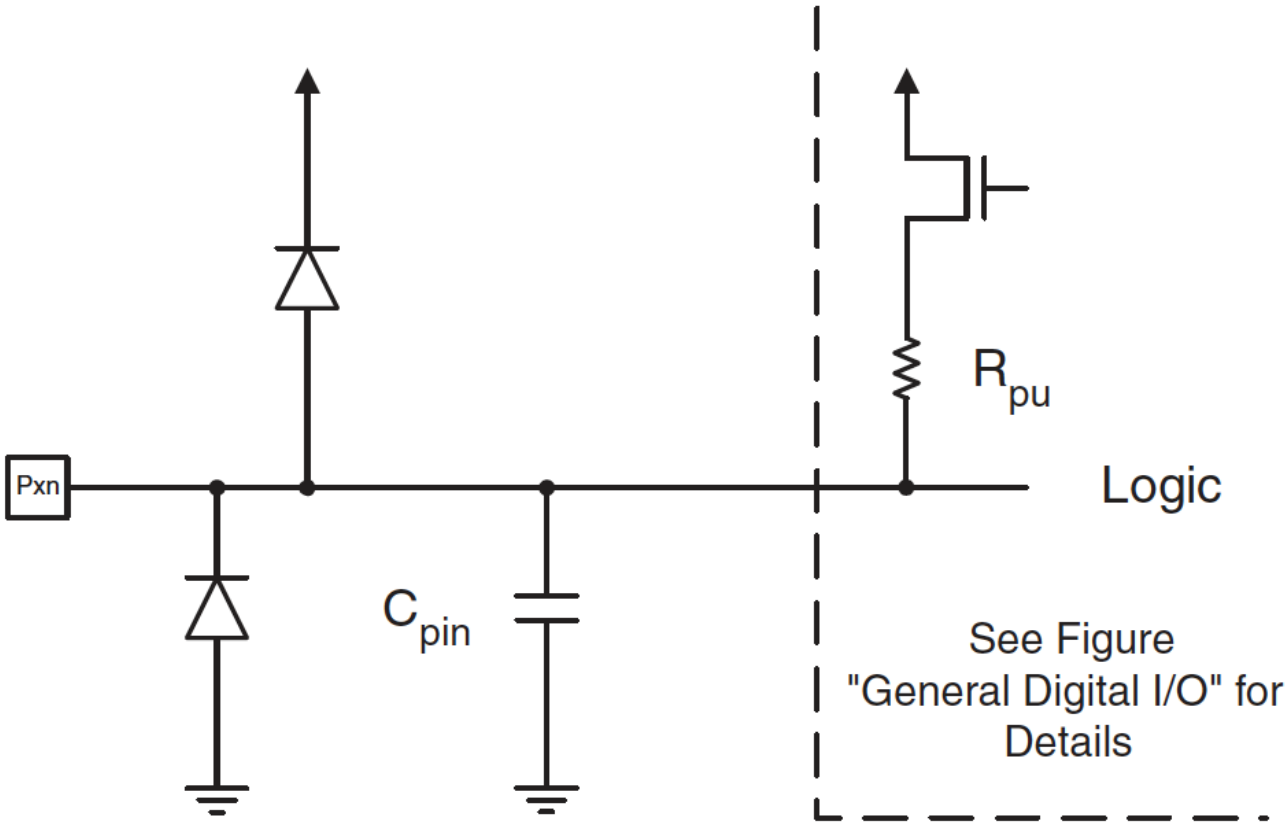
```
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>

int main()
{
    DDRC = (1 << PC4);


    while (1)
    {
        PORTC ^= (1 << PC4);
        _delay_ms(250);
    }
}
```

- Header-ul cu registrele I/O
- Frecvența de lucru (determinată de placă)
- Setarea pinului PC4 ca ieșire
- Alternarea stării pinului PC4 o dată la 250 de milisecunde

Un pin I/O


Pxn
C_{pin}
R_{pu}
Logic
See Figure
"General Digital I/O" for
Details



Configurarea unui PIN I/O

DDxn	PORTxn	PUD (in MCUCR)	I/O	Pull-up	Comment
0	0	X	Input	No	Tri-state (Hi-Z)
0	1	0	Input	Yes	Pxn will source current if ext. pulled low.
0	1	1	Input	No	Tri-state (Hi-Z)
1	0	X	Output	No	Output Low (Sink)
1	1	X	Output	No	Output High (Source)

Cum arată rezultatul?

```
00000000 <__vectors>:
   0:   0c 94 3e 00     jmp 0x7c            ; 0x7c <__ctors_end>
                                            ; care face jump catre main dupa initializar
00000094 <main>:
  94:   80 e1           ldi r24, 0x10       ; (1 << 4)
  96:   87 b9           out 0x07, r24       ; scriere in DDRC
  98:   90 e1           ldi r25, 0x10       ; (1 << 4)
  9a:   88 b1           in  r24, 0x08       ; citire din PORTC
  9c:   89 27           eor r24, r25        ; r24 = r24 XOR r25
  9e:   88 b9           out 0x08, r24       ; scriere in PORTC
  a0:   2f ef           ldi r18, 0xFF       ; 255
  a2:   34 e3           ldi r19, 0x34       ; 52
  a4:   8c e0           ldi r24, 0x0C       ; 12
  a6:   21 50           subi    r18, 0x01   ; 1 ciclu
  a8:   30 40           sbci    r19, 0x00   ; 1 ciclu
  aa:   80 40           sbci    r24, 0x00   ; 1 ciclu
  ac:   e1 f7           brne    .-8         ; 0xa6 <main+0x12>
                                            ; 1 ciclu daca nu face conditia, 2 altfel
                                            ; practic se numara de la 0x0C34FF la 0
                                            ; 0x0C34FF * 5 cicli - 1 de iesire
  ae:   00 c0           rjmp    .+0         ; 0xb0 <main+0x1c>
  b0:   00 00           nop                 ; nop + rjmp = 3 cicli
  b2:   f3 cf           rjmp    .-26        ; 0x9a <main+0x6>
```

- Ce sunt întreruperile?
  - Secvențe de cod care se execută asicron cu restul programului, ca urmare a unui stimul
- Cum apar ele?
  - Factori interni (timere, terminarea unor operații lungi)
  - Factori externi (serială, butoane)
- Cum se tratează?
  - Rutine de tratare a întreruperii - RTI (en. ISR)

- Cum descriu complet un program aflat în funcționare?
  - Program Counter (PC)
  - Status Register (flag de carry, flag de zero - SREG)
  - Stiva curentă
  - Registre + memorie
- Ce face procesorul pentru a trata întreruperea?
  - Salvez **contextul**
  - Găsesc adresa rutinei de tratare
  - Execut rutina de tratare
  - Refac **contextul**

- La declanșarea unei întreruperi
  - PC+2 pe stivă
  - Se dezactivează bitul I din SREG
- La revenirea din întrerupere (RETI)
  - PC ia valoarea de pe stivă
  - Se activează bitul I din SREG

- Ce trebuie să fac?
- Activez toate întreruperile
  - Întreruperile pot fi dezactivate complet cu flag-ul **I** din `SREG` - status register
- Activez întreruperea în cauză - fiecare are propriul pin de enable
- Scriu funcția de ISR

```
ISR(USART0_RX_vect)
{
    ...
}
```

EIMSK – External Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
0x1D (0x3D)	–	–	–	–	–	INT2	INT1	IINT0	EIMSK
Read/Write	R	R	R	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 2:0 – INT2:0: External Interrupt Request 2 - 0 Enable**

When an INT2:0 bit is written to one and the I-bit in the Status Register (SREG) is set (one), the corresponding external pin interrupt is enabled. The Interrupt Sense Control bits in the External Interrupt Control Register, EICRA, defines whether the external interrupt is activated on rising or falling edge or level sensed. Activity on any of these pins will trigger an interrupt request even if the pin is enabled as an output. This provides a way of generating a software interrupt.



Exemplu de folosire întreruperi

```
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>

ISR(INT0_vect)
{
    PORTC ^= (1 << PC4);
}

int main()
{
    DDRC = (1 << PC4);

    EIMSK |= (1 << INT0);

    sei();

    while (1);
}
```

- Enable la întreruperea pe care o dorim
- Enable la toate întreruperile `sei()`
- rutina de tratarea ISR
- Am păstrat bucla infinită din main!



Cum arată codul?

```
00000000 <__vectors>:
   0:   jmp     0x7c    ; 0x7c <__ctors_end>
   4:   jmp     0x94    ; 0x94 <__vector_1>
   8:   jmp     0x90    ; 0x90 <__bad_interrupt>
   c:   jmp     0x90    ; 0x90 <__bad_interrupt>
                        ...

00000094 <__vector_1>:
  94:   push    r1
  96:   push    r0
  98:   in      r0, 0x3f        ; 63
  9a:   push    r0
  9c:   eor     r1, r1
  9e:   push    r24
  a0:   push    r25
  a2:   in      r24, 0x08       ; 8
  a4:   ldi     r25, 0x10       ; 16
  a6:   eor     r24, r25
  a8:   out     0x08, r24       ; 8
  aa:   pop     r25
  ac:   pop     r24
  ae:   pop     r0
  b0:   out     0x3f, r0        ; 63
  b2:   pop     r0
  b4:   pop     r1
  b6:   reti
```

- Tabela de vectori de întrerupere este la început
- Orice intrare nedefinită din tabelă duce la un software restart
- Ce ar fi fost diferit dacă era altă întrerupere (nu `INT0`)?

De ce avem bucla infinită fără cod?

```
00000096 <main>:
  96:   ldi     r24, 0x10       ; 16
  98:   out     0x07, r24       ; 7
  9a:   sbi     0x1d, 0 ; 29
  9c:   sei
  9e:   rjmp    .-2
; 0x9e <main+0x8>

000000a0 <_exit>:
  a0:   cli

000000a2 <__stop_program>:
  a2:   rjmp    .-2
; 0xa2 <__stop_program>
```

- `cli` dezactivează întreruperile
- `rjmp .-2` este un loop pe loc, cu întreruperile dezactivate

```
ISR(INT0_vect, ISR_NAKED)
{
    PORTC |= (1 << PC4);
}

00000094 <__vector_1>:
  94:   sbi     0x08, 4 ; 8
```

- `ISR_NAKED` nu mai dezactivează întreruperile
- setarea unui bit pe 1 poate fi făcută cu o singură instrucțiune




```
#include <avr/interrupt.h>
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>

volatile int flag = 0;

ISR(INT0_vect)
{
    flag = 1;
}

int main()
{
    LCD_init();

    EIMSK |= (1 << INT0);

    sei();

    while (1)
    {
        if (flag)
        {
            LCD_print("PC4_pressed");
        }
    }
}
```

- Compilatorul nu poate crea dependențe de variabile între `main` și întreruperi
- `volatile` instruiește compilatorul să nu optimizeze în jurul acelei variabile
- Ce s-ar fi întâmplat dacă nu puneam `volatile`?

Alte detalii de boot

```
0000007c <__ctors_end>:
  7c:   eor     r1, r1
  7e:   out     0x3f, r1        ; 63
  80:   ldi     r28, 0xFF       ; 255
  82:   ldi     r29, 0x08       ; 8
  84:   out     0x3e, r29       ; 62
  86:   out     0x3d, r28       ; 61
  88:   call    0x96    ; 0x96 <main>
  8c:   jmp     0xa0    ; 0xa0 <_exit>
```

- 0x3f este SREG
- 0x3e și 0x3d sunt SPH și SPL, stiva pe 16 biți
- de ce punem în stivă valoarea 0x08FF?



- Avem un singur thread
  - Main function/loop
  - Asincron putem executa rutine
- Poate exista un sistem de multi-threading în AVR?
  - Timer + salvare de context

```
void thread_1() { ... }
void thread_2() { ... }

/* O data la 10ms */
ISR(Timer)
{
  save_context(peek(task_queue));
  task_t current = pop(task_queue);
  push(task_queue, current);
  restore_context(peek(task_queue));
}

void save_context(task_t task)
{
  push_all_registers();
  task.stack = SP;
}
void restore_context(task_t task)
{
  SP = task.stack;
  pop_all_registers();
}
```

- Scheduler Round Robin
- Cuantă de timp fixă
- Stive diferite



- Ce ar trebui să luăm în calcul pentru thread-uri?
  - Acces la resurse
  - Acces la memorie
  - Timp discontinuu



Proiectarea cu Microprocesoare
Curs 3

Andrei Voinescu

POLITEHNICA University of Bucharest

17 martie 2021



Outline

Introducere

Moduri de lucru

Alte Aplicații

Watchdog

- Timerele sunt dispozitive care ne permit să



- Timerele sunt dispozitive care ne permit să
  - știm cât timp a trecut între două evenimente



- Timerele sunt dispozitive care ne permit să
  - știm cât timp a trecut între două evenimente
  - numărăm evenimente externe



Timere: Cunoștințe de bază

- Timerele sunt dispozitive care ne permit să
  - știm cât timp a trecut între două evenimente
  - numărăm evenimente externe
  - să executăm cod/acțiuni după un anumit timp sau la intervale regulate (PWM)

- Timerele sunt dispozitive care ne permit să
  - știm cât timp a trecut între două evenimente
  - numărăm evenimente externe
  - să executăm cod/acțiuni după un anumit timp sau la intervale regulate (PWM)
- Timerele sunt implementate ca niște contoare
- Ca orice periferic I/O, vor avea registre I/O unde sunt configurate

- Timerele sunt dispozitive care ne permit să
  - știm cât timp a trecut între două evenimente
  - numărăm evenimente externe
  - să executăm cod/acțiuni după un anumit timp sau la intervale regulate (PWM)
- Timerele sunt implementate ca niște contoare
- Ca orice periferic I/O, vor avea registre I/O unde sunt configurate
  - Cum numără (până la cât, crescător/descrescător)



Timere: Cunoștințe de bază

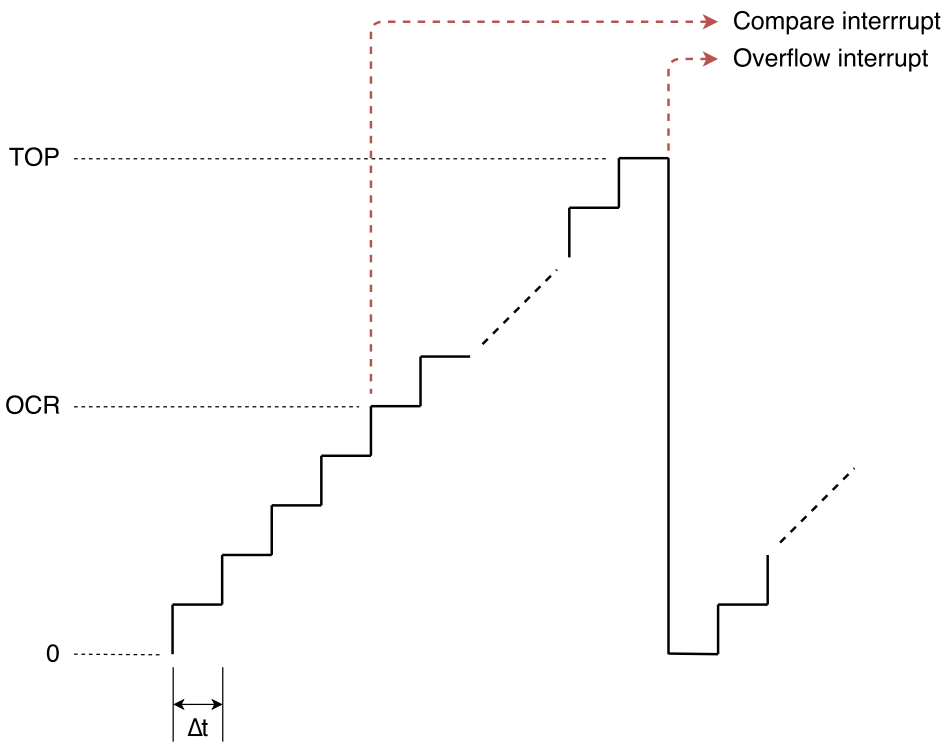
- Timerele sunt dispozitive care ne permit să
  - știm cât timp a trecut între două evenimente
  - numărăm evenimente externe
  - să executăm cod/acțiuni după un anumit timp sau la intervale regulate (PWM)
- Timerele sunt implementate ca niște contoare
- Ca orice periferic I/O, vor avea registre I/O unde sunt configurate
  - Cum numără (până la cât, crescător/descrescător)
  - Cât de repede numără

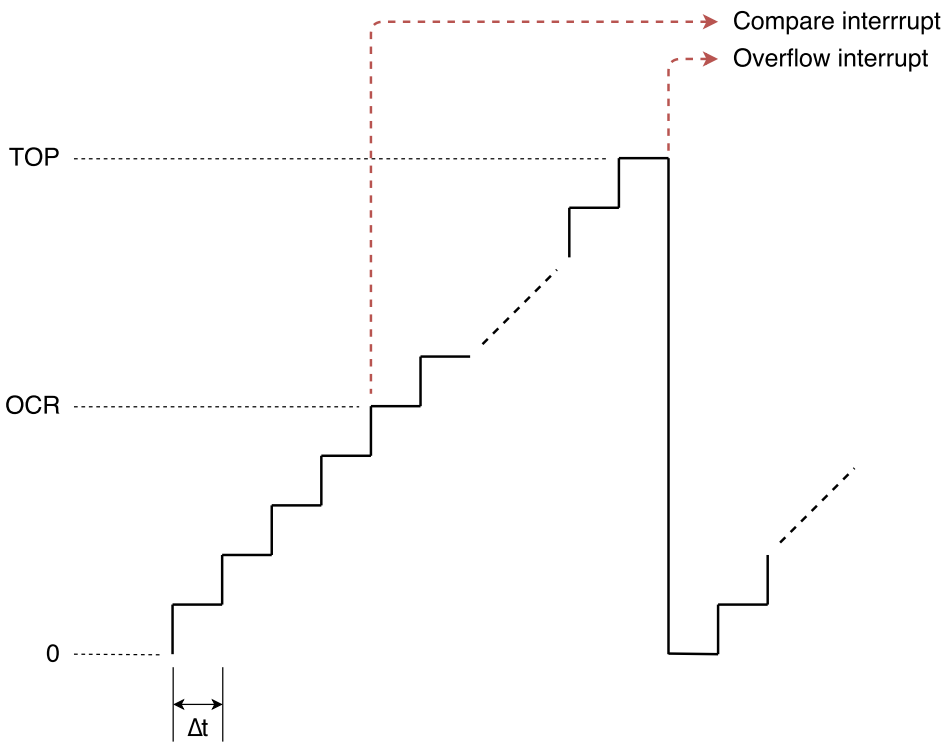
- Timerele sunt dispozitive care ne permit să
  - știm cât timp a trecut între două evenimente
  - numărăm evenimente externe
  - să executăm cod/acțiuni după un anumit timp sau la intervale regulate (PWM)
- Timerele sunt implementate ca niște contoare
- Ca orice periferic I/O, vor avea registre I/O unde sunt configurate
  - Cum numără (până la cât, crescător/descrescător)
  - Cât de repede numără
  - Când sunt asociate acțiuni



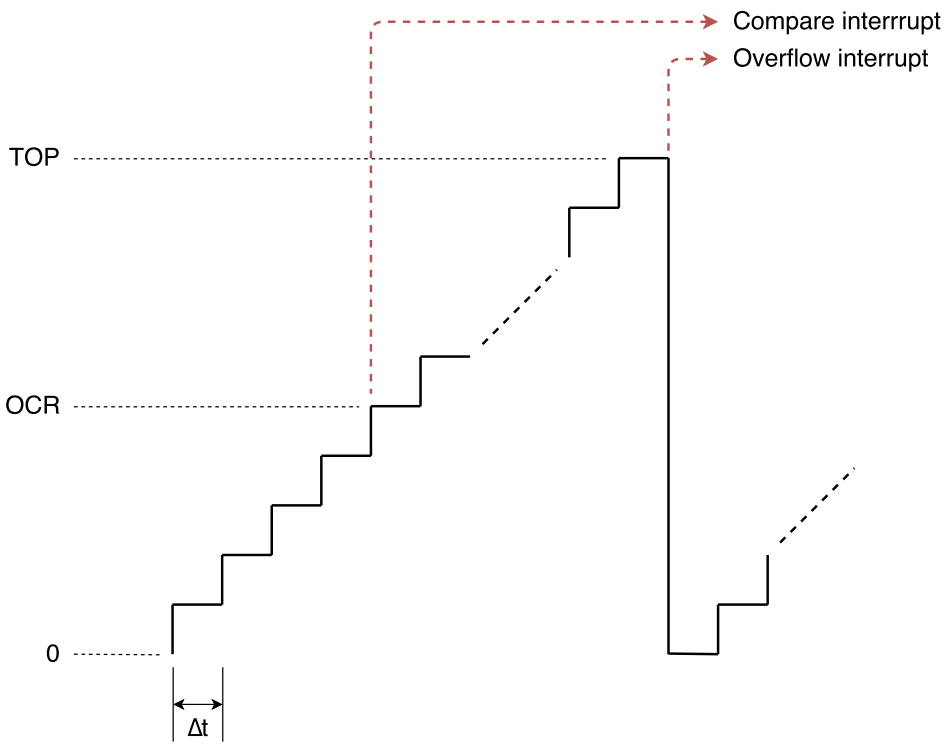
Timere: Cunoștințe de bază

- Timerele sunt dispozitive care ne permit să
  - știm cât timp a trecut între două evenimente
  - numărăm evenimente externe
  - să executăm cod/acțiuni după un anumit timp sau la intervale regulate (PWM)
- Timerele sunt implementate ca niște contoare
- Ca orice periferic I/O, vor avea registre I/O unde sunt configurate
  - Cum numără (până la cât, crescător/descrescător)
  - Cât de repede numără
  - Când sunt asociate acțiuni
  - Ce se execută în acele acțiuni

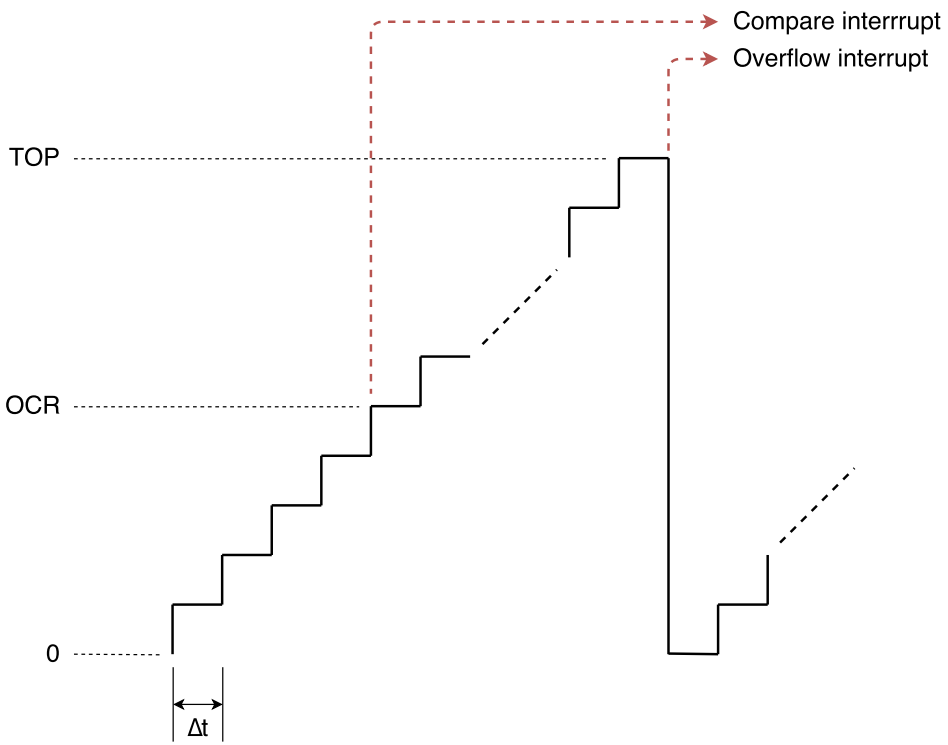

Compare interrrupt
Overflow interrupt
TOP
OCR
0
Δt



- TOP este valoarea până la care numără



- TOP este valoarea până la care numără
- OCR este un prag la care se poate întâmpla o acțiune (întrerupere sau PWM)



- TOP este valoarea până la care numără
- OCR este un prag la care se poate întâmpla o acțiune (întrerupere sau PWM)
- Δt este dat de viteza cu care numără

$$\Delta t = \frac{PS}{f_{CPU}}$$

- Pe ATMega324 timerele apar sub forma Timer/Counter
- Timer-ele 0 și 2 sunt timere pe 8 biți
- Timer1 este timer pe 16 biți
- TCCRxA și TCCRxB (x=0,1,2) - Control Register
  - Bitfield CS - Clock Select (cât de repede numără)
  - Bitfield WGM - Waveform Generation Mode (cum numără)
  - Bitfield COM - Compare Output (pentru PWM) Mode
- OCRxA, OCRxB - Output Compare Register (praguri)
- TCNTx - Counter (contorul efectiv)
- TIMSKx - interrupt mask register

- *No clock source* înseamnă că nu va număra
- Prescaler-ul are valori fixe



- *No clock source* înseamnă că nu va număra
- Prescaler-ul are valori fixe
- Poate fi legat un semnal extern de ceas pe Timer0 și 1

- *No clock source* înseamnă că nu va număra
- Prescaler-ul are valori fixe
- Poate fi legat un semnal extern de ceas pe Timer0 și 1
- Pe timer 2 poate fi legat un alt cristal de cuarț

Table 16-6. Clock Select bit description.

CSn2	CSn1	CSn0	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	$clk_{I/O}$/1 (No prescaling)
0	1	0	$clk_{I/O}$/8 (From prescaler)
0	1	1	$clk_{I/O}$/64 (From prescaler)
1	0	0	$clk_{I/O}$/256 (From prescaler)
1	0	1	$clk_{I/O}$/1024 (From prescaler)
1	1	0	External clock source on Tn pin. Clock on falling edge.
1	1	1	External clock source on Tn pin. Clock on rising edge.

Table 16-5. Waveform Generation mode bit description [(1)].

Mode	WGMn3	WGMn2 (CTCn)	WGMn1 (PWMn1)	WGMn0 (PWMn0)	Timer/Counter mode of operation	TOP	Update of OCRnx at	TOVn flag set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, Phase Correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, Phase Correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, Phase Correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCRnA	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	BOTTOM	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	BOTTOM	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	BOTTOM	TOP
8	1	0	0	0	PWM, Phase and Frequency Correct	ICRn	BOTTOM	BOTTOM
9	1	0	0	1	PWM, Phase and Frequency Correct	OCRnA	BOTTOM	BOTTOM
10	1	0	1	0	PWM, Phase Correct	ICRn	TOP	BOTTOM
11	1	0	1	1	PWM, Phase Correct	OCRnA	TOP	BOTTOM
12	1	1	0	0	CTC	ICRn	Immediate	MAX
13	1	1	0	1	(Reserved)	–	–	–
14	1	1	1	0	Fast PWM	ICRn	BOTTOM	TOP
15	1	1	1	1	Fast PWM	OCRnA	BOTTOM	TOP

Note: 1. The CTCn and PWMn1:0 bit definition names are obsolete. Use the WGMn2:0 definitions. However, the functionality and location of these bits are compatible with previous versions of the timer.

- Biții de COM activează semnalul de PWM pe canalul asociat (A sau B)
- Non-inverting generează un semnal cu duty cycle $D = \frac{1+prag}{1+TOP}$
- Inverting mode va avea $D = \frac{TOP-prag}{1+TOP}$
- Toggle mode va avea $D = 50\%$

Table 16-3. Compare Output mode, fast PWM (1).

COMnA1/COMnB1	COMnA0/COMnB0	Description
0	0	Normal port operation, OCnA/OCnB disconnected.
0	1	WGMn3:0 = 14 or 15: Toggle OC1A on Compare Match, OC1B disconnected (normal port operation). For all other WGM1 settings, normal port operation, OC1A/OC1B disconnected.
1	0	Clear OCnA/OCnB on Compare Match, set OCnA/OCnB at BOTTOM (non-inverting mode)
1	1	Set OCnA/OCnB on Compare Match, clear OCnA/OCnB at BOTTOM (inverting mode)



Accesul la registrele pe 16 biți

- AVR este o arhitectură pe 8 biți, dar se dorește totuși accesul "atomic" la valori care depășesc 255



Accesul la registrele pe 16 biți

- AVR este o arhitectură pe 8 biți, dar se dorește totuși accesul "atomic" la valori care depășesc 255
- Registrele pe 16 biți sunt împărțite într-un byte `high` și un byte `low`



Accesul la registrele pe 16 biți

- AVR este o arhitectură pe 8 biți, dar se dorește totuși accesul "atomic" la valori care depășesc 255
- Registrele pe 16 biți sunt împărțite într-un byte `high` și un byte `low`
- Pentru a face accesul atomic, perifericele intermediază accesul la aceste adrese cu un *latch*:

- AVR este o arhitectură pe 8 biți, dar se dorește totuși accesul "atomic" la valori care depășesc 255
- Registrele pe 16 biți sunt împărțite într-un byte `high` și un byte `low`
- Pentru a face accesul atomic, perifericele intermediază accesul la aceste adrese cu un *latch*:
  - La citirea byte-ului `low`, se citește automat și byte-ul high și se stochează într-o locație temporară



Accesul la registrele pe 16 biți

- AVR este o arhitectură pe 8 biți, dar se dorește totuși accesul "atomic" la valori care depășesc 255
- Registrele pe 16 biți sunt împărțite într-un byte `high` și un byte `low`
- Pentru a face accesul atomic, perifericele intermediază accesul la aceste adrese cu un *latch*:
  - La citirea byte-ului `low`, se citește automat și byte-ul high și se stochează într-o locație temporară
  - La citirea byte-ului `high` valoarea citită este tot timpul cea din locația temporară



Accesul la registrele pe 16 biți

- AVR este o arhitectură pe 8 biți, dar se dorește totuși accesul "atomic" la valori care depășesc 255
- Registrele pe 16 biți sunt împărțite într-un byte `high` și un byte `low`
- Pentru a face accesul atomic, perifericele intermediază accesul la aceste adrese cu un *latch*:
  - La citirea byte-ului `low`, se citește automat și byte-ul high și se stochează într-o locație temporară
  - La citirea byte-ului `high` valoarea citită este tot timpul cea din locația temporară
  - Scrierea se face în ordinea opusă (întâi byte-ul `high` și apoi `low`)



Accesul la registrele pe 16 biți

- AVR este o arhitectură pe 8 biți, dar se dorește totuși accesul "atomic" la valori care depășesc 255
- Registrele pe 16 biți sunt împărțite într-un byte `high` și un byte `low`
- Pentru a face accesul atomic, perifericele intermediază accesul la aceste adrese cu un *latch*:
  - La citirea byte-ului `low`, se citește automat și byte-ul high și se stochează într-o locație temporară
  - La citirea byte-ului `high` valoarea citită este tot timpul cea din locația temporară
  - Scrierea se face în ordinea opusă (întâi byte-ul `high` și apoi `low`)
  - Motivația este dată de variația în timp, vrem byte-ul `low` să corespundă în timp ultimei citiri/scrieri

- AVR este o arhitectură pe 8 biți, dar se dorește totuși accesul "atomic" la valori care depășesc 255
- Registrele pe 16 biți sunt împărțite într-un byte `high` și un byte `low`
- Pentru a face accesul atomic, perifericele intermediază accesul la aceste adrese cu un *latch*:
  - La citirea byte-ului `low`, se citește automat și byte-ul high și se stochează într-o locație temporară
  - La citirea byte-ului `high` valoarea citită este tot timpul cea din locația temporară
  - Scrierea se face în ordinea opusă (întâi byte-ul `high` și apoi `low`)
  - Motivația este dată de variația în timp, vrem byte-ul `low` să corespundă în timp ultimei citiri/scrieri
- Pentru C compilatorul poate face singur aceste operații în ordinea corectă

- Semnale pătratice periodice
  - Duty cycle
  - Frecvență

- Semnale pătratice periodice
  - Duty cycle
  - Frecvență
- De ce ne trebuie
  - Pot controla foarte multe dispozitive
    - Drivere de LED-uri
    - Drivere de motoare

- Semnale pătratice periodice
  - Duty cycle
  - Frecvență
- De ce ne trebuie
  - Pot controla foarte multe dispozitive
    - Drivere de LED-uri
    - Drivere de motoare
- Cum sunt implementate
  - Timer-ele au căi de output $\rightarrow$ canale
  - Timer-ele produc output PWM configurabil
    - Restul funcționalității de timer nu sunt afectate



Outline

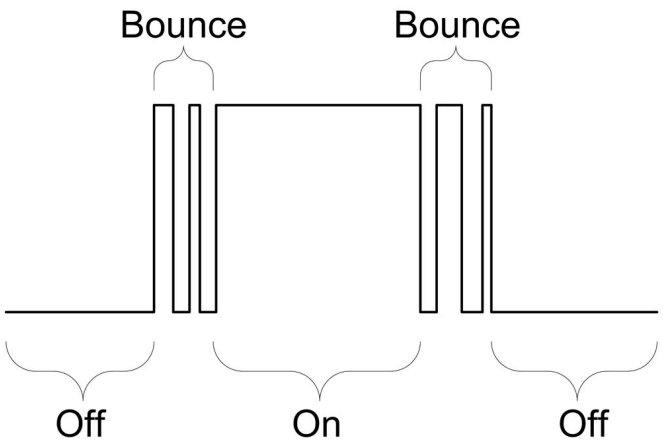
Introducere

Moduri de lucru

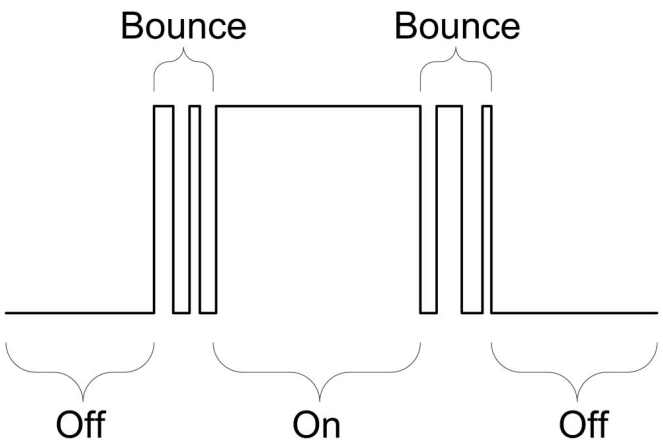
Alte Aplicații

Watchdog

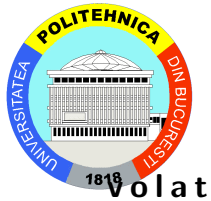
- Utilizări
  - Modul default pentru toate timerele
  - TOP fixat - frecvența fixă pentru evenimente generate
    - Pentru un timer de 8 biți, frecvența la care va face overflow va fi $f_{OVF} = \frac{f_{CPU}}{PS \cdot 256}$
    - Controlul frecvenței este limitat la prescaler sau la evenimente externe



- Folosiri $\rightarrow$ aplicații care nu au nevoie de o anumită frecvență
- Exemplu: o variantă de debouncing pentru butoane



- Folosiri $\rightarrow$ aplicații care nu au nevoie de o anumită frecvență
- Exemplu: o variantă de debouncing pentru butoane
- Cum am putea face debouncing cu ajutorul unui timer?
  - Nu avem nevoie de o frecvență anume, trebuie doar să știm cât timp a trecut de la ultima apăsare



Exemplu mod normal

```
volatile uint16_t    lastEvent = 0;
volatile uint8_t     buttonState = 0;  // 0 normal, 1 apasat

// 250 ms
#define DELAY    (16000000UL / 1024 / 4)

ISR(PCINT2_vect)
{
    if (buttonState != ((PINC & (1 << PC0)) == 0))
    { // daca butonul e apasat si noi credeam ca nu este
        if ((TCNT1 - lastEvent > DELAY))
        {
            buttonState = ((PINC & (1 << PC0)) == 0);
            buttonState? button_press() : button_release();
            lastEvent = TCNT1;
        }
    }
}

int main()
{
    TCCR1B = (5 << CS10); // prescaler 1024

    DDRC    &= ~(1 << PC0);      // PC0 ca intrare
    PCICR   |= (1 << PCINT2);    // PC0 este PCINT16, deci e in al
    PCMSK2  |= (1 << PCMSK16);   // treilea registru PCMSK

    sei();
    while (1);
    return 0;
}
```



- Modul de bază în care contorul se resetează când atinge un prag (fie OCR, fie ICR)

- Modul de bază în care contorul se resetează când atinge un prag (fie OCR, fie ICR)
- Putem controla frecvența cu care ajungem la *Compare match*

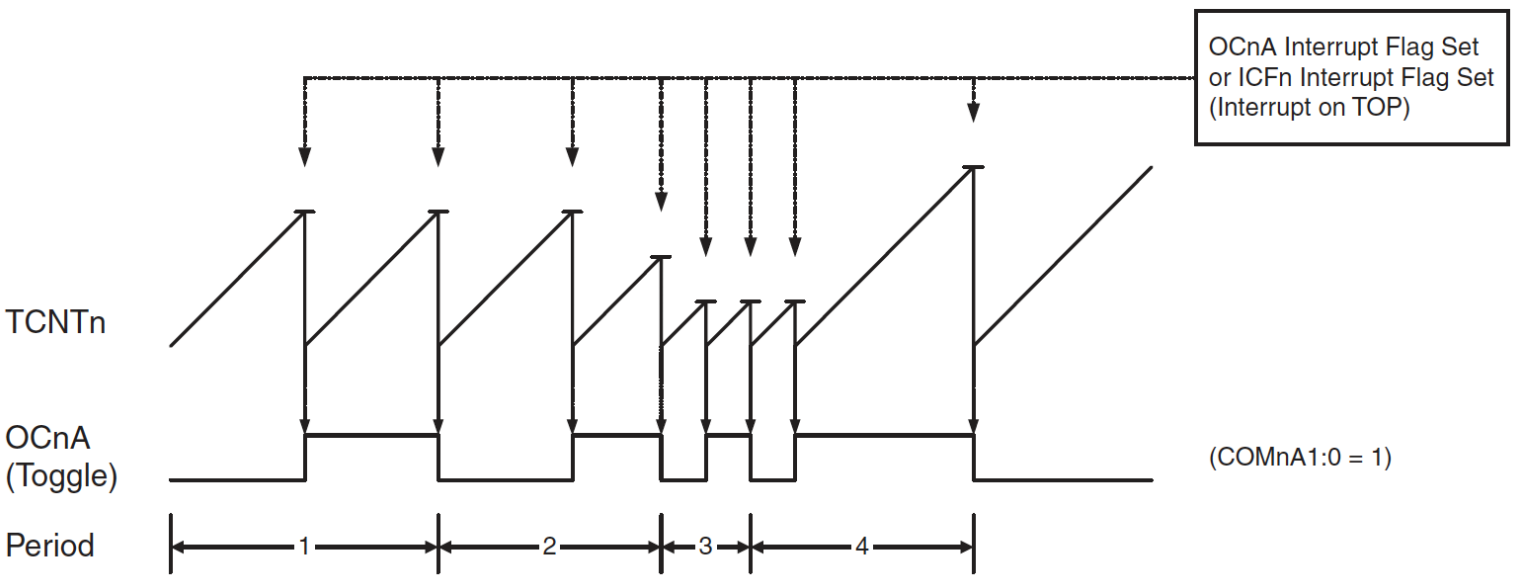
- Modul de bază în care contorul se resetează când atinge un prag (fie OCR, fie ICR)
- Putem controla frecvența cu care ajungem la *Compare match*
- Dar nu ajungem aproape niciodată la overflow

- Modul de bază în care contorul se resetează când atinge un prag (fie OCR, fie ICR)
- Putem controla frecvența cu care ajungem la *Compare match*
- Dar nu ajungem aproape niciodată la overflow
- $f_{CMP} = \frac{f_{CPU}}{PS \cdot (1 + OCRxA)}$

- Modul de bază în care contorul se resetează când atinge un prag (fie OCR, fie ICR)
- Putem controla frecvența cu care ajungem la *Compare match*
- Dar nu ajungem aproape niciodată la overflow
- $f_{CMP} = \frac{f_{CPU}}{PS \cdot (1 + OCRxA)}$
- Spre deosebire de alte moduri, modificările la prag se fac instantaneu
  - Ex: pragul este 100, contorul este 50, mutăm pragul la 25
  - La ce valoare se va reseta?

- Modul de bază în care contorul se resetează când atinge un prag (fie OCR, fie ICR)
- Putem controla frecvența cu care ajungem la *Compare match*
- Dar nu ajungem aproape niciodată la overflow
- $f_{CMP} = \frac{f_{CPU}}{PS \cdot (1 + OCRxA)}$
- Spre deosebire de alte moduri, modificările la prag se fac instantaneu
  - Ex: pragul este 100, contorul este 50, mutăm pragul la 25
  - La ce valoare se va reseta?
  - Pentru că update-ul se face imediat, nu la fiecare reset, contorul va ajunge la valoarea maximă (0xFF sau 0xFFFF)

Figure 16-6. CTC mode, timing diagram.



- Un ceas are nevoie să numere secundele

- Un ceas are nevoie să numere secundele
- Vrem să numărăm o secundă în hardware, iar *numărul de secunde* să îl numărăm în software
- cu prescalerul maxim de 1024, $f_{CMP_{MAX}} = \frac{f_{CPU}}{1024 \cdot 256} \simeq 61\,Hz$

- Un ceas are nevoie să numere secundele
- Vrem să numărăm o secundă în hardware, iar *numărul de secunde* să îl numărăm în software
- cu prescalerul maxim de 1024, $f_{CMP_{MAX}} = \frac{f_{CPU}}{1024 \cdot 256} \simeq 61\,Hz$
- Nu putem număra o secundă cu timerele de 8 biți
- Cu un timer de 16 biți $\rightarrow OCRxA = \frac{f_{CPU}}{PS \cdot 1\,Hz} - 1 = 15624$



Modul CTC - exemplu cod

```
...

volatile uint32_t nrSeconds = 0;

ISR(TIMER1_COMPA_vect)
{
    nrSeconds++;
}

int main()
{
    OCR1A  = (F_CPU / 1024) - 1;          // 15624
    TCCR1B = (5 << CS10) | (1 << WGM12); // prescaler 1024 si mod CTC cu TOP la OCR1A

    TIMSK |= (1 << OCIE1A);               // intrerupere pe OCR1A

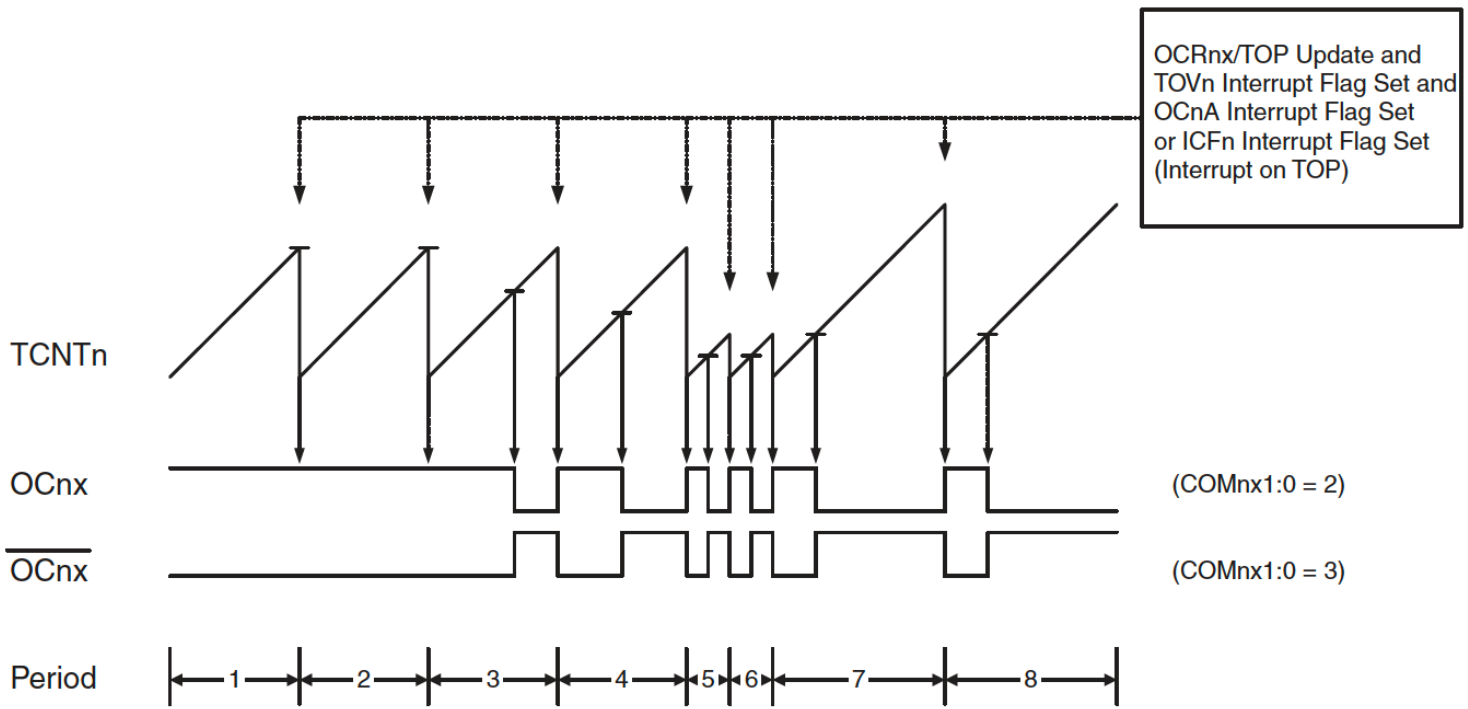
    sei();

    while (1);
    return 0;
}
```

- Moduri similare cu modul normal sau CTC, dar cu update de prag la reset

- Moduri similare cu modul normal sau CTC, dar cu update de prag la reset
  - Acest mod oferă garanții pentru semnalele PWM generate
- Modul cel mai folosit de la laborator!

Figure 16-7. Fast PWM mode, timing diagram.



- Vom da ca exemplu folosirea unui singur timer pentru două scopuri
  - Un canal de PWM este folosit pentru aprinderea unui LED
  - Alt canal de PWM este folosit pentru speaker
- Nevoi:
  - Canalul 1 (LED): Vrem să putem modifica ușor duty-cycle-ul, frecvența poate fi oricât atât timp cât este mai mare de 500Hz
  - Canalul 2 (Speaker): Vrem să putem modifica frecvența, duty cycle-ul trebuie să fie fixat la 50%



Modul FastPWM - exemplu

- 2 semnale $\rightarrow$ 2 canale PWM
- Pentru a modifica frecvența putem folosi doar `OCR1A` sau `ICR`

- 2 semnale $\rightarrow$ 2 canale PWM
- Pentru a modifica frecvența putem folosi doar `OCR1A` sau `ICR`
  - LEDul este pe canalul A, deci avem nevoie de `OCR1A` pentru acel canal (prag $\rightarrow$ duty cycle)
  - **trebuie** să folosim `ICR` ca prag (ICR este Input Capture Register, folosit pentru a reține momentul apariției unor evenimente externe)

- 2 semnale $\rightarrow$ 2 canale PWM
- Pentru a modifica frecvența putem folosi doar `OCR1A` sau `ICR`
  - LEDul este pe canalul A, deci avem nevoie de `OCR1A` pentru acel canal (prag $\rightarrow$ duty cycle)
  - **trebuie** să folosim `ICR` ca prag (ICR este Input Capture Register, folosit pentru a reține momentul apariției unor evenimente externe)
- Concluzii:
  - Frecvența este setată de `ICR1`
  - pragul pentru canalul A este `OCR1A` și determină luminozitatea pe LED
  - duty cycle-ul pe canalul B trebuie să fie 50%, putem face asta în două moduri diferite



Modul FastPWM - cod

```
...

uint32_t nrSeconds = 0;

void set_speaker_frequency(int fr)
{
    uint32_t threshold = F_CPU / 2 / fr - 1; // inversul formulei de mai devreme

    threshold = MIN(threshold, 0xFFFF);
    threshold = MAX(threshold, 3);

    ICR = threshold;     // cast implicit la 16 biti
}

int main()
{
    // prescaler 1 si mod CTC cu TOP la ICR1
    TCCR1B = (1 << CS10) | (1 << WGM13) | (1 << WGM12);
    TCCR1A = (1 << WGM11);

    TCCR1A |= (2 << COM1A0);     // non-inverting mode pe canalul A
    TCCR1A |= (3 << COM1B0);     // toggle mode pe canalul B

    DDRD |= (1 << PD4) | (1 << PD5); // canalele de PWM trebuie setate ca iesiri

    while (1);
    return 0;
}
```

- Care este gama de frecvențe posibilă pe speaker-ul din exemplul precedent?
  - Gama de praguri valide este $[3, 65535]$
  - $f_{Speaker} = \frac{f_{CPU}}{2 \cdot 1 \cdot (prag+1)}$
  - Gama de frecvențe este atunci $[122Hz, 2MHz]$
  - Cum putem schimba această gamă?

- Care este gama de frecvențe posibilă pe speaker-ul din exemplul precedent?
  - Gama de praguri valide este $[3, 65535]$
  - $f_{Speaker} = \frac{f_{CPU}}{2 \cdot 1 \cdot (prag+1)}$
  - Gama de frecvențe este atunci $[122Hz, 2MHz]$
  - Cum putem schimba această gamă?
  - Dacă am fi folosit prescaler-ul de 1024, gama era $[0.12Hz, 1.95kHz]$

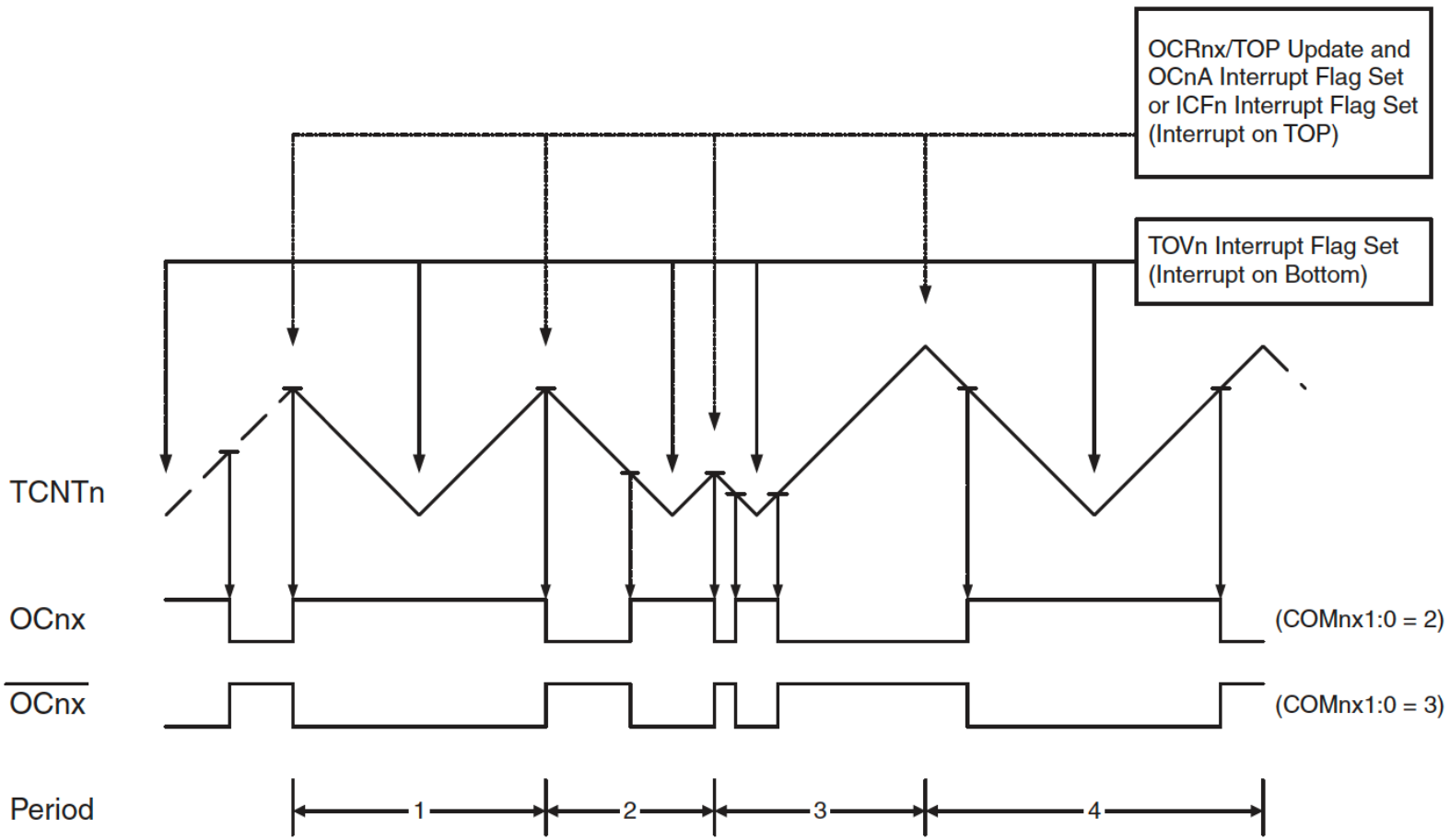
- Care este gama de frecvențe posibilă pe speaker-ul din exemplul precedent?
  - Gama de praguri valide este $[3, 65535]$
  - $f_{Speaker} = \frac{f_{CPU}}{2 \cdot 1 \cdot (prag+1)}$
  - Gama de frecvențe este atunci $[122Hz, 2MHz]$
  - Cum putem schimba această gamă?
  - Dacă am fi folosit prescaler-ul de 1024, gama era $[0.12Hz, 1.95kHz]$
- Trebuie să folosim un prescaler care ne permite să avem întreaga gamă de frecvențe audibile $[20Hz, 20kHz]$, dacă este posibil

- Folosit tot pentru generarea semnalelor PWM
- Acolo unde tranzițiile semnalului sunt în general nedorite
- Contorul numără între 0 și TOP, apoi înapoi la 0
  - Dacă facem tranziția pe semnalul PWM la prag, obținem un semnal cu același duty cycle ca fastPWM, dar cu frecvență de 2 ori mai mică

- Folosit tot pentru generarea semnalelor PWM
- Acolo unde tranzițiile semnalului sunt în general nedorite
- Contorul numără între 0 și TOP, apoi înapoi la 0
  - Dacă facem tranziția pe semnalul PWM la prag, obținem un semnal cu același duty cycle ca fastPWM, dar cu frecvență de 2 ori mai mică
  - *Clear when down-counting, set on up-counting*

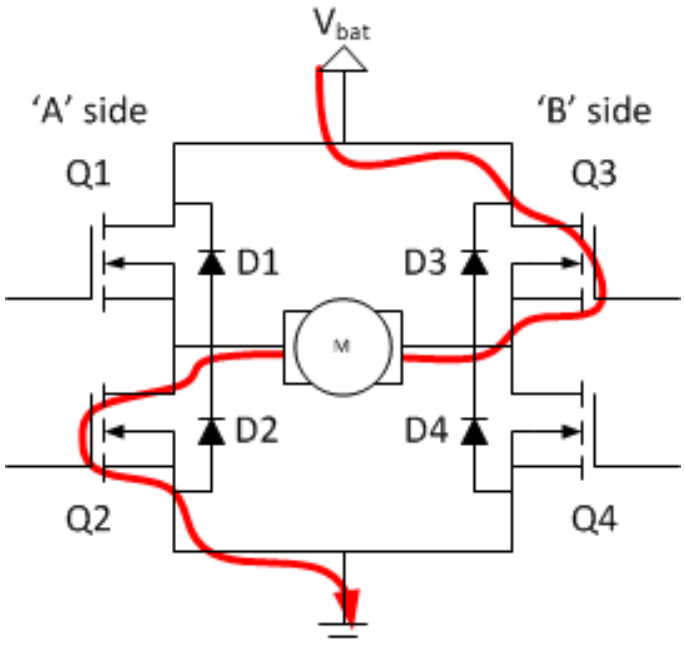
- Folosit tot pentru generarea semnalelor PWM
- Acolo unde tranzițiile semnalului sunt în general nedorite
- Contorul numără între 0 și TOP, apoi înapoi la 0
  - Dacă facem tranziția pe semnalul PWM la prag, obținem un semnal cu același duty cycle ca fastPWM, dar cu frecvență de 2 ori mai mică
  - *Clear when down-counting, set on up-counting*
  - *Set when down-counting, clear on up-counting*

Figure 16-8. Phase Correct PWM mode, timing diagram.



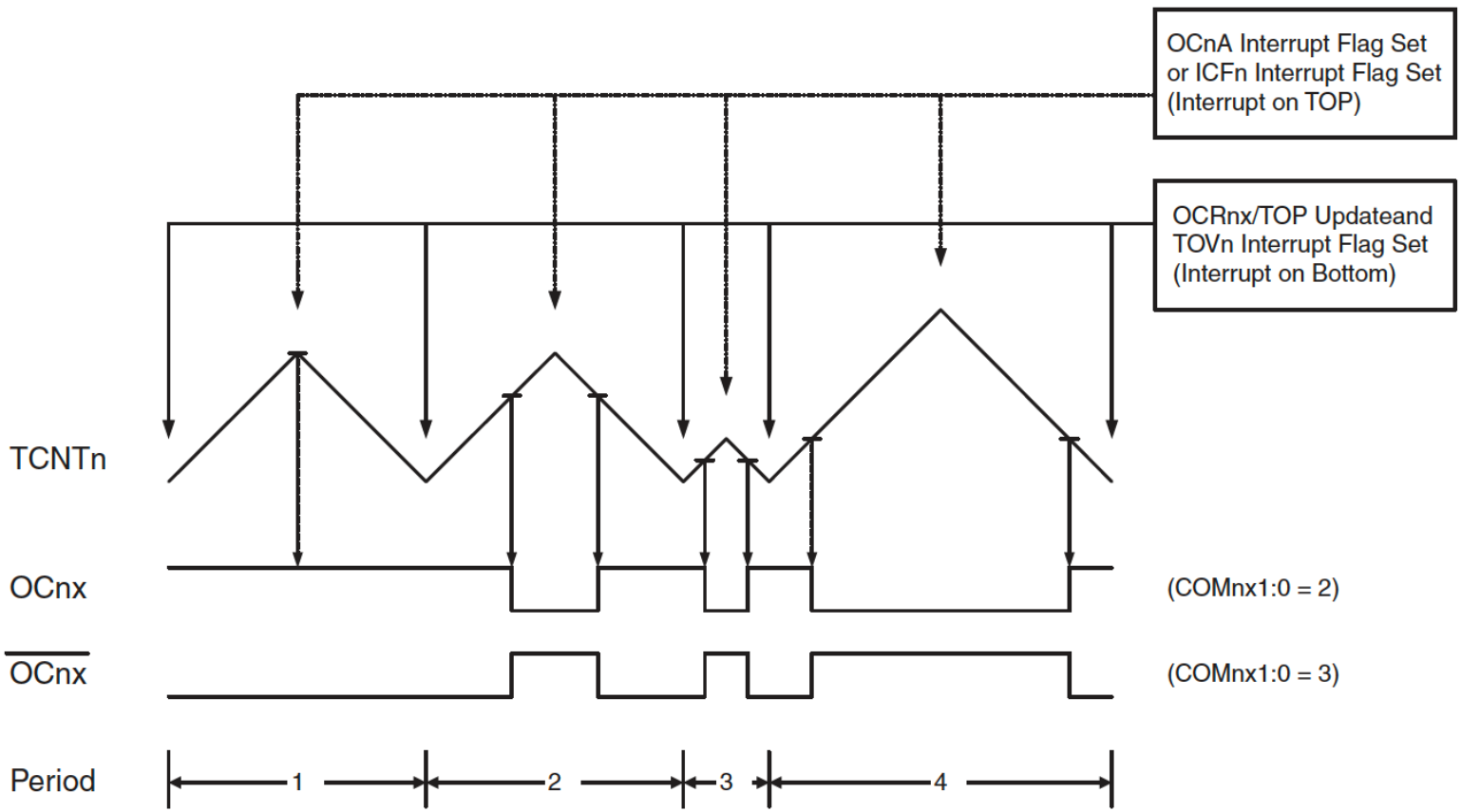
- Un motor nu poate fi controlat direct din pini
- Dacă folosim un singur tranzistor (*driver*), putem porni motorul într-un singur sens

- Un motor nu poate fi controlat direct din pini
- Dacă folosim un singur tranzistor (*driver*), putem porni motorul într-un singur sens
- Pentru a porni motorul DC în ambele sensuri este nevoie de o punte H



- La fel ca modul PWM phase correct, dar update-ul pe prag se întâmplă când se ajunge la 0 $\rightarrow$ pulsurile generate sunt întotdeauna consistente

Figure 16-9. Phase and Frequency Correct PWM mode, timing diagram.





Outline

Introducere

Moduri de lucru

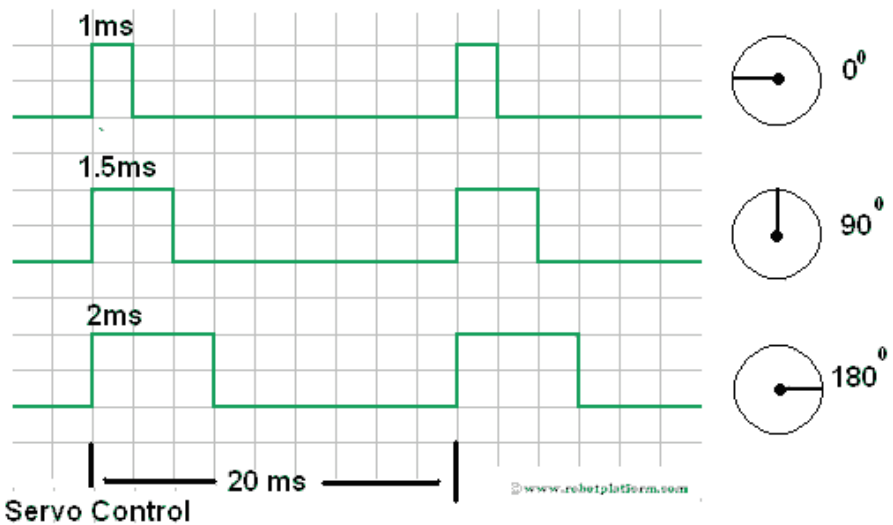
Alte Aplicații

Watchdog



- Motoare cu 3 pini: VCC, GND, semnal control





- Motoare cu 3 pini: VCC, GND, semnal control
- Semnalul de control este tipic la 5V și cu pulsuri de 1-2ms la intervale regulate care îi dau motorului *comanda*.
- Motorul încearcă să îndeplinească comenzile în timp util



Servomotoare - cod

```
...

void init_servo()
{
    // FastPWM cu TOP la OCR0A, prescaler de 1024
    TCCR0A = (1 << WGM01);
    TCCR0B = (5 << CS00) | (1 << WGM03) | (1 << WGM02);

    OCR0A = (F_CPU / 1024 / 50) - 1;     // 50Hz <==> 20ms perioada
}

// primeste un unghi intre 0 si 180
void rotate_servo(int angle) // in degrees
{
    angle = MAX(angle, 0);
    angle = MIN(angle, 180);

    OCR0B = OCR0A / 20;                                  // 1ms
    uint16_t fraction_times_100 = angle * 10 / 18; // intre 0 si 1 x100
    OCR0B += OCR0A * fraction_times_100 / 2000;

}

...
```

- Putem forma un timer de 24 (16 + 8) de biți pe ATMega324 înlănțuind Timer0 cu Timer1

- Putem forma un timer de 24 (16 + 8) de biți pe ATMega324 înlănțuind Timer0 cu Timer1
  - Unul dintre timere poate genera un semnal PWM când face overflow

- Putem forma un timer de 24 (16 + 8) de biți pe ATMega324 înlănțuind Timer0 cu Timer1
  - Unul dintre timere poate genera un semnal PWM când face overflow
  - Semnal poate fi "input" pentru al doilea timer - Bitfieldul de Clock Select poate folosi un pin extern ca ceas pentru counter în loc de ceasul sistemului

- Putem forma un timer de 24 (16 + 8) de biți pe ATMega324 înlănțuind Timer0 cu Timer1
  - Unul dintre timere poate genera un semnal PWM când face overflow
  - Semnal poate fi "input" pentru al doilea timer - Bitfieldul de Clock Select poate folosi un pin extern ca ceas pentru counter în loc de ceasul sistemului
  - Semnalul de PWM de la unul dintre timere poate fi considerat semnal de ceas pentru următorul



Outline

Introducere

Moduri de lucru

Alte Aplicații

Watchdog



Ce este un watchdog?

- Watchdog-ul este un timer special al sistemului



Ce este un watchdog?

- Watchdog-ul este un timer special al sistemului
- Când timer-ul ajunge la o anumită valoare timer-ul "expiră"
  - Controller-ul este fie resetat fie intră într-o întrerupere de watchdog
- Pentru a preveni asta, programul de pe microcontroller trebuie să reseteze contorul periodic *petting the watchdog*



Ce este un watchdog?

- Watchdog-ul este un timer special al sistemului
- Când timer-ul ajunge la o anumită valoare timer-ul "expiră"
  - Controller-ul este fie resetat fie intră într-o întrerupere de watchdog
- Pentru a preveni asta, programul de pe microcontroller trebuie să reseteze contorul periodic *petting the watchdog*
- Astfel se garantează că programul nu intră într-o stare de deadlock sau o altă stare invalidă

Watchdog - exemplu cod

```
...

ISR(WDT_vect)
{
    do_stuff();                          // retine informatii relevante despre
                                         // starea anormala in care se gaseste
}

int main()
{
    WDTCSR = (1 << WDTON) | (1 << WDIE); // watchdog enable, cu intrerupere
    WDTCSR |= (9 << WDP0);               // timeout de 8s

    sei();

    while (1)
    {
        wdt_reset();                     // pet/reset the watchdog
        do_normal_stuff();               // bucla normala
        _delay_ms(250);
    }
    return 0;
}
```



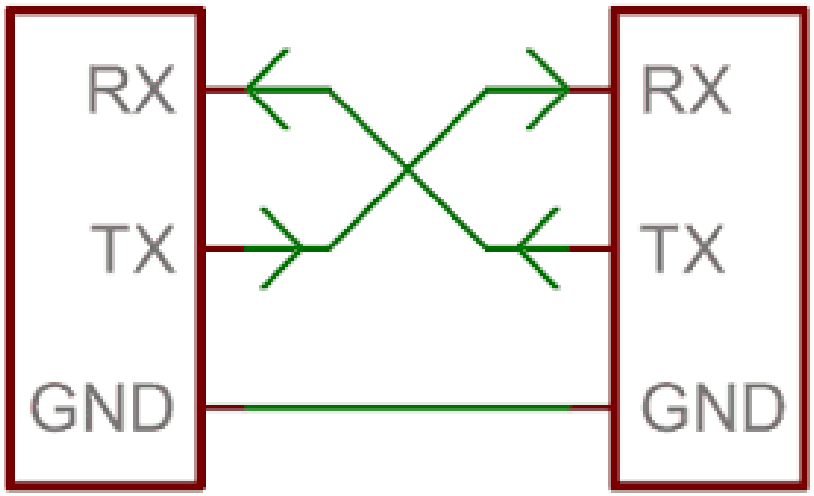
Proiectarea cu Microprocesoare
Curs 4

Andrei Voinescu

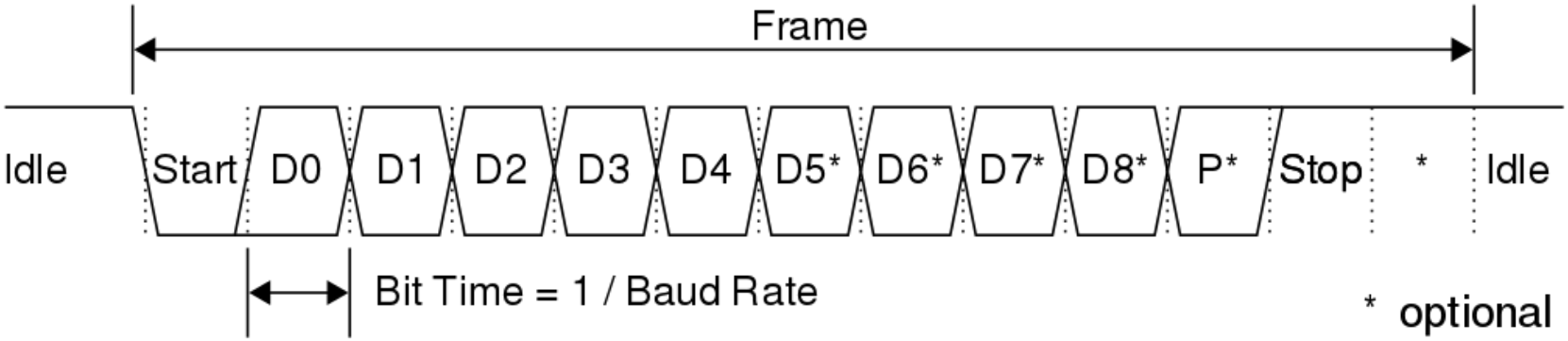
POLITEHNICA University of Bucharest

18 martie 2020

- Modalități de interconectare între dispozitive, sau între chip-uri din cadrul aceluiași dispozitiv
- În contrast cu interfețele paralele, presupune înșiruirea mai multor biți pe aceeași linie de date
- Duplex/ Half-duplex / Full-duplex
  - Duplex se referă la posibilitatea de a avea comunicație în 2 sensuri pe 2 canale diferite.
  - Comunicația Full-duplex permite comunicației să aibă loc simultan în ambele sensuri
  - Comunicația Half-duplex nu permite comunicația într-un singur sens la un moment dat



- Conexiuni doar între două dispozitive
- Semnalele sunt legate în cruce



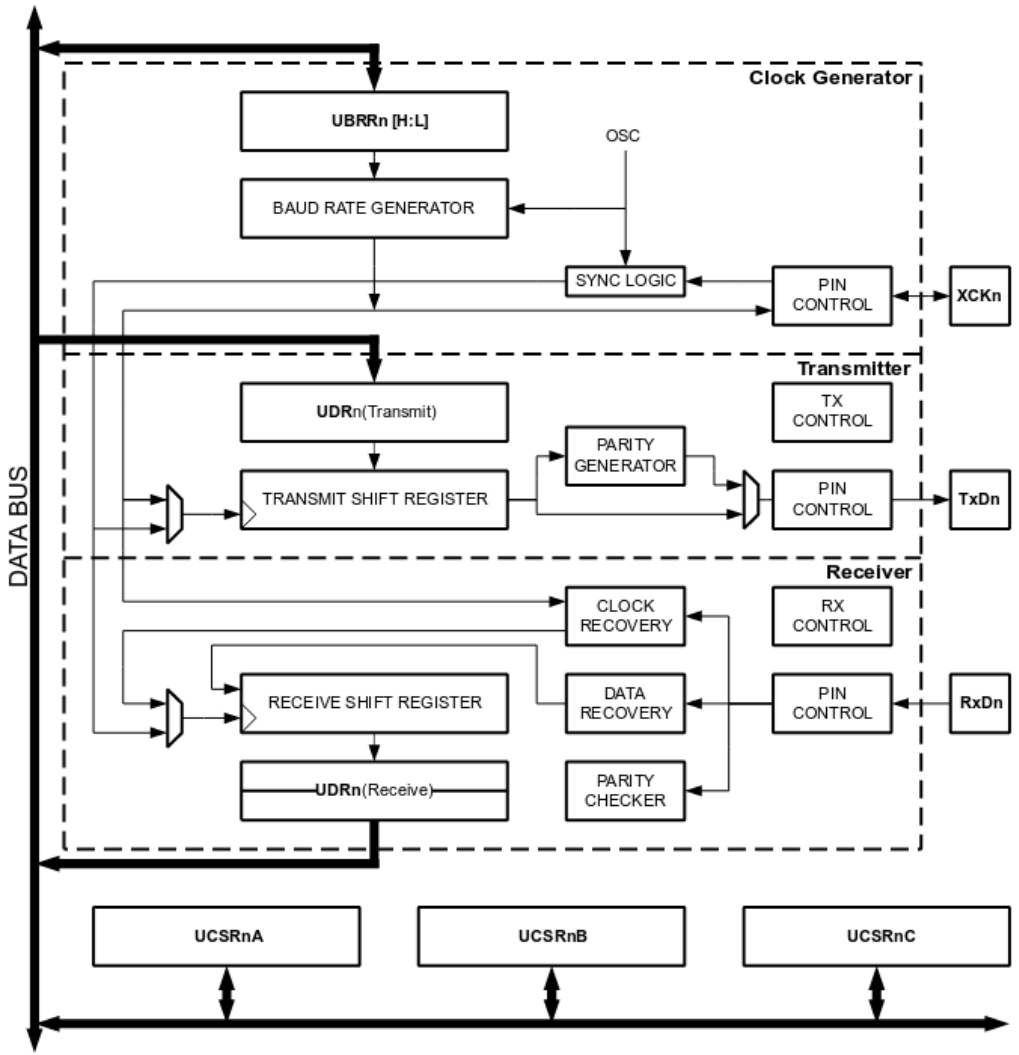
- Start-ul este dat de o tranziție a liniei între 1 și 0
- Datele pot avea între 5 și 9 biți
- Datele pot fi verificate cu un bit de paritate
- Sunt 1 sau 2 biți de STOP

[0]Sursă imagine: Wikipedia Commons

- Universal Synchronous/Asynchronous Receiver Transmitter
- Comunicație full-duplex
- UART - Protocol cu ceas **implicit**
  - Ambii parteneri în comunicație (cele 2 dispozitive) folosesc același ceas în transmisie/recepție
  - Similar, toți parametrii comunicației trebuie să fie identici
    - Frame size
    - Stop bits
    - Parity bit
- USART - ceas **explicit** - `XCK`
  - Generat de master
  - Slave-ul trebuie să se sincronizeze

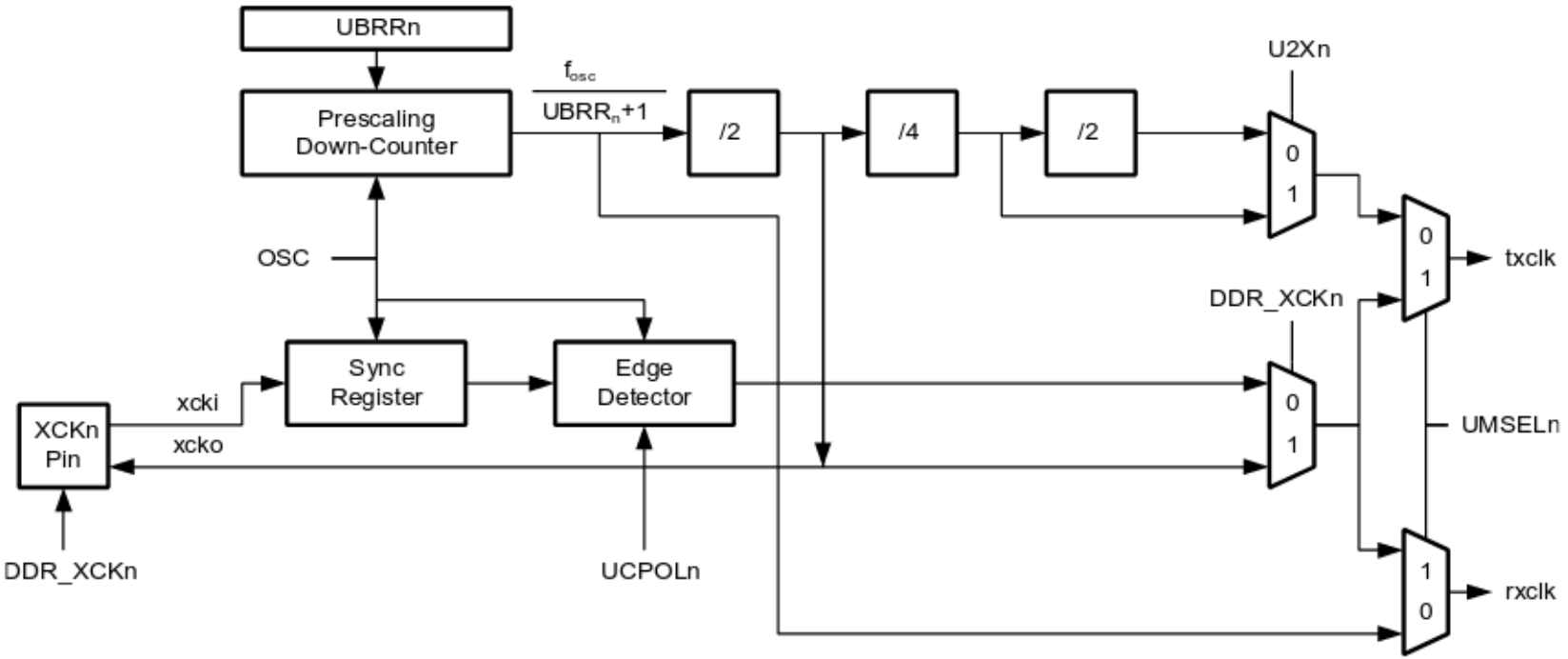
UART pe ATMega324a - arhitectură

Figure 21-1. USART Block Diagram



[0]Sursă imagine: Datasheet ATMega324a

- Ce trebuie să controlăm
  - Structura unui cadru (frame)
  - Viteza de tx/rx
- Ce trebuie să aflăm
  - Flow date în/din buffer
  - Verificări paritate



- Datorită sincronizării între domenii de timp

$$f_{XCK} < \frac{f_{OSC}}{4}$$

[0]Sursă imagine: Datasheet ATMega324a

1. Receptorul caută START
   - Detectează o tranziție 1→0
   - Citiri repetate (3) în mijlocul bitului
   - 3 x 0 → START valid
2. Citește fiecare bit din frame
3. Citește STOP bit ca 1 (altfel *Frame Error*)

- Surse de erori:
  - Frecvențele de tx/rx nu sunt egale
  - Frecvențe instabile
  - Glitch-uri pe linia de transmisie (START fals)
  - All of the above
- Frame Error - bit în registru de status
- Parity Error - bit în registru de status

- Format cadru - identic
- Două câmpuri de biți de configurare în plus
  - UMSEL - mode select
  - UCPOL - clock polarity
- O linie de semnal în plus
- Frecvențe de comunicație mai mari
- Sursele de eroare mult diminuate

- UBRR0
  - Registru prescaler pentru generarea ceasului de transmisie
  - Prescaler-ul de la f_{CPU} este $16 \cdot UBRR0$
  - `UBRR0` poate fi calculat sau poate fi extras din tabele existente în *datasheet*
  - $$UBRR = \frac{f_{CPU}}{16 \cdot baud} - 1$$
  - putem dubla viteza cu același `UBRR`
  - $$UBRR_{U2X} = \frac{f_{CPU}}{8 \cdot baud} - 1$$
  - $$Error_{[\%]} = \left(\frac{Baudrate_{closest}}{Baudrate} - 1\right) \cdot \%$$

- UCSRnA (status)
  - `UDRE` - 1 când buffer-ul de ieșire este liber
  - `RXC` - 1 când am primit un frame nou
  - `U2X` - 1 dacă micșorăm prescalerul la 8
  - `FE` - 1 dacă a fost Frame Error
  - `UPE` - 1 dacă a fost Parity Error
- UCSRnB (biți enable)
  - `TXEN` - Activarea transmițătorului
  - `RXEN` - Activarea receptorului
  - `RXCIE`, `TXCIE`, `UDRIE` - Activarea întreruperilor pentru RX complete, TX commplete, buffer empty
- UCSRnC (parametri frame)
  - `USBS` - numărul de biți de stop
  - `UCSZ` - dimensiunea unui frame
  - `UPM` - configurarea bitului de paritate

- inițializare USART0 9600,8N1

```
UCSR0C |= (1 << TXEN0) | (1 << RXEN0);
UBRR0 = 103;
```

- inițializare USART0 115200,8E2

```
UCSR0B |= (1 << TXEN0) | (1 << RXEN0);
UCSR0C |= (1 << UPM01) | (1 << USBS0) | (3 << UCSZ00);
UBRR0 = 8;
```

- inițializare USART1 2000000 (2Mbaud), 8O2

```
UCSR1A |= (1 << U2X1);
UCSR1B |= (1 << TXEN1) | (1 << RXEN1);
UCSR1C |= (1 << UPM10) | (1 << USBS1) | (3 << UCSZ10);
UBRR1 = 0;
```

- Receive simplu

```
loop_until_bit_is_set(UCSR0A, RXC0);
char c = UDR0;
```

  - Busy-waiting

- Receive pe întrerupere

```
volatile char c;
ISR(USART0_RX_vect)
{
    c = UDR0;
}
```

  - Fără busy-waiting, dar se pierd date dacă nu le citește nimeni din variabila shared



UART snippets

```
#define BUFSIZE     (100)

volatile uint8_t rx_buffer[BUFSIZE];
volatile uint8_t read_index = 0, write_index = 0;

ISR(USART0_RX_vect)
{
    uint8_t next_write = (write_index + 1) % BUFSIZE;

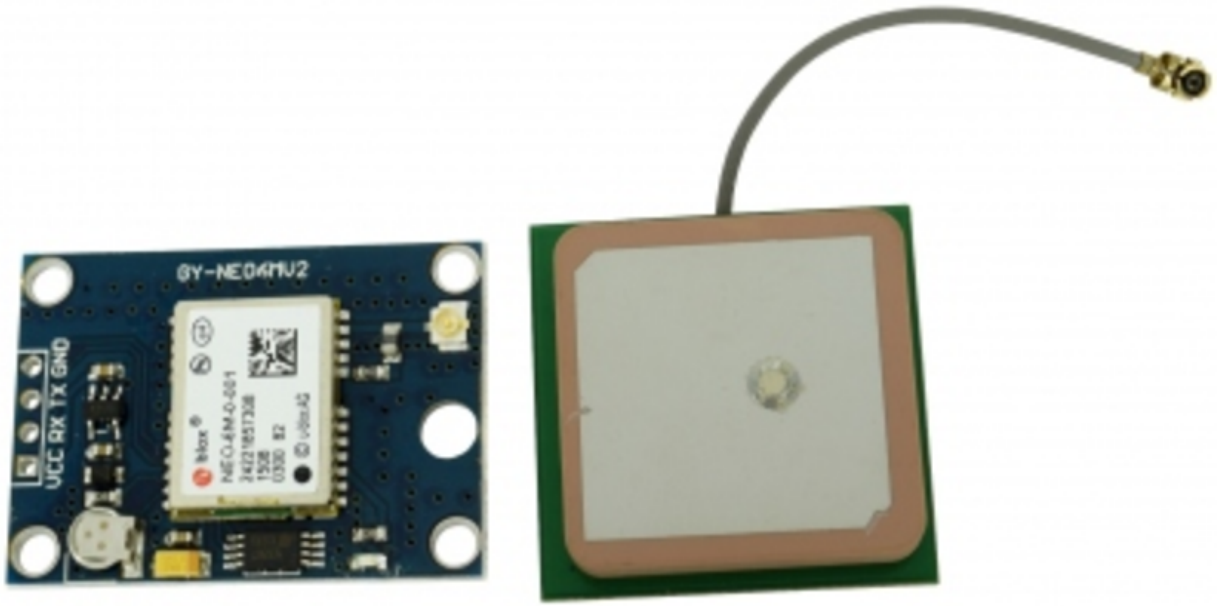
    if (next_write != read_index) /* daca nu e plin bufferul */
    {
        rx_buffer[write_index] = UDR0;
        write_index = next_write;
    }
}

int main()
{
    init_stuff();
    sei();

    while(1)
    {
        while(read_index != write_index) /* cata vreme avem elemente */
        {
            LCD_putChar(rx_buffer[read_index]);
            read_index = (read_index + 1) % BUFSIZE;
        }
    }
    return 0;
}
```

- Necesitatea unui buffer
  - Dacă folosim busy-waiting nu avem multă nevoie, dar avem multe alte dezavantaje
  - Bufferul din periferic reține un singur frame (5-9 biți)
  - Un buffer software ne va garanta că nu pierdem caractere

UART exemplu - GPS



- Modulele GSM/GPS de multe ori folosesc UART
- De ce?

CFG_COM1	CFG_COM0	Protocol	Messages	UARTBaud rate	USB power
1	**1**	**NMEA**	**GSV, RMC, GSA, GGA, GLL, VTG, TXT**	**9600**	**BUS Powered**
1	0	NMEA	GSV, RMC, GSA, GGA, GLL, VTG, TXT	38400	Self Powered
0	1	NMEA	GSV[14], RMC, GSA, GGA, VTG, TXT	4800	BUS Powered
0	0	UBX	NAV-SOL, NAV-STATUS, NAV-SVINFO, NAV-CLOCK, INF, MON-EXCEPT, AID-ALPSERV	57600	BUS Powered

- Configurația pinilor dictează protocoalele
- Pentru 1 și 1 avem UART 9600 baud cu formatul GGA

Table 1-3 contains the values for the following example:

$GPGGA,161229.487,3723.2475,N,12158.3416,W,1,07,1.0,9.0,M, , , ,0000*18

Table 1-3 GGA Data Format

Name	Example	Units	Description
Message ID	$GPGGA		GGA protocol header
UTC Time	161229.487		hhmmss.sss
Latitude	3723.2475		ddmm.mmmm
N/S Indicator	N		N=north or S=south
Longitude	12158.3416		dddmm.mmmm
E/W Indicator	W		E=east or W=west
Position Fix Indicator	1		See Table 1-4
Satellites Used	07		Range 0 to 12
HDOP	1.0		Horizontal Dilution of Precision
MSL Altitude	9.0	meters	
Units	M	meters	
Geoid Separation		meters	
Units	M	meters	
Age of Diff. Corr.		second	Null fields when DGPS is not used
Diff. Ref. Station ID	0000		
Checksum	*18		
<CR> <LF>			End of message termination



Checklist

- Conectăm modulul la ATMega324 (TX la RX, RX la TX)
- Masa și alimentarea, pinii de configurare pentru UART 9600 baud
- Configurăm seriala UART 0/1 pentru comunicație la 9600 baud
- Așteptăm frameuri pe serială
- Citim frameurile într-un buffer, parsăm atunci când am primit "tot"

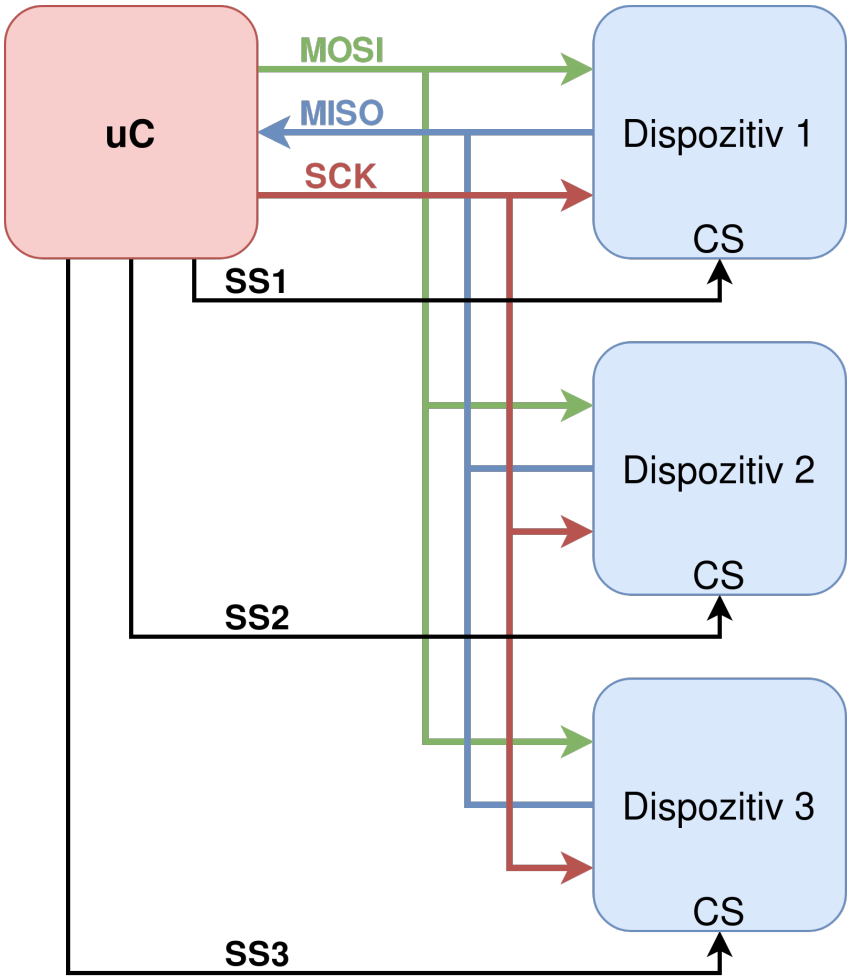
- Cum se rezolvă?
- Cum se face inițializarea?
- Cum se face recepționarea datelor?
- Unde/Când sunt parsate datele?
- Ce facem cu rezultatul procesării datelor?



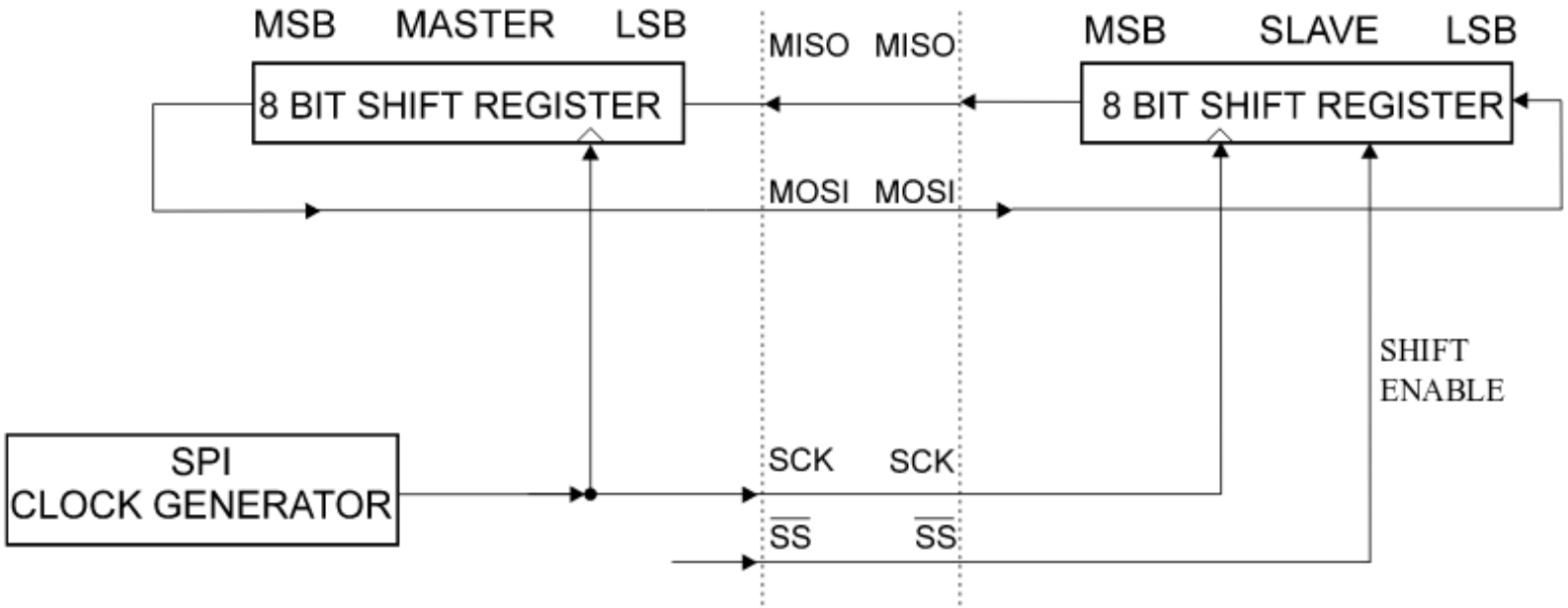
UART - concluzii

- Ceas implicit
- Transmițător independent de receptor
- Susceptibil la erori
- Error correction
- Mod sincron

- Serial Peripheral Interface
- Ce presupune
  - Comunicație sincronă 1:N
- Când este folosită
  - Dispozitive periferice externe cu bandwidth mare



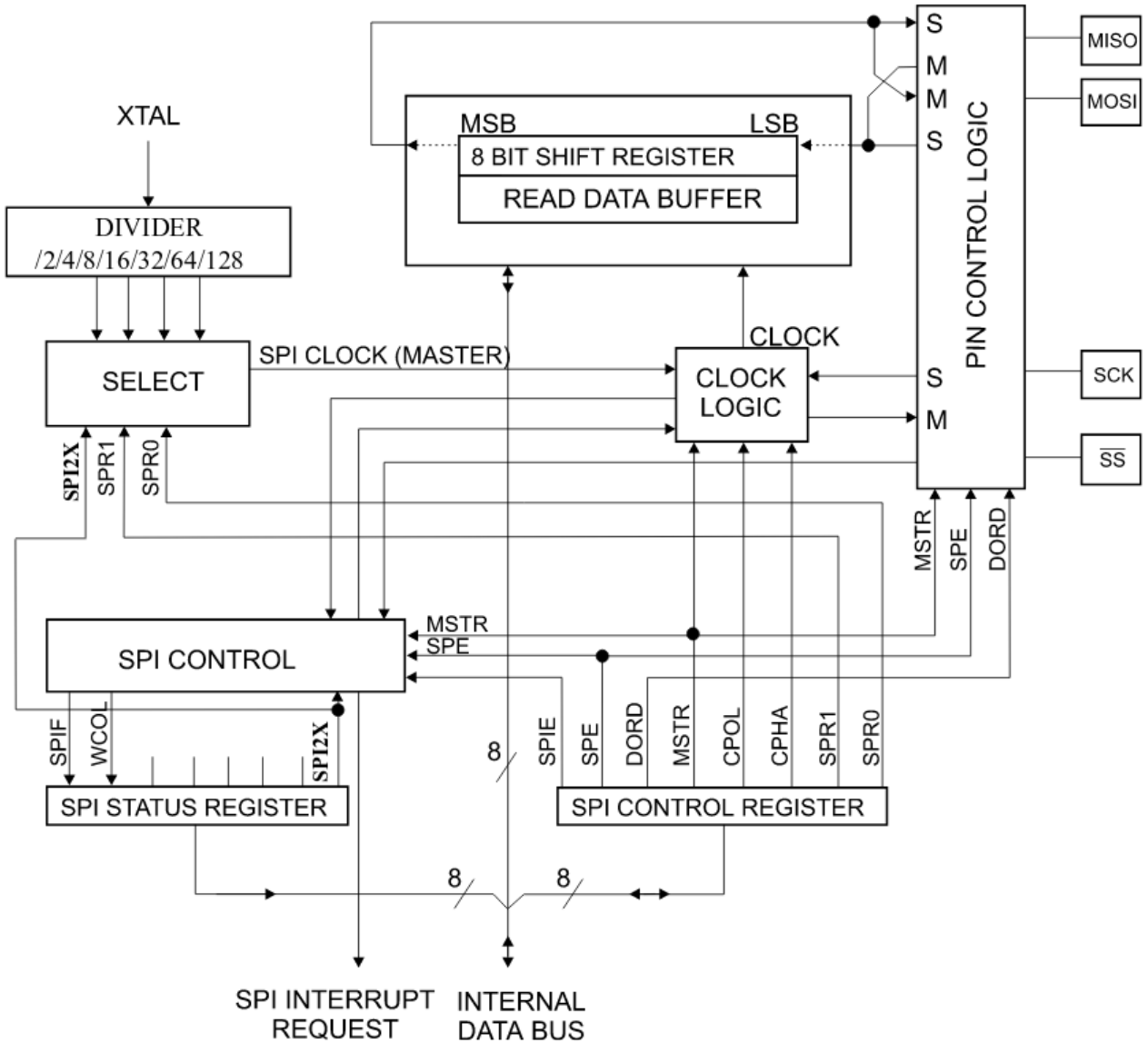
- 4 linii de date pt device
- 3 linii comune
- câte o linie de select pt fiecare device
- linii unidirecționale
- `MOSI` - Master Output Slave Input
- `MISO` - Master Input Slave Output
- `SCK` - Serial Clock
- `SS/CS` - Slave/Chip Select



- MSB de la master devine LSB la Slave (configurabil invers)
- Se formează practic un Rotate Left pe 16 biți
- Cu 8 biți pe Master și 8 biți pe Slave

[0]Sursă imagine: Datasheet ATMega324a

SPI pe ATMega324a - arhitectură



[0]Sursă imagine: Datasheet ATMega324a

- Ce trebuie să controlăm
  - Modul Master/Slave
  - Viteza de transmisie/polaritate
  - Select Chip - se face manual cu GPIO (la fiecare tranzacție)
- Ce trebuie să aflăm
  - Dacă o transmisie s-a terminat

- SPCRn - registru de control
  - SPIE - 1 dacă întreruperea SPIF este activă
  - SPE - 1 dacă SPI este pornit
  - DORD - 1 dacă LSB se trimite primul
  - MSTR - 1 pentru master
  - CPOL:CPHA - Clock polarity & phase
  - SPR1:0 - SPI Data Rate
- SPSRn - registru de status
  - SPIF - 1 dacă un transfer e complet
  - SPI2X - dublează data rate-ul din SPR1:0
- SPDRn - buffer de date

- Comunicație pe SPI full-duplex

```
uint8_t spi_xchg(uint8_t date) {
  SPDR = date;
  loop_until_bit_is_set(SPSR, SPIF);
  return SPDR;
}
```

- Citire pe SPI

```
uint8_t spi_read() {
  return spi_xchg(0xFF);
}
```

- Scriere pe SPI

```
spi_write(uint8_t in) {
  spi_xchg(in);
}
```

- Full-duplex sau half-duplex?
  - Documentație periferic (ex: Accelerometru ADXL362)

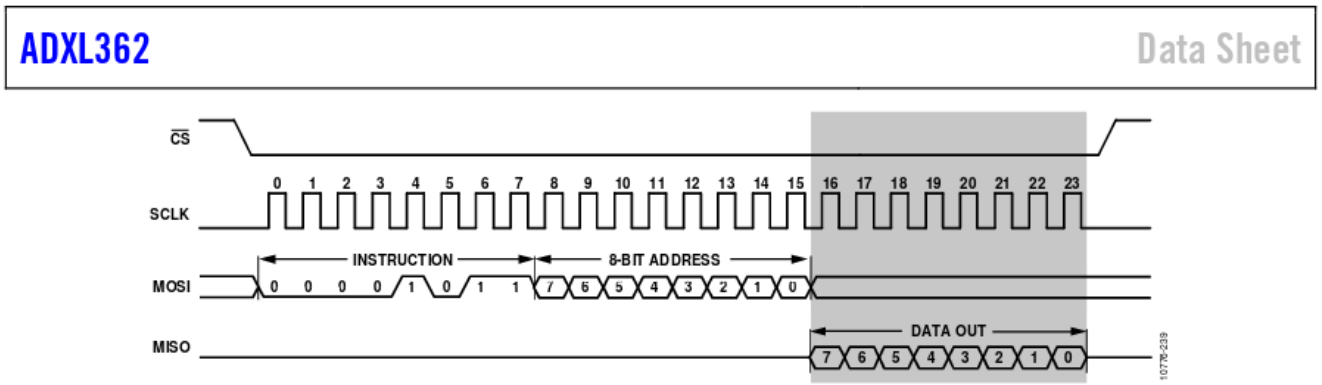


Figure 36. Register Read

  - Vrem să citim de la adresa 0x08 din memoria perifericului (XDATA)

```
spi_write(0x0B); //comanda register read
uint8_t valoare = spi_xchg(0x08); // gresit!
```

  - Dispozitivul nu are cum să răspundă instant la adresă

```
spi_write(0x0B); // comanda register read
spi_write(0x08); // adresa de citit
uint8_t valoare = spi_read(); // corect!
```

[0]Sursă imagine: Datasheet ADXL362

- Senzori (accelerometre, altimetre, giroscop etc.)
- Display-uri
- Memorii (Flash, carduri SD/MMC)
- Este extins pentru bandwidth
  - Dual, Quad, Octo - SPI
  - 2, 4, 8 liniile de date bidirecționale
  - Inițial pe Single SPI



SPI - concluzii

- Ceas explicit - fără erori
- Viteză de transmisie mare (f. mare pentru QSPI OSPI)
- Diferențe mici față de UART sincron
- Necesită multe linii pentru mai multe dispozitive (n+3)



Proiectarea cu Microprocesoare
I^2C, USB

Andrei Voinescu

POLITEHNICA University of Bucharest

2021

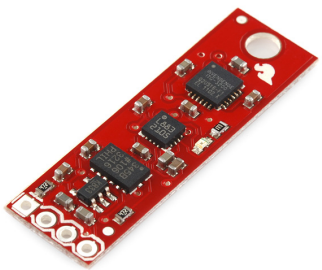
Outline

I²C

USB

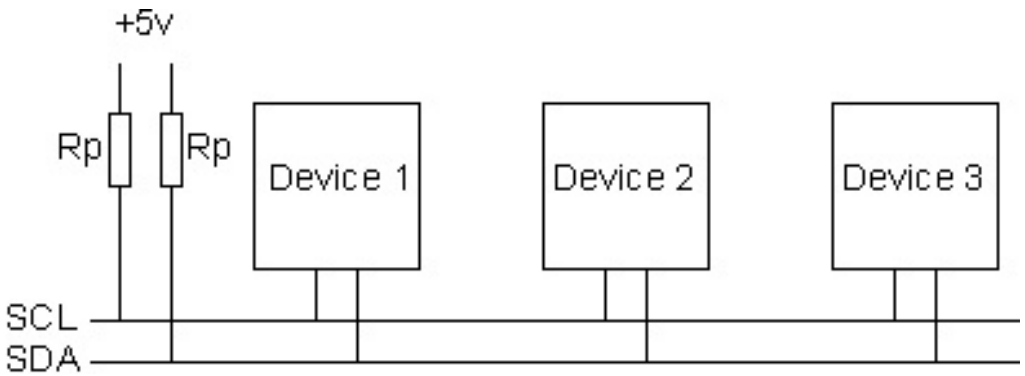
- **Inter-Integrated Circuit**, sau Two-Wire Interface (TWI)
  - Standard dezvoltat de Phillips, proprietar
- Comunicație între microcontrollere
- Comunicație cu senzori digitali

- **Inter-Integrated Circuit**, sau Two-Wire Interface (TWI)
  - Standard dezvoltat de Phillips, proprietar
- Comunicație între microcontrollere
- Comunicație cu senzori digitali
- Altele: memorii EEPROM seriale, ADC-uri, display-uri, ceasuri de timp real, date de monitorizare în PC-uri

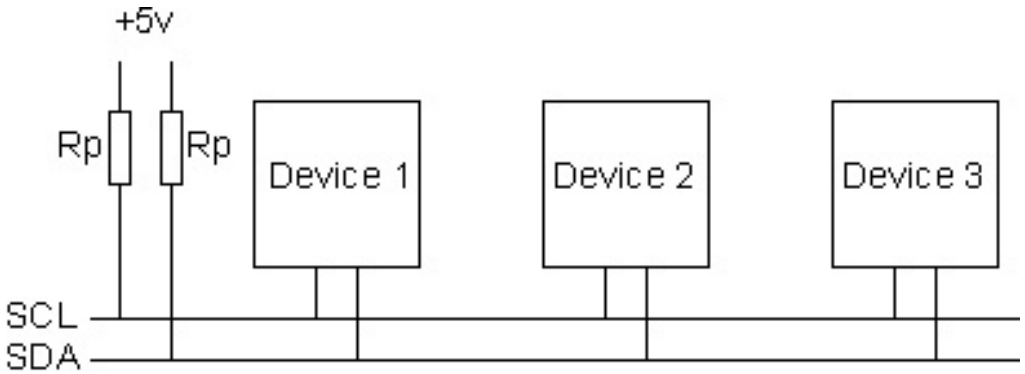




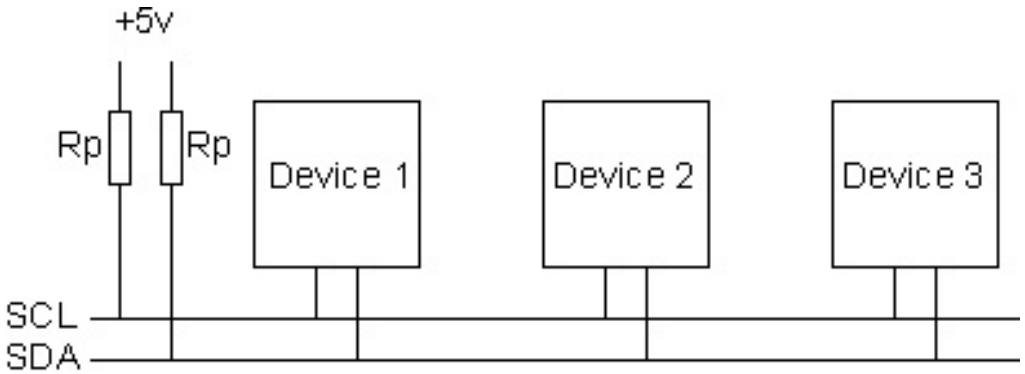




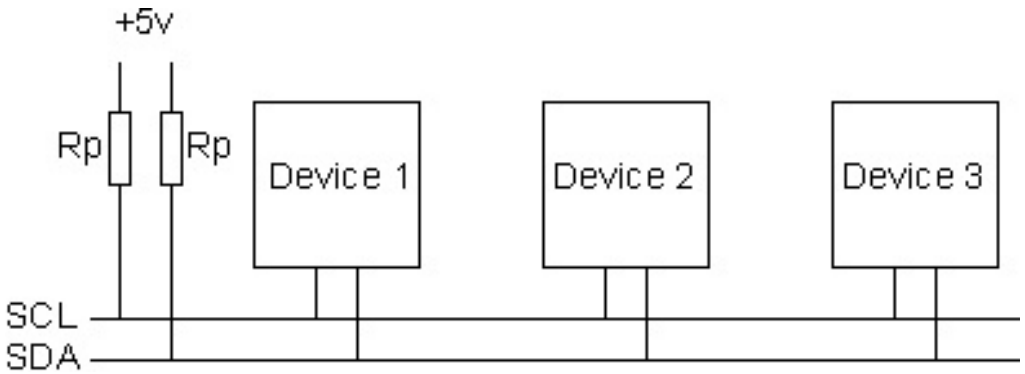
- Protocol master-slave



- Protocol master-slave
- Comunicație sincronă (semnal de ceas explicit)



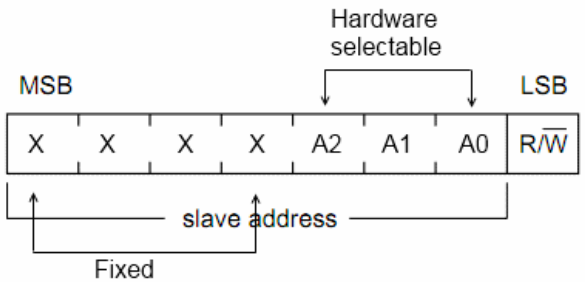
- Protocol master-slave
- Comunicație sincronă (semnal de ceas explicit)
- 2 linii de date: `SCL` și `SDA`

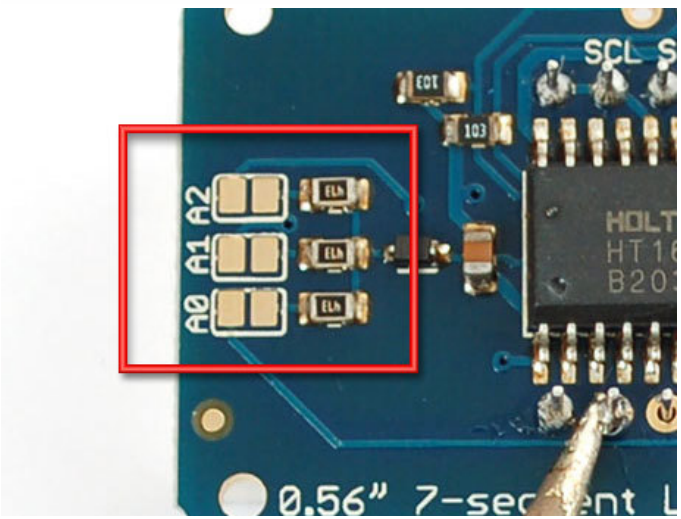


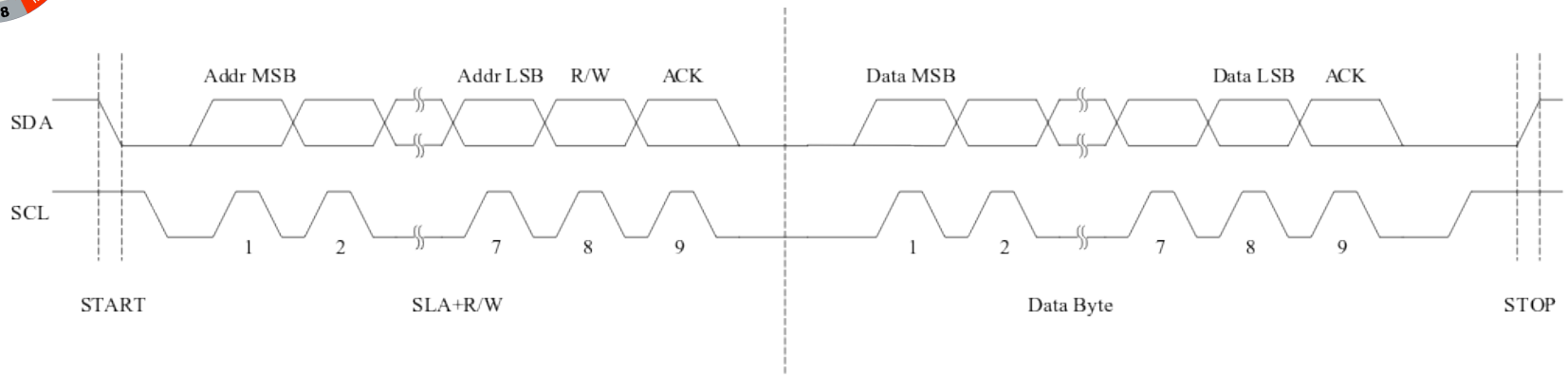
- Conexiuni open-collector $\rightarrow$ pull-up
- 1 - linie lăsată în aer, 0 - linie "trasă" la masă
- Un singur master activ

Adresele în I^{2}C

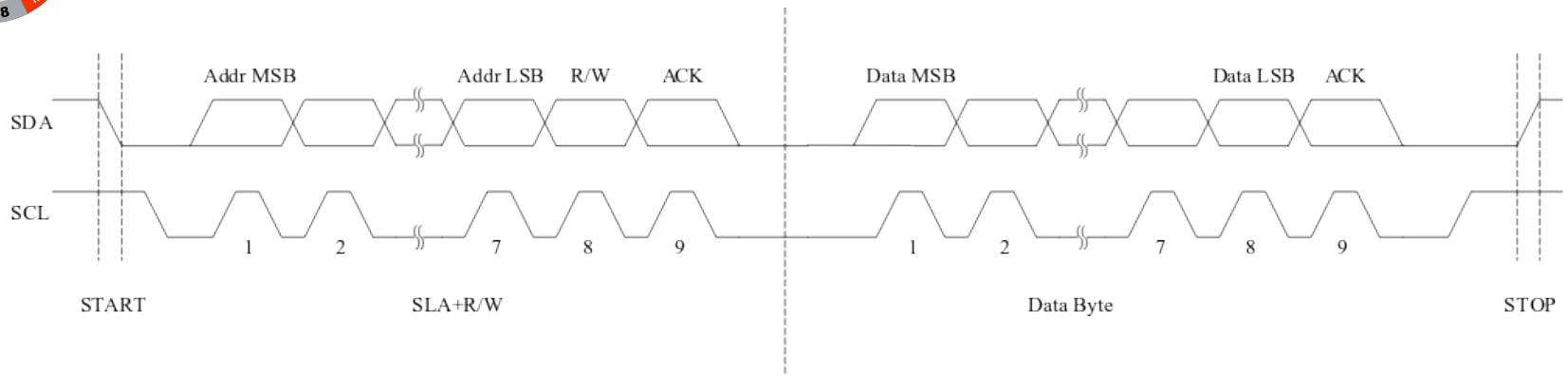
- Adrese pe 7 biți
  - Unii biți fixați
  - Alți biți configurabili hardware (imagine)
- Biții de configurare elimină conflicte între
  - Dispozitive de același fel
  - Dispozitive diferite cu aceeași adresă
- Ultimul bit (8) determină direcția comunicației



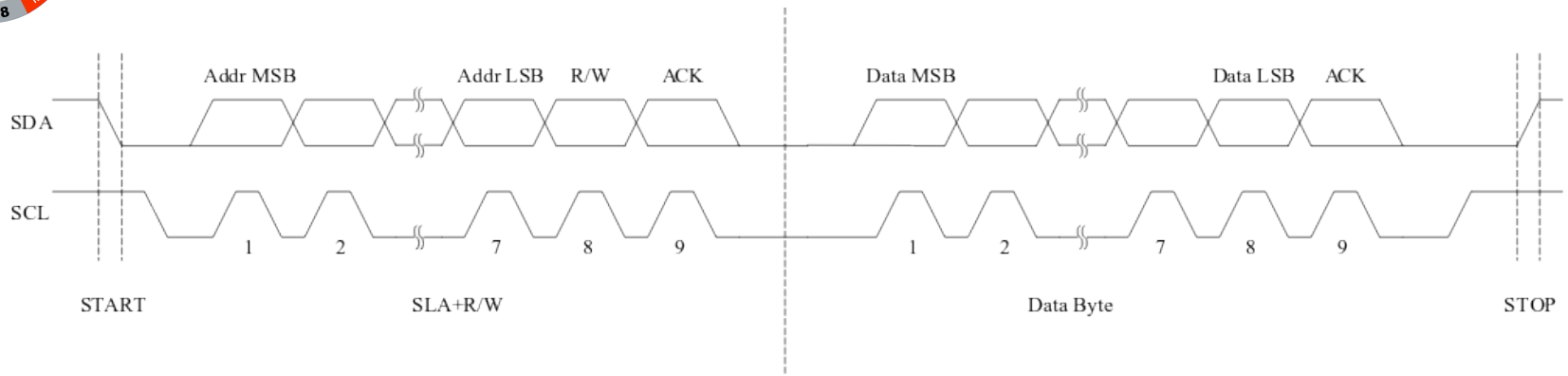




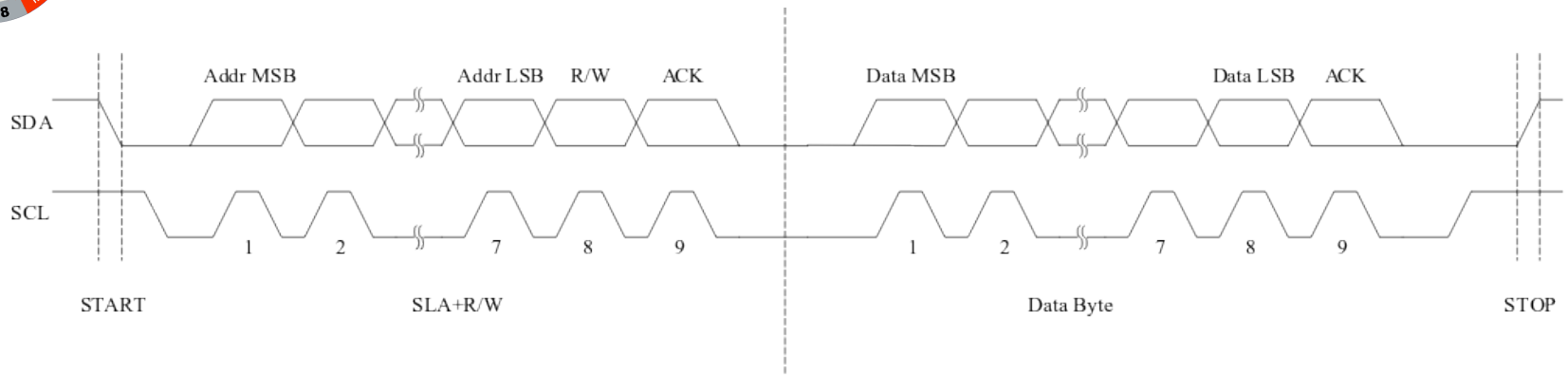
- START este 0 pe SDA când SCL este 1



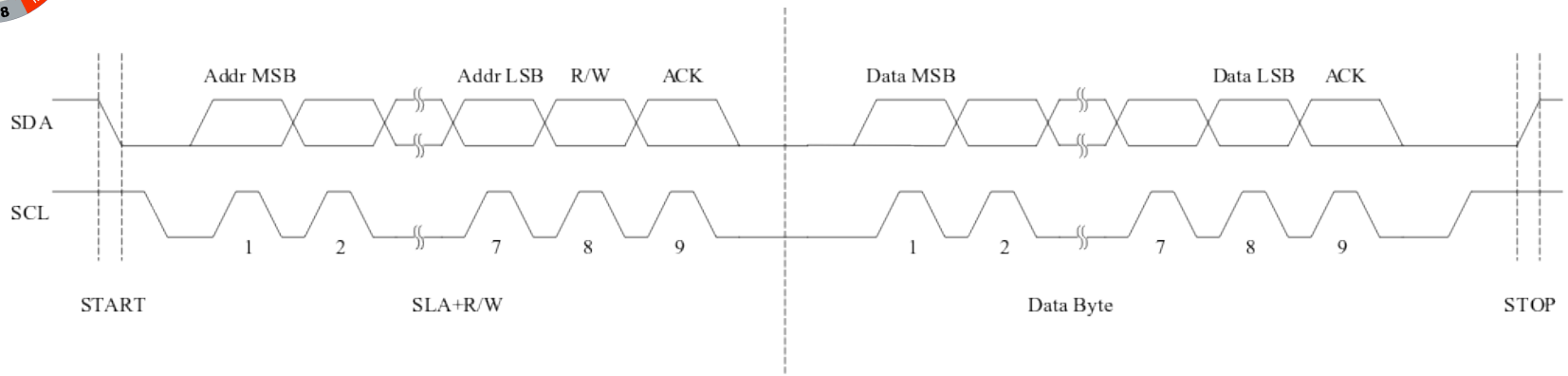
- START este 0 pe SDA când SCL este 1
- STOP este o tranziție către 1 a SDA atunci când SCL este 1



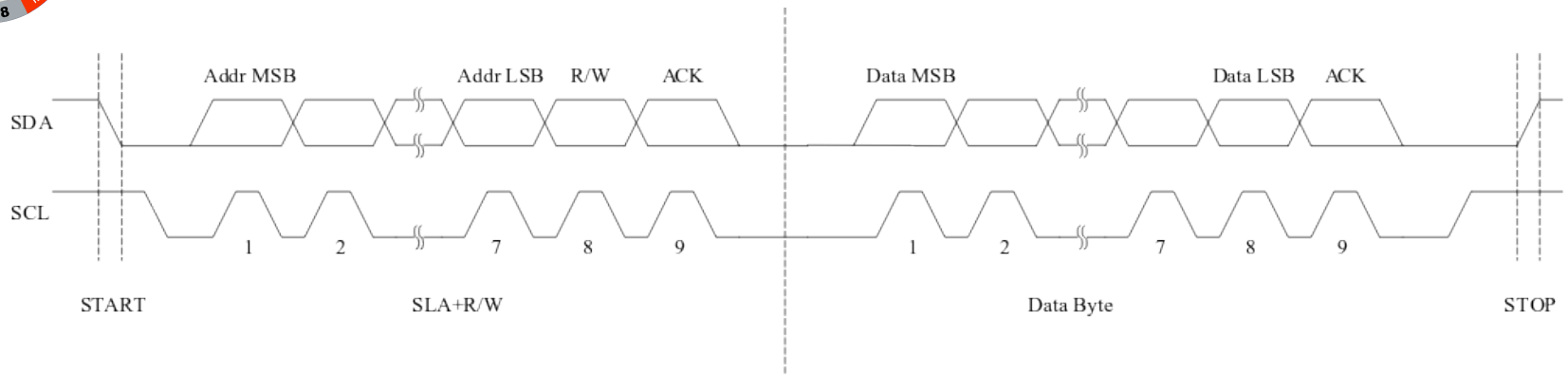
- START este 0 pe SDA când SCL este 1
- STOP este o tranziție către 1 a SDA atunci când SCL este 1
- REPEATED START == START



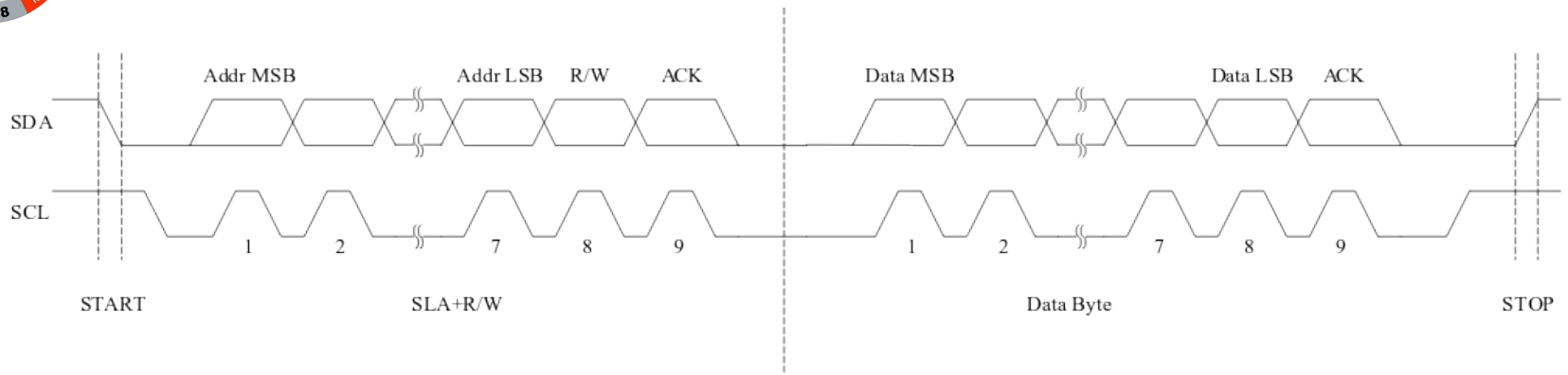
- START este 0 pe SDA când SCL este 1
- STOP este o tranziție către 1 a SDA atunci când SCL este 1
- REPEATED START == START
- ACK
  - Receiver-ul comunicației după 8 biți de date
  - Receiver ține SDA pe 0 când și SCK este pe 0



- START este 0 pe SDA când SCL este 1
- STOP este o tranziție către 1 a SDA atunci când SCL este 1
- REPEATED START == START
- ACK
  - Receiver-ul comunicației după 8 biți de date
  - Receiver ține SDA pe 0 când și SCK este pe 0
- NACK
  - Dacă nu are nimeni adresa



- START este 0 pe SDA când SCL este 1
- STOP este o tranziție către 1 a SDA atunci când SCL este 1
- REPEATED START == START
- ACK
  - Receiver-ul comunicației după 8 biți de date
  - Receiver ține SDA pe 0 când și SCK este pe 0
- NACK
  - Dacă nu are nimeni adresa
  - Trimis înainte de oprirea comunicației



- START este 0 pe SDA când SCL este 1
- STOP este o tranziție către 1 a SDA atunci când SCL este 1
- REPEATED START == START
- ACK
  - Receiver-ul comunicației după 8 biți de date
  - Receiver ține SDA pe 0 când și SCK este pe 0
- NACK
  - Dacă nu are nimeni adresa
  - Trimis înainte de oprirea comunicației
- Datele sunt valide pe frontul crescător al SCL

Exemplu semnal I^2C

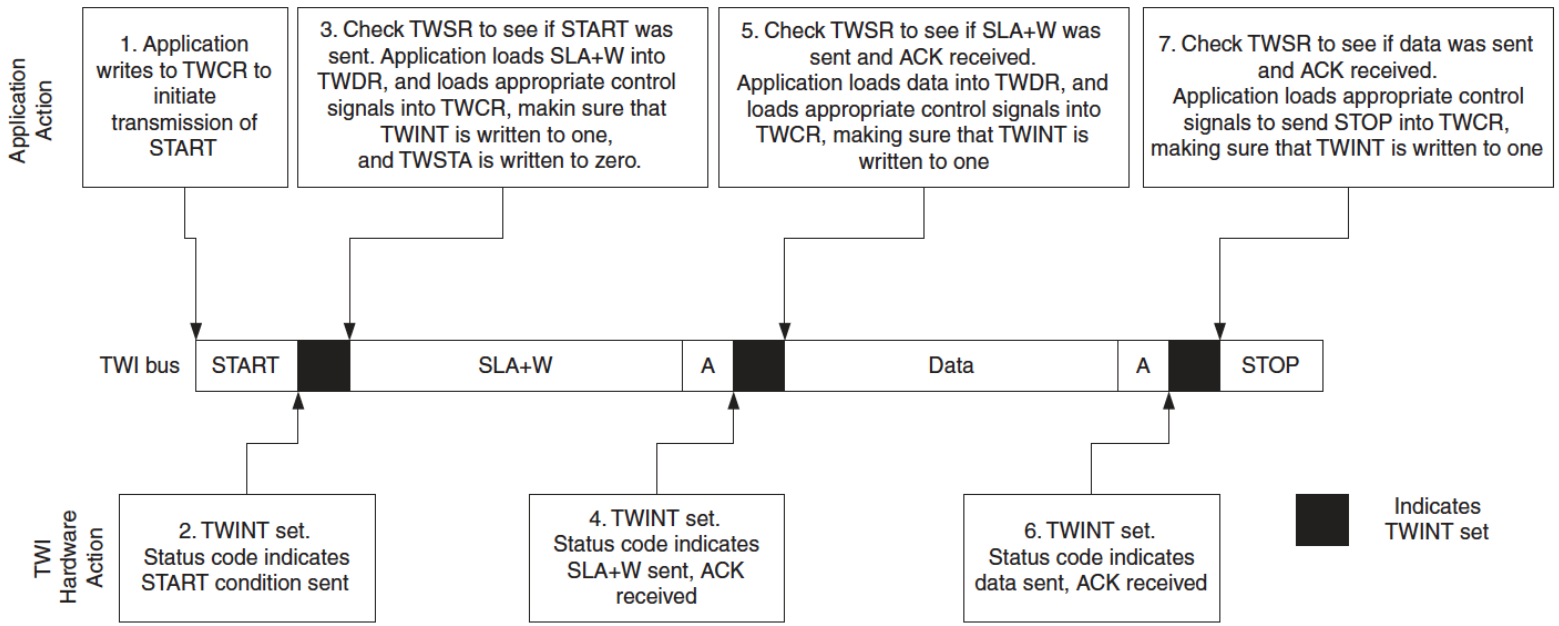


- Liniile de I^2C formează electric un ȘI logic
- Multi-master
  - Fiecare master monitorizează linia de clock (SCL)
  - În puține cazuri vor porni chiar în același timp
    - În momentul în care diferă comunicația "câștigă" cel cu 0 (își vede propriul output)

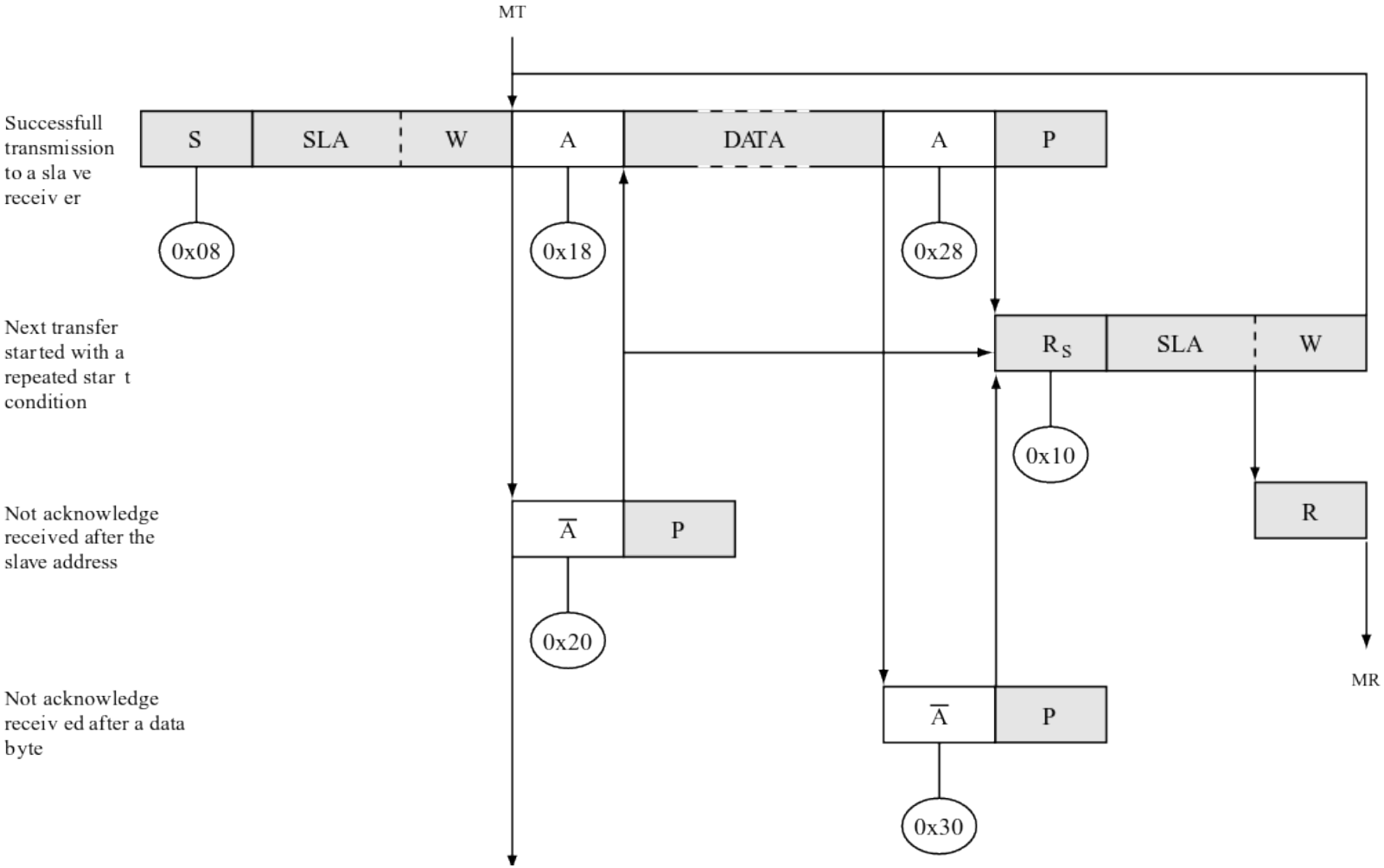
- Registre:
  - `TWCR` - flag-urile de control
  - `TWSR` - cu biți de status
  - `TWDR` - bufferul de date
  - `TWAR` - adresa dispozitivului, dacă este slave
  - `TWBR` - viteza de transmisie

- Registre:
  - `TWCR` - flag-urile de control
  - `TWSR` - cu biți de status
  - `TWDR` - bufferul de date
  - `TWAR` - adresa dispozitivului, dacă este slave
  - `TWBR` - viteza de transmisie
- Întrerupere atunci când perifericul a terminat operația curentă și așteaptă următoarea comandă

Exemplu flow transmisie



Exemplu flowchart



- mașină de stări în software



I²C - exemplu cod

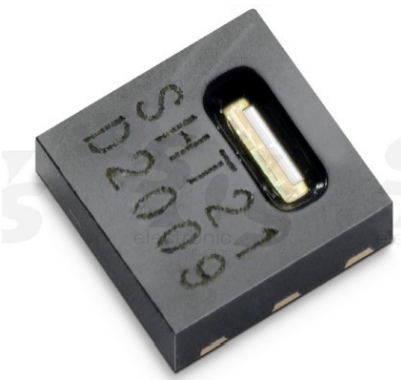
```
void i2c_writeb(uint8_t address, uint8_t value) {
  uint8_t state = STATE_START, next_state;

  TWCR0 = (1 << TWINT) | (1 << TWSTA) | (1 << TWEN);

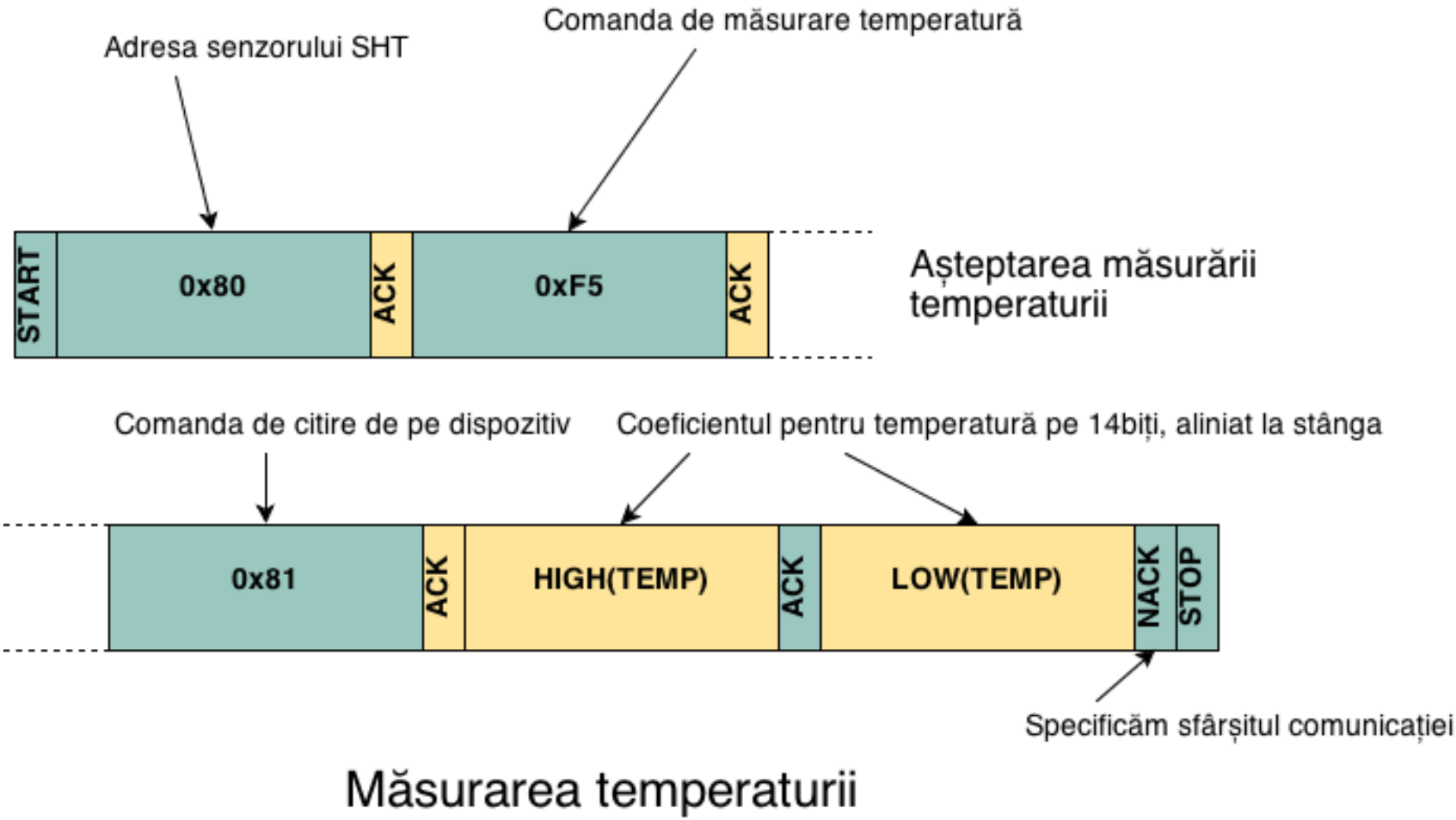
  while(1) {
    loop_until_bit_is_set(TWCR0, TWINT);
    switch(state) {
      case STATE_START:
        if ((TWSR0 & 0xf8) != STATUS_START_OK) // 0x08
          return ERROR_NO_START;
        TWDR0 = (address << 1) + SLA_W; // 0
        next_state = STATE_SLA;
        break;
      case STATE_SLA:
        if ((TWSR0 & 0xf8) != STATUS_SLA_ACK) // 0x18
          return ERROR_NO_SLA_ACK;
        TWDR0 = value;
        next_state = STATE_DATA;
        break;
      case STATE_DATA:
        if ((TWSR0 & 0xf8) != STATUS_DATA_ACK) // 0x18
          return ERROR_NO_DATA_ACK;
        next_state = STATE_STOP;
        TWCR0 = (1 << TWINT) | (1 << TWEN) | (1 << TWSTO);
        return OK;
    }

    TWCR0 = (1 << TWINT) | (1 << TWEN);
    state = next_state;
  }
}
```

- **SHT21**
  - Senzor auto-calibrat de temperatură și umiditate
  - Adresa I²C pe 7 biți este 0x40
  - Măsurarea temperaturii durează cam 100ms (așteptare sau clock stretching)
    - Așteptarea presupune ca master-ul să știe cât este delay-ul între pornirea măsurătorii și terminarea ei, pentru a cere rezultatul după terminare
    - Clock stretching este metoda non-standard prin care slave-ul prelungește un bit 0 de pe SCK pentru a ține ocupat masterul și magistrala

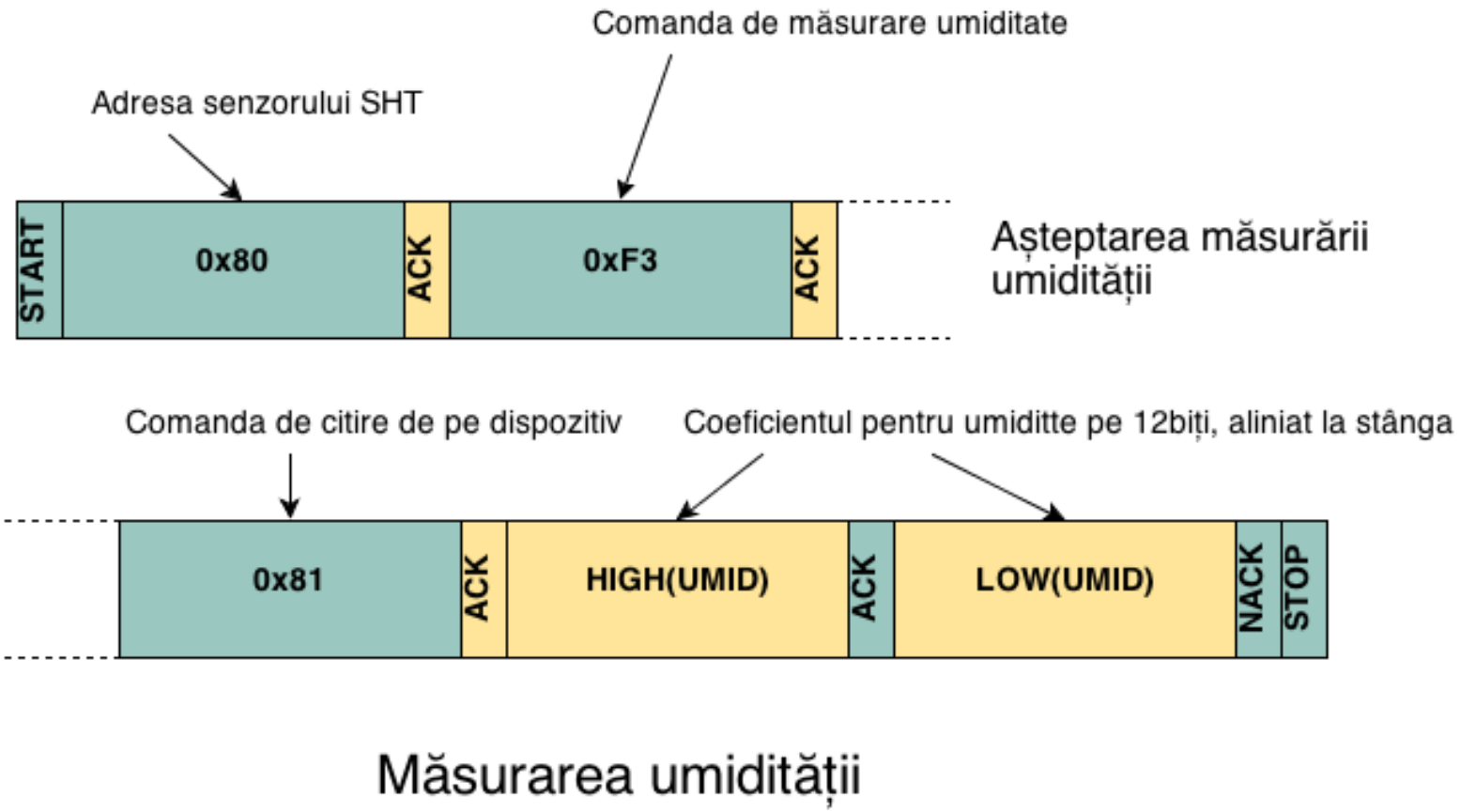


Exemplu comunicație I^{2}C (II)

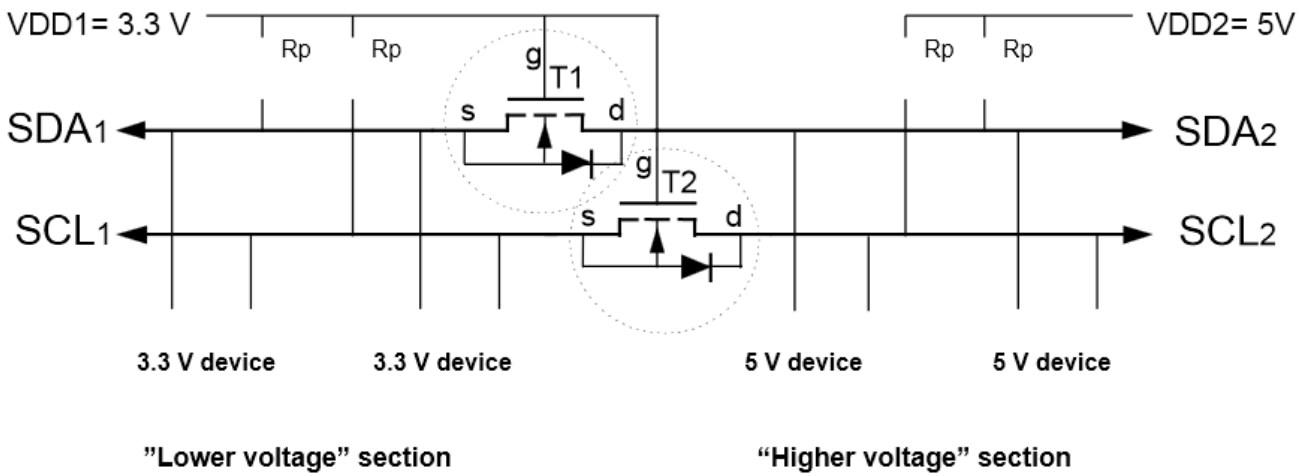


Măsurarea temperaturii

Exemplu comunicație I^{2}C (III)



Ajustare nivele de tensiune pe I^{2}C



- Dacă avem dispozitive cu nivele logice diferite?
- Adaptarea nu se poate face cu divizor rezistiv
- Schema de mai sus garantează adaptarea nivelelor logice
  - În repaus este 1 logic pe ambele părți, MOSFET-ul închis
  - În 0 logic pe tensiunea mică va apărea un $v_{gs} >$ prag, deci MOSFET-ul va fi deschis $\rightarrow$ 0 logic pe tensiunea mare
  - Cu 0 logic pe tensiunea mare va fi leakage current pe dioda de protecție $\rightarrow$ $v_{gs} >$ prag



- SMBus (System Management Bus)

- SMBus (System Management Bus)
  - Încărcarea bateriei, temperaturi interne, ventilatoare
- I^3C (Improved I^2C)

- SMBus (System Management Bus)
  - Încărcarea bateriei, temperaturi interne, ventilatoare
- I^3C (Improved I^2C)
  - Standard apărut 2017
  - Mai eficient energetic
  - Viteze de transfer mai mari decât I^2C
  - Feature-uri avansate



Concluzii I^2C

- Serială cu puține fire (2)



- Serială cu puține fire (2)
- Adresarea se face prin firul de date



- Serială cu puține fire (2)
- Adresarea se face prin firul de date
- Suport multi-master



- Serială cu puține fire (2)
- Adresarea se face prin firul de date
- Suport multi-master
- Viteze relativ mici (400kHz vs. 8MHz la SPI)



Concluzii I^2C

- Serială cu puține fire (2)
- Adresarea se face prin firul de date
- Suport multi-master
- Viteze relativ mici (400kHz vs. 8MHz la SPI)
- Necesită software mai complex



Outline

I^2C

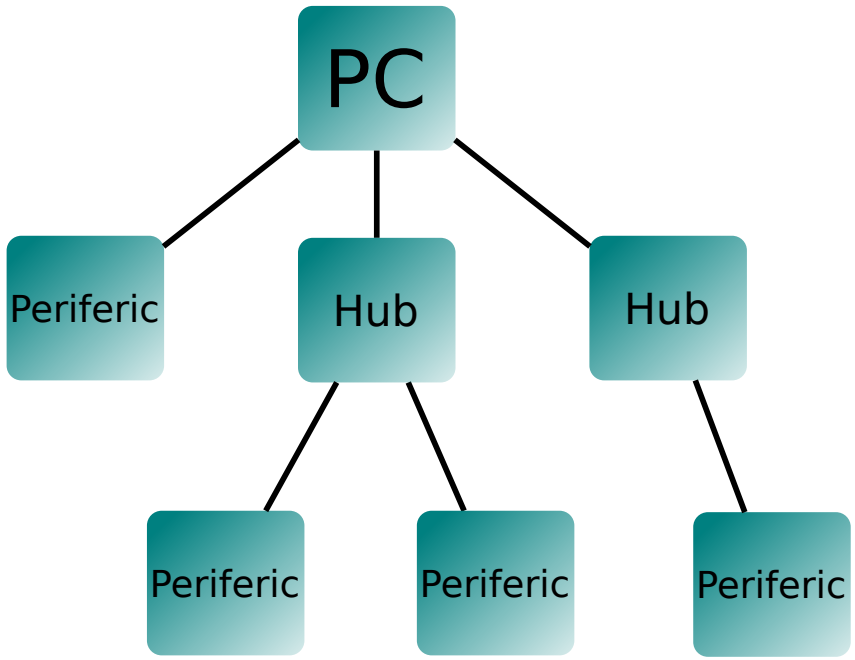
USB

- Standard definit în anii '90
- Unifică interfețe și alimentări
- USB1.0 1996
- USB2.0 2000
  - Viteze de transfer îmbunătățite (480Mbps)
  - Conectori mini
  - OTG în 2006
- USB3.0 2011
  - 5Gbps
  - Comunicație full-duplex
- USB3.1 2014
  - 10Gbps
  - Conector unic, simetric(!)

- Număr redus de conectori (A/B/microAB/miniAB/C)
- Fără cablu de alimentare
- Număr de porturi extensibil
- Rate de transfer bune
- Hot plugging
- Fără configurări manuale

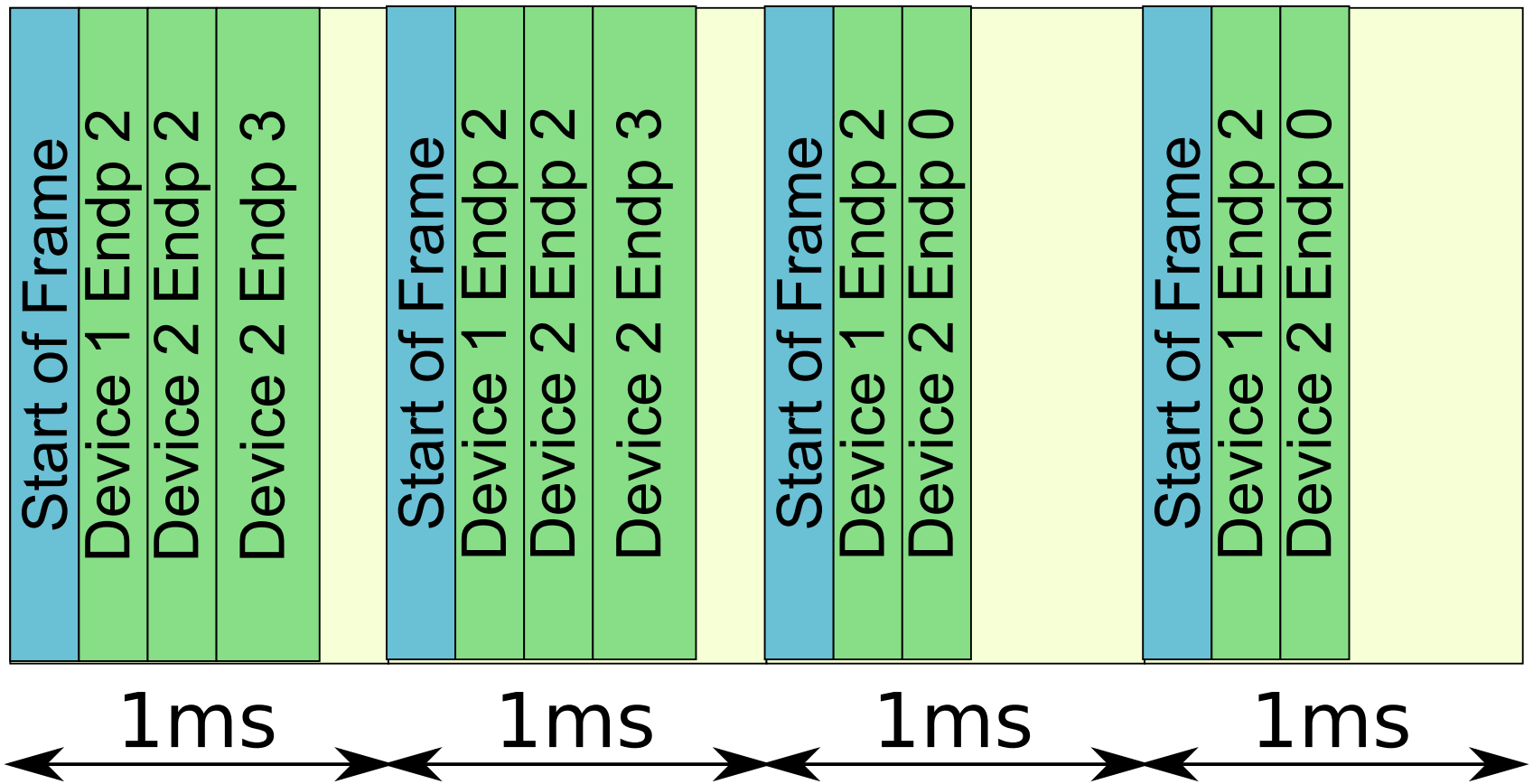
- Avantaje
  - Fără alimentare separată
  - Interfațare unică pentru dispozitive diferite
    - Se pot implementa chip-uri dedicate
  - Suport software în SO
  - Documentație din partea USB-IF
- Dezavantaje
  - Complexitate mărită (de ex. față de rs232)

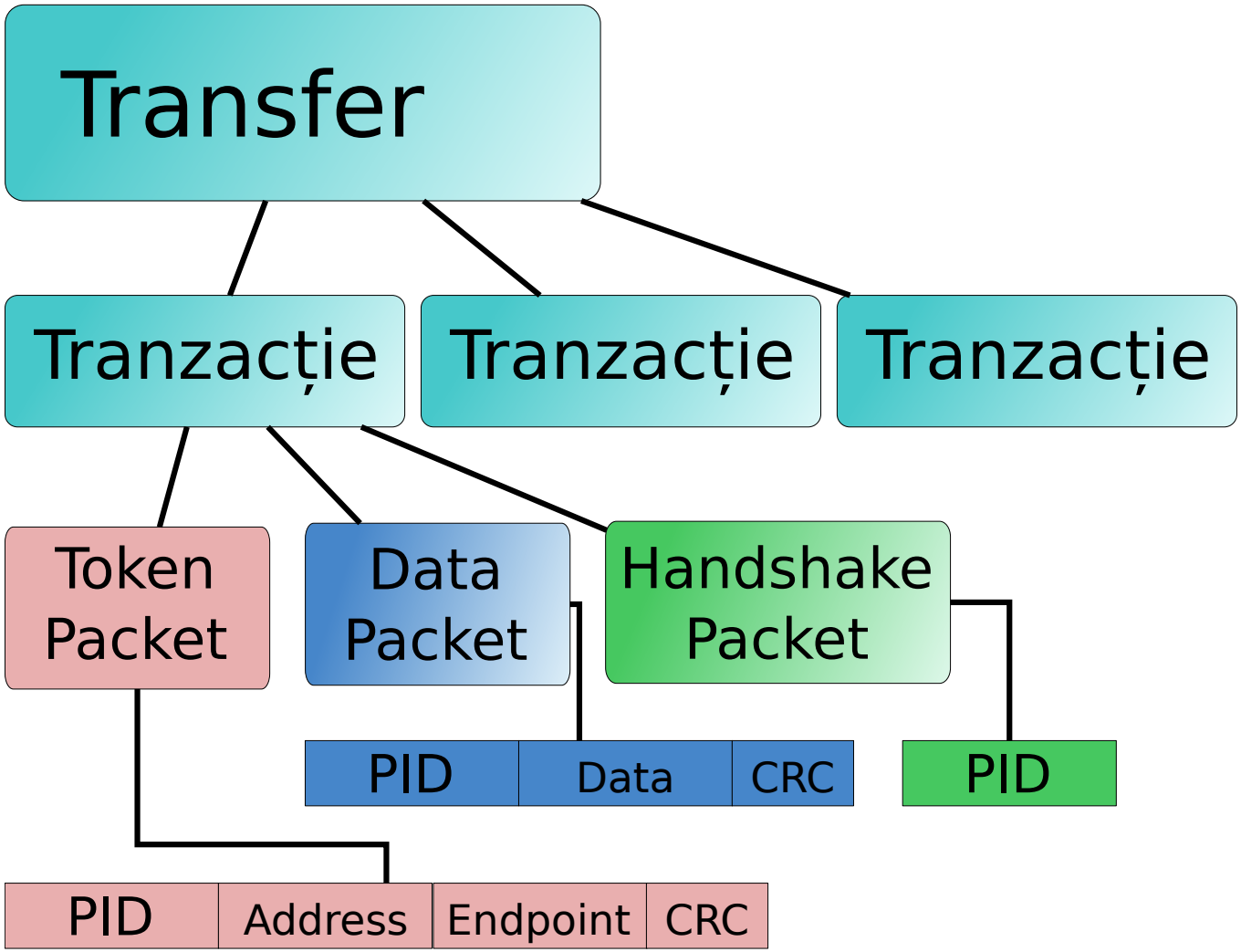
- Arbore:
  - Perifericele - frunze
  - Host,hub-uri - noduri
- Comunicație:
  - Half-duplex
  - Condiționată de host
  - Numai între host și un dispozitiv



- Cunoaște toate dispozitivele
  - capabilitățile
  - nevoile de transfer de date
  - cerințele de consum
- Administrează comunicația
  - Alocă lățime de bandă pentru anumite dispozitive
  - Anunță comunicația dispozitiv→host
  - Interoghează dispozitive

- Responsabil să se identifice host-ului
- Canale de comunicare (endpoints $\leftarrow$ ports)
- Recunoaște comunicația adresată lui
- Răspunde la cererile host-ului
- Menține consumul în limitele declarate
  - Reduce consumul când bus-ul este în SLEEP
- Comunică cu host-ul atunci când acesta cere


Start of Frame
Device 1 Endp 2
Device 2 Endp 2
Device 2 Endp 3
Start of Frame
Device 1 Endp 2
Device 2 Endp 2
Device 2 Endp 3
Start of Frame
Device 1 Endp 2
Device 2 Endp 0
Start of Frame
Device 1 Endp 2
Device 2 Endp 0
1ms
1ms
1ms
1ms


Transfer
Tranzacție
Tranzacție
Tranzacție
Token Packet
Data Packet
Handshake Packet
PID
Data
CRC
PID
PID
Address
Endpoint
CRC

Tip	**Control**	**Interrupt**	**Isochronous**	**Bulk**
Folosire	Identificare Configurare	Mouse Keyboard	Webcam	Storage
Obligatoriu	Da	Nu	Nu	Nu
Low-speed	Posibil	Posibil	Nu	Nu
Transfer	Medie	Mică	Medie	Mare
Trafic	Min 20%	-	Max 80%	-



Endpoints

- Analog porturilor de la socket
- Endpoint-ul este unidirecțional - poate fi IN sau OUT
- Poate avea un singur tip de transfer $\rightarrow$ tip endpoint
- Endpoint 0 - de control
- Fiecare funcție a dispozitivului - endpoints
- `/sys/bus/usb/devices` conține toate informațiile
- `$ cat /sys/bus/usb/devices/usb1/ep_00/type`



- Procedură de înregistrare a unui device la host
- Dispozitivul este atașat fizic

- Procedură de înregistrare a unui device la host
- Dispozitivul este atașat fizic
- Hub-ul
  - detectează dispozitivul
  - informează host-ul
  - detectează viteza cu care se poate comunica
  - resetează device-ul
  - Host-ul află dacă device-ul este high-speed
  - conectează dispozitivul la bus

- Procedură de înregistrare a unui device la host
- Dispozitivul este atașat fizic
- Hub-ul
  - detectează dispozitivul
  - informează host-ul
  - detectează viteza cu care se poate comunica
  - resetează device-ul
  - Host-ul află dacă device-ul este high-speed
  - conectează dispozitivul la bus
- Host-ul
  - alocă o adresă
  - cere descriptorii device-ului
  - selectează un driver potrivit
- Driver-ul selectează o configurație din cele disponibile

- Procedură de înregistrare a unui device la host
- Dispozitivul este atașat fizic
- Hub-ul
  - detectează dispozitivul
  - informează host-ul
  - detectează viteza cu care se poate comunica
  - resetează device-ul
  - Host-ul află dacă device-ul este high-speed
  - conectează dispozitivul la bus
- Host-ul
  - alocă o adresă
  - cere descriptorii device-ului
  - selectează un driver potrivit
- Driver-ul selectează o configurație din cele disponibile
- Încercați să urmăriți acasă cu Wireshark!

- Descriptorii sunt structuri care prezintă capabilitățile și configurările unui dispozitiv către host
- Device descriptor
  - Vendor ID, Product ID, link către string descriptors cu numele
  - Nivelul de compatibilitate USB
  - Numărul de configurații posibile
- Configuration Descriptor
  - Descrie un set de funcționalități care pot fi active simultan
  - Configurația curentă este selectată de Host bazându-se pe compatibilitate
- Interface Descriptor
- Endpoint Descriptor
- String Descriptor



Exemplu de Descriptor

Type	Field	Value
uint8_t	bLength	18
uint8_t	bDescriptorType	0x01 (Device descriptor)
uint16_t	bscUSB	0x0200
uint8_t	bDeviceClass	0x00 (Composite)
uint8_t	bDeviceSubclass	0x00 (No specific subclass)
uint8_t	bDeviceProtocol	0x00 (No specific protocol)
uint8_t	bMaxPacketSize	64
uint8_t	idVendor	0x03eb (VID of Atmel)
uint8_t	idProduct	0x2018
uint16_t	bcdDevice	0x1000
uint8_t	iManufacturer	0 (no manufacturer string)
uint8_t	...	...



- Communication Device Class
  - Abstract Control Model
    - `/dev/ttyACM0`



- Communication Device Class
  - Abstract Control Model
    - `/dev/ttyACM0`
- Mass Storage - stick-uri de memorie



Exemple de clase

- Communication Device Class
  - Abstract Control Model
    - `/dev/ttyACM0`
- Mass Storage - stick-uri de memorie
- Monitor



Exemple de clase

- Communication Device Class
  - Abstract Control Model
    - `/dev/ttyACM0`
- Mass Storage - stick-uri de memorie
- Monitor
- Video class - webcams



Exemple de clase

- Communication Device Class
  - Abstract Control Model
    - `/dev/ttyACM0`
- Mass Storage - stick-uri de memorie
- Monitor
- Video class - webcams
- Audio class - microfoane



Exemple de clase

- Communication Device Class
  - Abstract Control Model
    - `/dev/ttyACM0`
- Mass Storage - stick-uri de memorie
- Monitor
- Video class - webcams
- Audio class - microfoane
- Printer

- ATMega32U4 este printre puținele chip-uri de AVR care au periferic USB, niciunul dintre controlllere care se găsesc în pachet DIP nu au
- Challenges:
  - Funcționarea full-speed necesită un ceas de 48MHz care este format din frecvența inferioară pe care o are controller-ul
  - Transmisia la viteze superioare necesită mult buffering $\rightarrow$ memorie dual-port
  - Endpoint-uri limitate ca memorie și număr, deoarece orice endpoint necesită un buffer
    - 1 control endpoint de 64 bytes
    - 1 endpoint de 256 bytes
    - 5 endpointuride 64 bytes

- Dacă este prezent perifericul de USB
  - Management alimentare
  - Management conectare la bus (+enumerare)
  - Power management periferic
  - Management endpoint-uri
  - Management-ul memoriei alocate endpoint-urilor
- 7 + 8 posibile întreruperi * 7 endpoints = 63
- 20 de registre I/O pentru control/status
- Biblioteci:
  - LUFA - `http://www.fourwalledcubicle.com/files/LUFA/Doc/120219/html/index.html`
  - Atmel - `http://atmel.com/dyn/products/app_notes.asp?family_id=607#USB`



USB - exemplu cod LUFA

```
/* Exemplu HID - emulator mouse/tastatura */

// (...) Structuri configurare

void main()
{
  SetupHardware();
  GlobalInterruptEnable();

  for (;;)
  {
    HID_Device_USBTask(&Generic_HID_Interface);
    USB_USBTask();
  }
}

// (...) SetupHardware (apeleaza si USB_Init)

// (...) Event callbacks (e.g. EVENT_USB_Device_Disconnect

CALLBACK_HID_Device_CreateReport(USB_ClassInfo_HID_Device_t* const HIDInterfaceInfo,
                                 uint8_t* const ReportID,
                                 const uint8_t ReportType,
                                 void* ReportData,
                                 uint16_t* const ReportSize)
{
  /* TODO date puse in ReportData si ReportSize */
}

// (...) Callback host->device CALLBACK_HID_Device_ProcessHIDReport
```

- Pe ATMega324(p)a putem folosi USB software:
  - V-USB - `https://www.obdev.at/products/vusb/index.html`
  - Implementează funcționalitate de low-speed device
    - USB HID - mouse/tastatură/joystick
    - USB ACM - serială
    - Clase custom
  - Folosește `PD2` și `PD3` (external interrupts)

```c
/* Exemplu HID - emulator mouse/tastatura */

// (...) Structuri configurare ex: descriptorii

void main()
{
  (...) // initializare

  for(;;)
  {
    usbPoll();

    if(usbInterruptIsReady())
    {
      /* TODO pus date in reportBuffer */

      usbSetInterrupt((void *) &reportBuffer, sizeof(reportBuffer));
    }
  }
}
```

Exemplu disponibil pe `https://github.com/obdev/v-usb/blob/master/examples/hid-mouse/firmware/main.c`



- Serială cu puține fire
- Hot-plugging
- Gestionare dificilă pe un control cu putere de procesare limitată
- Nu există periferic USB pe ATMega324a
- Folosim biblioteci pur software sau peste periferic



Proiectarea cu Microprocesoare
ADC și memorii ATMega324a

Andrei Voinescu

POLITEHNICA University of Bucharest

2021



Outline

ADC

EEPROM

Fuse bits

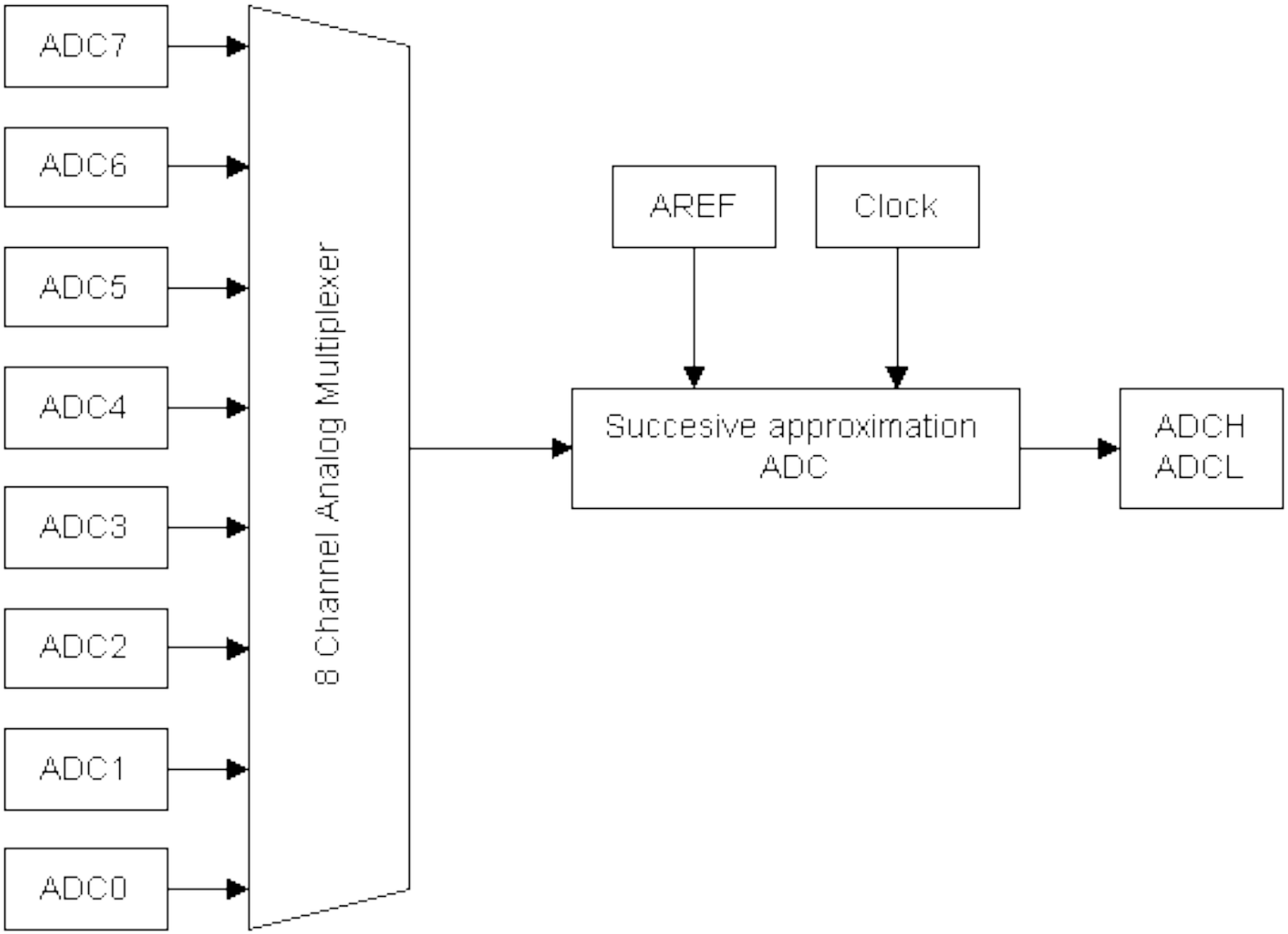
Bootloader support

- Analog to Digital Converter
  - Periferic care se ocupă de conversia unor tensiuni pe pini în valori digitale.

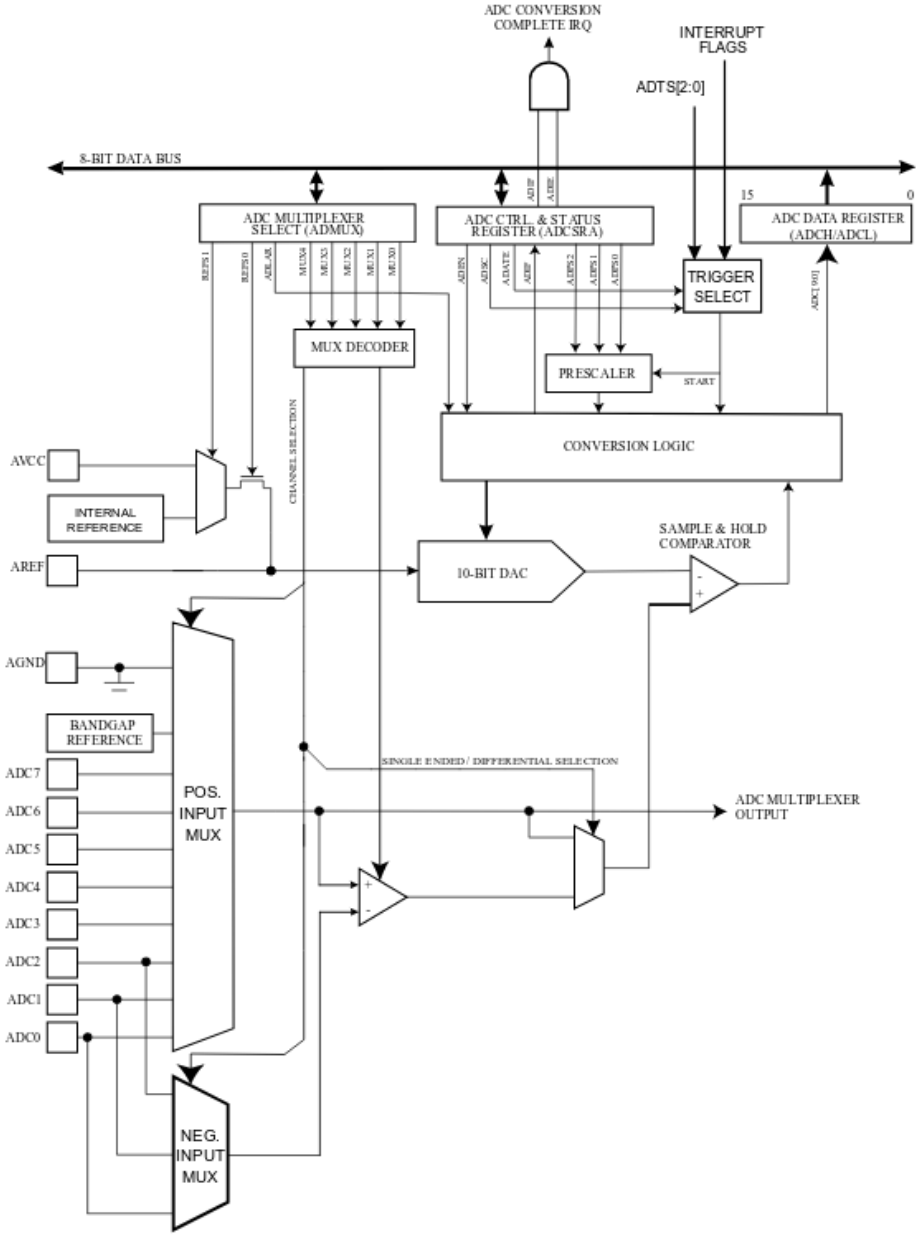
- Analog to Digital Converter
  - Periferic care se ocupă de conversia unor tensiuni pe pini în valori digitale.
  - Util pentru senzori analogici (temperatură, presiune, lumină, curent etc.)
- Rezoluție de 10 biți pe ATMega324

- Analog to Digital Converter
  - Periferic care se ocupă de conversia unor tensiuni pe pini în valori digitale.
  - Util pentru senzori analogici (temperatură, presiune, lumină, curent etc.)
- Rezoluție de 10 biți pe ATMega324
  - Rezoluție vs precizie
  - Rezoluție 10 biți - 1024 valori
  - Domeniu $[0..5V] \rightarrow 4.88mV$
  - Precizie $2LSB = 9.76mV$

ADC - funcționare



ADC - schemă bloc completă



[0] Sursă: Datasheet ATMega324a



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz
  - Frecvență mai mare = precizie mai mică



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz
  - Frecvență mai mare = precizie mai mică
  - Scădem rezoluția = se mărește precizia



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz
  - Frecvență mai mare = precizie mai mică
  - Scădem rezoluția = se mărește precizia
- Care este domeniul $[0..x]$?



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz
  - Frecvență mai mare = precizie mai mică
  - Scădem rezoluția = se mărește precizia
- Care este domeniul $[0..x]$?
  - V_{cc}
  - referință externă
  - 2.56V generați intern
  - 1.1V generați intern



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz
  - Frecvență mai mare = precizie mai mică
  - Scădem rezoluția = se mărește precizia
- Care este domeniul $[0..x]$?
  - V_{cc}
  - referință externă
  - 2.56V generați intern
  - 1.1V generați intern
- De unde luăm tensiunea



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz
  - Frecvență mai mare = precizie mai mică
  - Scădem rezoluția = se mărește precizia
- Care este domeniul $[0..x]$?
  - V_{cc}
  - referință externă
  - 2.56V generați intern
  - 1.1V generați intern
- De unde luăm tensiunea
  - Un pin - PA0..PA7
  - Diferența între doi pini amplificată



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz
  - Frecvență mai mare = precizie mai mică
  - Scădem rezoluția = se mărește precizia
- Care este domeniul $[0..x]$?
  - V_{cc}
  - referință externă
  - 2.56V generați intern
  - 1.1V generați intern
- De unde luăm tensiunea
  - Un pin - PA0..PA7
  - Diferența între doi pini amplificată
- Când facem conversia

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz
  - Frecvență mai mare = precizie mai mică
  - Scădem rezoluția = se mărește precizia
- Care este domeniul $[0..x]$?
  - V_{cc}
  - referință externă
  - 2.56V generați intern
  - 1.1V generați intern
- De unde luăm tensiunea
  - Un pin - PA0..PA7
  - Diferența între doi pini amplificată
- Când facem conversia
  - La comandă
  - La un anumit semnal

- ADMUX
  - REFS - referința de tensiune
  - ADLAR - 10biți aliniați la stânga sau dreapta
  - MUX - selectarea canalului de pe care se face conversia
- ADCSRA
  - ADEN - activarea perifericului
  - ADSC - '1' pentru pornirea unei conversii
  - ADATE - activarea auto-triggering
  - ADIF - '1' când se termină o conversie
  - ADIE - activarea întreruperii de conversion complete
  - ADPS - prescaler pentru aproximațiile succesive ale rezultatului
- ADCSRB
  - ADTS - sursa de trigger
- ADC - Rezultatul conversiei

- Folosire simplă: citire de pe canalul 1

```
uint16_t val;
// initializare domeniu 0..Vcc, canalul 1
ADMUX = (1 << REFS0) | (1 << MUX0);
ADCSRA = (1 << ADEN);

// start conversie
ADCSRA |= (1 << ADSC);

// asteapta conversia sa se termine
loop_until_bit_is_set(ADCSRA, ADIF);
// valoarea este in registrul de 16 biti ADC
val = ADC;
```

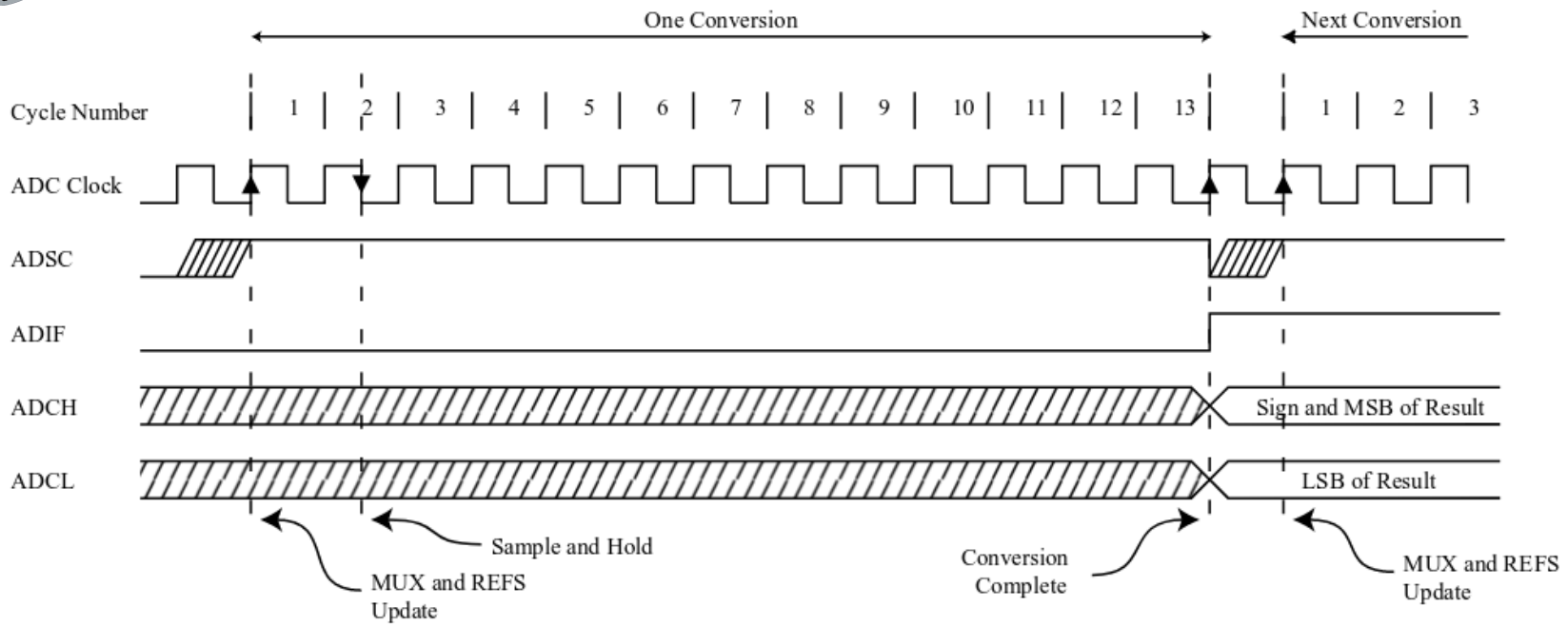
Folosire cu intreruperi - citire de pe canalul 1

```
volatile int16_t val = -1;
ISR(ADC_vect)
{
  val = ADC;
}
void init_adc()
{
  // initializare domeniu 0..Vcc, canalul 1
  ADMUX = (1 << REFS0) | (1 << MUX0);
  ADCSRA = (1 << ADEN);
}
void main()
{
  init_adc();
  sei();

  while(1)
  {
    // start conversie
    ADCSRA |= (1 << ADSC);

    if (val != -1)
    {
      // use val
    }
    _delay_ms(500);
  }
}
```

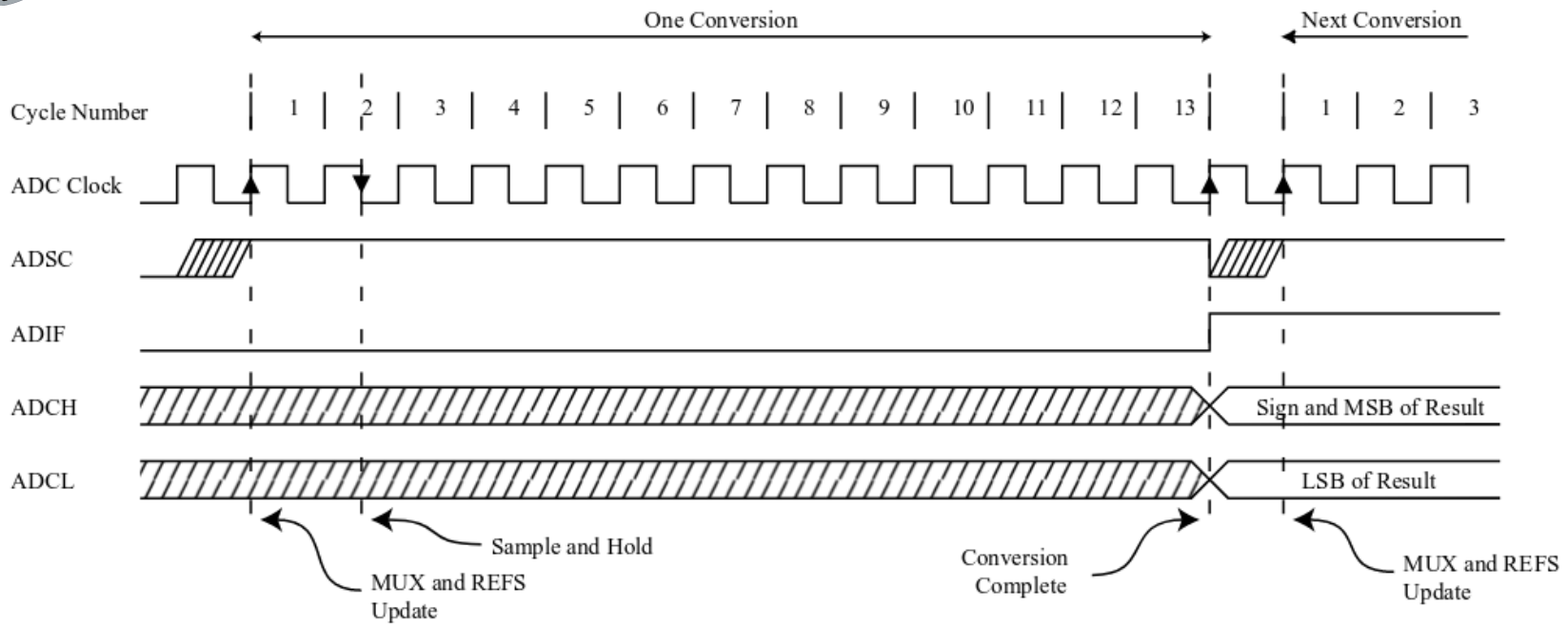
ADC - timing pentru o conversie simplă



- O conversie durează 13.x cicli ADC (prima 25)
- Timing special pentru schimbarea de canal/referință

[0]Sursă: Datasheet ATMega324a

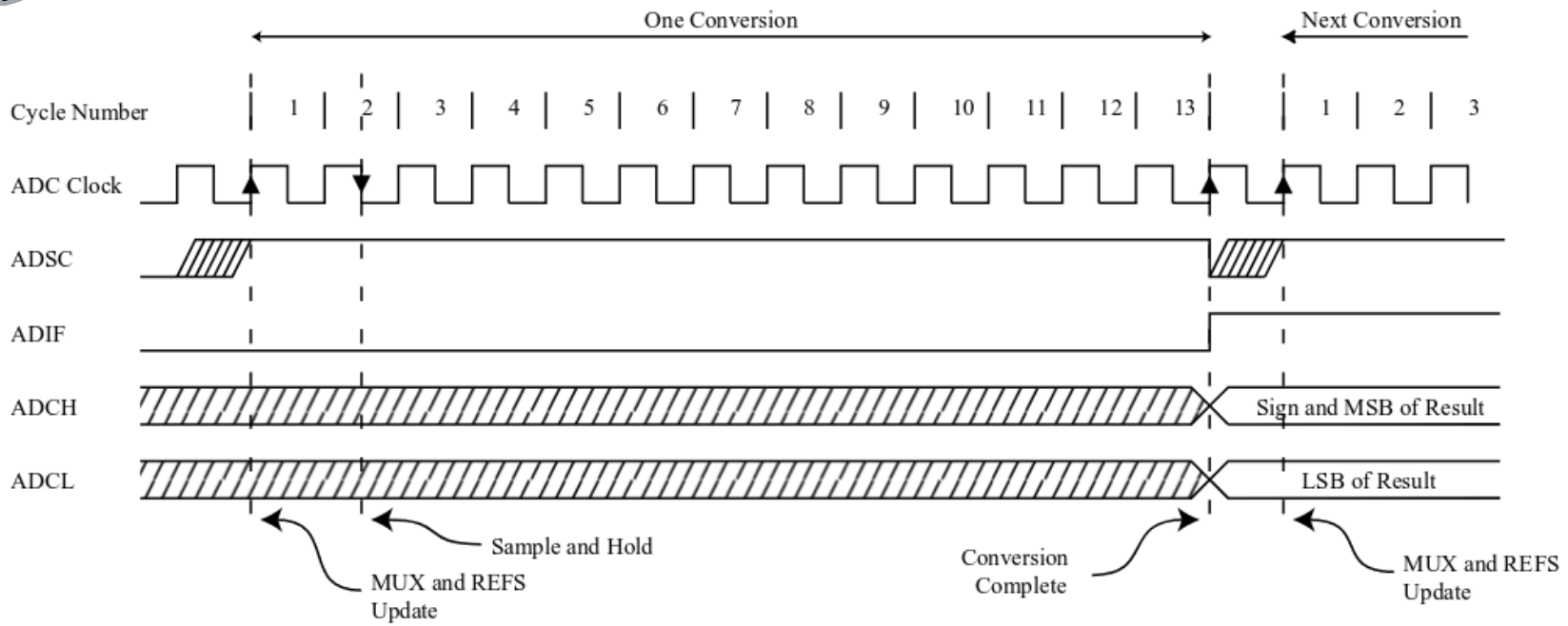
ADC - timing pentru o conversie simplă



- O conversie durează 13.x cicli ADC (prima 25)
- Timing special pentru schimbarea de canal/referință
  - Trebuie evitate schimbări de la startul conversiei până la un ciclu ADC după (maxim $8\mu s$ sau 128 cicli procesor)

[0]Sursă: Datasheet ATMega324a

ADC - timing pentru o conversie simplă

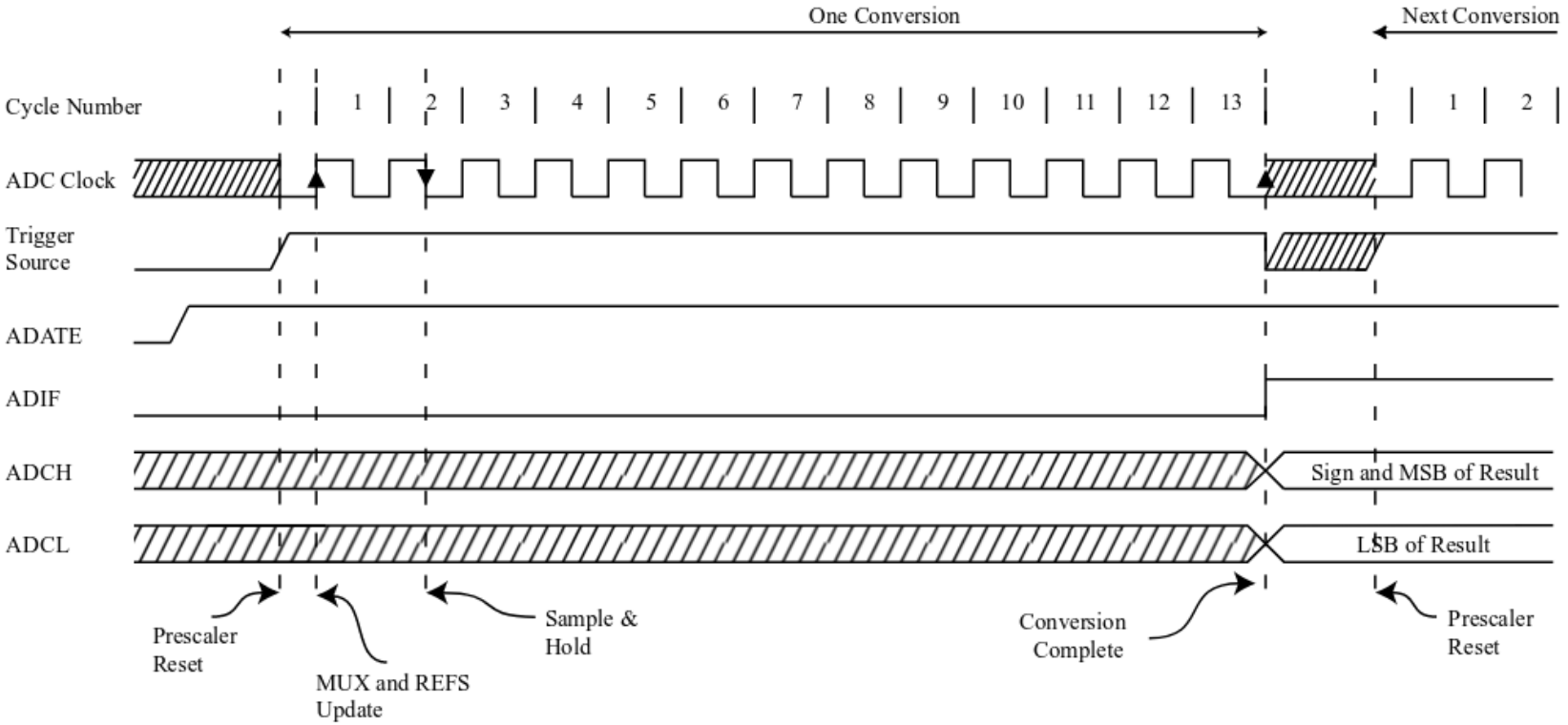


- O conversie durează 13.x cicli ADC (prima 25)
- Timing special pentru schimbarea de canal/referință
  - Trebuie evitate schimbări de la startul conversiei până la un ciclu ADC după (maxim $8\mu s$ sau 128 cicli procesor)
- După 1.5 cicli ADC se face *Sample&Hold*

[0]Sursă: Datasheet ATMega324a

- Permite pornirea automată de conversii
- Surse posibile:
  - La terminarea conversiei precedente
  - Întrerupere externă `INT0`
  - Timer0 compA sau overflow
  - Timer1 compB/overflow/input capture
  - Comparatorul analogic
- Metodă rapidă de a porni conversii

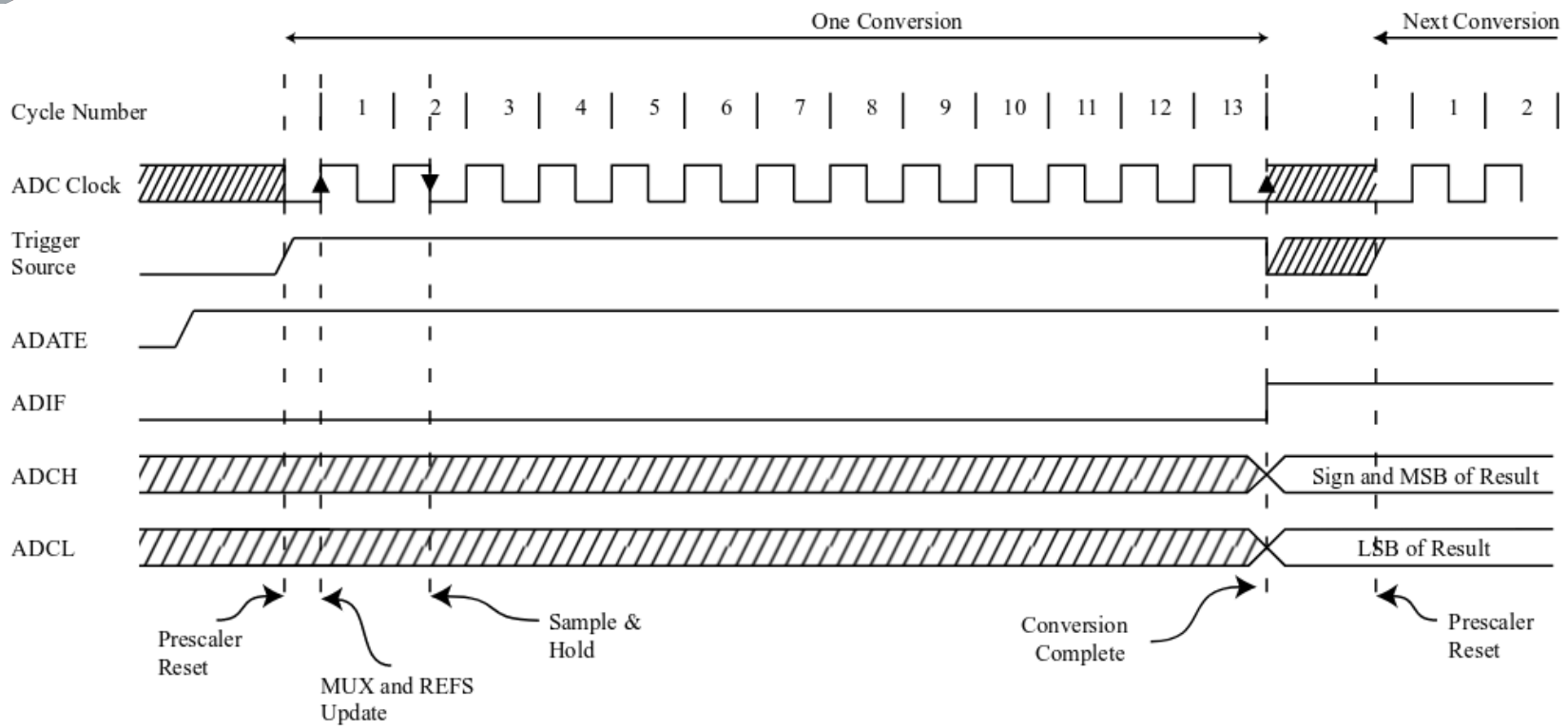
ADC - timing auto-triggering



- Durează 13.5 cicli ADC
- Mai sensibilă la schimbări de canal/referință

[0]Sursă: Datasheet ATMega324a

ADC - timing auto-triggering



- Durează 13.5 cicli ADC
- Mai sensibilă la schimbări de canal/referință
  - Schimbăm canalul în timpul unei conversii, la un ciclu ADC după pornire

[0]Sursă: Datasheet ATMega324a

ADC - exemplu auto-triggering

```
init_adc()
{

    ADMUX  = (1 << REFS0);          // initializare domeniu 0..Vcc, canalul 0
    ADCSRA = (1 << ADEN)  |         // activare ADC
             (5 << ADPS0) |         // prescaler 128
             (1 << ADATE);          // auto-triggering enabled
    ADCSRB = (5 << ADTS0);          // auto-trigger Timer 1 comp B
}
init_timer()
{
    TCCR1B = (1 << WGM12) |             // CTC, TOP la OCR1A
             (5 << CS10);               // prescaler 1024

    OCR1B = 16000000UL/1024/10 - 1; // 10Hz - 100ms
    OCR1A = OCR1B;                  // trigger-ul e pe OCR1B
}
int main()
{
  init_timer();
  init_adc();

  while(1)
  {
    // dupa primele 100ms ADC este tot timpul
    // o valoare recenta
  }
}
```

- Folosire cu mai mulți senzori
- Avem mai multe variante:
  - Pornim o conversie de fiecare dată când avem nevoie de o valoare
  - Facem conversiile pentru toți senzorii și stocăm cea mai recentă valoare

- Folosire cu mai mulți senzori
- Avem mai multe variante:
  - Pornim o conversie de fiecare dată când avem nevoie de o valoare
  - Facem conversiile pentru toți senzorii și stocăm cea mai recentă valoare
- Dacă vrem să facem toate conversiile, putem face în mai multe moduri
  - Auto-running mode: Lăsăm fiecare conversie să "cascadeze" în următoarea. Avem grijă de fiecare dată să schimbăm canalul pentru a face conversiile pentru toți senzorii
  - Timer care pornește o cascadă de conversii (o dată la X ms se pornește prima conversie, care o antrenează pe următoarea etc.)

ADC - exemplu complex

```
volatile uint8_t channel = 0;
volatile uint8_t adc_values[NRSENSORS];

ISR(TIMER1_COMPA_vect)
{
    ADCSRA |= (1 << ADSC);
}
ISR(ADC_vect)
{
    adc_values[channel] = ADC;                  // luam rezultatul precedent
    channel = (channel + 1) % NRSENSORS;        // trecem la urmatorul canal
    ADMUX = (ADMUX & 0xE0) | channel;           // schimbam canalul si punem in ADMUX
    if (channel != 0)                           // pornim urmatoarea conversie daca nu
    {                                           // este ultima
        ADCSRA |= (1 << ADSC);
    }
}
init_adc()
{
    ADMUX = (1 << REFS0);                       // referinta de tensiune AVCC
    ADCSRA = (1 << ADEN) | (1 << ADIE) | (5 << ADPS0);
}
init_timer()
{
    TCCR1B = (1 << WGM12) | (5 << CS10);        // CTC, TOP OCR1A, prescaler 1024
    TIMSK1 = (1 << OCIE1A);                     // activare intrerupere compare

    OCR1A = 16000000UL/1024/10 - 1;             // 10Hz - 100ms
}
int main()
{
    ...
}
```

ADC - exemplu complex (II)

```
volatile uint8_t channel = 0;
volatile uint8_t adc_values[NRSENSORS];

ISR(ADC_vect)
{
    adc_values[channel] = ADC;                  // luam rezultatul precedent
    channel = (channel + 1) % NRSENSORS;        // trecem la urmatorul canal
    ADMUX = (ADMUX & 0xE0) | channel;           // schimbam canalul si punem in ADMUX
    if (channel != 0)                           // pornim urmatoarea conversie daca nu
    {                                           // este ultima
        ADCSRA |= (1 << ADSC);
    }
}
init_adc()
{
    ADMUX = (1 << REFS0);                       // referinta de tensiune AVCC
    ADCSRA = (1 << ADEN) | (1 << ADATE) | (1 << ADIE) | (5 << ADPS0);
    ADCSRB = (6 << ADTS0);                      // trigger pe timer 1 overflow
}
init_timer()
{
    TCCR1A = (3 << WGM11);
    TCCR1B = (3 << WGM12) | (5 << CS10);        // Fast PWM, TOP OCR1A, prescaler 1024

    OCR1A = 16000000UL/1024/10 - 1;             // 10Hz - 100ms
}
int main()
{
    ...
}
```

- Putem converti o tensiune de pe pinii PA0..PA7
- Precizie vs. Rezoluție
- Proces lung $\rightarrow$ trebuie evitat busy-waiting
- Update-ul de configurație are perioade critice neprotejate
- Conversiile pot dura diferit în funcție de stare/configurare



Outline

ADC

EEPROM

Fuse bits

Bootloader support

- Electronically-Erasable Programmable Read-Only Memory
- Memorie non-volatilă
- Folosită sub diferite forme înainte apariției memoriei Flash
- În prezent folosită doar pentru memorie de configurare

- 1KB de memorie, adresabilă la byte
- 100000 cicli de scriere
- `EEAR` - Adresa pe 10 biți
- `EEDR` - buffer date
- `EECR` - Control Register
  - EEPM - Mode bits
  - EEMPE - Master write
  - EEPE - Write enable
  - EERE - Read enable

EEPM[1:0]	Programming Time	Operation
00	3.4ms	Erase and Write in one operation (Atomic Operation)
01	1.8ms	Erase Only
10	1.8ms	Write Only
11	-	Reserved for future use

- Operația de write durează mult

EEPM[1:0]	Programming Time	Operation
00	3.4ms	Erase and Write in one operation (Atomic Operation)
01	1.8ms	Erase Only
10	1.8ms	Write Only
11	-	Reserved for future use

- Operația de write durează mult
  - Trebuie verificat statusul `EEPE`

EEPM[1:0]	Programming Time	Operation
00	3.4ms	Erase and Write in one operation (Atomic Operation)
01	1.8ms	Erase Only
10	1.8ms	Write Only
11	-	Reserved for future use

- Operația de write durează mult
  - Trebuie verificat statusul `EEPE`
- Operația de read durează o instrucțiune

EEPM[1:0]	Programming Time	Operation
00	3.4ms	Erase and Write in one operation (Atomic Operation)
01	1.8ms	Erase Only
10	1.8ms	Write Only
11	-	Reserved for future use

- Operația de write durează mult
  - Trebuie verificat statusul `EEPE`
- Operația de read durează o instrucțiune
  - Nu trebuie verificat statusul `EERE`

EEPM[1:0]	Programming Time	Operation
00	3.4ms	Erase and Write in one operation (Atomic Operation)
01	1.8ms	Erase Only
10	1.8ms	Write Only
11	-	Reserved for future use

- Operația de write durează mult
  - Trebuie verificat statusul `EEPE`
- Operația de read durează o instrucțiune
  - Nu trebuie verificat statusul `EERE`
- Operația de read nu poate avea loc concomitent

EEPM[1:0]	Programming Time	Operation
00	3.4ms	Erase and Write in one operation (Atomic Operation)
01	1.8ms	Erase Only
10	1.8ms	Write Only
11	-	Reserved for future use

- Operația de write durează mult
  - Trebuie verificat statusul `EEPE`
- Operația de read durează o instrucțiune
  - Nu trebuie verificat statusul `EERE`
- Operația de read nu poate avea loc concomitent
  - Trebuie verificat statusul `EEPE`

EEPM[1:0]	Programming Time	Operation
00	3.4ms	Erase and Write in one operation (Atomic Operation)
01	1.8ms	Erase Only
10	1.8ms	Write Only
11	-	Reserved for future use

- Operația de write durează mult
  - Trebuie verificat statusul `EEPE`
- Operația de read durează o instrucțiune
  - Nu trebuie verificat statusul `EERE`
- Operația de read nu poate avea loc concomitent
  - Trebuie verificat statusul `EEPE`
- `EEMPE` trebuie activat cu 4 cicli înainte de `EEPE`
- `EEAR` trebuie să rămână valid

EEPM[1:0]	Programming Time	Operation
00	3.4ms	Erase and Write in one operation (Atomic Operation)
01	1.8ms	Erase Only
10	1.8ms	Write Only
11	-	Reserved for future use

- Operația de write durează mult
  - Trebuie verificat statusul `EEPE`
- Operația de read durează o instrucțiune
  - Nu trebuie verificat statusul `EERE`
- Operația de read nu poate avea loc concomitent
  - Trebuie verificat statusul `EEPE`
- `EEMPE` trebuie activat cu 4 cicli înainte de `EEPE`
- `EEAR` trebuie să rămână valid
  - Trebuie dezactivate întreruperile

- Scriere simplă

```
void EEPROM_write( unsigned int uiAddress, unsigned char ucData)
{
  /* Wait for completion of previous write */
  while (EECR & (1<<EEPE)) ;
  /* Set up address and Data Registers */
  EEAR = uiAddress;
  EEDR = ucData;
  /* Write logical one to EEMPE */
  EECR |= (1<<EEMPE);
  /* Start eeprom write by setting EEPE */
  EECR |= (1<<EEPE);
}
```

- Citire simplă

```
unsigned char EEPROM_read( unsigned int uiAddress)
{
  /* Wait for completion of previous write */
  while (EECR & (1<<EEPE)) ;
  /* Set up address register */
  EEAR = uiAddress;
  /* Start eeprom read by writing EERE */
  EECR |= (1<<EERE);
  /* Return data from Data Register */
  return EEDR;
}
```

[0]Sursă: Datasheet ATMega324a

- Singura memorie non-volatilă ușor accesibilă
- Constrângerile memoriei $\rightarrow$ folosire anevoioasă
- Sensibilitate mare la întreruperi în faza de comandă



Outline

ADC

EEPROM

Fuse bits

Bootloader support



Ce sunt fuse bits?

- Biți de configurare de bază
- Accesibili (mai ales) din exterior

- `JTAGEN` - enable programare pe JTAG
- `OCDEN` - enable debug pe JTAG
- `SPIEN` - enable programare prin SPI
- `WDTON` - watchdog timer always on
- `EESAVE` - EEPROM se păstrează chiar și la erase
- `BOOTSZ` - dimensiune bootloader
- `BOOTRST` - funcționare cu bootloader

- `CKDIV8` - împarte ceasul la 8
- `CKOUT` - ceasul vizibil pe `PB0`
- `SUT` - timp de start-up
- `CKSEL` - sursă ceas
  - Cel mai periculos câmp
  - Internal oscillator
  - External clock
  - Low Power crystal Oscillator
  - Low Frequency crystal oscillator
  - ...



Fuse byte extended

- `BODLEVEL` - Brown-out detection
- Nivele de tensiune la 1.8V, 2.7V, 4.3V

- CKDIV8
  - CLKPS este prescaler-ul ceasului [1..256]

```
CLKPR = (1 << CLKPCE); // change enable
CLKPR = 0;             // CLKPS3..0 = 0 - prescaler 1
```

- JTAGEN
  - Bit-ul JTD din MCUCR

```
uint8_t temp = MCUCR;
MCUCR = temp | (1 << JTD);
MCUCR = temp | (1 << JTD); // in maxim 4 cicli
```

- BOOTRST
  - Bit-ul IVSEL din MCUCR

```
uint8_t temp = MCUCR;
MCUCR = temp | (1 << IVCE);    // change enable
MCUCR = temp & ~(1 << IVSEL); // in maxim 4 cicli
```



Concluzii fuse bits

- Configurații importante
- Set & forget



Outline

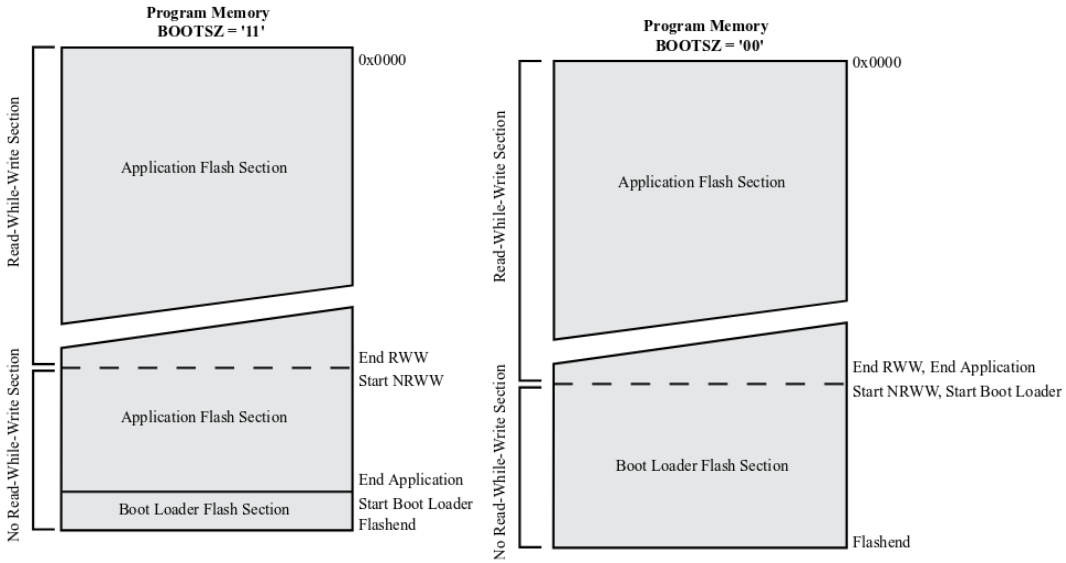
ADC

EEPROM

Fuse bits

Bootloader support

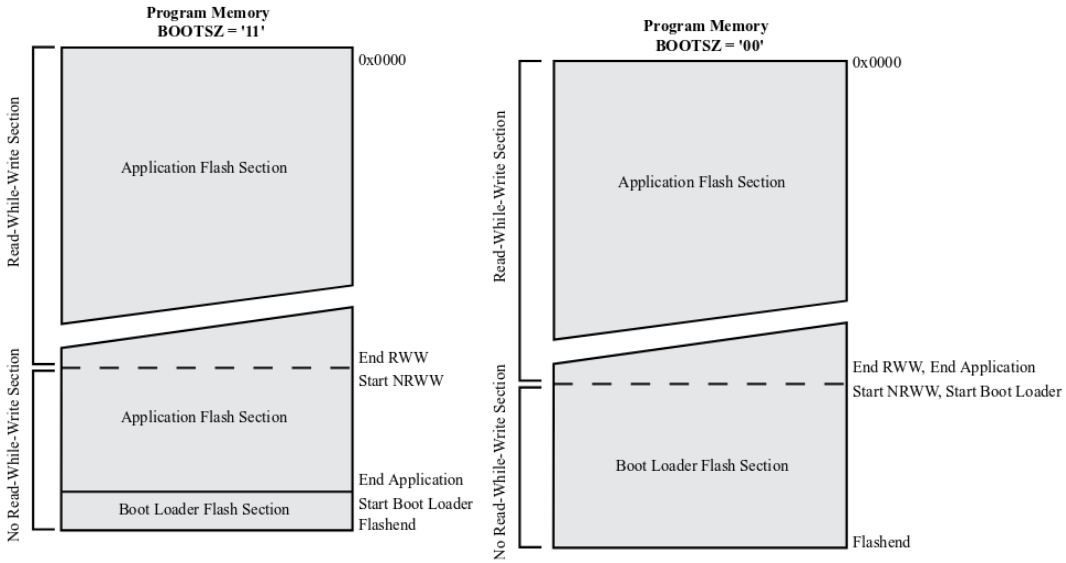
- Constrângeri specifice
  - Număr limitat de scrieri
  - Scrierile pot schimba doar biți de 1 în 0
  - Erase duce biții la 1
  - Erase se face doar pe pagini - 64 cuvinte - 128 bytes
  - Erase durează între $3.7ms$ și $4.5ms$
- Pentru bootloader
  - Memoria Flash pe care o scrie e aceeași din care execută
  - Scrierea e mult mai lentă decât ceea ce poate primi



- Este permisă citirea în zona bootloader-ului când se scrie/șterge în alte zone

[0]Sursă: Datasheet ATMega324a

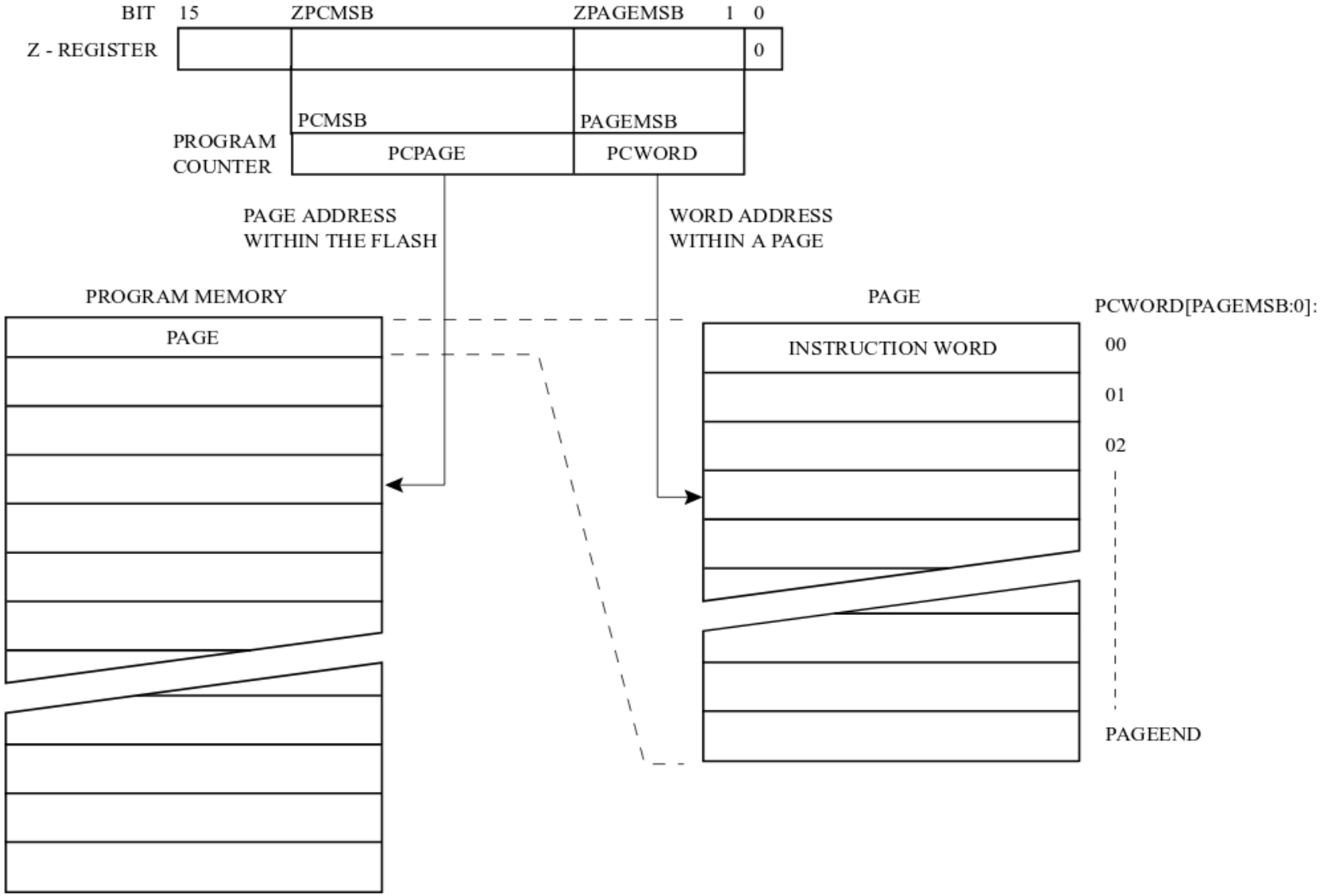
ATMega324a - Read-While-Write



- Este permisă citirea în zona bootloader-ului când se scrie/șterge în alte zone
- Uneori însă bootloader-ul nu acoperă întreaga zonă

[0]Sursă: Datasheet ATMega324a

Adresare



[0]Sursă: Datasheet ATMega324a

- Scriere RAM → page buffer
  - Câte un word (2 bytes)
  - `Buffer[PCWORD] = R1:R0`

- Scriere RAM → page buffer
  - Câte un word (2 bytes)
  - `Buffer[PCWORD] = R1:R0`
- Ștergerea unei pagini
  - Câte o pagină (256 bytes)
  - Adresa este `PCPAGE`

- Scriere RAM → page buffer
  - Câte un word (2 bytes)
  - `Buffer[PCWORD] = R1:R0`
- Ștergerea unei pagini
  - Câte o pagină (256 bytes)
  - Adresa este `PCPAGE`
- Scrierea unei pagini
  - Câte o pagină (256 bytes)
  - `Flash[PCPAGE] = Buffer`

- Scriere RAM → page buffer
  - Câte un word (2 bytes)
  - `Buffer[PCWORD] = R1:R0`
- Ștergerea unei pagini
  - Câte o pagină (256 bytes)
  - Adresa este `PCPAGE`
- Scrierea unei pagini
  - Câte o pagină (256 bytes)
  - `Flash[PCPAGE] = Buffer`
- Fiecare operație presupune

- Scriere RAM → page buffer
  - Câte un word (2 bytes)
  - `Buffer[PCWORD] = R1:R0`
- Ștergerea unei pagini
  - Câte o pagină (256 bytes)
  - Adresa este `PCPAGE`
- Scrierea unei pagini
  - Câte o pagină (256 bytes)
  - `Flash[PCPAGE] = Buffer`
- Fiecare operație presupune
  - Pregătire parametri în `Z (R31:R30)`, `R1`, `R0`



Trei tipuri de operații

- Scriere RAM → page buffer
  - Câte un word (2 bytes)
  - `Buffer[PCWORD] = R1:R0`
- Ștergerea unei pagini
  - Câte o pagină (256 bytes)
  - Adresa este `PCPAGE`
- Scrierea unei pagini
  - Câte o pagină (256 bytes)
  - `Flash[PCPAGE] = Buffer`
- Fiecare operație presupune
  - Pregătire parametri în `Z (R31:R30)`, `R1`, `R0`
  - Selectare operație în `SPMCSR`



Trei tipuri de operații

- Scriere RAM → page buffer
  - Câte un word (2 bytes)
  - `Buffer[PCWORD] = R1:R0`
- Ștergerea unei pagini
  - Câte o pagină (256 bytes)
  - Adresa este `PCPAGE`
- Scrierea unei pagini
  - Câte o pagină (256 bytes)
  - `Flash[PCPAGE] = Buffer`
- Fiecare operație presupune
  - Pregătire parametri în `Z (R31:R30)`, `R1`, `R0`
  - Selectare operație în `SPMCSR`
  - Instrucțiune `SPM`



Pseudocod bootloader

```
ISR(USART0_RX_vect)
{
  // Add to buf, buf_size
}
int main()
{
  init_usart();

  while(1)
  {
    while(buf_size < PAGESIZE * 2);

    bl_fill_pagebuffer(buf);
    bl_erasepage(current_page);
    bl_writepage(current_page++);

    // presupunem ca exista un flow control, pe seriala
    // se transmite cate o singura pagina si
    // se asteapta un ack pentru a trimite urmatoare pagina
    // si se transmit paginile in ordine, fara a depasi capacitatea
    send_ack();
  }
}
```

- `spm` nu poate fi executată decât din zona de bootloader
- Lock bits
  - Asemănători cu fuse bits
  - 2 biți pentru Application section
  - 2 biți pentru Bootloader section
  - Pot preveni citiri/scrieri de memorie de program
  - Previn suprascrieri accidentale ale bootloader-ului
  - Previn accesul la codul precedent

- Scrierea pe flash
  - Erase pe blocuri
  - Write cu pregătirea unui buffer
- Read-While-Write ne ajută
  - Putem executa cod de bootloader cât timp el șterge un bloc
  - Nu este însă valabil pe toată memoria
- Instrucțiunile de spm nu pot fi executate decât din bootloader



Proiectarea cu Microprocesoare
Power Management pe ATMega324a

Andrei Voinescu

POLITEHNICA University of Bucharest

2021



Outline

Power Management



- În tehnologia CMOS: $P \sim V_{DD}^2 \cdot f$
  - Tranzițiile costă



Consumul de energie

- În tehnologia CMOS: $P \sim V_{DD}^2 \cdot f$
  - Tranzițiile costă
- Ce tranziții avem când în cod avem `while(1) ;`?



Consumul de energie

- În tehnologia CMOS: $P \sim V_{DD}^2 \cdot f$
  - Tranzițiile costă
- Ce tranziții avem când în cod avem `while(1) ;`?
  - Încă se execută instrucțiuni, PC se modifică etc.



Consumul de energie

- În tehnologia CMOS: $P \sim V_{DD}^2 \cdot f$
  - Tranzițiile costă
- Ce tranziții avem când în cod avem `while(1) ;`?
  - Încă se execută instrucțiuni, PC se modifică etc.
  - Toate timerele active numără



Consumul de energie

- În tehnologia CMOS: $P \sim V_{DD}^2 \cdot f$
  - Tranzițiile costă
- Ce tranziții avem când în cod avem `while(1) ;`?
  - Încă se execută instrucțiuni, PC se modifică etc.
  - Toate timerele active numără
  - ADC-ul poate să fie activ



Consumul de energie

- În tehnologia CMOS: $P \sim V_{DD}^2 \cdot f$
  - Tranzițiile costă
- Ce tranziții avem când în cod avem `while(1) ;`?
  - Încă se execută instrucțiuni, PC se modifică etc.
  - Toate timerele active numără
  - ADC-ul poate să fie activ
  - Seriala poate transmite/recepționa



Consumul de energie

- În tehnologia CMOS: $P \sim V_{DD}^2 \cdot f$
  - Tranzițiile costă
- Ce tranziții avem când în cod avem `while(1) ;`?
  - Încă se execută instrucțiuni, PC se modifică etc.
  - Toate timerele active numără
  - ADC-ul poate să fie activ
  - Seriala poate transmite/recepționa
  - Circuitele de input GPIO sunt active

- În tehnologia CMOS: $P \sim V_{DD}^2 \cdot f$
  - Tranzițiile costă
- Ce tranziții avem când în cod avem `while(1) ;`?
  - Încă se execută instrucțiuni, PC se modifică etc.
  - Toate timerele active numără
  - ADC-ul poate să fie activ
  - Seriala poate transmite/recepționa
  - Circuitele de input GPIO sunt active
- Power management - procedeul prin care restricționăm ce poate face procesorul

- Mai multe stări de power management
  - Mai puține periferice
  - Mai puține ceasuri active
  - Mai puține metode de revenire
- Periferice dezactivabile manual
  - `PRR0` - Power Reduction Register
    - Un bit pt fiecare periferic

Sleep Mode	Active Clock Domains					Oscillators		Wake-up Sources						
	clk_{CPU}	clk_{FLASH}	clk_{IO}	clk_{ADC}	clk_{ASY}	Main Clock Source Enabled	Timer Oscillator Enabled	INT and PCINT	TWI Address Match	Timer2	SPM/EEPROM Ready	ADC	WDT	Other I/O
Idle			Yes	Yes	Yes	Yes	Yes(2)	Yes	Yes	Yes	Yes	Yes	Yes	Yes
ADC Noise Reduction				Yes	Yes	Yes	Yes(2)	Yes(3)	Yes	Yes(2)	Yes	Yes	Yes	
Power-down								Yes(3)	Yes				Yes	
Power-save					Yes	Yes	Yes(2)	Yes(3)	Yes	Yes			Yes	
Standby(1)						Yes		Yes(3)	Yes				Yes	
Extended Standby					Yes(2)	Yes	Yes(2)	Yes(3)	Yes	Yes			Yes	

Note:

1. Only recommended with external crystal or resonator selected as clock source.
2. If Timer/Counter2 is running in asynchronous mode.
3. For INT2, INT1 and INT0, only level interrupt.

[0]Sursă: Datasheet ATMega324a

Symbol	Parameter	Condition	Min.	Typ.(2)	Max.	Units
I_{CC}	Power Supply Current(1)	Active 1MHz, V_{CC} = 2V	-	0.3	0.55	mA
		Active 4MHz, V_{CC} = 3V	-	1.5	3.5	
		Active 8MHz, V_{CC} = 5V	-	5.2	12	
		Idle 1MHz, V_{CC} = 2V	-	0.06	0.5	
		Idle 4MHz, V_{CC} = 3V	-	0.35	1.5	
		Idle 8MHz, V_{CC} = 5V	-	1.3	5.5	
	Power-save mode(3)	32 kHz TOSC enabled, V_{CC} = 1.8V	-	0.5	-	
		32 kHz TOSC enabled, V_{CC} = 3V	-	0.6	-	
	Power-down mode(3)	WDT enabled, V_{CC} = 3V	-	4.2	15	µA
		WDT disabled, V_{CC} = 3V	-	0.15	3.0	

Note:

1. All bits set in the "PRR – Power Reduction Register ".
2. Typical values at 25°C. Maximum values are test limits in production.
3. The current consumption values include input leakage current.

[0]Sursă: Datasheet ATMega324a

- SMCR - registru control sleep
  - biți SM - modul în care să intre
  - SE - sleep enable (se poate intra in sleep)
- `sleep` - instrucțiunea care intră în sleep efectiv

- Fără ajutor

```
void main ()
{
  while(1)
  {
    do_stuff();
    // ...

    SMCR = (1 << SE); // pentru bitii SM2..0 -- 0 este IDLE
    __asm__ __volatile__ ( "sleep" "\n\t" :: );
    SMCR = 0;
  }
}
```

- Fără ajutor

```
void main ()
{
  while(1)
  {
    do_stuff();
    // ...

    SMCR = (1 << SE); // pentru bitii SM2..0 -- 0 este IDLE
    __asm__ __volatile__ ( "sleep" "\n\t" :: );
    SMCR = 0;
  }
}
```

- Cu ajutor

```
void main ()
{
  while(1)
  {
    do_stuff();
    // ...

    set_sleep_mode(SLEEP_MODE_IDLE);
    sleep_mode();
  }
}
```



Exemple aplicații - Termometru digital

- Un termometru cu afișaj digital
- O dată la 1s măsoară și schimbă afișajul
- Afișaj conectat prin SPI foarte eficient
- Cum minimizăm consumul microcontroller-ului?

- Presupunem un consum mediu de $10\mu A$ pentru senzor și display
- Avem la dispoziție o baterie de ceas CR2032 de $200mAh$ și 3.3V
- Presupunem că putem folosi toată capacitatea
- Avem un cristal de cuarț de 8MHz
- ADC configurat inițial cu prescaler maxim 128

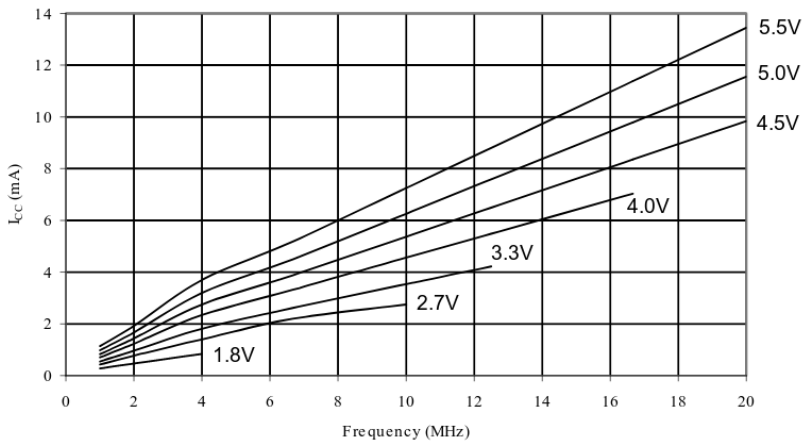
$$t_{conversie} = 13 cicli_{ADC} \cdot 128 \cdot t_{ciclu CPU}$$
$$t_{conversie} = 208\mu s \simeq 200\mu s$$

- Consumul pe SPI presupune schimbul a 100 bytes la 4MHz.

$$t_{comunicare} = \frac{100 * 8}{4000000} s$$
$$t_{comunicare} = 200\mu s$$

- Neglijăm restul operațiilor

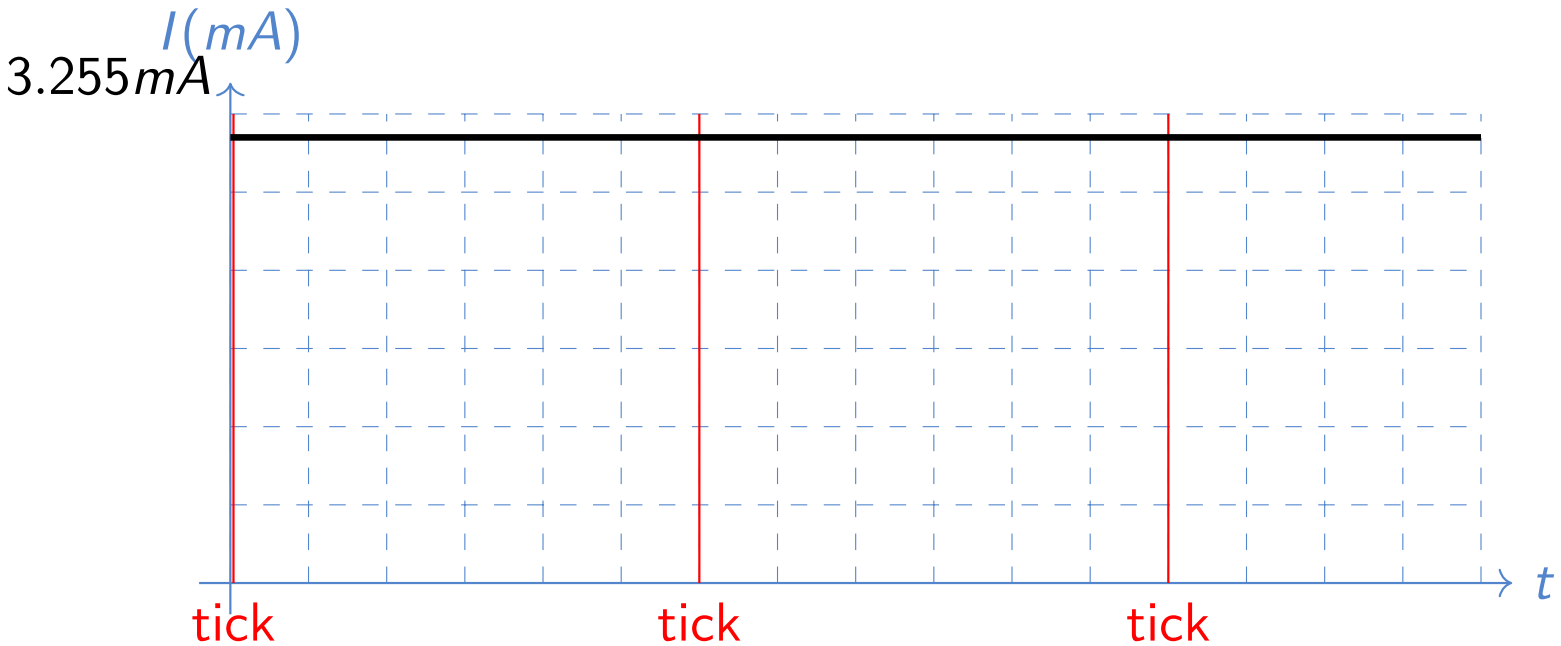
Calcul consum activ ATMega324a



PRR bit	Additional Current consumption compared to Active with external clock (See Figure 30-1 and Figure 30-2)
PRTIM2	2.5%
PRTIM1	1.6%
PRTIM0	0.7%
PRADC	4.2%
PRSPI	2.7%

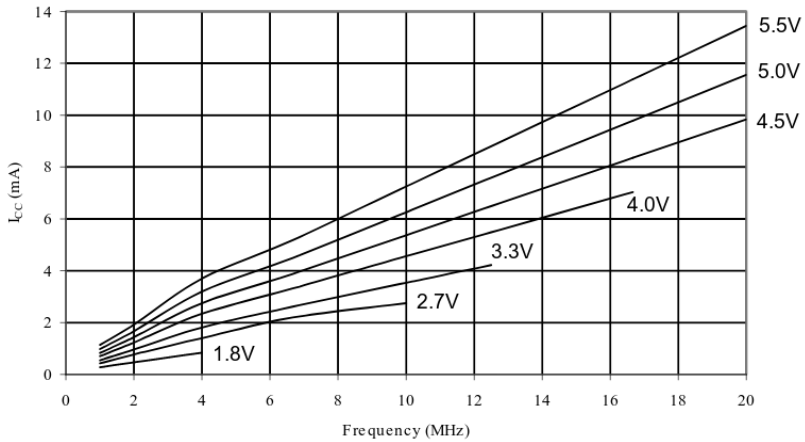
$$I_{CC_{total}} = I_{CC_{Active,8MHz,3.3V}}(1 + P_{ADC} + P_{SPI} + P_{TIM1})$$
$$I_{CC_{total}} = 3mA \cdot (1 + 0.042 + 0.027 + 0.016)$$
$$I_{CC_{total}} = 3mA \cdot (1 + 0.042 + 0.027 + 0.016) = 3.255mA$$

[0]Sursă: Datasheet ATMega324a



$$t_{baterie} = \frac{Capacitate}{Consum} = \frac{200}{3.26 + 0.01} \simeq 61.1h$$

Calcul consum activ ATMega324a



PRR bit	Additional Current consumption compared to Active with external clock (See Figure 30-1 and Figure 30-2)
PRTIM2	2.5%
PRTIM1	1.6%
PRTIM0	0.7%
PRADC	4.2%
PRSPI	2.7%

- Dacă folosim PRR0 să închidem ADC și SPI când nu avem nevoie?

$$I_{CC_{conversie}} = I_{CC_{Active,8MHz,3.3V}}(1 + P_{ADC} + P_{TIM1}) = 3.174mA$$
$$I_{CC_{comunicatie}} = I_{CC_{Active,8MHz,3.3V}}(1 + P_{SPI} + P_{TIM1}) = 3.129mA$$
$$I_{CC_{rest}} = I_{CC_{Active,8MHz,3.3V}}(1 + P_{TIM1}) = 3.048mA$$

[0]Sursă: Datasheet ATMega324a

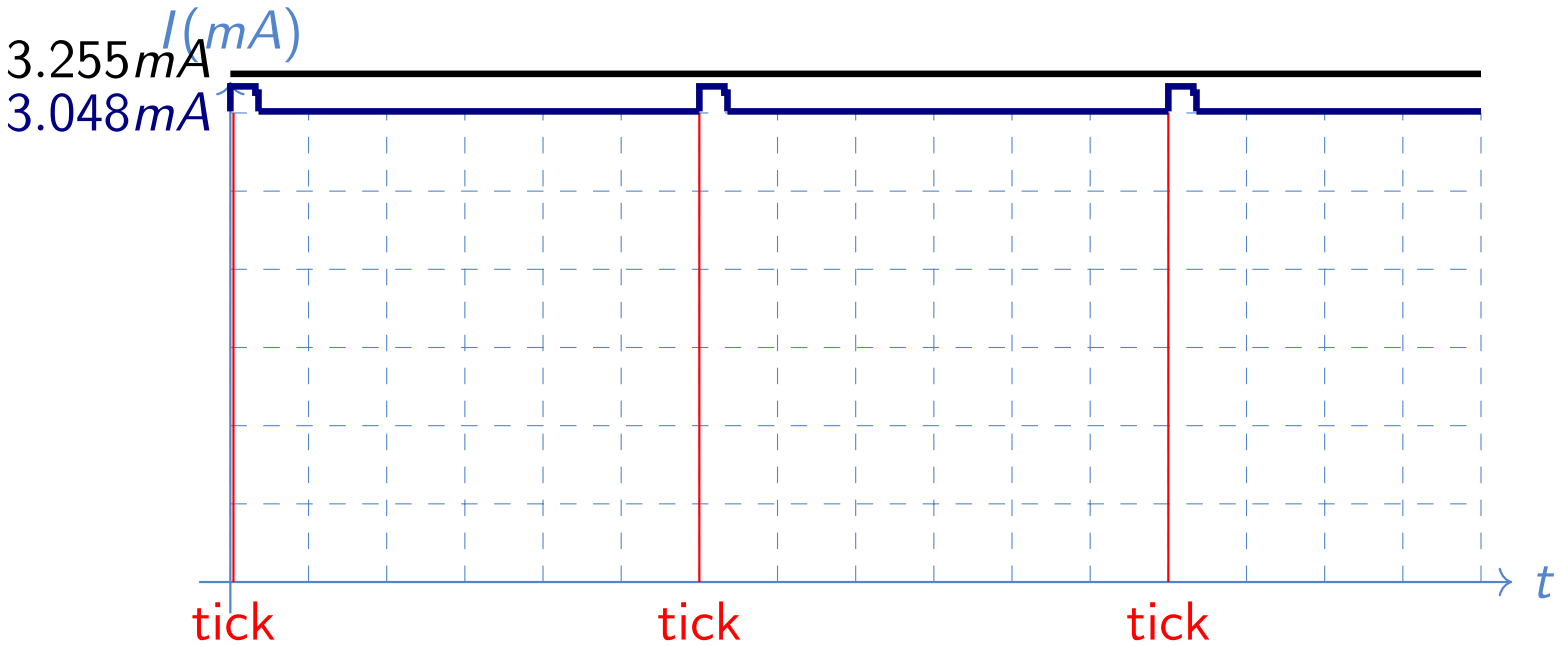
- Curentul mediu este media ponderată pe timpi

$$I_{CC_{mediu}} = \frac{\sum I_{CC_x} \cdot t_x}{\sum t_x}$$

$$I_{CC_{mediu}} = \frac{I_{CC_{conversie}} \cdot t_{coversie} + I_{CC_{comunicatie}} \cdot t_{comunicatie} + I_{CC_{rest}} \cdot t_{rest}}{t_{total}}$$

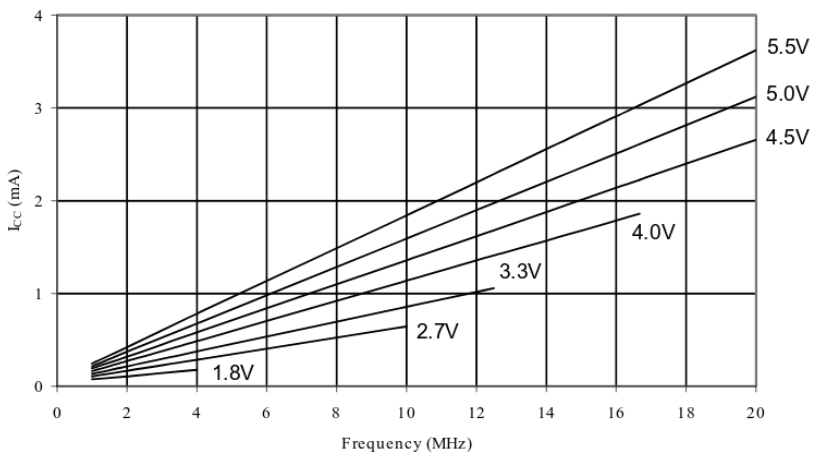
$$I_{CC_{mediu}} = \frac{3.174 \cdot 200 + 3.129 \cdot 200 + 3.048 \cdot 999600}{1000000}$$

$$I_{CC_{mediu}} = 3.048 \mu A$$



$$t_{baterie} = \frac{Capacitate}{Consum} = \frac{200}{3.05 + 0.01} \simeq 65.3h$$

- Putem folosi Idle Mode, cu wake-up pe întreruperea de Timer1
- Putem număra maxim 4s (vezi curs Timere) cu Timer1
- Alegem întrerupere de Timer1 la 1s
- Neglijăm timpii de intrare/ieșire din Idle



$$I_{CC_{Idle}} = I_{CC_{Idle,8Mhz,3.3V}} \cdot (1 + P_{TIM1,Idle}) = 0.8 \cdot 1.06 = 0.848mA$$

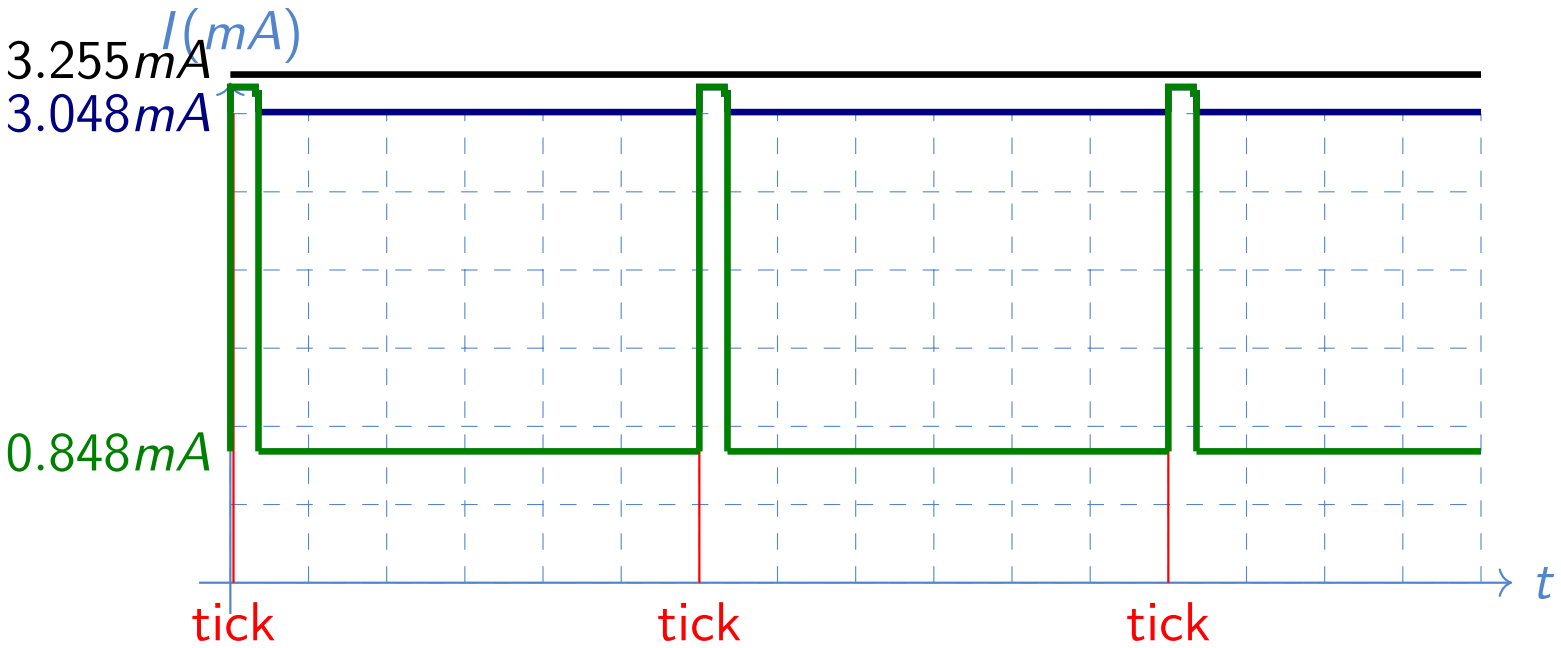
- Curentul mediu este media ponderată pe timpi

$$I_{CC_{mediu}} = \frac{\sum I_{CC_x} \cdot t_x}{\sum t_x}$$

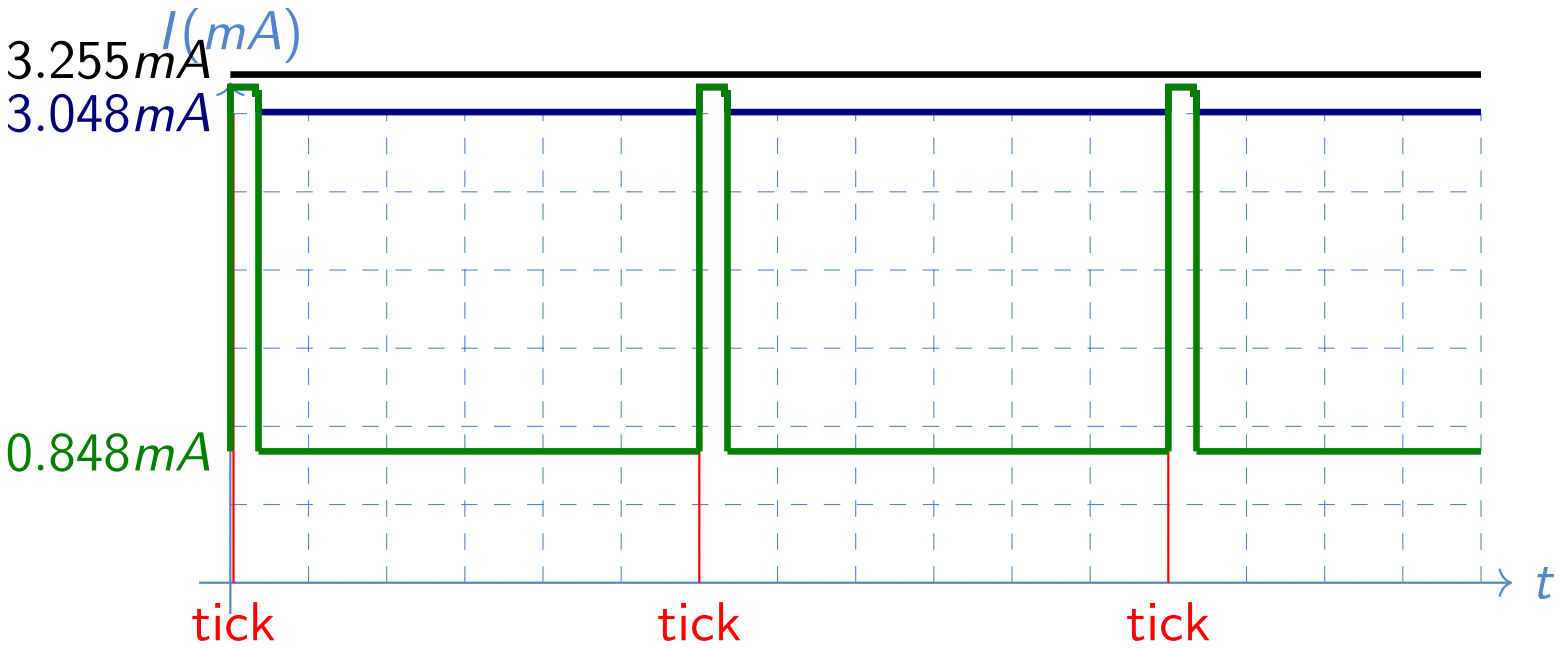
$$I_{CC_{mediu}} = \frac{I_{CC_{conversie}} \cdot t_{coversie} + I_{CC_{comunicatie}} \cdot t_{comunicatie} + I_{CC_{rest}} \cdot t_{rest}}{t_{total}}$$

$$I_{CC_{mediu}} = \frac{3.174 \cdot 200 + 3.129 \cdot 200 + 0.848 \cdot 999800}{1000000}$$

$$I_{CC_{mediu}} = 0.848mA$$



$$t_{baterie} = \frac{Capacitate}{Consum} = \frac{200}{0.85 + 0.01} \simeq 232.56h$$



$$t_{baterie} = \frac{Capacitate}{Consum} = \frac{200}{0.85 + 0.01} \simeq 232.56h$$

- Deja de 4 ori mai bine!

- Power-down - cel mai eficient mod de (ne)lucru
  - Poate fi trezit cu: întrerupere externă, i2c, watchdog
  - Consum la $25^{\circ}C$ de $0.2\mu A$ fără watchdog

- Power-down - cel mai eficient mod de (ne)lucru
  - Poate fi trezit cu: întrerupere externă, i2c, watchdog
  - Consum la $25^{\circ}C$ de $0.2\mu A$ fără watchdog
  - Noi am avea nevoie de watchdog să-l trezim $\rightarrow 4\mu A$
- Power-save - Adăugă timer-ul 2 dacă funcționează cu cristal propriu
  - Consum la $25^{\circ}C$, 3.3V de $0.75\mu A$ fără watchdog

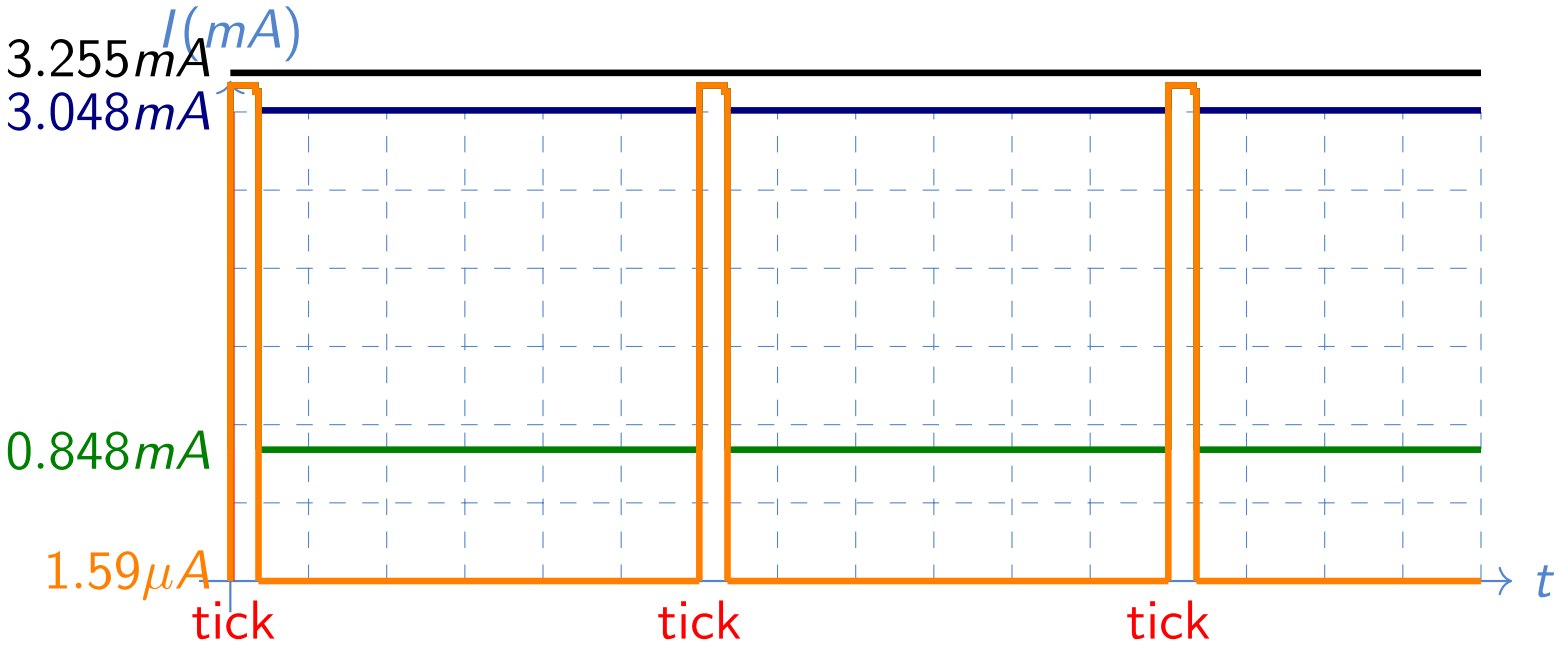
- Curentul mediu este media ponderată pe timpi

$$I_{CC_{mediu}} = \frac{\sum I_{CC_x} \cdot t_x}{\sum t_x}$$

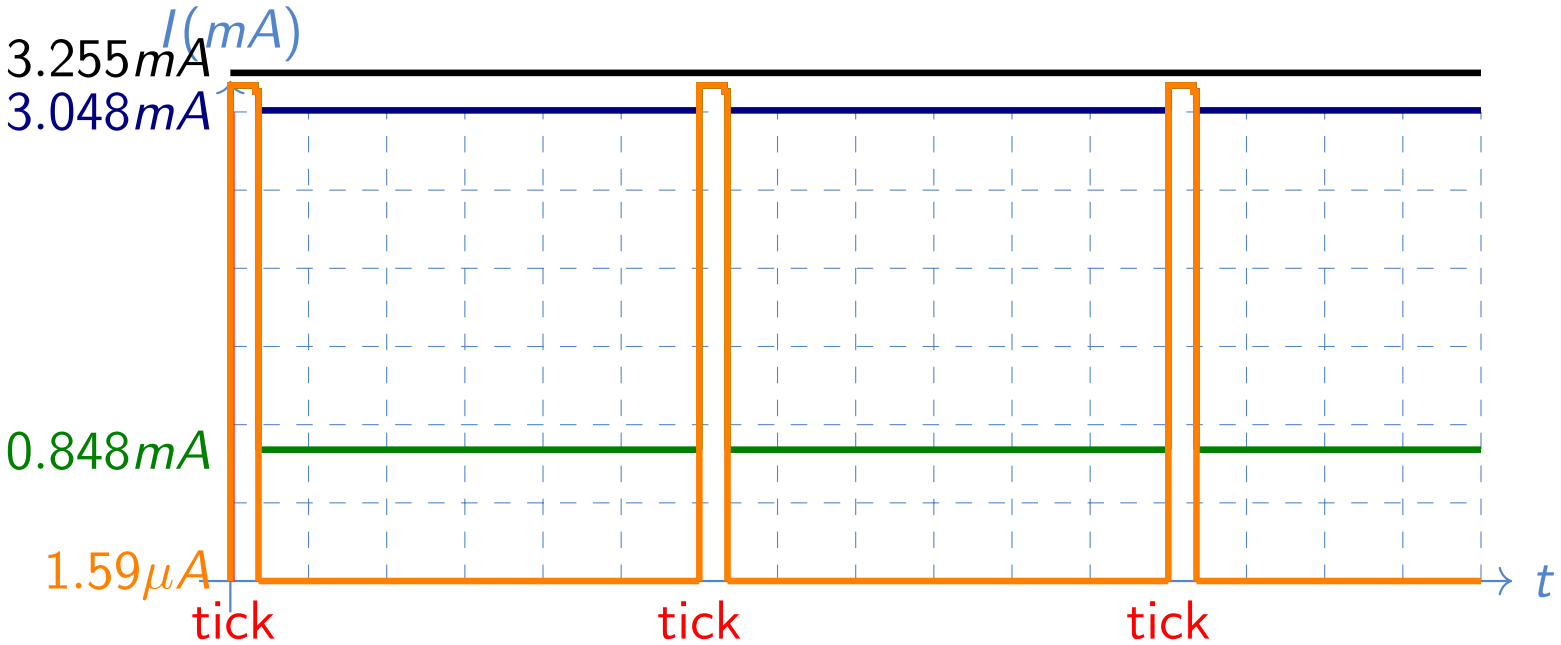
$$I_{CC_{mediu}} = \frac{I_{CC_{conversie}} \cdot t_{coversie} + I_{CC_{comunicatie}} \cdot t_{comunicatie} + I_{CC_{rest}} \cdot t_{rest}}{t_{total}}$$

$$I_{CC_{mediu}} = \frac{3.174 \cdot 200 + 3.129 \cdot 200 + 0.00075 \cdot 999600}{1000000}$$

$$I_{CC_{mediu}} = 1.59 \mu A$$

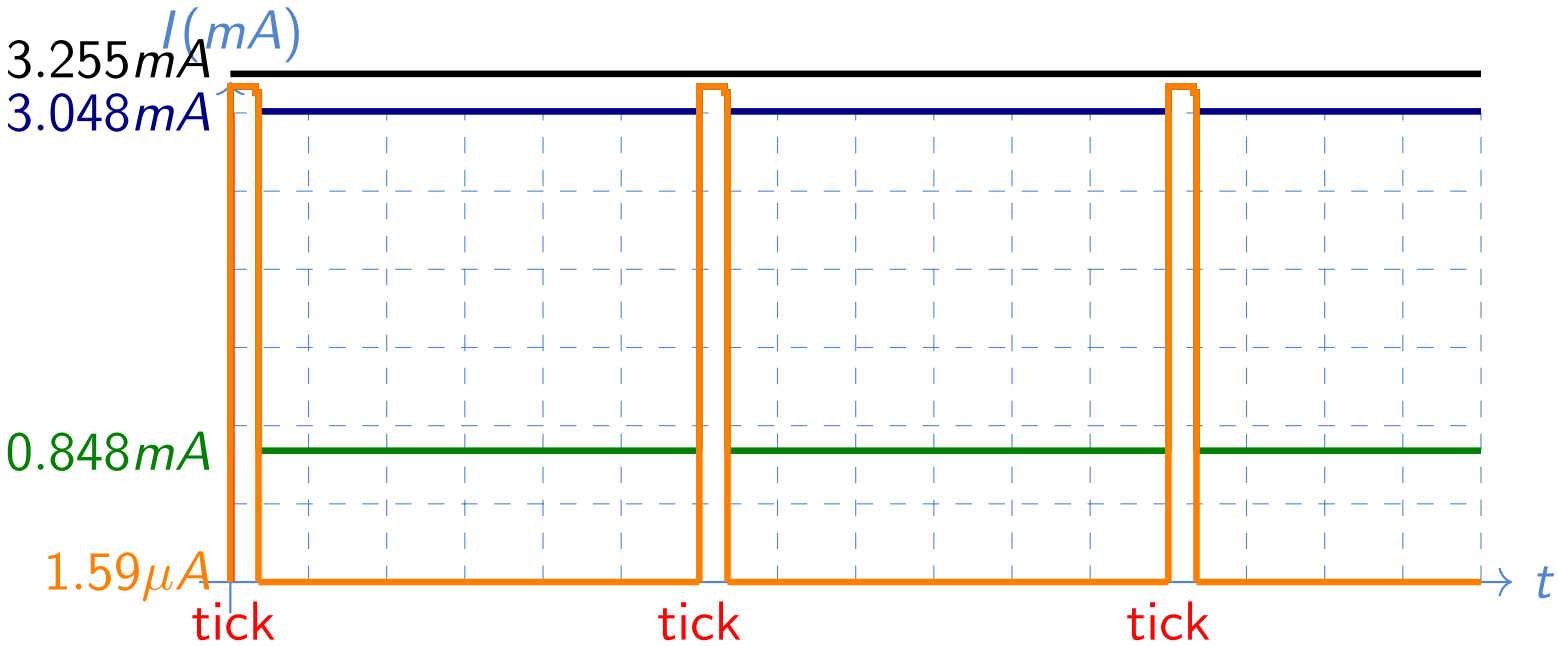


$$t_{baterie} = \frac{Capacitate}{Consum} = \frac{200}{0.00159 + 0.01} \simeq 17256h \simeq 719zile$$



$$t_{baterie} = \frac{Capacitate}{Consum} = \frac{200}{0.00159 + 0.01} \simeq 17256h \simeq 719zile$$

- De 313 ori mai bine decât estimarea inițială!



$$t_{baterie} = \frac{Capacitate}{Consum} = \frac{200}{0.00159 + 0.01} \simeq 17256h \simeq 719zile$$

- De 313 ori mai bine decât estimarea inițială!
- Consumul MCU-ului este de 2047 ori mai mic

- Senzorul oferă o scară de $10mV/1°C$, cu o precizie de $0.5°C$
- Rezoluția ADC-ului este de 10 biți, la $3.3V \rightarrow 3.22mV$
  - Nu putem reduce rezoluția ADC-ului
  - Frecvența ADC-ului trebuie să fie între 50 și 200kHz
  - Cu PS 128 la 8Mhz este la 62.5kHz $\rightarrow$ putem reduce prescaler-ul de 2 ori
  - Pentru rezoluție mai mică, puteam reduce la PS 16 sau 32

- Am redus prescaler-ul de la 128 la 64

$$t_{conversie} = 13 cicli_{ADC} \cdot 64 \cdot t_{ciclu CPU}$$
$$t_{conversie} = 104\mu s \simeq 100\mu s$$

- Curentul mediu este media ponderată pe timpi

$$I_{CC_{mediu}} = \frac{\sum I_{CC_x} \cdot t_x}{\sum t_x}$$
$$I_{CC_{mediu}} = \frac{I_{CC_{conversie}} \cdot t_{coversie} + I_{CC_{comunicatie}} \cdot t_{comunicatie} + I_{CC_{rest}} \cdot t_{rest}}{t_{total}}$$
$$I_{CC_{mediu}} = \frac{3.174 \cdot 100 + 3.129 \cdot 200 + 0.00075 \cdot 999700}{10000000}$$
$$I_{CC_{mediu}} = 1.27\mu A$$

- Dar dacă am face totul mai încet?



Optimizări frecvență

- Dar dacă am face totul mai încet?
- 1MHz MCU, SPI 500kHz (minim /2 prescaler)

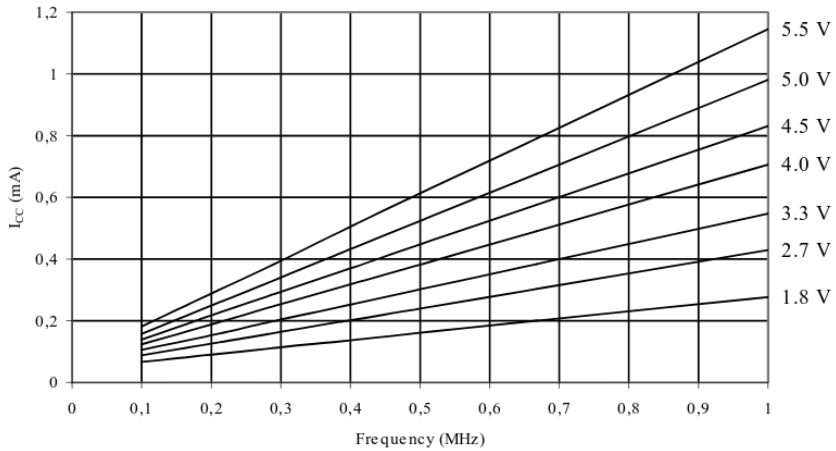
- Dar dacă am face totul mai încet?
- 1MHz MCU, SPI 500kHz (minim /2 prescaler)
- ADC prescaler 8 $\rightarrow$ 125000

$$t_{conversie} = 13 cicli_{ADC} \cdot 8 \cdot t_{ciclu CPU}$$
$$t_{conversie} = 104\mu s \simeq 100\mu s$$

- SPI durează mai mult

$$t_{comunicare} = \frac{100 * 8}{500000} s = 1.6ms$$

Calcul consum activ 1MHz



PRR bit	Additional Current consumption compared to Active with external clock (See Figure 30-1 and Figure 30-2)
PRTIM2	2.5%
PRTIM1	1.6%
PRTIM0	0.7%
PRADC	4.2%
PRSPI	2.7%

$$I_{CC_{total}} = I_{CC_{Active,1MHz,3.3V}}(1 + P_{ADC} + P_{SPI} + P_{TIM1})$$
$$I_{CC_{total}} = 0.55mA \cdot (1 + 0.042 + 0.027 + 0.016) = 0.597mA$$
$$I_{CC_{conversie}} = I_{CC_{Active,1MHz,3.3V}}(1 + P_{ADC} + P_{TIM1}) = 0.588mA$$
$$I_{CC_{comunicatie}} = I_{CC_{Active,1MHz,3.3V}}(1 + P_{SPI} + P_{TIM1}) = 0.573mA$$

[0]Sursă: Datasheet ATMega324a

- Curentul mediu este media ponderată pe timpi

$$I_{CC_{mediu}} = \frac{\sum I_{CC_x} \cdot t_x}{\sum t_x}$$

$$I_{CC_{mediu}} = \frac{I_{CC_{conversie}} \cdot t_{coversie} + I_{CC_{comunicatie}} \cdot t_{comunicatie} + I_{CC_{rest}} \cdot t_{rest}}{t_{total}}$$

$$I_{CC_{mediu}} = \frac{0.588 \cdot 100 + 0.573 \cdot 1600 + 0.00075 \cdot 998300}{1000000}$$

$$I_{CC_{mediu}} = 0.98\mu A$$

- Consumul mediu al MCU-ului de 3300 ori mai mic



Ce mai puteam face?

- Estimarea întregului cod (am ignorat mult)



Ce mai puteam face?

- Estimarea întregului cod (am ignorat mult)
- Scăderea tensiunii de alimentare (altă baterie?)



Ce mai puteam face?

- Estimarea întregului cod (am ignorat mult)
- Scăderea tensiunii de alimentare (altă baterie?)
- Modelarea consumului pentru nivele de baterie diferite



Ce mai puteam face?

- Estimarea întregului cod (am ignorat mult)
- Scăderea tensiunii de alimentare (altă baterie?)
- Modelarea consumului pentru nivele de baterie diferite
- Schimbarea cerințelor pentru optimizări în plus



Ce mai puteam face?

- Estimarea întregului cod (am ignorat mult)
- Scăderea tensiunii de alimentare (altă baterie?)
- Modelarea consumului pentru nivele de baterie diferite
- Schimbarea cerințelor pentru optimizări în plus
  - Trebuie neapărat la 1s pentru temperatură?



Ce mai puteam face?

- Estimarea întregului cod (am ignorat mult)
- Scăderea tensiunii de alimentare (altă baterie?)
- Modelarea consumului pentru nivele de baterie diferite
- Schimbarea cerințelor pentru optimizări în plus
  - Trebuie neapărat la 1s pentru temperatură?
  - Trebuie neapărat făcut update la ecran dacă nu s-a schimbat nimic?



Ce mai puteam face?

- Estimarea întregului cod (am ignorat mult)
- Scăderea tensiunii de alimentare (altă baterie?)
- Modelarea consumului pentru nivele de baterie diferite
- Schimbarea cerințelor pentru optimizări în plus
  - Trebuie neapărat la 1s pentru temperatură?
  - Trebuie neapărat făcut update la ecran dacă nu s-a schimbat nimic?
- Optimizări pe părțile ignorate (senzor, LCD)



Concluzii

- Cele mai mari economii - Idle sau Power-save



Concluzii

- Cele mai mari economii - Idle sau Power-save
- Premisa - avem deja închise toate perifericele nefolosite



Concluzii

- Cele mai mari economii - Idle sau Power-save
- Premisa - avem deja închise toate perifericele nefolosite
- Premisa2 - Avem un timp de repaus mare între procesări



Concluzii

- Cele mai mari economii - Idle sau Power-save
- Premisa - avem deja închise toate perifericele nefolosite
- Premisa2 - Avem un timp de repaus mare între procesări
- ATMega324a în general e mai eficient per MIPS la frecvențe mai mari *pentru aceeași tensiune*



Concluzii

- Cele mai mari economii - Idle sau Power-save
- Premisa - avem deja închise toate perifericele nefolosite
- Premisa2 - Avem un timp de repaus mare între procesări
- ATMega324a în general e mai eficient per MIPS la frecvențe mai mari *pentru aceeași tensiune*
  - Depinde și de ce alte operații mai facem



Concluzii

- Cele mai mari economii - Idle sau Power-save
- Premisa - avem deja închise toate perifericele nefolosite
- Premisa2 - Avem un timp de repaus mare între procesări
- ATMega324a în general e mai eficient per MIPS la frecvențe mai mari *pentru aceeași tensiune*
  - Depinde și de ce alte operații mai facem
  - Degeaba reducem frecvența dacă nu reducem și tensiunea



Proiectarea cu Microprocesoare
Curs 9

Andrei Voinescu

POLITEHNICA University of Bucharest

2021



Outline

Introducere

Memorii pe interfață paralelă



Context

- Până acum am studiat un microcontroller și posibilitățile de aplicații în jurul lui

- Până acum am studiat un microcontroller și posibilitățile de aplicații în jurul lui
- Nu toate problemele se rezolvă cu un sistem de calcul bazat pe microcontrollere, cu atât mai puțin pe unul de 8 biți



- Până acum am studiat un microcontroller și posibilitățile de aplicații în jurul lui
- Nu toate problemele se rezolvă cu un sistem de calcul bazat pe microcontrollere, cu atât mai puțin pe unul de 8 biți
- Microcontrollerul are memorie nonvolatilă, memorie RAM periferice într-un singur pachet pentru a reduce costurile unui sistem de calcul bazat pe el



- Până acum am studiat un microcontroller și posibilitățile de aplicații în jurul lui
- Nu toate problemele se rezolvă cu un sistem de calcul bazat pe microcontrollere, cu atât mai puțin pe unul de 8 biți
- Microcontrollerul are memorie nonvolatilă, memorie RAM periferice într-un singur pachet pentru a reduce costurile unui sistem de calcul bazat pe el
- La capătul opus al spectrului, un microprocesor conține doar capabilități de calcul și control, cu foarte puțină memorie



Context

- Până acum am studiat un microcontroller și posibilitățile de aplicații în jurul lui
- Nu toate problemele se rezolvă cu un sistem de calcul bazat pe microcontrollere, cu atât mai puțin pe unul de 8 biți
- Microcontrollerul are memorie nonvolatilă, memorie RAM periferice într-un singur pachet pentru a reduce costurile unui sistem de calcul bazat pe el
- La capătul opus al spectrului, un microprocesor conține doar capabilități de calcul și control, cu foarte puțină memorie
- Studiul unei plăci de bază contemporane este dificil datorită complexității și a informației non-publice



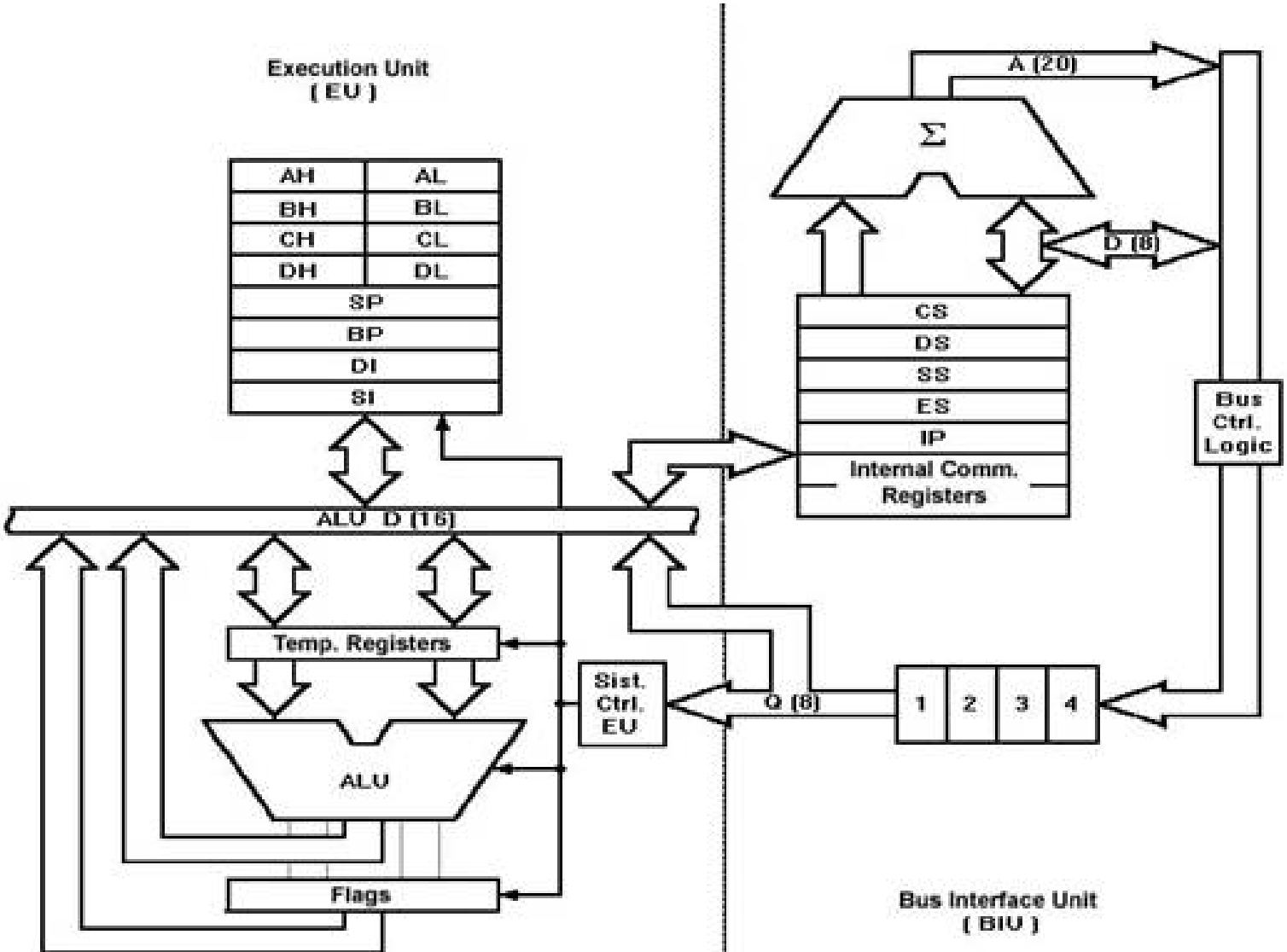
Context

- Până acum am studiat un microcontroller și posibilitățile de aplicații în jurul lui
- Nu toate problemele se rezolvă cu un sistem de calcul bazat pe microcontrollere, cu atât mai puțin pe unul de 8 biți
- Microcontrollerul are memorie nonvolatilă, memorie RAM periferice într-un singur pachet pentru a reduce costurile unui sistem de calcul bazat pe el
- La capătul opus al spectrului, un microprocesor conține doar capabilități de calcul și control, cu foarte puțină memorie
- Studiul unei plăci de bază contemporane este dificil datorită complexității și a informației non-publice
- Vom studia conceptele care au stat la baza plăcii cu 8086

- Dezvoltat în anul 1978
- Procesor de 16 biți cu 20 biți de adresă
- Magistrala de adrese multiplexată cu cea de date

- Dezvoltat în anul 1978
- Procesor de 16 biți cu 20 biți de adresă
- Magistrala de adrese multiplexată cu cea de date
- Pachet 40 pini - alimentare 5V - la fel ca ATMega324a
- 14 registre de 16 biți
- Cache de instrucțiuni cu 6 bytes
- Două moduri de interfațare externă (ne uităm doar la unul)




Execution Unit
(EU)
AH
AL
BH
BL
CH
CL
DH
DL
SP
BP
DI
SI
ALU D (16)
Temp. Registers
ALU
Flags
Sist. Ctrl. EU
Q (8)
A (20)
Σ
D (8)
CS
DS
SS
ES
IP
Internal Comm. Registers
Bus Ctrl. Logic
1
2
3
4
Bus Interface Unit
(BIU)

- Arhitectură CISC $\rightarrow$ registrele au funcții speciale

- Arhitectură CISC $\rightarrow$ registrele au funcții speciale
- Registre acumulatori - destinații + surse automate în unele instrucțiuni

- Arhitectură CISC $\rightarrow$ registrele au funcții speciale
- Registre acumulatori - destinații + surse automate în unele instrucțiuni
- Registre index - folosite ca index automat în anumite instrucțiuni (ex. instrucțiuni stringuri)

- Arhitectură CISC $\rightarrow$ registrele au funcții speciale
- Registre acumulatori - destinații + surse automate în unele instrucțiuni
- Registre index - folosite ca index automat în anumite instrucțiuni (ex. instrucțiuni stringuri)
- Registre segment - identifică **segmente**

- Arhitectură CISC $\rightarrow$ registrele au funcții speciale
- Registre acumulatori - destinații + surse automate în unele instrucțiuni
- Registre index - folosite ca index automat în anumite instrucțiuni (ex. instrucțiuni stringuri)
- Registre segment - identifică **segmente**
  - **segmentele** sunt bucăți de 64kB referențiate de anumite instrucțiuni



Registre (II)

- `AX (AH|AL)` - Acumulator principal
- `BX (BH|BL)` - Acumulator și bază
- `CX (CH|CL)` - Acumulator/Numărător
- `DX (DH|DL)` - Acumulator/Numărător/Date IO



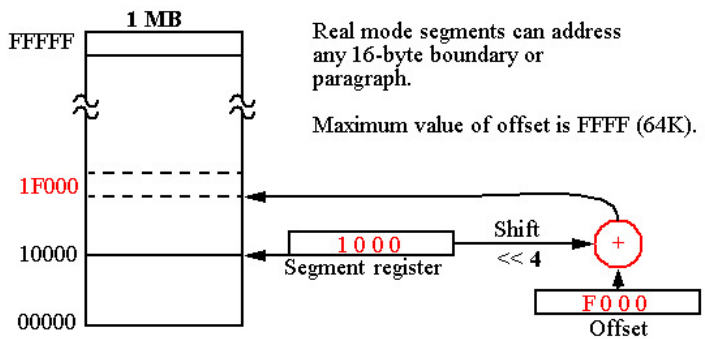
Registre (II)

- `AX (AH|AL)` - Acumulator principal
- `BX (BH|BL)` - Acumulator și bază
- `CX (CH|CL)` - Acumulator/Numărător
- `DX (DH|DL)` - Acumulator/Numărător/Date IO
- `SP` - Stack pointer
- `BP` - Base pointer
- `SI` - Source index
- `DI` - Destination index

- CS - Code Segment
- DS - Data Segment
- SS - Stack Segment
- ES - Extended Segment

- `CS` - Code Segment
- `DS` - Data Segment
- `SS` - Stack Segment
- `ES` - Extended Segment
- `IP` - Program Counter/ Instruction Pointer
- `FLAGS` - Registrul cu flags

- Segmentele sunt:
  - CS:IP
  - SS:SP
  - DS:DI
  - ES:SI



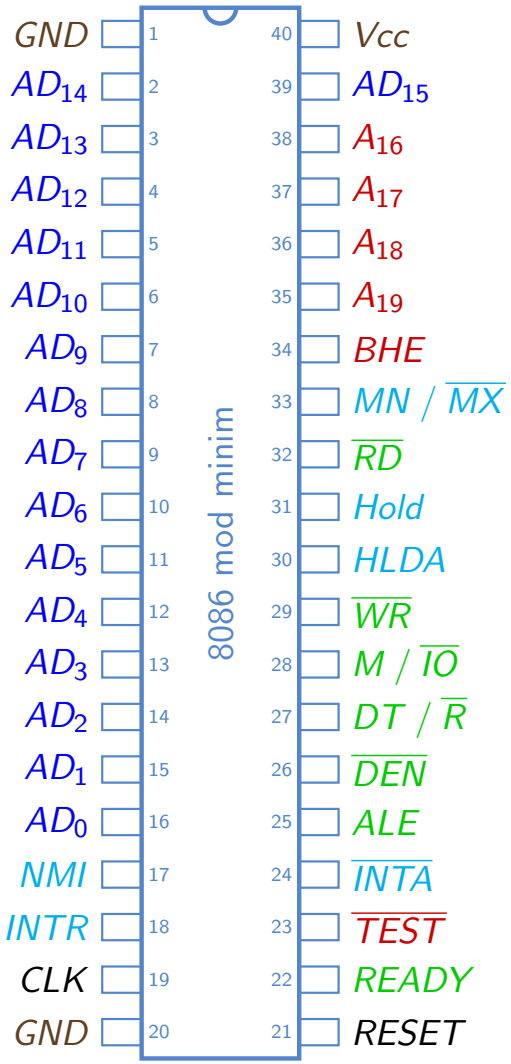
- Adresare directă - `MOV AX,BX`
- Adresare indirectă de registru - `MOV AX,[BX]`
- Bazată sau indexată - se folosește (SS, SI, DI)
- Bazată și indexată - și index și bază
- Toate Variantele de adresare pot avea și offset imediat (mai puțin cea directă)



Flags

- Carry - carry/borrow la cel mai semnificativ bit
- Parity - 1 dacă suma biților este pară
- Zero - 1 dacă rezultatul este 0
- Sign - bitul cel mai semnificativ al rezultatului
- Direction - 1 dacă se face auto-decrement
- Interrupt Enable - 1 dacă întreruperile mascabile sunt activate
- Trap/Single step - dacă e 1 întreruperea de single-step va apărea după următoarea instrucțiune
- Overflow
- Auxiliary - carry/borrow pentru biții 0-3

Pinout


8086 mod minim
GND 1
AD14 2
AD13 3
AD12 4
AD11 5
AD10 6
AD9 7
AD8 8
AD7 9
AD6 10
AD5 11
AD4 12
AD3 13
AD2 14
AD1 15
AD0 16
NMI 17
INTR 18
CLK 19
GND 20
40 Vcc
39 AD15
38 A16
37 A17
36 A18
35 A19
34 BHE
33 MN / MX
32 RD
31 Hold
30 HLDA
29 WR
28 M / IO
27 DT / R
26 DEN
25 ALE
24 INTA
23 TEST
22 READY
21 RESET

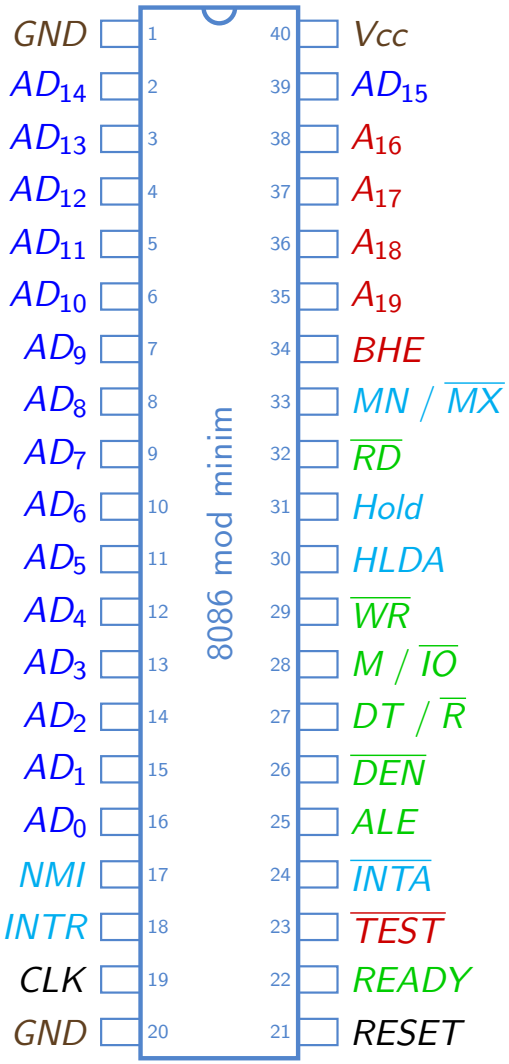
- Procesorul are două moduri, pinii pot avea funcționalități diferite între cele două



- Procesorul are două moduri, pinii pot avea funcționalități diferite între cele două
- Modul este selectabil hardware, printr-un pin

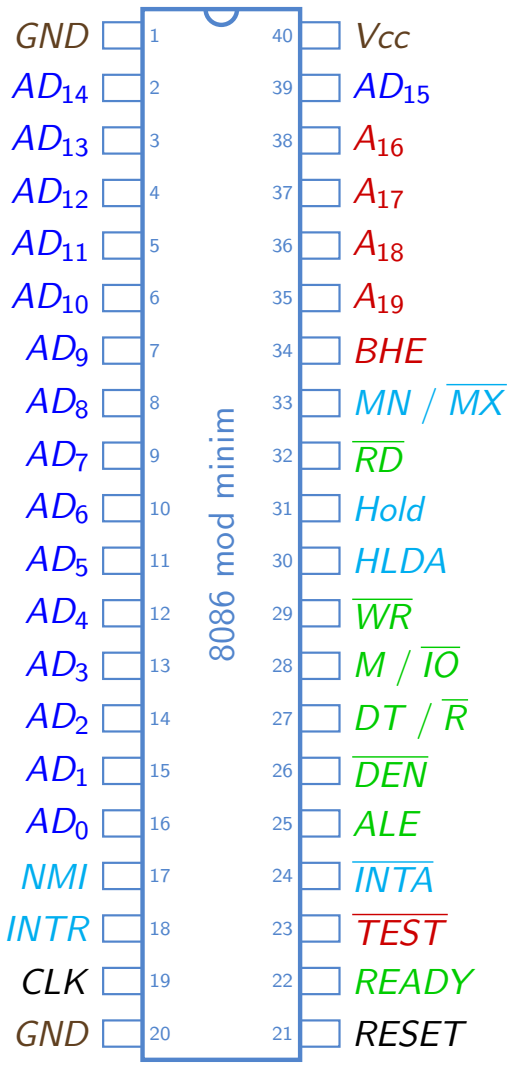
- Procesorul are două moduri, pinii pot avea funcționalități diferite între cele două
- Modul este selectabil hardware, printr-un pin
- Diferența constă în felul în care se accesează magistrală (ce fel de semnale generează procesorul)

Pinout (I)



- *(I/O/Z)* `AD15..0` Address/Data Bus
- *(O/Z)* `A19/S6..A16/S3` Address
- *(O)* $\overline{BHE}/S7$ Byte High Enable
- *(I)* $MN/\overline{MX}$ Minimum/Maximum Mode
- *(O/Z)* $M/\overline{IO}$ acces memorie sau I/O
- *(O)* $\overline{RD}$ Semnal read
- *(O)* $\overline{WR}$ Semnal write
- *(O)* $\overline{ALE}$ Address latch
- *(I)* $\overline{TEST}$ instrucțiunea wait $\leftarrow$ 0 pe pin
- *(I)* `READY` semnal de la alt dispozitiv

Pinout (I)



- *(I)* `INTR` apariția unei întreruperi
  - Verificată la sfârșitul ultimului ciclu al unei instrucțiuni
  - Ajungem la tabela de vectori
- *(I)* `NMI` întrerupere non-mascabilă, edge-triggered
- *(I)* `RESET` 4 cicli $\rightarrow$ execuție 0xFFFF0
- *(I)* `CLK` 33% duty cycle (2-8 MHz)
- *(O)* $\overline{INTA}$ interrupt ack
- *(O/Z)* $DT/\overline{R}$ Data Transmit/Receive
- *(O/Z)* $\overline{DEN}$ Data enable
- *(I)* `HOLD` cineva cere hold pe magistrală
- *(O)* `HOLDA` ack hold

- Microprocesorul extrage cod prin cicli de magistrală
  - Grupuri de perioade de ceas fără legătură cu instrucțiunile

- Microprocesorul extrage cod prin cicli de magistrală
  - Grupuri de perioade de ceas fără legătură cu instrucțiunile
- **T1** - MPU pune adresele + ALE, pe $ALE \downarrow$ se garantează adresele

- Microprocesorul extrage cod prin cicli de magistrală
  - Grupuri de perioade de ceas fără legătură cu instrucțiunile
- **T1** - MPU pune adresele + ALE, pe $ALE \downarrow$ se garantează adresele
- **T2** - MPU pregătește transferul de date ($S7..4, \overline{RD}, \overline{WR}$)

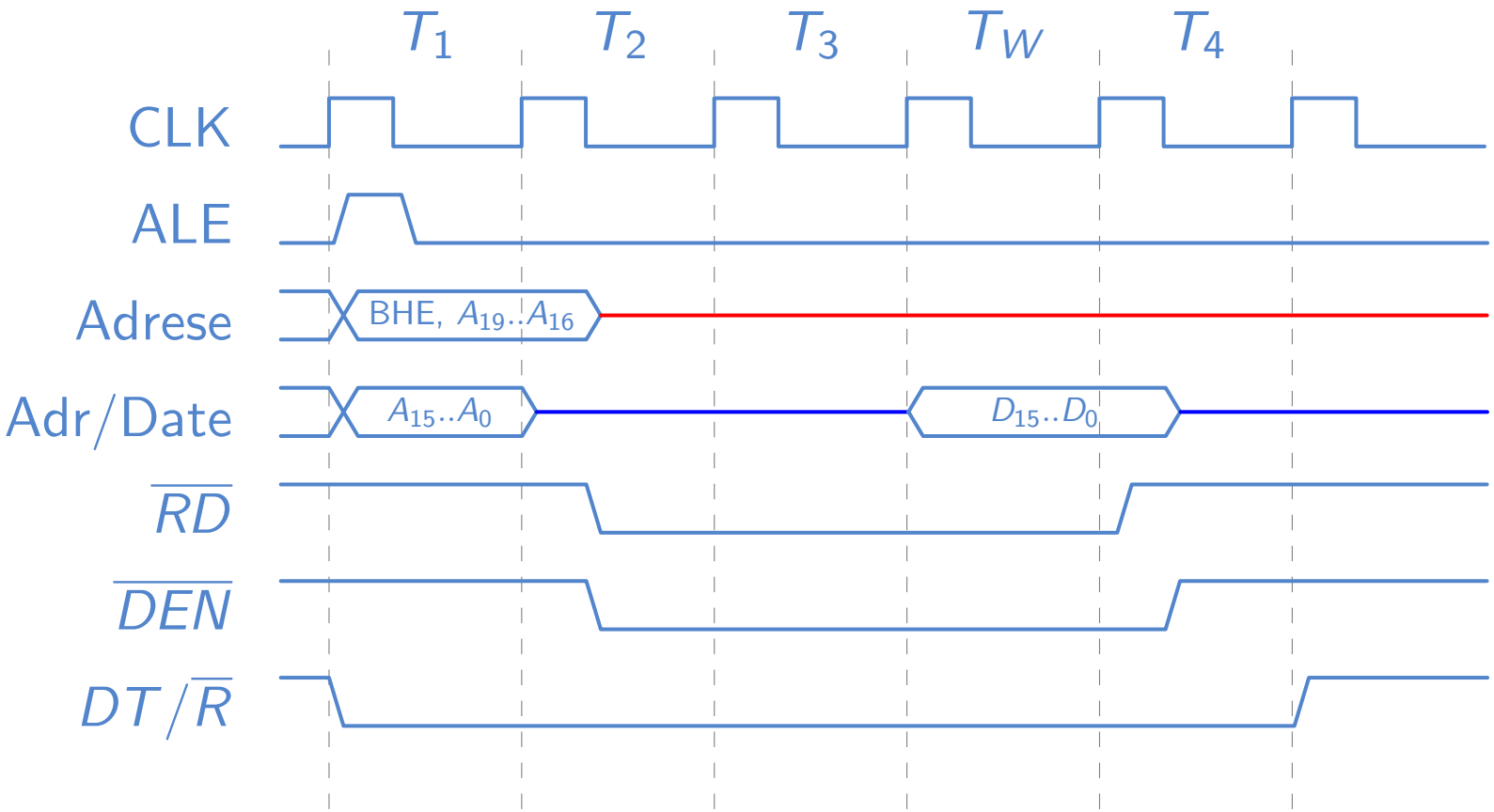
- Microprocesorul extrage cod prin cicli de magistrală
  - Grupuri de perioade de ceas fără legătură cu instrucțiunile
- **T1** - MPU pune adresele + ALE, pe $ALE \downarrow$ se garantează adresele
- **T2** - MPU pregătește transferul de date ($S7..4, \overline{RD}, \overline{WR}$)
- **T3** - MPU scrie sau citește, citirea poate fi extinsă prin READY=0 (**TW**)

- Microprocesorul extrage cod prin cicli de magistrală
  - Grupuri de perioade de ceas fără legătură cu instrucțiunile
- **T1** - MPU pune adresele + ALE, pe $ALE \downarrow$ se garantează adresele
- **T2** - MPU pregătește transferul de date ($S7..4, \overline{RD}, \overline{WR}$)
- **T3** - MPU scrie sau citește, citirea poate fi extinsă prin READY=0 (**TW**)
- **T4** - dezactivarea comenzilor și eliberarea magistralei locale

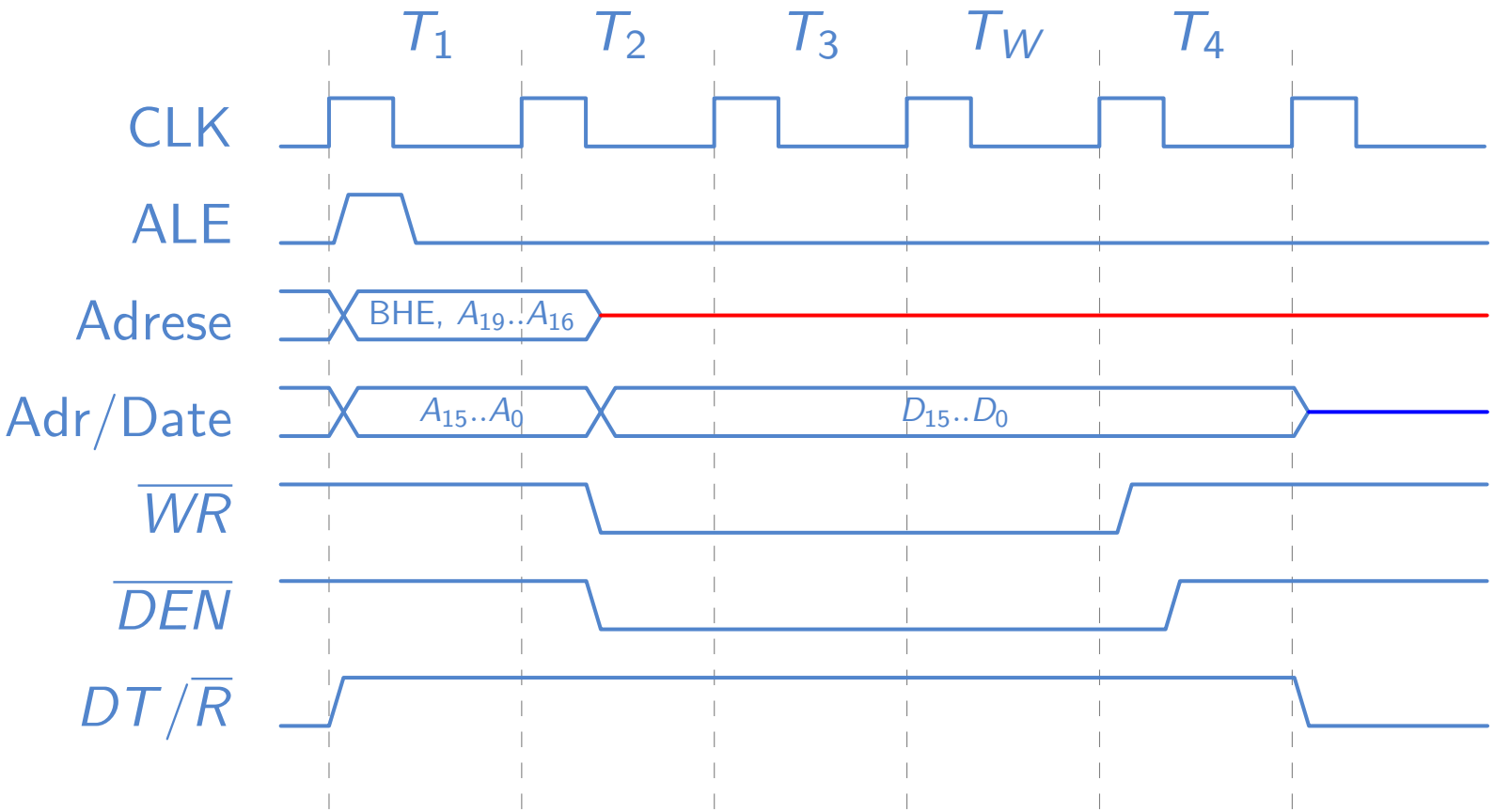
- MPU accesează magistrală cu ajutorul unor semnale

$M/\overline{IO}$	$\overline{RD}$	$\overline{WR}$	Transfer
0	0	1	Citire I/O
0	1	0	Scriere I/O
1	0	1	Citire memorie
1	1	0	Scriere memorie

Accesul la magistrala externă - Citirea

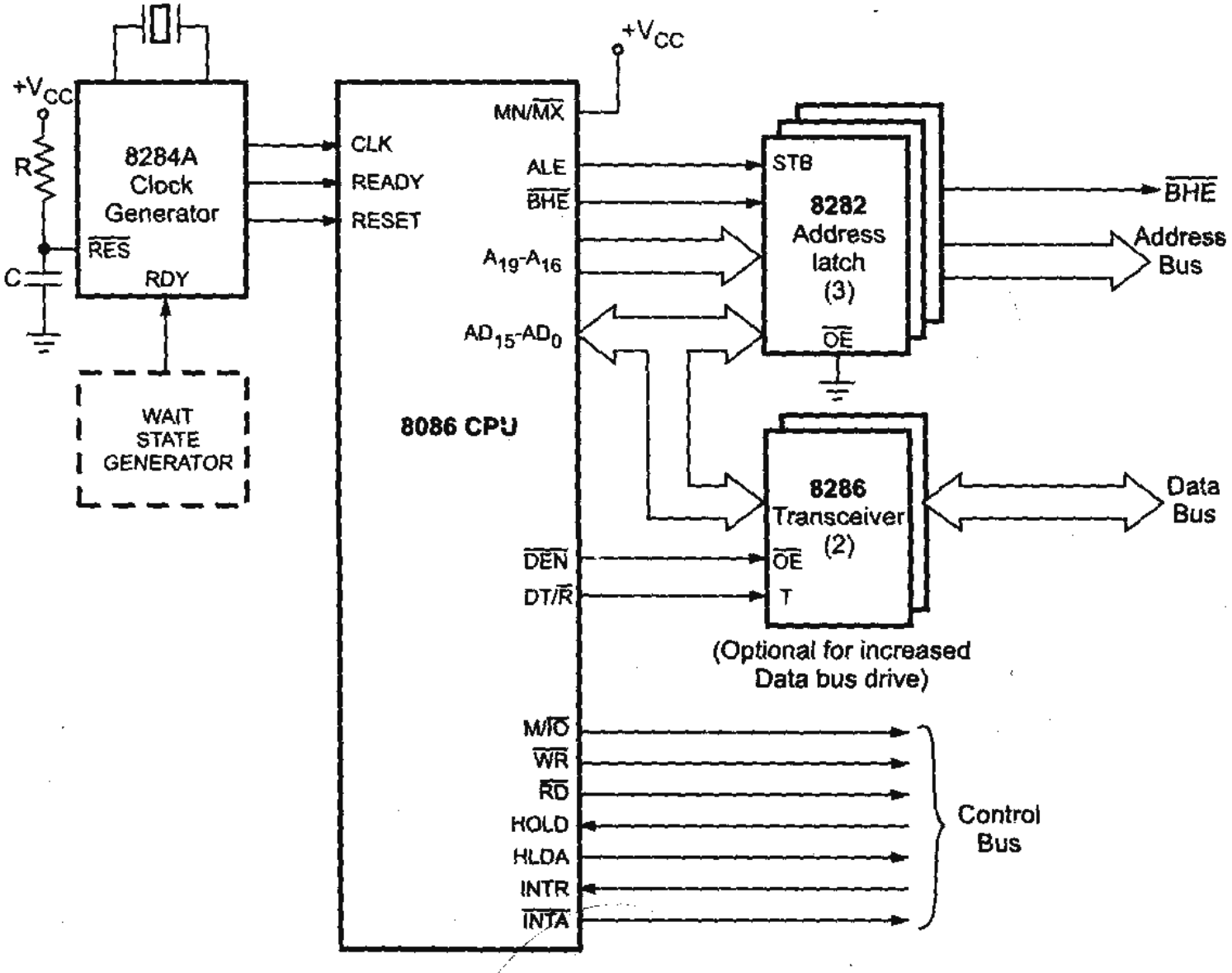


Accesul la magistrala externă - Scrierea



- Magistrala de date este de 16 biți, dar putem scrie și câte 8 la un moment dat.
- Adresele noastre accesează câte un octet

$\overline{BHE}$	$A0$	tip transfer
0	0	16 biți de la adresă pară
0	1	8 biți semnificativi de la adresă impară
1	0	8 biți nesemnificativi de la adresă pară
1	1	nimic


+VCC
R
C
8284A
Clock
Generator
RES
RDY
WAIT
STATE
GENERATOR
CLK
READY
RESET
MN/MX
ALE
BHE
A19-A16
AD15-AD0
8086 CPU
DEN
DT/R
M/IO
WR
RD
HOLD
HLDA
INTR
INTA
STB
8282
Address
latch
(3)
OE
BHE
Address
Bus
8286
Transceiver
(2)
OE
T
Data
Bus
(Optional for increased
Data bus drive)
Control
Bus

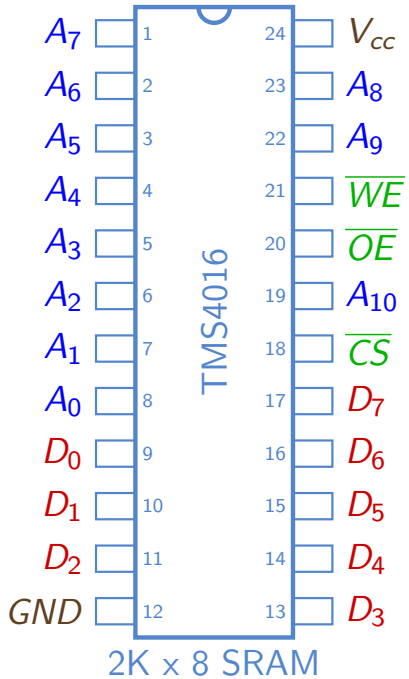


Introducere

Memorii pe interfață paralelă

Pinout general

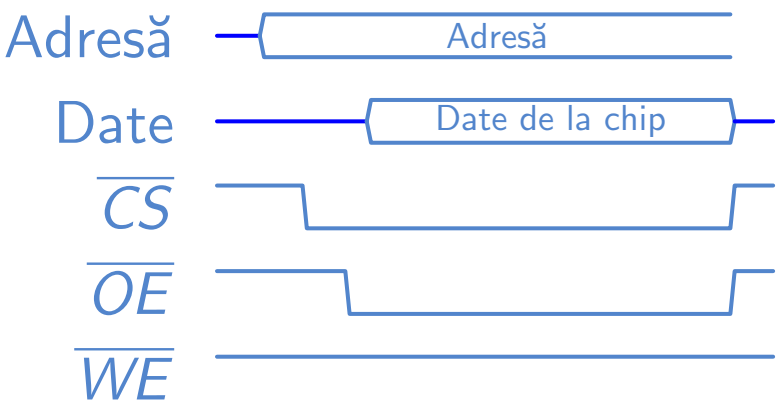
- Pini de adresă: Pe acești pini un alt dispozitiv codifică locația cuvântului de memorie dorit (N biți de adresă codifică 2^N locații
- Pini de date: La o locație sunt stocați un anumit număr de biți
- CS: pinul care selectează chip-ului. Nu se poate comunica decât cu un chip selectat

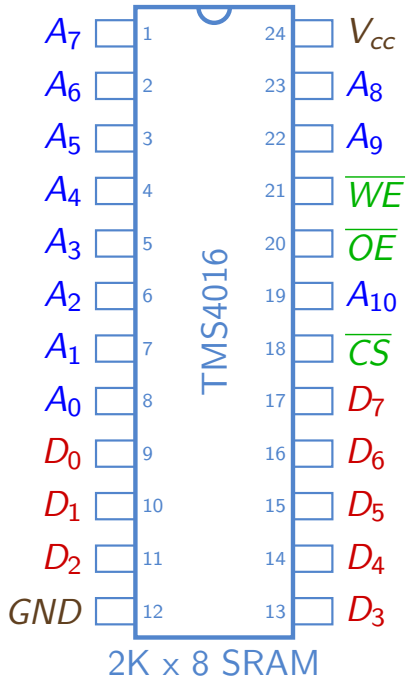


Pin(s)	Type	Function
$A_{10}..A_0$	I	Address
$D_7..D_0$	I/O	Data In/Out
$\overline{CE}$	I	Chip Enable
$\overline{OE}$	I	Output Enable
$\overline{WE}$	I	Write Enable

Pinout general

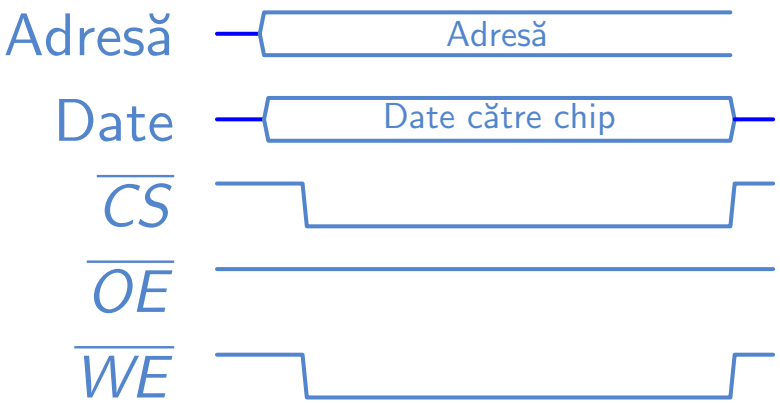
- `OE` - Output Enable: reflectă faptul că se dorește citirea din chip, iar chipului îi este permis să scrie pe magistrala de date
- Adresele sunt puse înainte de OE
- Datele sunt primite la Δt după OE

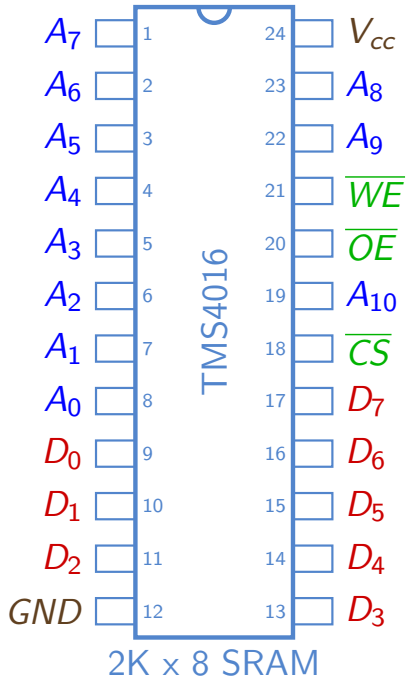




Pin(s)	Type	Function
$A_{10}..A_0$	I	Address
$D_7..D_0$	I/O	Data In/Out
$\overline{CE}$	I	Chip Enable
$\overline{OE}$	I	Output Enable
$\overline{WE}$	I	Write Enable

- `WE` - Write Enable: reflectă faptul că se dorește să se scrie în locația de memorie
- Adresele și datele sunt puse înainte de WE
- Este lăsat un timp după WE

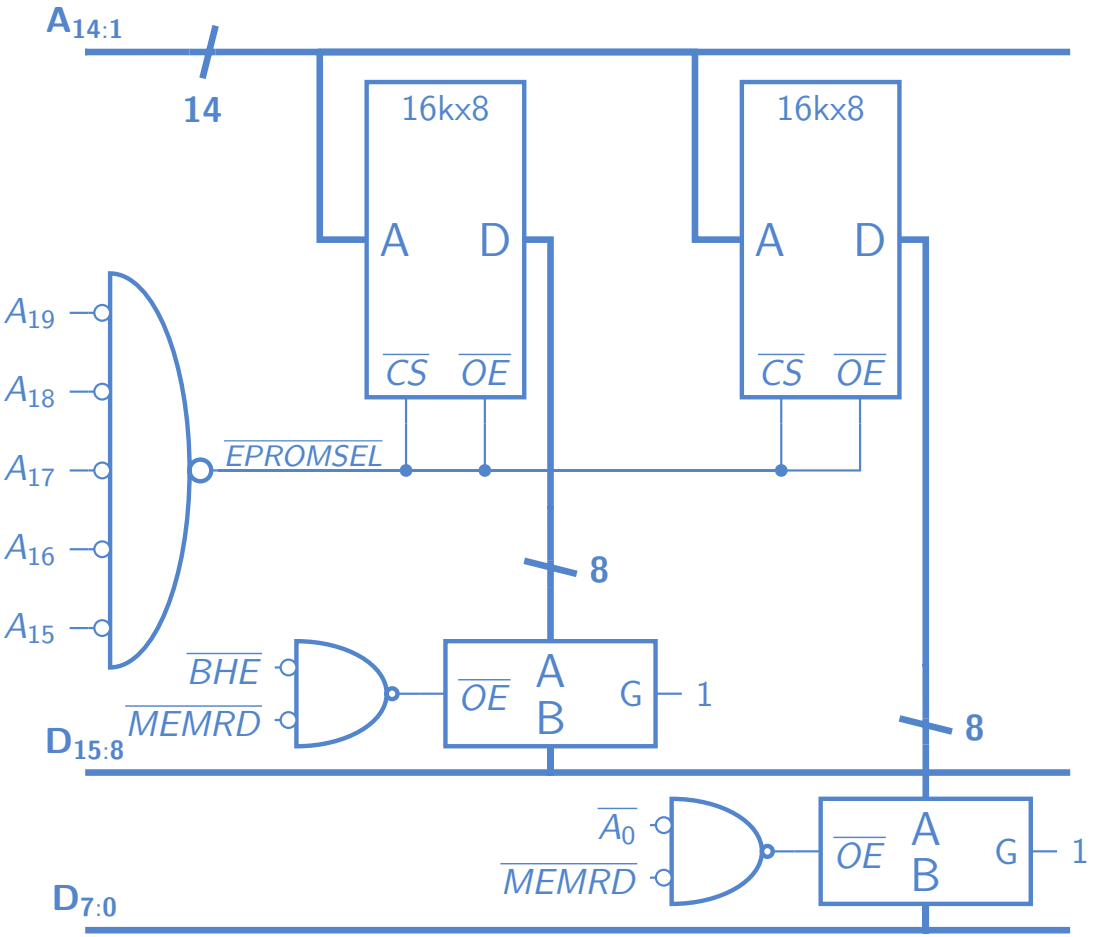




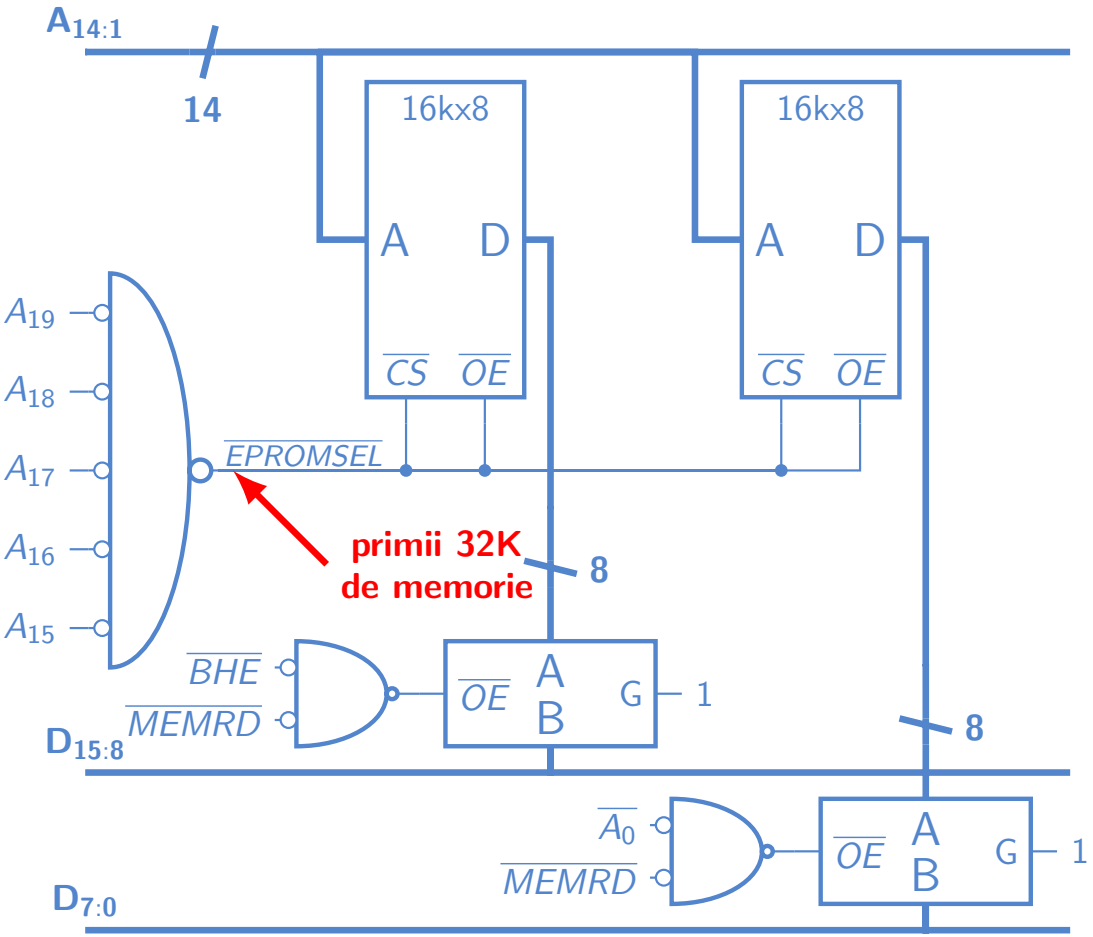
Pin(s)	Type	Function
$A_{10}..A_0$	I	Address
$D_7..D_0$	I/O	Data In/Out
$\overline{CE}$	I	Chip Enable
$\overline{OE}$	I	Output Enable
$\overline{WE}$	I	Write Enable

- 8086 are magistrala pe 16 biți
  - Conectăm câte două chip-uri x8 în paralel
  - Folosim ultimul bit de adresă A_0 pentru a selecta între ele
- 8086 are magistrala pe 16 biți

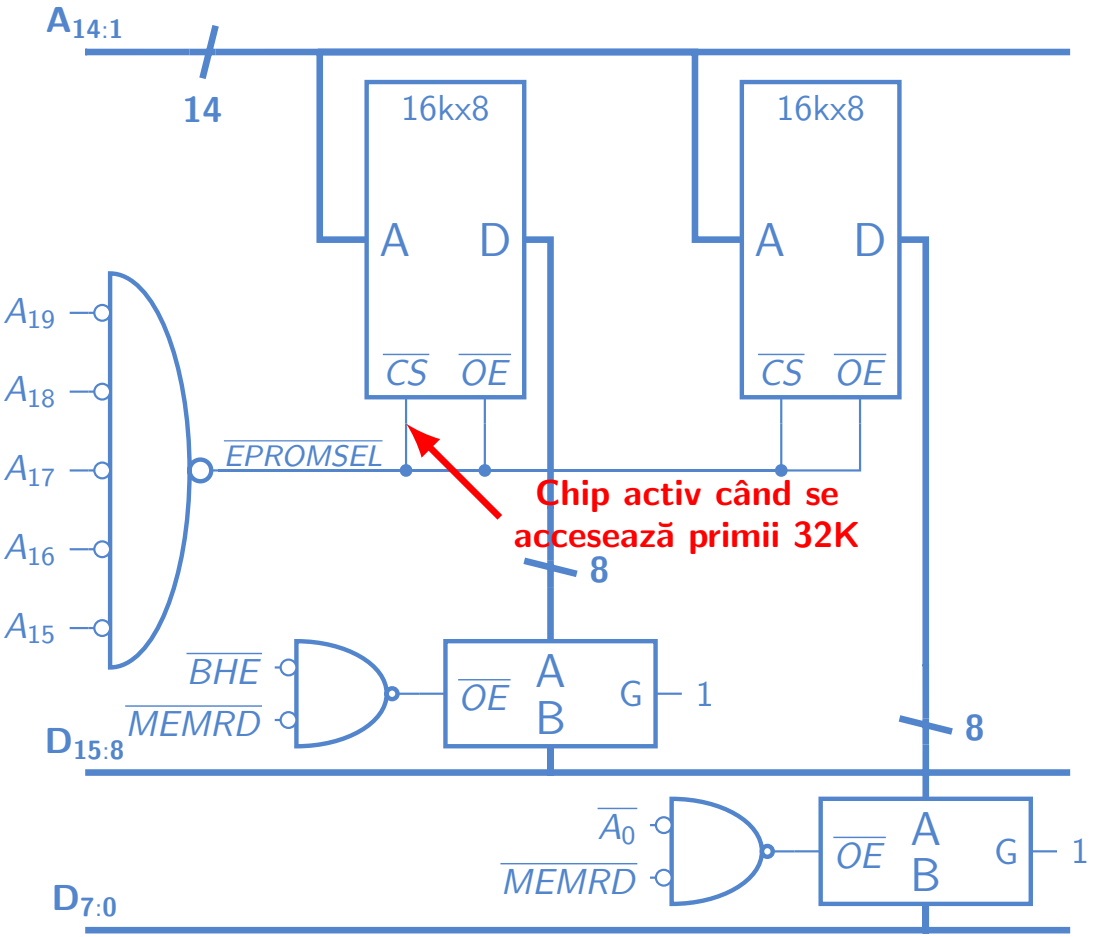
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000-0x07FFF



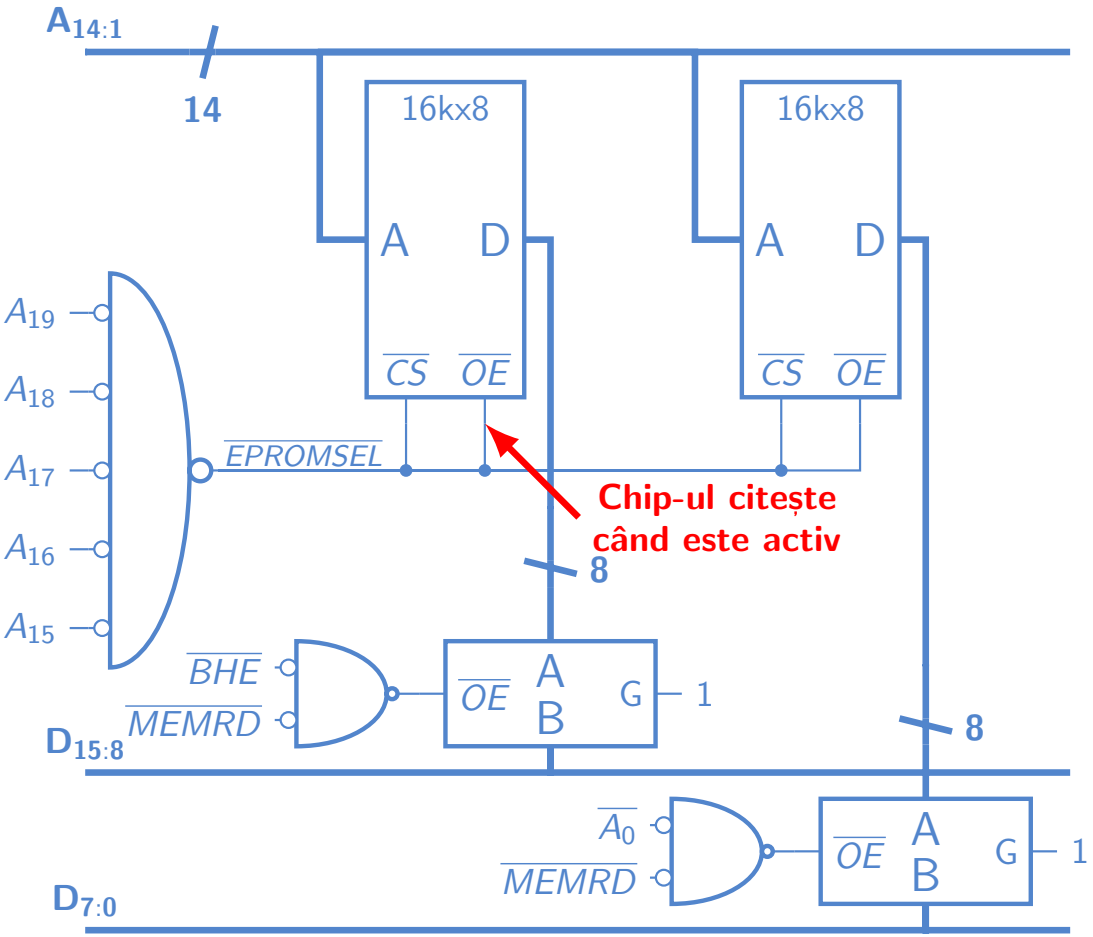
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000–0x07FFF



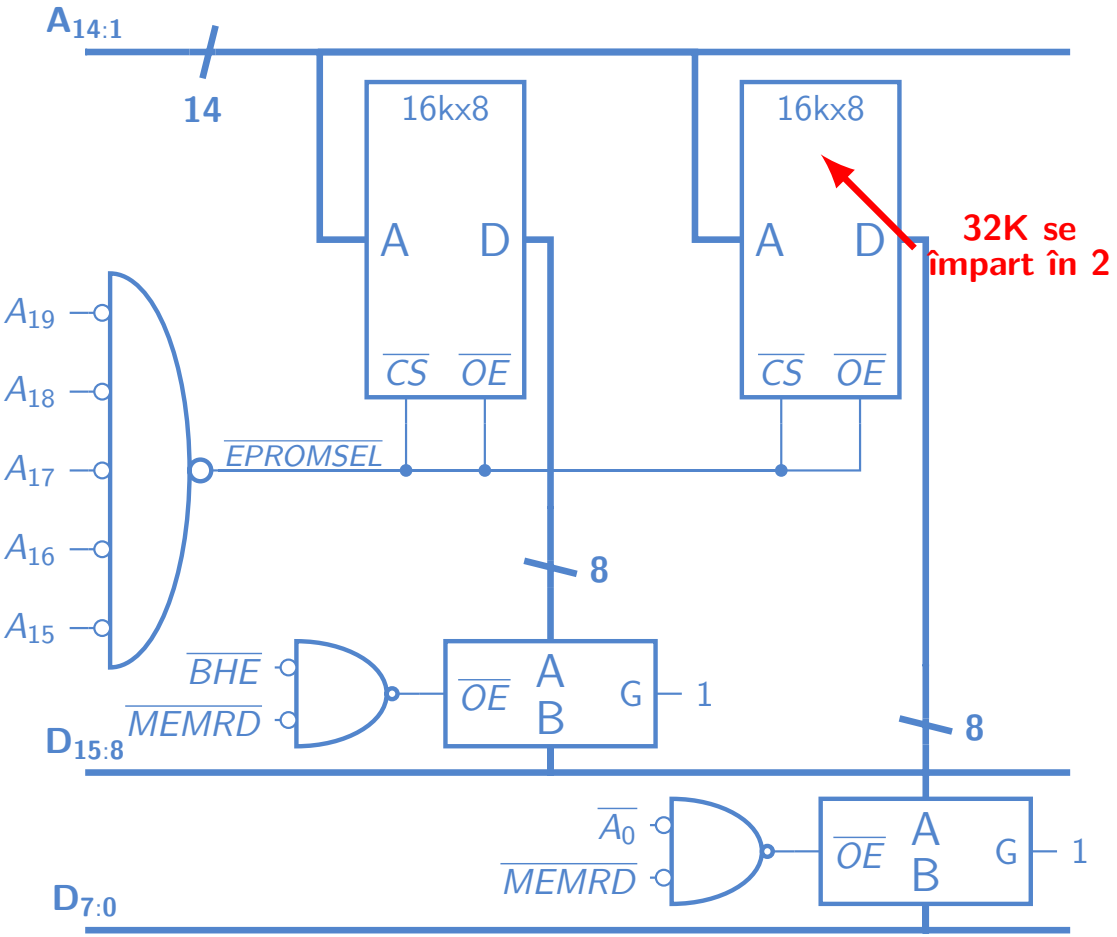
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000-0x07FFF



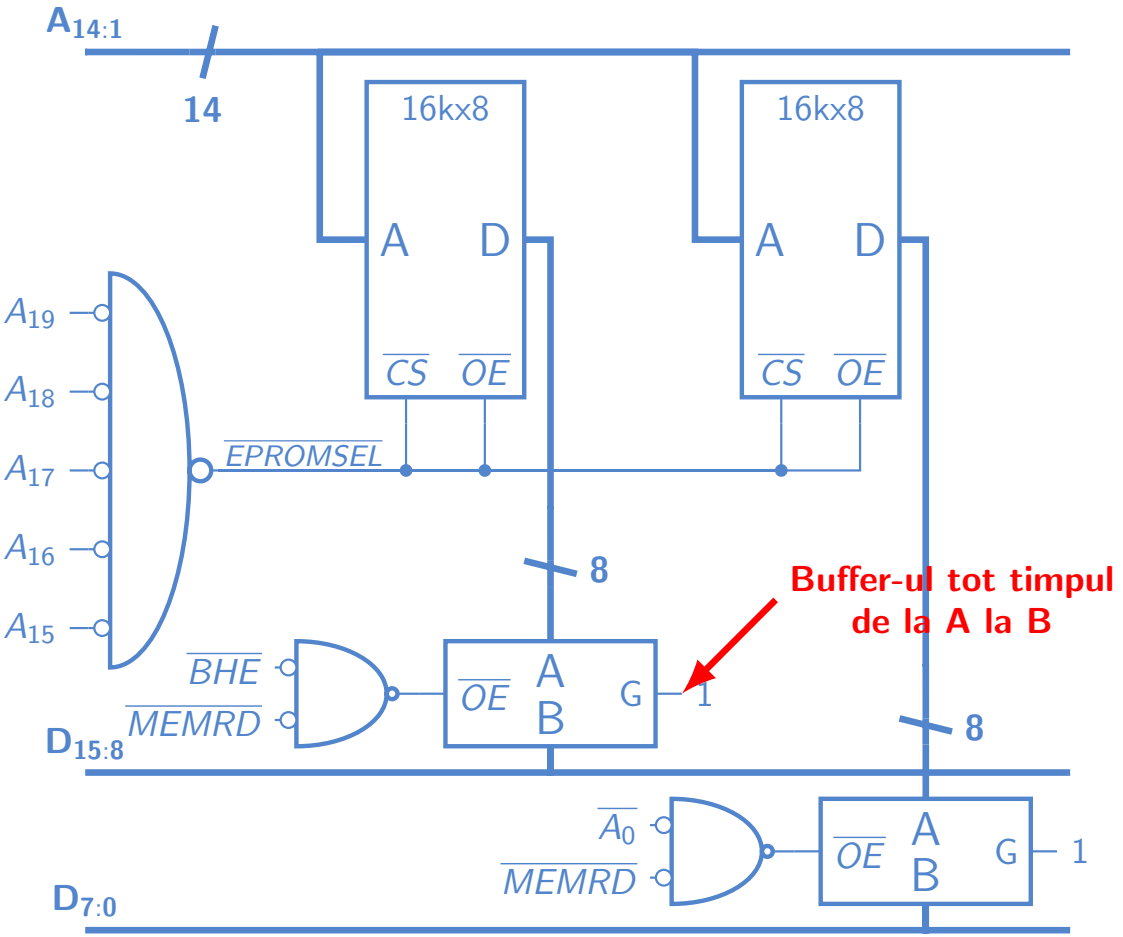
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000–0x07FFF



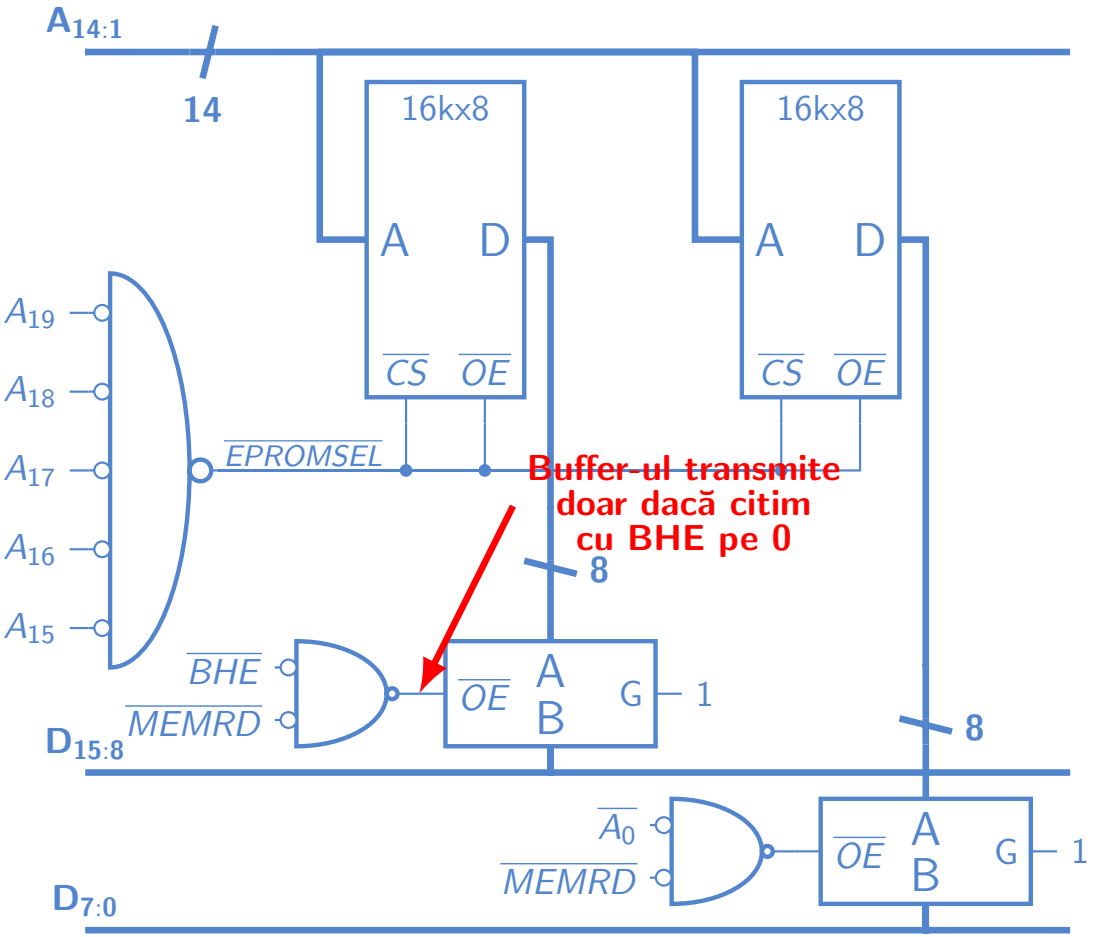
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000-0x07FFF



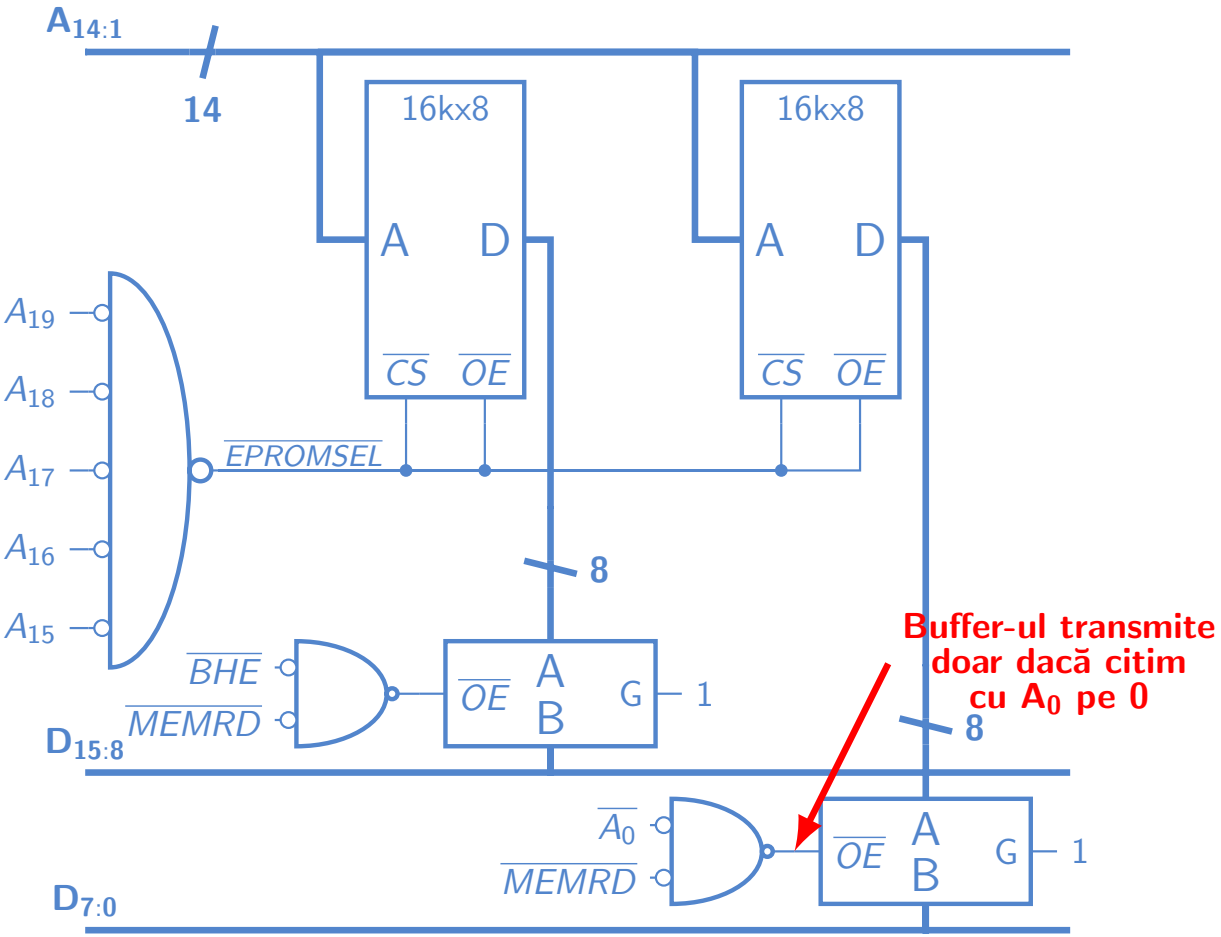
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000-0x07FFF



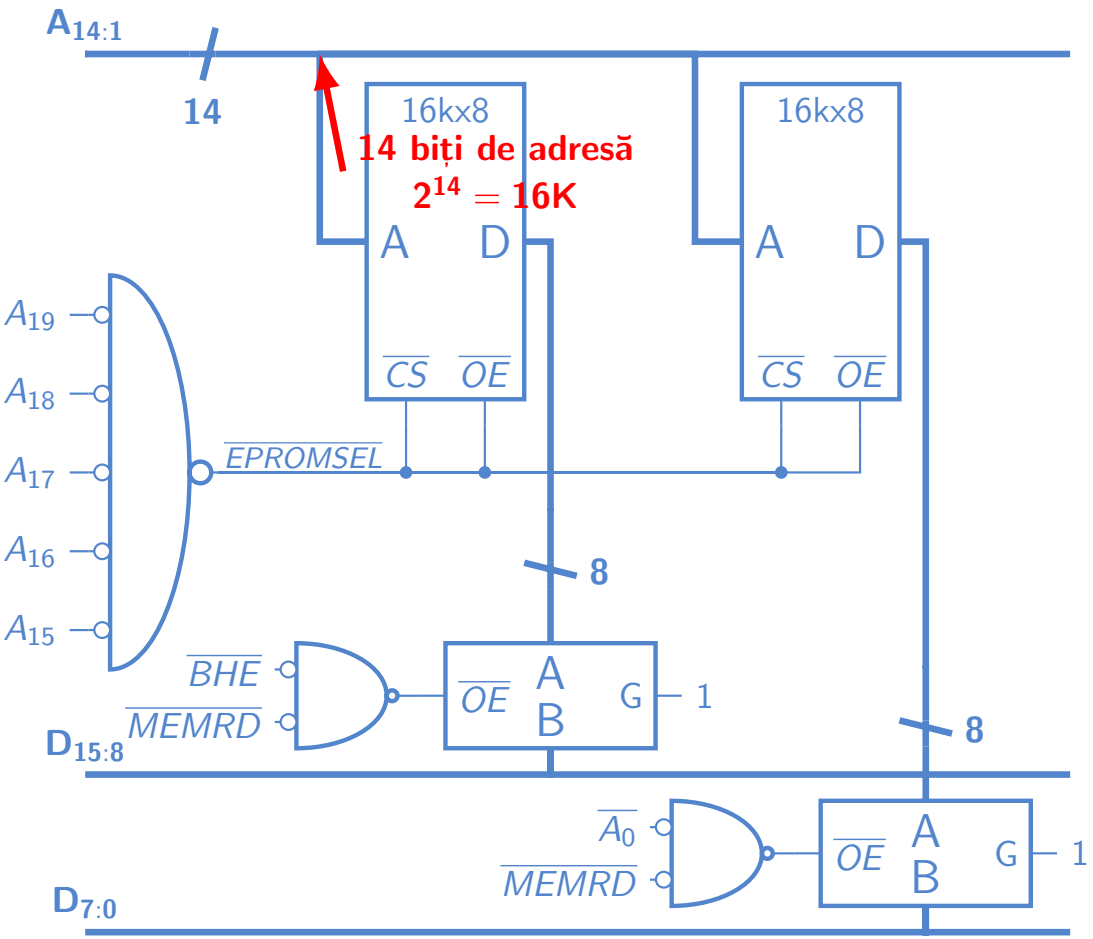
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000-0x07FFF



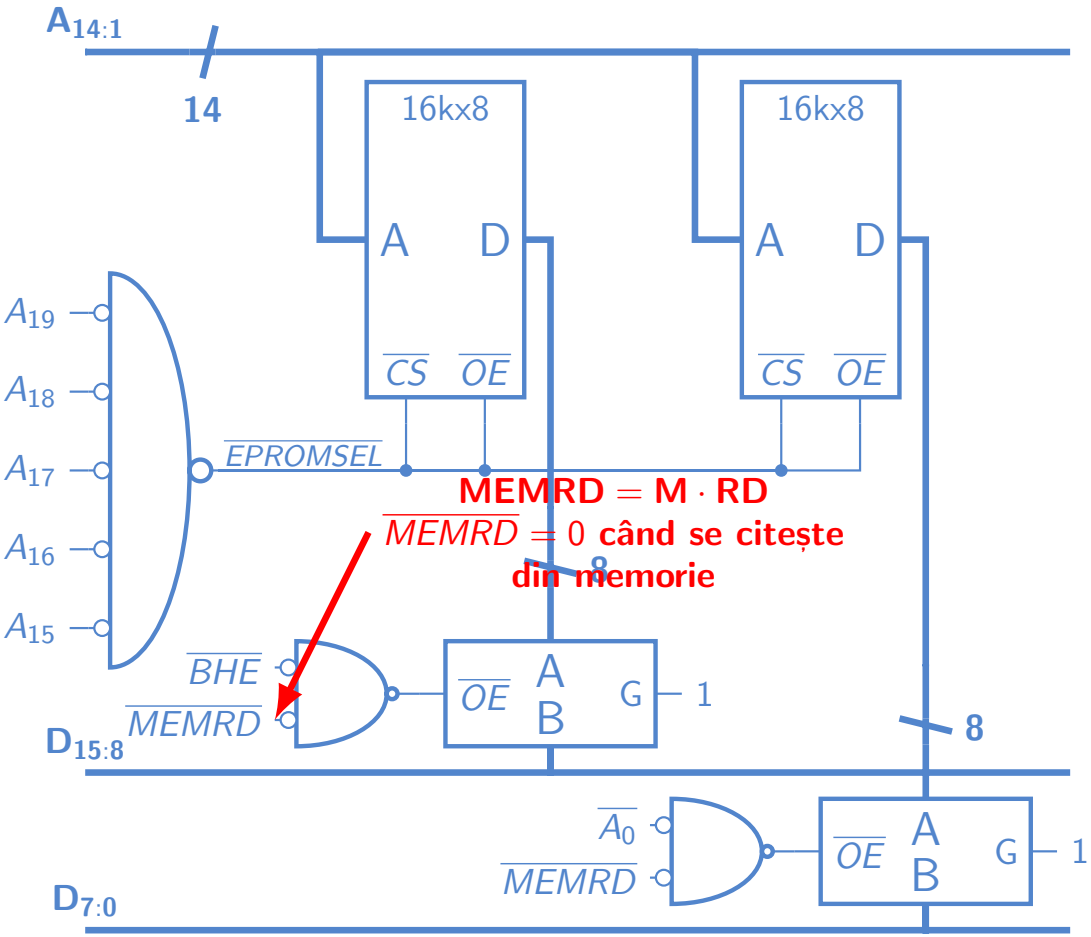
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000–0x07FFF



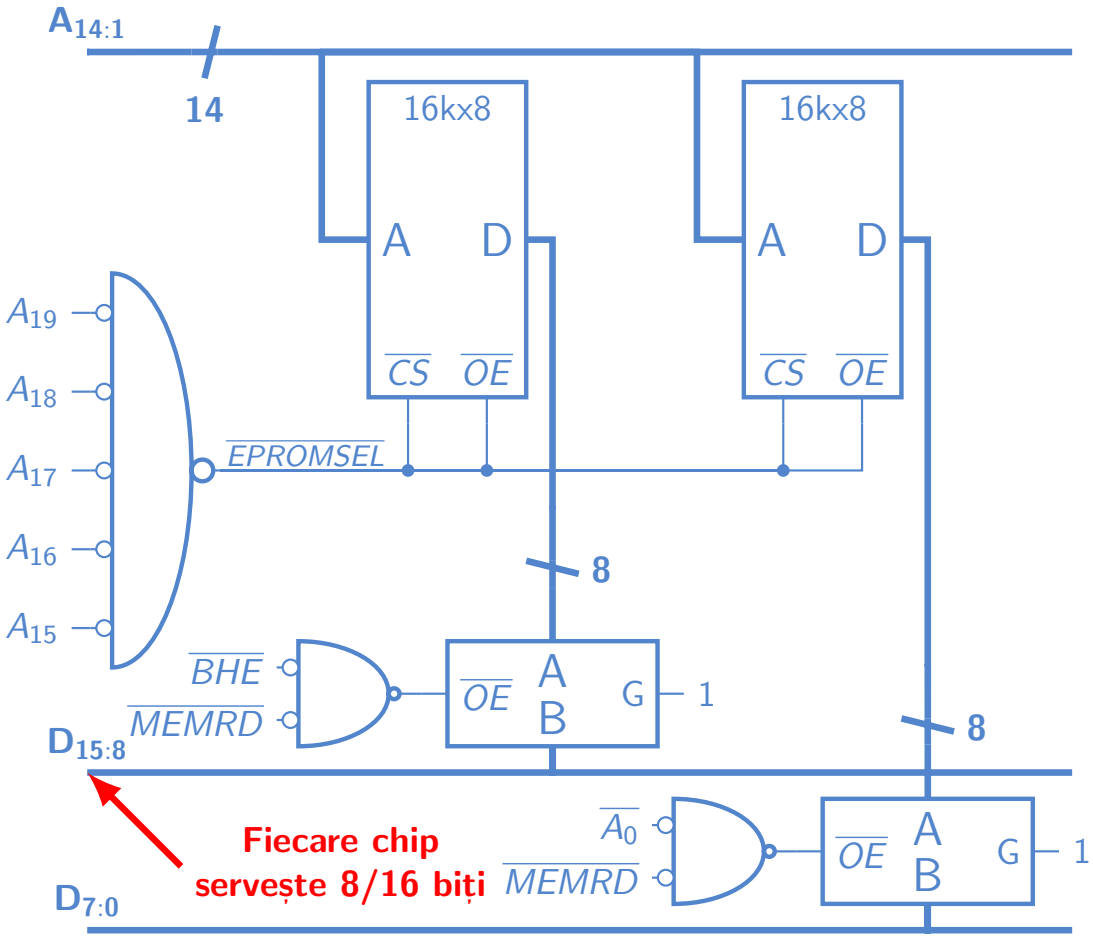
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000–0x07FFF



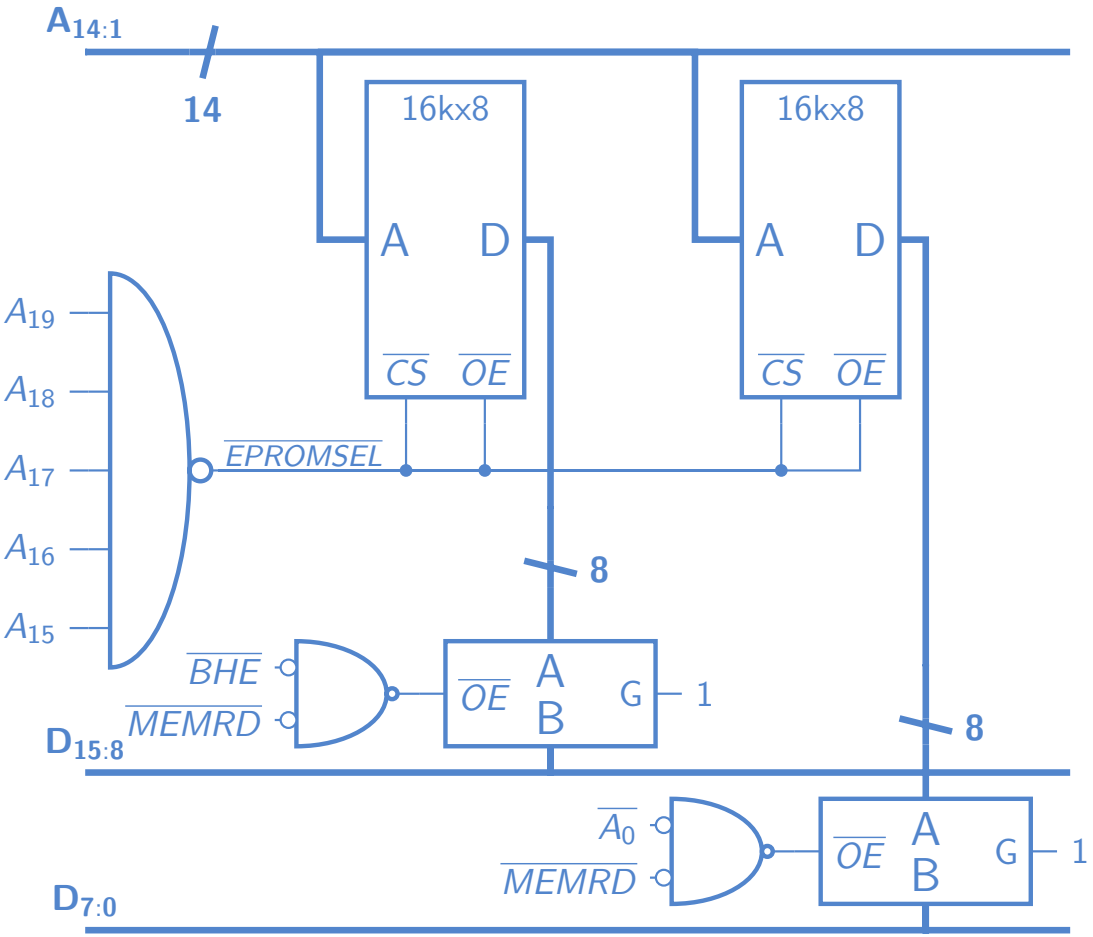
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000–0x07FFF



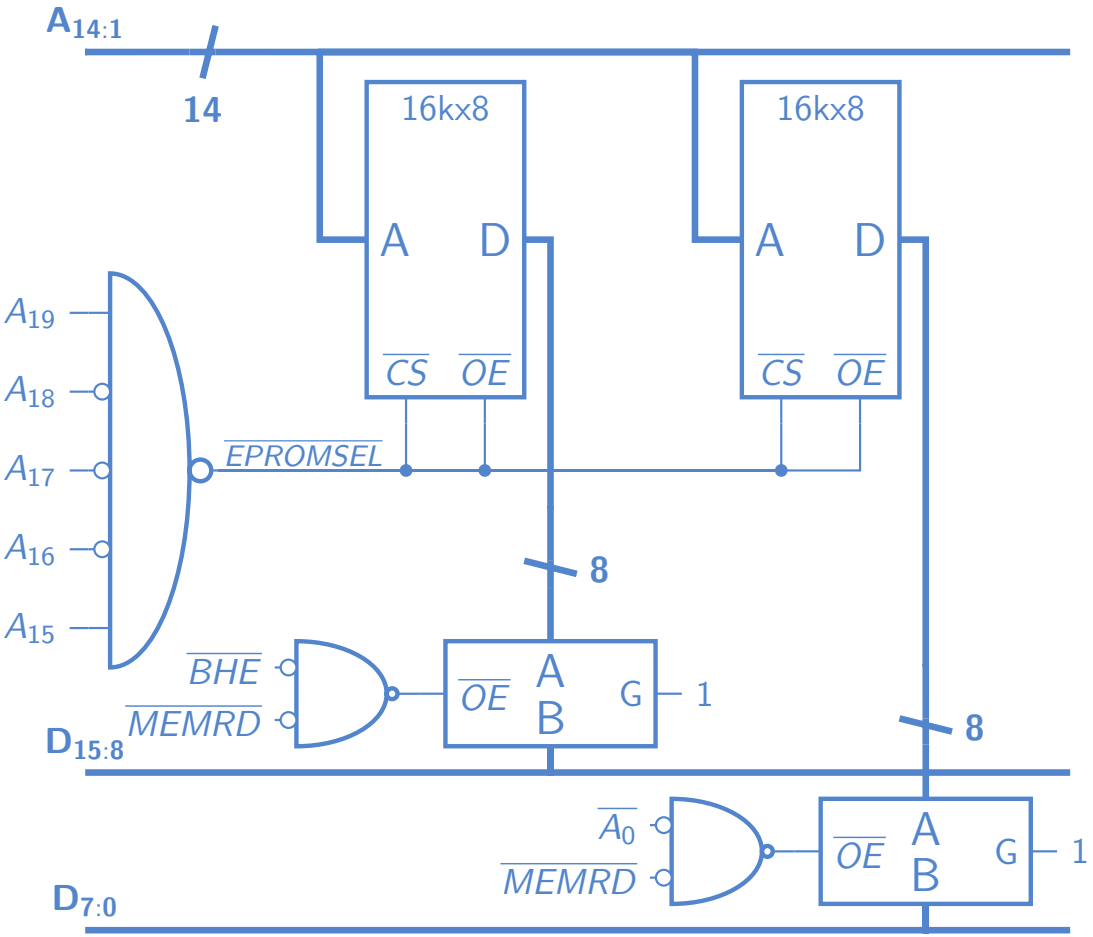
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000–0x07FFF



- Mapăm în **ultimii** 32K EPROM (read-only)
- Cu Chip-uri x8
- $EPROMSEL = A_{19} \cdot A_{18} \cdot A_{17} \cdot A_{16} \cdot A_{15}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0xF8000–0xFFFFF



- Mapăm 32K EPROM (read-only) de la adresa 0x88000
- $\overline{EPROMSEL} = A_{19} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot A_{15}$
- Range 0x88000–0x8FFFF



- Dar dacă nu vrem de la o adresă multiplu de 32K?



32K de la orice adresă

- Dar dacă nu vrem de la o adresă multiplu de 32K?
- EPROMSEL va deveni mai complicat



32K de la orice adresă

- Dar dacă nu vrem de la o adresă multiplu de 32K?
- EPROMSEL va deveni mai complicat
- EPROMSEL este un "factor comun" pentru toate locațiile
- Memoriile vor face wrap-around intern

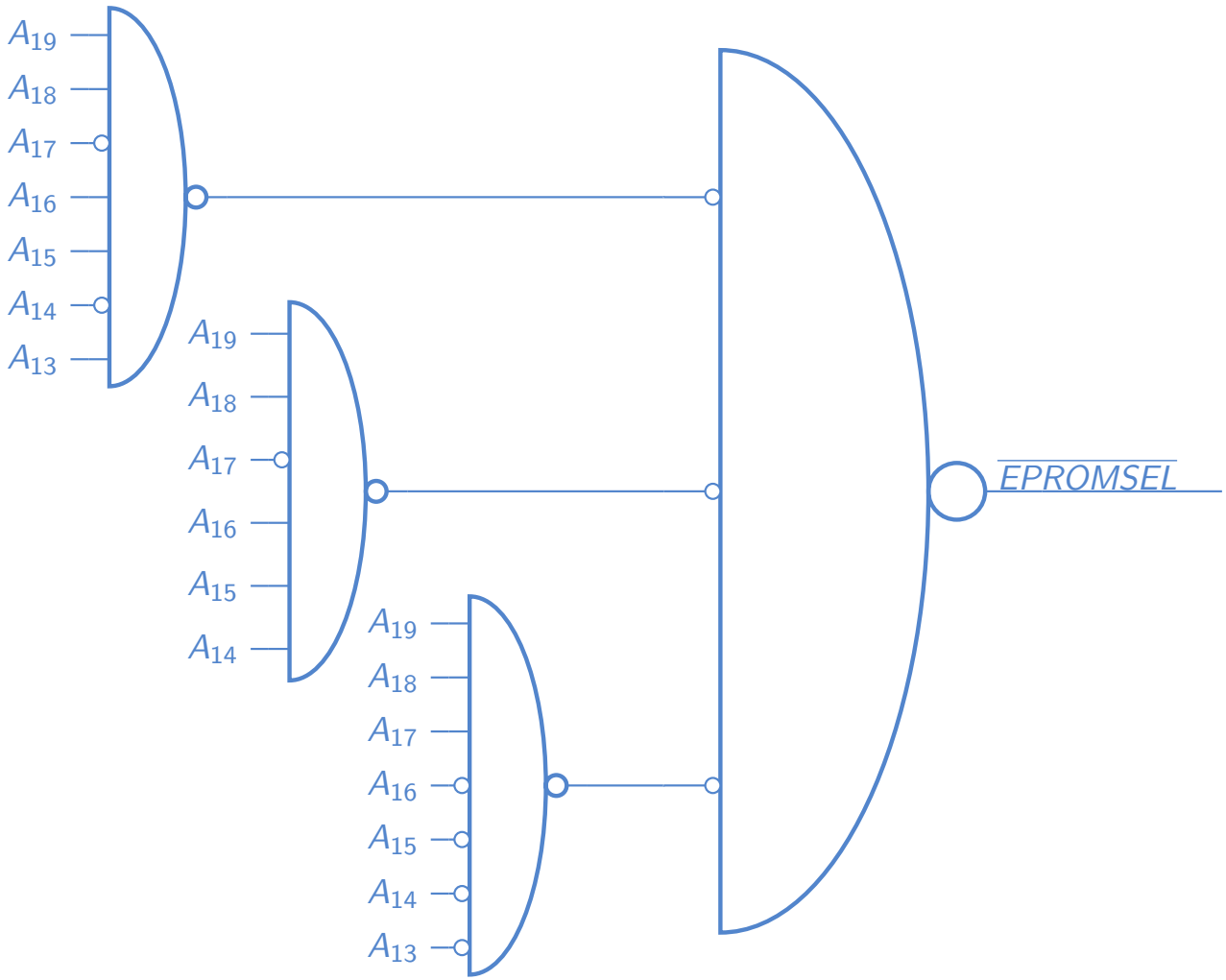
- Dar dacă nu vrem de la o adresă multiplu de 32K?
- EPROMSEL va deveni mai complicat
- EPROMSEL este un "factor comun" pentru toate locațiile
- Memoriile vor face wrap-around intern
- Ex: 32K de la `0xDA000`

$$0xDA000 + 2^{15} - 1 = 0xE1FFF$$

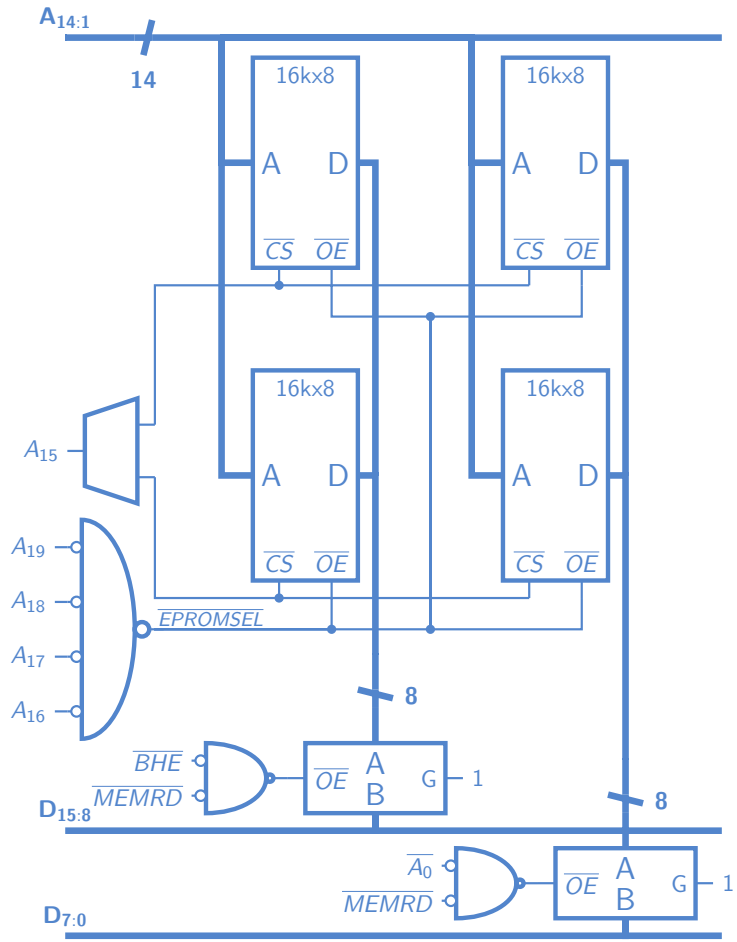
$$0xDA000 \rightarrow 0xDBFFF = A_{19} \cdot A_{18} \cdot \overline{A_{17}} \cdot A_{16} \cdot A_{15} \cdot \overline{A_{14}} \cdot A_{13}$$

$$0xDC000 \rightarrow 0xDFFFF = A_{19} \cdot A_{18} \cdot \overline{A_{17}} \cdot A_{16} \cdot A_{15} \cdot A_{14}$$

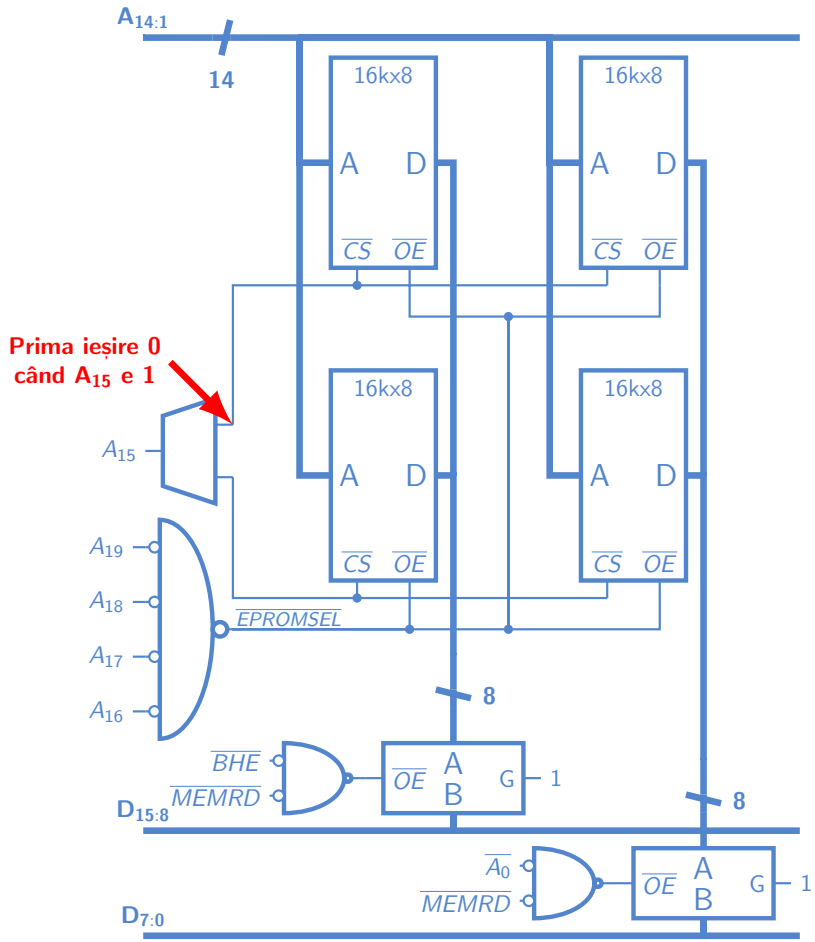
$$0xE0000 \rightarrow 0xE1FFF = A_{19} \cdot A_{18} \cdot A_{17} \cdot \overline{A_{16}} \cdot \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}}$$


A_{19}
A_{18}
A_{17}
A_{16}
A_{15}
A_{14}
A_{13}
A_{19}
A_{18}
A_{17}
A_{16}
A_{15}
A_{14}
A_{19}
A_{18}
A_{17}
A_{16}
A_{15}
A_{14}
A_{13}
$\overline{EPROMSEL}$

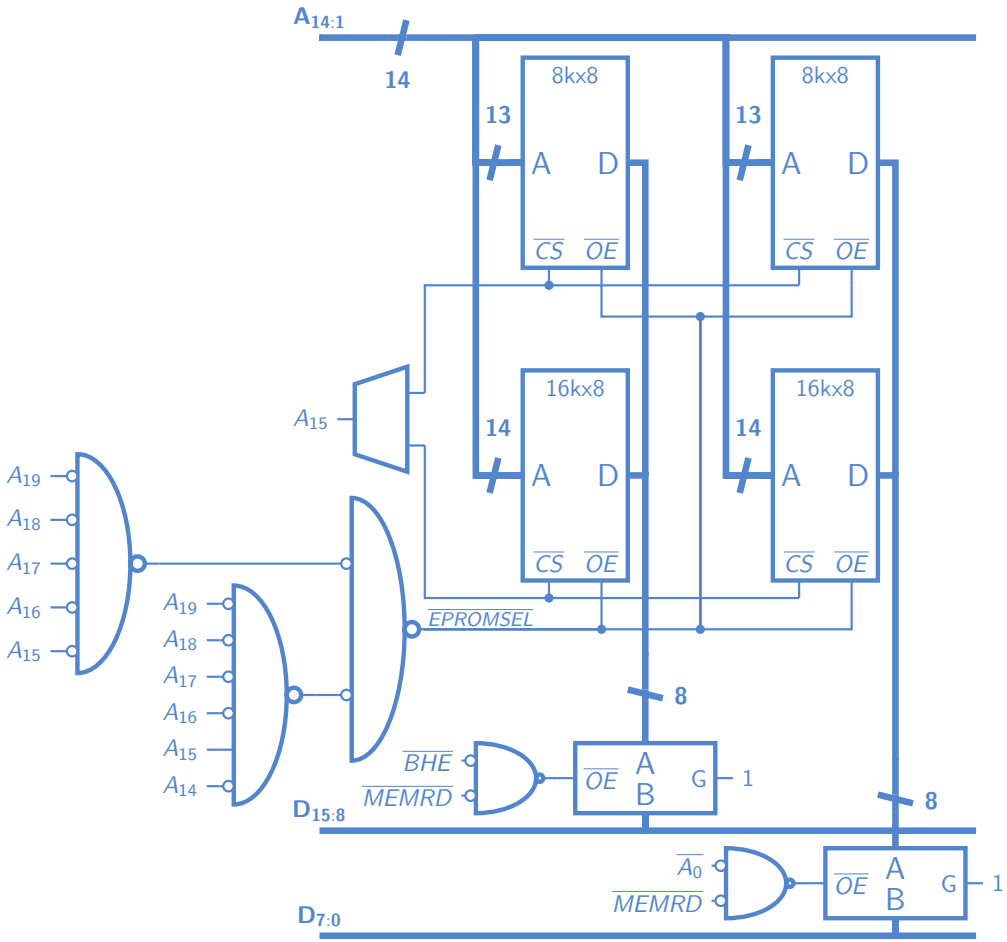
- Mapăm în primii 64K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}}}$
- *EPROMSEL* activ pe 0
- Range 0x00000–0x0FFFF



- Mapăm în primii 64K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}}}$
- *EPROMSEL* activ pe 0
- Range 0x00000–0x0FFFF

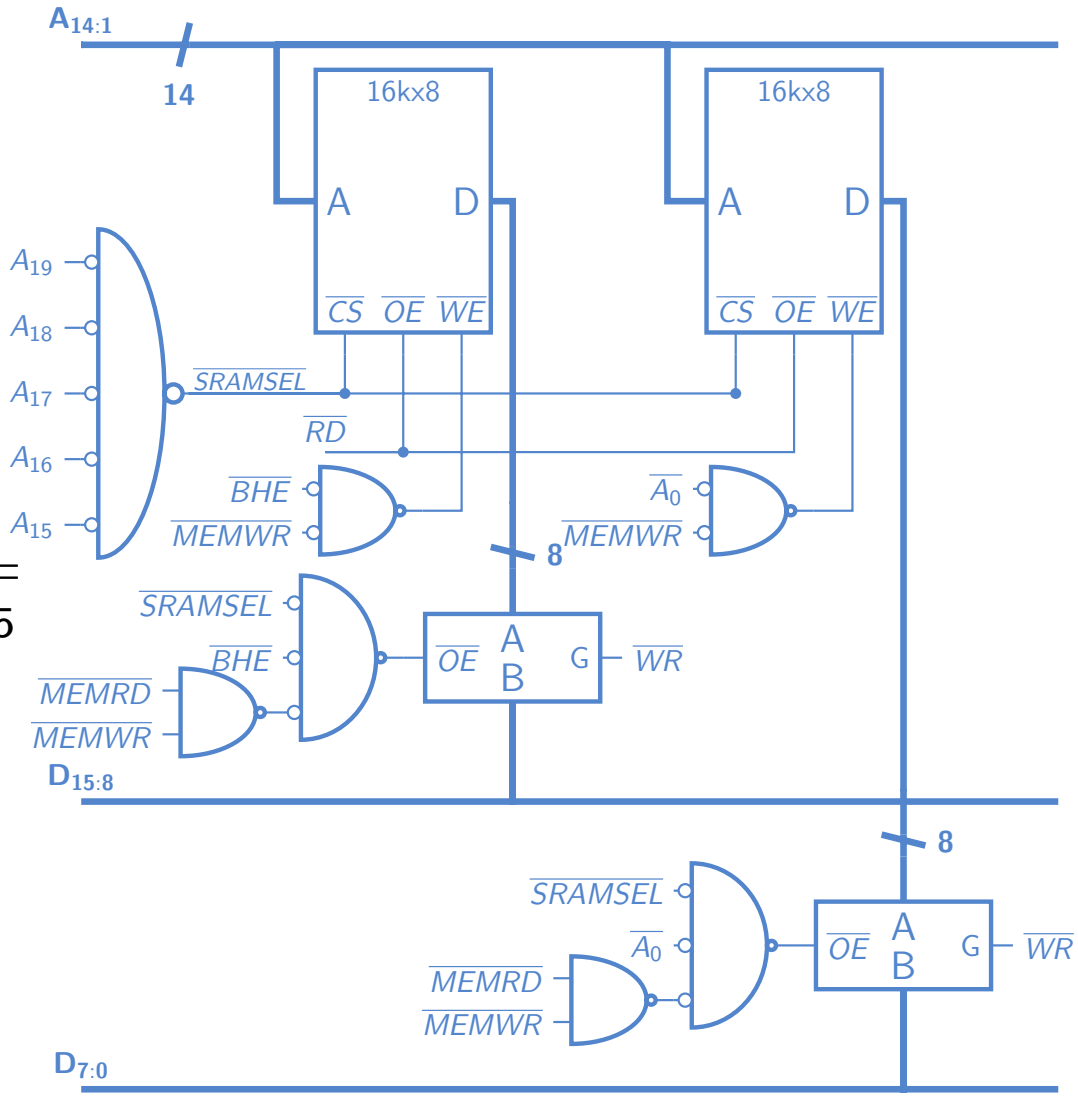


- Mapăm în primii 48K EPROM (read-only)
- Range 0x00000-0x0BFFF = 0x0000-0x07FFF + 0x08000-0x0BFFF
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}} + \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot A_{15} \cdot \overline{A_{14}}$



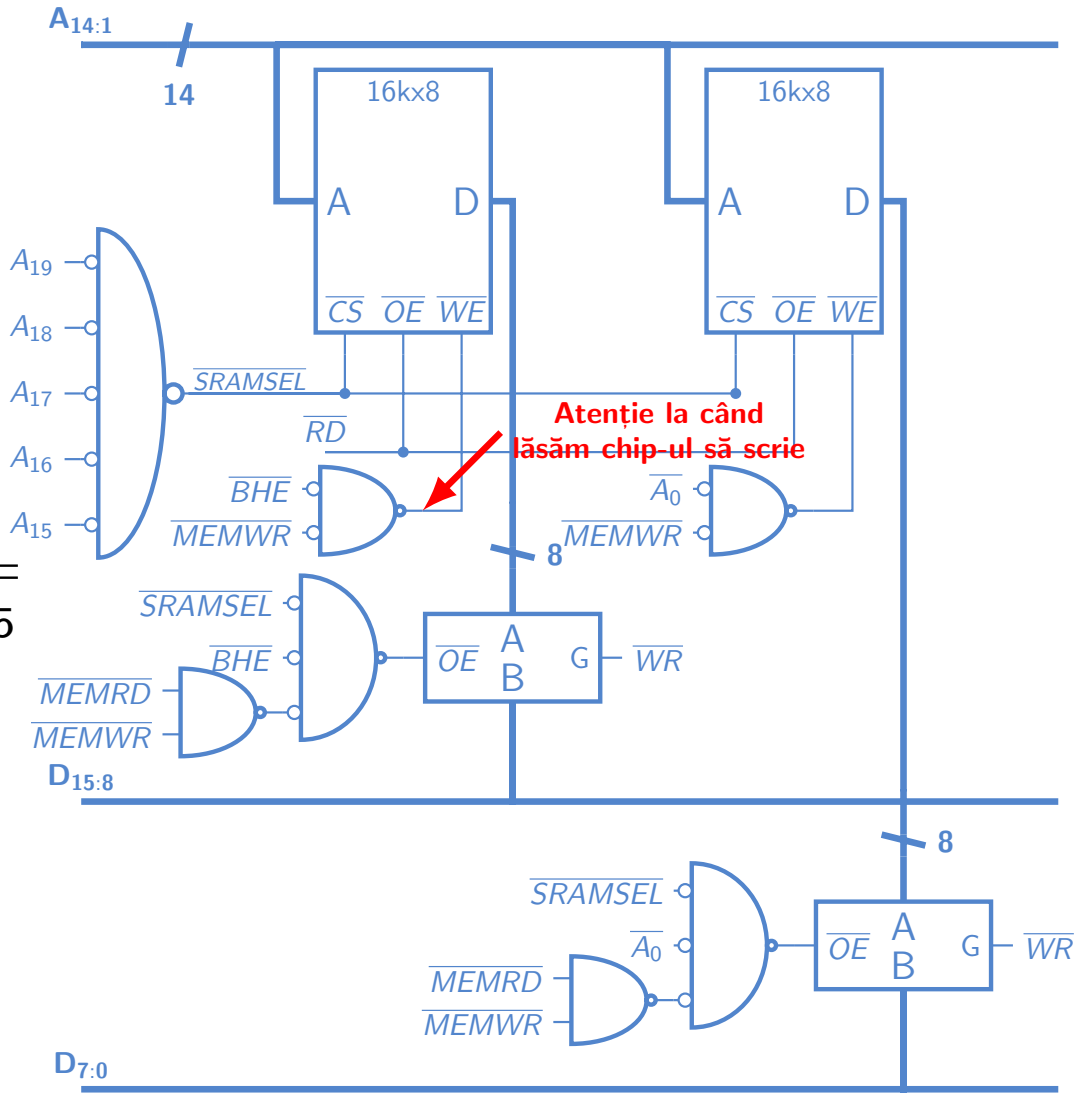
Mapare 32K SRAM

- Mapăm în primii 32K SRAM (read-write)
- Cu Chip-uri x8
- $\overline{SRAMSEL} = \overline{A_{19} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}}$
- *SRAMSEL* activ pe 0
- Range 0x00000–0x07FFF



Mapare 32K SRAM

- Mapăm în primii 32K SRAM (read-write)
- Cu Chip-uri x8
- $\overline{SRAMSEL} = \overline{\overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}}$
- *SRAMSEL* activ pe 0
- Range 0x00000–0x07FFF



- Mapăm în primii 32K SRAM (read-write)
- Cu Chip-uri x8
- $\overline{SRAMSEL} = \overline{A_{19} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}}$
- *SRAMSEL* activ pe 0
- Range 0x00000–0x07FFF

